CSCI-GA 2420 / MATH-GA 2010

Numerical Methods I

Course Notes

Fall 2026

Instructor: Prof. Florian Schaefer

Notes by You Li

Acknowledgements

These notes follow the Fall 2026 lectures of CSCI-GA 2420 / MATH-GA 2010 (Numerical Methods I) at the Courant Institute, taught by Prof. Florian Schaefer. Each chapter follows one lecture. Sections marked (SUPPLEMENT) go beyond the blackboard and draw on Trefethen and Bau [1], Demmel [2], and Golub and Van Loan [5]; background from linear algebra follows Axler [4].

Notation. Throughout, $\mathbb{K}$ denotes either $\mathbb{R}$ or $\mathbb{C}$. Vectors carry an arrow, $\vec{x} \in \mathbb{K}^n$; for $A \in \mathbb{K}^{m\times n}$, A^* is the conjugate transpose (equal to $A^\top$ when $\mathbb{K} = \mathbb{R}$), $\mathcal{R}(A)$ is the range (column space), and $\mathcal{N}(A)$ is the null space. The symbol $:=$ means "is defined as", with the colon on the side of the name being defined ($=:$ when that name stands on the right). Unless stated otherwise, $\|\cdot\|$ is the Euclidean norm on vectors and the induced 2-norm on matrices. Absolute values and inequalities between matrices, such as $|E| \leq |L|\,|U|$, are taken entrywise.

Contents

1 Floating-Point Representation, Conditioning, and Stability **6**
- 1.1 Overview . . . 6
- 1.2 Vectors and Matrices . . . 6
- 1.3 Representing Real Numbers . . . 7
- 1.4 The Geometry of Floating-Point Numbers . . . 9
- 1.5 Cancellation . . . 9
- 1.6 Conditioning . . . 10
- 1.7 Stability . . . 10
- 1.8 (SUPPLEMENT) Further Topics . . . 12

2 Cache Hierarchy and Tiling/Blocking, Solving Linear Systems **14**
- 2.1 Flops Are Not the Whole Story . . . 14
- 2.2 Memory Is One-Dimensional . . . 15
- 2.3 The Memory Hierarchy . . . 15
- 2.4 Loop Reordering . . . 16
- 2.5 Tiling . . . 16
- 2.6 Linear Systems . . . 17
- 2.7 Triangular Systems . . . 17
- 2.8 Gaussian Elimination . . . 18
- 2.9 (SUPPLEMENT) Further Topics . . . 19

3 LU Factorization and Its Stability **21**
- 3.1 Recap: From Triangular Systems to Elimination . . . 21
- 3.2 The Algebra of Lower Triangular Matrices . . . 21
- 3.3 Answering the Two Questions . . . 23
- 3.4 The In-Place Algorithm . . . 23
- 3.5 Zero Pivots . . . 24
- 3.6 Small Pivots Are Also Dangerous . . . 24
- 3.7 Error Analysis of the LU Factorization . . . 25
- 3.8 (SUPPLEMENT) Further Topics . . . 25

4 LU Factorization with Pivoting **27**
- 4.1 Backward Error of the LU Factorization . . . 27
- 4.2 Pivots and the Growth of Products . . . 29
- 4.3 Partial Pivoting . . . 29
- 4.4 The In-Place Algorithm with Partial Pivoting . . . 30

4.5 (SUPPLEMENT) Further Topics . . . 31

5 Least Squares Problems, Gram–Schmidt, and QR 33
5.1 Is Partial Pivoting Enough? . . . 33
5.2 Non-Square Systems . . . 34
5.3 Least Squares Problems . . . 34
5.4 The Geometry of Least Squares . . . 35
5.5 Orthogonalization: The QR Factorization . . . 36
5.6 Classical Gram–Schmidt . . . 36
5.7 Modified Gram–Schmidt . . . 38
5.8 (SUPPLEMENT) Further Topics . . . 40

6 Product View on Error Bounds, Householder QR 42
6.1 Why Write Algorithms as Matrix Products? . . . 42
6.2 The Product Form of Gram–Schmidt . . . 42
6.3 Error Amplification in Products . . . 45
6.4 Lower Triangularization: The Case of LU . . . 46
6.5 Gram–Schmidt Is Also a Lower Triangularization . . . 47
6.6 Householder Triangularization . . . 47
6.7 (SUPPLEMENT) Further Topics . . . 51

7 Householder QR in Practice, Givens Rotations, Existence and Uniqueness of QR 53
7.1 Picking Up from Householder QR . . . 53
7.2 Using Q . . . 53
7.3 Givens Rotations . . . 55
7.4 Full and Reduced QR . . . 58
7.5 Existence . . . 58
7.6 Uniqueness . . . 59
7.7 (SUPPLEMENT) Further Topics . . . 60

8 Overdetermined Least Squares and the Normal Equations 62
8.1 Recap and Motivation . . . 62
8.2 Conventions . . . 63
8.3 The Optimality Condition . . . 64
8.4 What the Optimality Condition Says . . . 66
8.5 The Normal Equations . . . 67
8.6 Gram Matrices and Positive Semidefinite Matrices . . . 68
8.7 (SUPPLEMENT) Further Topics . . . 70

9 Cholesky Factorization 71
9.1 Recap and Motivation . . . 71
9.2 Deriving Cholesky by Symmetric Elimination . . . 72
9.3 The Cholesky Algorithm . . . 74
9.4 Existence: Cholesky Never Breaks Down . . . 77
9.5 Rounding Errors . . . 78
9.6 Schur Complements . . . 80
9.7 Cholesky and QR . . . 83
9.8 (SUPPLEMENT) Further Topics . . . 84

10 Eigen-like Decompositions, Computing a Single Eigenvalue **86**
10.1 Eigen-Type Factorizations . . . 86
10.2 Eigenpairs of Hermitian Matrices . . . 87
10.3 A Negative Result . . . 88
10.4 Power Iteration . . . 89
10.5 Matrix Functions: Changing the Spectrum . . . 93
10.6 Inverse Iteration . . . 94
10.7 Rayleigh Quotient Iteration . . . 96
10.8 (SUPPLEMENT) Further Topics . . . 99

11 Schur Factorization **100**
11.1 Recap and Motivation . . . 100
11.2 Deflation . . . 100
11.3 Subspace Iteration: Motivation . . . 101
11.4 Eigenvalue Problems as Matrix Factorizations . . . 102
11.5 Defective Matrices and Invariant Subspaces . . . 102
11.6 The Jordan Normal Form . . . 104
11.7 The Schur Factorization . . . 105
11.8 Computing Eigenvectors from the Schur Form . . . 107
11.9 (SUPPLEMENT) Further Topics . . . 108

12 Orthogonal Iteration **111**
12.1 Recap and Motivation . . . 111
12.2 From Schur Factorization to Diagonalization . . . 111
12.3 Orthogonal Iteration . . . 112
12.4 Orthogonal Iteration in General . . . 114
12.5 Convergence of Eigenvector Estimates . . . 115
12.6 Convergence to Schur Factorization . . . 116
12.7 (SUPPLEMENT) Further Topics . . . 116

13 Convergence of Orthogonal Iteration and the Basic QR Iteration **118**
13.1 Recap and Motivation . . . 118
13.2 Schur and Eigendecomposition . . . 118
13.3 Convergence of Orthogonal Iteration . . . 119
13.4 Convergence to Schur Factorization . . . 120
13.5 Basic QR Iteration . . . 120
13.6 (SUPPLEMENT) Further Topics . . . 121

14 Hessenberg QR **124**
14.1 Recap and Motivation . . . 124
14.2 Improvements to QR . . . 124
14.3 Reduction to Hessenberg Form . . . 125
14.4 QR Iteration on H . . . 125
14.5 Bulge Chasing . . . 126
14.6 The Implicit Q Theorem . . . 127
14.7 (SUPPLEMENT) Further Topics . . . 127

15 Shifted QR **129**

15.1 Recap and Motivation . . . 129
15.2 Proof of the Implicit Q Theorem . . . 129
15.3 Application to Bulge Chasing . . . 130
15.4 The Shifted QR Iteration . . . 130
15.5 Choosing the Shift . . . 131
15.6 Relationship with (Inverse) Power Iteration . . . 132
15.7 (SUPPLEMENT) Further Topics . . . 133

A Linear Algebra Basics **135**
A.1 Matrix Multiplication . . . 135
A.2 Transpose and Conjugate Transpose . . . 136
A.3 Inverses, Elementary and Triangular Matrices . . . 137
A.4 Determinants . . . 138
A.5 Subspaces and Rank . . . 138
A.6 Eigenvalues and Diagonalization . . . 139
A.7 Orthogonality and Projections . . . 139
A.8 Hermitian Matrices, Quadratic Forms, and the SVD . . . 140

B Norms and Unitary Matrices **141**
B.1 Vector and Matrix Norms . . . 141
B.2 Unitary Transformations . . . 146
B.3 The 2-Norm, Singular Values, and Condition Numbers . . . 148
B.4 The Frobenius Norm . . . 149

C The Floating-Point Model **152**
C.1 IEEE 754 Formats . . . 152
C.2 Machine Epsilon and the Model of Arithmetic . . . 153
C.3 Backward Stability . . . 154

Lecture 1

Floating-Point Representation, Conditioning, and Stability

Supplementary reading:

- [1] Lectures 12, 13, and 14.
- [2] Sections 1.3–1.5.
- [5] Section 2.4.

1.1 Overview

The topics of the semester are: numerical representation of vectors and matrices; solving linear systems by Gaussian elimination; solving least squares problems by orthogonalization; computing eigendecompositions; low-rank approximation; iterative methods; and structured matrices.

This lecture covers the first topic. A computer cannot store real numbers exactly: every number is rounded when it is stored, and every arithmetic operation rounds again, so even $(x + a) - a = x$ can fail. We proceed in two parts.

1. *Representation* (Sections 1.3 to 1.5): how real numbers are stored (floating point) and what errors this causes.
2. *Error analysis* (Sections 1.6 and 1.7): how to tell whether an error comes from the problem (*conditioning*) or from the algorithm (*stability*).

1.2 Vectors and Matrices

We write $\mathbb{K}$ for either $\mathbb{R}$ or $\mathbb{C}$, so that every result only needs to be stated once. A *vector* $\vec{v} \in \mathbb{K}^m$ is an m-tuple of elements of $\mathbb{K}$, representing a position or direction in an m-dimensional coordinate system. A *matrix* $A \in \mathbb{K}^{m \times n}$ is an $m \times n$ array of elements of $\mathbb{K}$, which may be thought of as n positions in an m-dimensional

coordinate system. We identify $\mathbb{K}^{m\times 1} \cong \mathbb{K}^m$, so a column vector is a matrix with one column:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \\ A_{31} & A_{32} \end{bmatrix} = \begin{bmatrix} \vec{a}_1 & \vec{a}_2 \end{bmatrix}. \tag{1.1}$$

Scalar multiplication and addition act entrywise. For $A \in \mathbb{K}^{m\times n}$, $\vec{v} \in \mathbb{K}^n$, and $B \in \mathbb{K}^{n\times \ell}$,

$$(A\vec{v})_i = \sum_k A_{ik} v_k, \qquad (AB)_{ij} = \sum_k A_{ik} B_{kj}, \tag{1.2}$$

so A is a linear map $\mathbb{K}^n \to \mathbb{K}^m$ and $\mathbb{K}^{n\times\ell} \to \mathbb{K}^{m\times\ell}$: it carries a region, together with the vectors $\vec{x}$ in it, to the image vectors $A\vec{x}$.

The entrywise formula is rarely the most useful way to think. Numerical linear algebra relies much more on the *column view*:

$$A\vec{v} = \sum_{j=1}^{n} v_j \vec{a}_j, \tag{1.3}$$

that is, $A\vec{v}$ is the linear combination of the columns of A with coefficients given by the entries of $\vec{v}$. This view appears throughout the course. The column space $\mathcal{R}(A)$ decides whether $A\vec{x} = \vec{b}$ is solvable, the QR factorization orthogonalizes the columns, and low-rank approximation asks how few directions suffice to approximately span them. Likewise, the j-th column of AB is A applied to the j-th column of B, so a matrix–matrix product is ℓ matrix–vector products done at once.

1.3 Representing Real Numbers

To store $\vec{v} \in \mathbb{K}^m$ or $A \in \mathbb{K}^{m\times n}$ in memory, we must represent real numbers. A computer's native objects are bits, $\mathbb{B} \coloneqq \{0, 1\}$, so every number must be built from finitely many of them. We get there in steps, each fixing a shortcoming of the previous one: unsigned integers, signed integers, fixed point, and floating point.

1.3.1 Unsigned and Signed Integers

A string $d \in \mathbb{B}^b$, with bits $d_i \in \{0, 1\}$, represents the integers 0 through $2^b - 1$ via

$$\mathrm{UInt}(d) \coloneqq \sum_{i=1}^{b} d_i 2^{i-1}. \tag{1.4}$$

To represent negative numbers, we use an offset and give the highest bit a negative weight:

$$\mathrm{Int}(d) \coloneqq \sum_{i=1}^{b-1} d_i 2^{i-1} - d_b 2^{b-1}, \tag{1.5}$$

which covers -2^{b-1} through $2^{b-1} - 1$. This is *two's complement*. Nonnegative numbers keep their unsigned encoding and the adder circuit is shared with unsigned integers. The price is an asymmetric range: the most negative number, such as `typemin(Int8)` $= -128$, has no positive counterpart, so $-x$ can overflow back to x. (A different kind of offset, the *biased* encoding that stores $x + K$, is used for floating-point exponents; see Section C.1.)

1.3.2 Fixed-Point Numbers

Fixing the position of the binary point and using $f = \lfloor b/c \rfloor$ of the bits as fractional digits gives

$$\mathrm{Fixed}(d) \coloneqq \mathrm{Int}(d) \cdot 2^{-f}. \tag{1.6}$$

That is, about $1/c$ of the bits represent the fractional part (e.g., $c = 2$: half of them).

The problem. Fixed-point numbers are equally spaced, with spacing 2^{-f} everywhere. To represent both a large number s and a small number s^{-1}, we need about $\log_2(s)$ bits before the binary point and about as many after it. So the number of bits grows *linearly* with the number of orders of magnitude we want to cover. Physical simulations routinely span a dozen orders of magnitude, so fixed point quickly becomes impractical.

1.3.3 Floating-Point Numbers

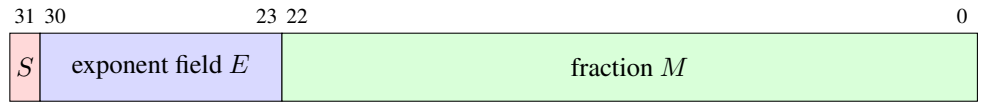
The idea of floating point is to store the order of magnitude separately as an exponent. Given $(p, e) \in \mathbb{B}^{b_p} \times \mathbb{B}^{b_e}$, define

$$\mathrm{Float}(p, e) \coloneqq (-1)^{p_1} \left(1 + \sum_{i=2}^{b_p} p_i 2^{-(i-1)}\right) 2^{\mathrm{Int}(e)}. \tag{1.7}$$

The three parts are the sign bit p_1, the *significand* (or *mantissa*) in $[1, 2)$, and the power of two. The leading 1 of the significand is *implicit* and takes no storage. The decimal analogue is $\pm 1.123 \cdot 10^{-4}$.

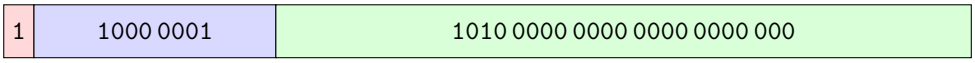
This solves the problem of fixed point: to cover numbers from s^{-1} to s, the exponent only has to reach $\pm \log_2(s)$, which takes about $\log_2(\log_2(s))$ bits. Fixed point needs about $\log_2(s)$ bits for the same range. So floating point covers huge ranges cheaply, which is why scientific computing uses it. A complex number is stored as two floating-point numbers. For details, see [6].

In memory. The picture below shows how the bits are laid out, for single precision (Float32, $b_p = 24$, $b_e = 8$).



$$\underbrace{(-1)^{p_1}}_{\text{sign}} \times \underbrace{\left(1 + \sum_{i=2}^{b_p} p_i 2^{-(i-1)}\right)}_{\text{significand } 1.M \in [1,2)} \times \underbrace{2^{\mathrm{Int}(e)}}_{\text{IEEE stores } E = \text{exponent}+127}$$

For example, $-6.5 = -110.1_2 = -1.101_2 \times 2^2$ is stored as

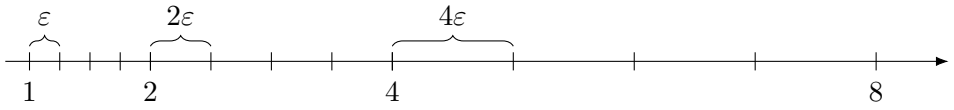


that is, `0xC0D00000`: the sign is $(-1)^1$, the exponent is $129 - 127 = 2$, and the significand is $1.101_2 = 1 + \frac{1}{2} + \frac{1}{8} = 1.625$, so the value is $-1.625 \times 2^2 = -6.5$.

The only difference from the actual IEEE format is the exponent. The formula writes it as $\mathrm{Int}(e)$ for simplicity, while IEEE stores it with a bias, as $E - 127$ (or $E - 1023$ for Float64), and reserves the all-zeros and all-ones exponent fields for 0, subnormals, ∞, and `NaN` (Section C.1).

1.4 The Geometry of Floating-Point Numbers

Plotting all floating-point numbers on the real line shows that they are *not equally spaced*: within each *binade* $[2^E, 2^{E+1})$ they are equally spaced, but crossing a power of two doubles the spacing: if the spacing near 1 is ε, it is 2ε near 2 and 4ε near 4.



Why? Within one binade the exponent E is fixed, and only the significand $1.M$ changes.

- In $[1, 2)$: $1.M$ has $b_p - 1$ fraction bits, so it takes 2^{b_p-1} equally spaced values, with spacing $\varepsilon := 2^{-(b_p-1)}$.
- In $[2^E, 2^{E+1})$: the same values times 2^E, so the spacing is $2^E\varepsilon$.

Every binade holds the same number of floating-point numbers, but each is twice as long as the previous one. Hence the spacing doubles.

So the absolute spacing grows with the numbers, but the relative spacing (spacing divided by the number) always stays between $\varepsilon/2$ and ε. In other words, floating point gives every number about the same *relative* accuracy. This suits real data, whose uncertainty is usually proportional to its size.

Because every result is rounded to this grid, the usual rules of arithmetic can fail.

Example 1 (Floating-Point Addition Is Not Associative)**.** Suppose ties are rounded up, and evaluate $(1 + \varepsilon) + 1 + 2$ in two ways.

- Left to right: $(1 + \varepsilon) + 1 = 2 + \varepsilon$ lies exactly halfway between 2 and $2 + 2\varepsilon$ and rounds up to $2 + 2\varepsilon$. Adding 2 gives $4 + 2\varepsilon$, halfway between 4 and $4 + 4\varepsilon$, which rounds up to $4 + 4\varepsilon$.
- Last two terms first: $1 + 2 = 3$ exactly, and $(1 + \varepsilon) + 3 = 4 + \varepsilon$ rounds down to 4.

The same expression yields 4 or $4 + 4\varepsilon$. Multiplication is not associative either, and the distributive law fails as well; commutativity of addition and of multiplication does hold.

1.5 Cancellation

Relative accuracy has one weak point: subtracting two nearly equal numbers can destroy it.

Example 2 (Cancellation Exposes Existing Error)**.** Suppose the stored value of $10^6 + 1$ is already off by a relative error ε, so it is $(10^6 + 1)(1 + \varepsilon)$. Subtract 10^6:

$$(10^6 + 1)(1 + \varepsilon) - 10^6 \approx 1 + 10^6\varepsilon. \tag{1.8}$$

The exact answer is 1, but we get $1 + 10^6\varepsilon$.

- Before the subtraction: the error is about $10^6\varepsilon$, on a number of size 10^6. Relative error: ε.
- After the subtraction: the error is still $10^6\varepsilon$, but now on a number of size 1. Relative error: $10^6\varepsilon$.

The error itself did not grow. The result got smaller, so the same error is now a million times larger relative to it.

1.6 Conditioning

How do errors *propagate*? Consider computing $y = f(x)$ for $f : \mathbb{R} \to \mathbb{R}$. Since the input is rounded before anything else happens, the best we can hope for is $\hat{y} = f(\hat{x})$ with $|\hat{x} - x| / |x| < \varepsilon$. Comparing relative errors,

$$\frac{|\hat{y} - y| / |y|}{|\hat{x} - x| / |x|} = \frac{|f(\hat{x}) - f(x)|}{|\hat{x} - x|} \cdot \frac{|x|}{|f(x)|} \xrightarrow[\varepsilon \to 0]{} \frac{|x|\,|f'(x)|}{|f(x)|}. \tag{1.9}$$

Definition 3 (Condition Number)

The (relative) *condition number* of f at x is

$$\kappa_f(x) := \frac{|x|\,|f'(x)|}{|f(x)|}. \tag{1.10}$$

The problem is *well-conditioned* at x if $\kappa_f(x) \approx 1$ and *ill-conditioned* if $\kappa_f(x) \gg 1$.

The condition number depends only on the problem and the input, not on the algorithm. This is the most important point here: no algorithm can rescue an ill-conditioned problem, because the difficulty is intrinsic.

Example 4 (Conditioning of Subtraction)**.** Let $f(x) = x - 10^6$ at $x = 10^6 + 1$. Then

$$\kappa_f(10^6 + 1) = \frac{|10^6 + 1| \cdot 1}{1} \approx 10^6 \gg 1. \tag{1.11}$$

This is Example 2 seen as a problem: cancellation is a conditioning phenomenon, not a defect of floating point. Even an algorithm with infinite precision turns a relative uncertainty of 10^{-16} in x into a relative uncertainty of 10^{-10} in the output.

1.7 Stability

Conditioning describes the problem; stability describes the algorithm. We write $\hat{f}$ for the algorithm as it actually runs on the computer: it starts from the stored input $\hat{x} = \mathrm{fl}(x)$ and rounds after every operation.

Definition 5 (Stability)

An algorithm $\hat{f}$ for f is *stable* if

$$\frac{\left|\hat{f}(\hat{x}) - y\right| / |y|}{|\hat{x} - x| / |x|} \lesssim \kappa_f(x). \tag{1.12}$$

The left-hand side is the relative error of the output divided by the relative error of the input. In words: the algorithm amplifies input errors no more than the problem itself does. Otherwise $\hat{f}$ is *unstable*.

The form used in practice. The input error comes from rounding, so it is at most the unit roundoff $u = \varepsilon/2$ (Definition 109), and it is 0 if x is already a floating-point number, which would make the ratio undefined. Replacing the denominator by u gives

$$\frac{\left|\hat{f}(\hat{x}) - y\right|}{|y|} \lesssim \kappa_f(x)\, u. \tag{1.13}$$

The right-hand side is the error that *any* algorithm must make: rounding the input costs u, and the problem amplifies it by $\kappa_f(x)$. A stable algorithm adds nothing beyond that. (Here $\lesssim$ means "at most a modest constant times".)

Example 6 (An Unstable Algorithm for the Identity)**.** Let $f(x) = x$, the identity, with $\kappa_f = 1$, and compute it as $\hat{f}(x) = (x + 10^6) - 10^6$. Mathematically $\hat{f} = f$. Take x to be a floating-point number, so $\hat{x} = x$ and $y = x$.

- Step 1: $x + 10^6$ is rounded, $\mathrm{fl}(x + 10^6) = (x + 10^6)(1 + \delta)$ with $|\delta| \le u$. The low-order bits of x are lost here.
- Step 2: subtracting 10^6 is exact (Lemma 7).

Hence

$$\hat{f}(x) = x + (x + 10^6)\delta, \qquad \frac{\left|\hat{f}(x) - x\right|}{|x|} = \frac{|x + 10^6|}{|x|}\,|\delta| \approx \frac{10^6}{|x|}\,|\delta|\,. \tag{1.14}$$

For $x \approx 1$ the error is about $10^6 u$, but a stable algorithm must reach $\kappa_f u = u$. So $\hat{f}$ is unstable, by a factor of about 10^6; for instance, $\hat{f}(0.1) = 0.09999999997671694$. ($x = 1$ is an exception: $10^6 + 1$ is a floating-point number, so $\delta = 0$.)

Why it fails. Look at the two steps separately: $\hat{f} = g_2 \circ g_1$ with $g_1(x) = x + 10^6$ and $g_2(z) = z - 10^6$. For $x \approx 1$ and $z = x + 10^6$, their condition numbers (Definition 3) are

$$\kappa_{g_1}(x) = \frac{|x|}{|x + 10^6|} \approx 10^{-6}, \qquad \kappa_{g_2}(z) = \frac{|z|}{|z - 10^6|} \approx 10^6. \tag{1.15}$$

Step 1 shrinks relative errors by a factor 10^6; step 2 is a cancellation and magnifies them by 10^6. What happens to an error depends on where it enters:

- An error already present in x passes through both steps: it is shrunk, then magnified, so overall it is unchanged ($\kappa_f = 1$).
- The rounding error δ of step 1 enters *after* the shrinking step. It passes only through step 2, so it is magnified by 10^6.

Lesson. A well-conditioned problem can still be solved by an unstable algorithm if one of its intermediate steps is ill-conditioned. We will see this again with Gram–Schmidt and the normal equations.

Unstable vs. ill-conditioned. These are different diagnoses with different fixes.

- *Unstable*: the algorithm is at fault. Use a different algorithm.
- *Ill-conditioned*: the problem itself is sensitive, and no algorithm can help. Change the problem (reformulate or regularize it), or use higher precision.

The IEEE formats, the two conventions for machine epsilon, the standard model of floating-point arithmetic, and the notion of backward stability are collected in Appendix C.

1.8 (SUPPLEMENT) Further Topics

1.8.1 Rewriting Formulas to Avoid Cancellation

The recipe: find the step that subtracts two nearly equal numbers, and use an algebraic identity to rewrite the formula without it. Mathematically nothing changes; numerically the error drops to $O(u)$.

Roots of a quadratic. The roots of $ax^2 + bx + c$ are $x_{1,2} = (-b \pm \sqrt{b^2 - 4ac})/(2a)$. Suppose $b > 0$ and $b^2 \gg 4ac$. Then $\sqrt{b^2 - 4ac} \approx b$, so the "+" root computes $-b + (\text{almost } b)$: a cancellation. The "−" root adds two numbers of the same sign and is accurate. So compute that one first and get the other from Vieta's relation $x_1 x_2 = c/a$:

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}, \qquad x_1 = \frac{c}{a\, x_2} = \frac{2c}{-b - \sqrt{b^2 - 4ac}}. \tag{1.16}$$

(If $b < 0$, swap the roles of the two roots.)

Variance. The formula $\mathrm{Var} = \overline{x^2} - \bar{x}^2$ (mean of squares minus square of mean) subtracts two large, nearly equal numbers when the data have a large mean and a small spread. For the data $10^8 + 1$, $10^8 + 2$, $10^8 + 3$ it returns 0 in Float64, while the true variance is $2/3$; it can even return a negative number. The fixes avoid squaring the large mean:

- *Two passes*: first compute $\bar{x}$, then $\mathrm{Var} = \frac{1}{n}\sum_i (x_i - \bar{x})^2$. The differences $x_i - \bar{x}$ are small, and all terms are nonnegative.
- *Welford's recurrence*: one pass over the data, updating the mean and the sum of squared deviations as each x_k arrives,

$$\bar{x}_k = \bar{x}_{k-1} + \frac{x_k - \bar{x}_{k-1}}{k}, \qquad M_k = M_{k-1} + (x_k - \bar{x}_{k-1})(x_k - \bar{x}_k), \tag{1.17}$$

  and finally $\mathrm{Var} = M_n/n$.

Three standard cases.

- $1 - \cos x$ for small x: for $x = 10^{-8}$, $\cos x$ rounds to exactly 1, giving 0 instead of $5 \cdot 10^{-17}$. Use $1 - \cos x = 2\sin^2(x/2)$.
- $\log(1 + x)$ for small x: already $1 + x$ loses the low bits of x (as in Example 6); for $x = 10^{-17}$ the result is 0. `log1p(x)` computes $\log(1 + x)$ directly from x (likewise `expm1(x)` for $e^x - 1$).
- $\sqrt{x+1} - \sqrt{x}$ for large x: for $x = 10^{16}$ the result is 0 instead of $5 \cdot 10^{-9}$. Rationalize: $\sqrt{x+1} - \sqrt{x} = 1/(\sqrt{x+1} + \sqrt{x})$.

1.8.2 Sterbenz's Lemma: Close Numbers Subtract Exactly

Lemma 7 (Sterbenz)

If x, y are floating-point numbers with $y/2 \le x \le 2y$, then $x - y$ is a floating-point number, so $\mathrm{fl}(x - y) = x - y$ exactly.

Intuition. Think of decimal numbers with three significant digits. $1.23 - 1.21 = 0.02$: both inputs lie on the grid of spacing 0.01, so their difference does too, and it is *smaller* than the inputs, so it needs fewer digits, not more. Nothing has to be rounded.

Proof. Say $y \le x \le 2y$, and let 2^e be the spacing of floating-point numbers near y, so y is an integer multiple of 2^e with at most b_p bits. The spacing near $x \ge y$ is 2^e or a larger power of 2, so x is an integer multiple of 2^e as well. Hence $x - y = k\,2^e$ with an integer $k \ge 0$. From $x \le 2y$ we get $x - y \le y < 2^{b_p} 2^e$, so $k < 2^{b_p}$ and $x - y$ fits in b_p bits. □

Consequence: the subtraction is not to blame. In Example 2 the subtraction itself is exact by the lemma: all of the error was already in the input, and the subtraction only made it visible. Rule: if the inputs are inexact, do not compute a small result by subtracting two large numbers.

1.8.3 Useful Techniques

Kahan summation. Adding n numbers one by one can make an error of order $nu \sum_i |x_i|$: adding a small term to a large running sum rounds away the term's low bits. Kahan's compensated summation keeps those lost bits in a variable c and feeds them back:

```
s = 0.0; c = 0.0
for x in xs
    y = x - c        # add back what was lost last time
    t = s + y        # rounded sum: low bits of y are lost
    c = (t - s) - y  # = (what was really added) - y = the lost part
    s = t
end
```

This reduces the error to about $2u \sum_i |x_i|$, essentially independent of n [7], at about four times the cost. Example: adding 0.1 ten million times gives an error of $1.6 \cdot 10^{-4}$ naively, 0 (to the printed digits) with Kahan, and $1.4 \cdot 10^{-9}$ with Julia's `sum`, which adds *pairwise* in a binary tree so that its error grows only like $u \log_2 n$.

Fused multiply-add (FMA). $\mathrm{fma}(a, b, c)$ computes $ab + c$ with a single rounding instead of two: faster and more accurate. Since a compiler may or may not use it, the same code can give slightly different results.

`-ffast-math`. This compiler flag (Julia: `@fastmath`) lets the compiler treat floating point as associative, so it may reorder sums (e.g., to vectorize them). This changes results and can break Kahan summation, whose $c = (t - s) - y$ is algebraically 0 and may be deleted. Keep it off.

1.8.4 Low-Precision Formats

Two 16-bit formats split their bits differently. Float16 has 5 exponent bits and $b_p = 11$: $\varepsilon \approx 10^{-3}$, but the largest number is only 65504. BFloat16 has 8 exponent bits, hence the same range as Float32 (up to $3.4 \cdot 10^{38}$), but only $b_p = 8$: $\varepsilon \approx 8 \cdot 10^{-3}$, about two decimal digits. Deep learning prefers BFloat16: gradients span many orders of magnitude, so what fails first is *range* (overflow, underflow), not precision; Float16 training needs *loss scaling* (multiplying the loss by a constant), BFloat16 does not. Sums are still accumulated in Float32: by Proposition 111, a dot product of length n has error up to about $nu\ |\vec{x}|^\top |\vec{y}|$. For $n = 4096$, BFloat16 ($u = 2^{-8}$) gives $nu = 16 \ge 1$, so the bound guarantees nothing, while Float32 ($u = 2^{-24}$) gives $nu \approx 2.4 \cdot 10^{-4}$. Hence tensor cores take BFloat16 (or FP8) inputs but accumulate in Float32.

Lecture 2

Cache Hierarchy and Tiling/Blocking, Solving Linear Systems

Supplementary reading:

- [2] Sections 2.3 and 2.6.
- [1] Lectures 17 and 20.

2.1 Flops Are Not the Whole Story

In Lecture 1 we asked whether we compute the *right* answer (conditioning: the problem's fundamental limit; stability: does the algorithm do something stupid?). Now we ask how *fast* we compute it.

Algorithms and computational cost. Consider the algorithm for matrix multiplication

$$A \mathrel{+}= BC, \qquad A \in \mathbb{R}^{\ell \times n},\ B \in \mathbb{R}^{\ell \times m},\ C \in \mathbb{R}^{m \times n}: \tag{2.1}$$

```
function matmuladd!(A, B, C)          # A += B*C
    for i = 1:l
        for j = 1:m
            for k = 1:n
                A[i,k] += B[i,j] * C[j,k]
            end
        end
    end
end
```

The innermost line runs $\ell \times m \times n$ times, so the computational cost is $\ell \times m \times n$. We set $\ell = 2 \cdot 10^4$, $m = 3 \cdot 10^4$ and time it with `@btime`:

n	Cost ℓmn	Time
1	$6 \cdot 10^8$	$6 \cdot 10^{-2}$ s
10^3	$6 \cdot 10^{11}$	4 s

The cost grows by a factor of 1000, so we would expect 1000×0.06 s $= 60$ s. It takes only 4 s: *more than* 10 *times faster than expected*. (The timings are for Julia's built-in multiplication, which calls BLAS.)

Flops and memory access. Does Julia use fewer operations? No: for $\ell = m = n$, the time grows roughly like the cost,

$\ell = m = n$	Cost ℓmn	Time
10^3	10^9	$7 \cdot 10^{-3}$ s
10^4	10^{12}	6 s

that is, about 1000 times the cost in about 1000 times the time. The explanation is that four operations appear in the algorithm: additions, multiplications, and reading and writing entries of the arrays (both written $[\,]$). Their costs are very different:

$$\text{cost}(+) < \text{cost}(*) \ll \text{cost}([\,]). \tag{2.2}$$

(On modern CPUs, $+$ and $*$ in fact cost about the same.) *Memory access is much slower than flops on modern hardware.*

- For $n = 1$, every entry of B is read and then used only once, so almost all of the time goes into memory access.
- For $n = 10^3$, every entry of B is used 10^3 times, so the cost of reading it is shared by many operations.

2.2 Memory Is One-Dimensional

Memory is a long string of bits, and a vector is a contiguous run of floating-point numbers $u_1, u_2, u_3, \ldots$. There is no such thing as a two-dimensional matrix in memory; it must be flattened in some order. Julia, Fortran, BLAS, and LAPACK use *column-major* order, storing entire columns contiguously:

$$A_{11},\ A_{21},\ A_{31},\ A_{41},\ A_{12},\ A_{22},\ A_{32},\ A_{42},\ \ldots \tag{2.3}$$

so that A_{ij} sits at linear index $i + (j-1)m$. C and NumPy default to row-major order. Besides the data, one also stores metadata such as sizes, strides, and preallocated buffers.

Since the storage is just a contiguous block, the same memory can be *reinterpreted* as an array of shape $m \times n \times k \times l$, $mn \times kl$, or $mnk \times l$ without any copying. Reshaping is free; transposing is not.

2.3 The Memory Hierarchy

Main memory is much slower than arithmetic, so hardware adds *caches*: small, fast memory close to the processor. Two bottlenecks remain:

- *Latency*: how long one access waits (a few cycles for L1, a few hundred for main memory).
- *Bandwidth*: how many bytes can be moved per second.

Latency is hidden by *prefetching*: the hardware fetches nearby memory, or memory in a strided pattern, before it is needed. This only works for regular access patterns, so the *regularity* of memory access often matters more than its volume. GPUs instead hide latency by running many threads. When threads access contiguous elements, the hardware merges their accesses into one, so on a GPU non-contiguous access is very slow.

2.4 Loop Reordering

Example 8 (Two Loop Orders for a Matrix–Vector Product)**.** Both loops compute $\vec{u} = A\vec{v}$ with the same number of operations.

```
# i outer: inner loop along a row of A
for i = 1:m
    for j = 1:n
        u[i] += A[i,j] * v[j]
    end
end
```

```
# j outer: inner loop down a column
for j = 1:n
    for i = 1:m
        u[i] += A[i,j] * v[j]
    end
end
```

In column-major storage, the left loop reads A along a row, in steps of m; in the right loop, every access is contiguous.

The left loop computes inner products of the rows of A with $\vec{v}$ (`dot`); the right loop adds up the columns of A times v_j (`axpy`). Julia stores matrices by columns, so the right loop is faster. The gap grows with the size of A: once A no longer fits in the cache, each strided access uses only one number from every cache line it loads. (The board says the order matters most for small matrices; we measured the opposite: the left loop is $3\times$ slower at $m = 64$ and $20\times$ at $m = 4096$.)

2.5 Tiling

Even without cache misses, the limited bandwidth slows us down. The key observation: multiplying two $m \times m$ matrices takes about m^3 operations but involves only about m^2 numbers, so each number is used about m times.

- If all m^2 numbers fit into the cache, we load each of them from main memory only once, and then do all m^3 operations in the fast cache.
- If the matrices are too large, we *tile*: split A, B, C into $b \times b$ blocks, $N = m/b$ along each side, and multiply block by block. (For simplicity, assume b divides m; otherwise the blocks at the edges are smaller.) We choose b so that the three blocks we are working on fit into the cache; Section 2.9.2 quantifies the gain.

Written out entry by entry, the tiled algorithm has *six* nested loops: the outer three choose the blocks (I, J, K count blocks, not entries), and the inner three multiply them.

```
for I = 1:N, J = 1:N, K = 1:N              # which blocks: C_IJ += A_IK * B_KJ
    for i = (I-1)*b+1 : I*b                # multiply the two b x b blocks
        for j = (J-1)*b+1 : J*b
            for k = (K-1)*b+1 : K*b
                C[i,j] += A[i,k] * B[k,j]
            end
        end
    end
end
```

We do the same m^3 operations as before, just in a different order. The inner three loops only use three blocks, $3b^2$ numbers in total. They stay in the cache and are each used about b times.

2.6 Linear Systems

Problem. Given $A \in \mathbb{K}^{m\times n}$ and $\vec{b} \in \mathcal{R}(A) \subseteq \mathbb{K}^m$, find $\vec{x} \in \mathbb{K}^n$ with

$$A\vec{x} = \vec{b}. \tag{2.4}$$

The condition $\vec{b} \in \mathcal{R}(A)$ just says that a solution exists.

- Linear systems appear in virtually all of mathematics.
- They *can* be solved by computing A^{-1}, but they *need not* be. Computing A^{-1} is in fact the wrong way: it costs more ($2m^3$ flops for a square matrix, versus $\frac{2}{3}m^3$ for the LU factorization of Lecture 3), and it usually gives a less accurate answer.

Square matrices. We begin with $A \in \mathbb{K}^{m\times m}$. Then the following three conditions are equivalent:

$$\underbrace{\mathcal{N}(A) = \{\vec{0}\}}_{\vec{x}\mapsto A\vec{x}\text{ injective}} \iff \text{the columns of } A \text{ are linearly independent} \iff \underbrace{\mathcal{R}(A) = \mathbb{K}^m}_{\vec{x}\mapsto A\vec{x}\text{ surjective}} \tag{2.5}$$

In words:

- *Left*: only $\vec{x} = \vec{0}$ is mapped to $\vec{0}$. Then different inputs give different outputs: if $A\vec{x} = A\vec{y}$, then $A(\vec{x} - \vec{y}) = \vec{0}$, so $\vec{x} = \vec{y}$. Hence a solution, if it exists, is unique.
- *Right*: every $\vec{b} \in \mathbb{K}^m$ is of the form $A\vec{x}$. Hence a solution always exists.

If these conditions hold, we say that A *has full rank*, *is invertible*, or *is nonsingular*; then $A\vec{x} = \vec{b}$ has exactly one solution for every $\vec{b}$. For rectangular matrices, injective and surjective are no longer the same; they are treated in Lecture 5.

2.7 Triangular Systems

We begin with the easy case. The small example

$$\begin{aligned} x_1 + 5 &= 15 \\ x_1 + 5x_2 &= 20 \end{aligned} \qquad \Longrightarrow \qquad x_1 = 10,\ x_2 = 2 \tag{2.6}$$

already shows the whole idea: x_1 does not depend on x_2, so the triangular structure *decouples* the unknowns and they can be solved for one at a time. For lower triangular A, *forward substitution* computes

$$x_1 = \frac{b_1}{A_{11}}, \qquad x_i = \frac{1}{A_{ii}}\left(b_i - \sum_{j=1}^{i-1} A_{ij}x_j\right), \quad i = 2, \dots, m. \tag{2.7}$$

Upper triangular systems are solved in reverse order by *back substitution*. The same formula admits two implementations, both costing $\sim m^2$ flops but with different access patterns.

Algorithm 1 Forward substitution, row-oriented (`dot` form).

1: **for** $i = 1, \dots, m$ **do**
2: $\quad x_i \leftarrow b_i$
3: $\quad$ **for** $j = 1, \dots, i-1$ **do**
4: $\quad\quad x_i \leftarrow x_i - A_{ij}x_j$
5: $\quad x_i \leftarrow x_i / A_{ii}$

Algorithm 2 Forward substitution, column-oriented (`axpy` form), the efficient variant.

$\vec{x} \leftarrow \vec{b}$
for $j = 1, \dots, m$ **do**
    $x_j \leftarrow x_j / A_{jj}$
    for $i = j+1, \dots, m$ **do**
        $x_i \leftarrow x_i - A_{ij} x_j$ ▷ remove the contribution of x_j at once

For upper triangular A, x_m is the only unknown in the last equation, so back substitution starts from the bottom:

$$x_m = \frac{b_m}{A_{mm}}, \qquad x_i = \frac{1}{A_{ii}} \left(b_i - \sum_{j=i+1}^{m} A_{ij} x_j \right), \quad i = m-1, \dots, 1. \tag{2.8}$$

The two implementations mirror those of forward substitution, with every loop over the unknowns reversed.

Algorithm 3 Back substitution, row-oriented (`dot` form).

for $i = m, \dots, 1$ **do**
    $x_i \leftarrow b_i$
    for $j = i+1, \dots, m$ **do**
        $x_i \leftarrow x_i - A_{ij} x_j$
    $x_i \leftarrow x_i / A_{ii}$

Algorithm 4 Back substitution, column-oriented (`axpy` form).

$\vec{x} \leftarrow \vec{b}$
for $j = m, \dots, 1$ **do**
    $x_j \leftarrow x_j / A_{jj}$
    for $i = 1, \dots, j-1$ **do**
        $x_i \leftarrow x_i - A_{ij} x_j$ ▷ remove the contribution of x_j at once

In column-major storage the row forms walk along rows (strided) and the column forms walk down columns (contiguous); this is the matrix–vector discussion again.

2.8 Gaussian Elimination

A general matrix is not triangular. Gaussian elimination *transforms A into upper triangular form by a sequence of row operations*, reducing the general problem to a triangular one. A 2×2 example:

$$\begin{aligned} x_1 + 2x_2 &= 10 \\ 2x_1 + x_2 &= 8 \end{aligned} \xrightarrow{\text{row } 2 - 2\times\text{row } 1} \begin{aligned} x_1 + 2x_2 &= 10 \\ -3x_2 &= -12 \end{aligned} \quad \Longrightarrow \quad x_2 = 4,\ x_1 = 2. \tag{2.9}$$

In matrix language, this elimination step is a *left multiplication*:

$$\begin{bmatrix} 1 & 0 \\ -2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -2 & 1 \end{bmatrix} \begin{bmatrix} 10 \\ 8 \end{bmatrix}. \tag{2.10}$$

Theorem 9 (Elementary Lower Triangular Matrices)

Let $A \in \mathbb{K}^{m\times m}$ and suppose the first $k-1$ columns of A are already zero below the diagonal, and $A_{kk} \neq 0$. Define the multipliers $\ell_{ik} := A_{ik}/A_{kk}$ for $i = k+1, \dots, m$, the vector

$$\vec{\ell}_k := \begin{bmatrix} 0 & \cdots & 0 & \ell_{k+1,k} & \cdots & \ell_{mk} \end{bmatrix}^\top, \tag{2.11}$$

whose first k entries vanish, and the *elementary lower triangular matrix*

$$L_k := I - \vec{\ell}_k \vec{e}_k^* = \begin{bmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & -\ell_{k+1,k} & 1 & & \\ & & \vdots & & \ddots & \\ & & -\ell_{mk} & & & 1 \end{bmatrix} \leftarrow \text{row } k \tag{2.12}$$

Then $L_k A$ is zero below the diagonal in its first k columns. Its first k rows equal those of A; besides zeroing column k below the diagonal, only the trailing $(m-k)\times(m-k)$ block is updated.

Proof. Left multiplication by L_k performs, for each $i > k$, the row operation "row i minus ℓ_{ik} times row k". The (i,k) entry becomes $A_{ik} - \ell_{ik}A_{kk} = 0$. Row k is zero in its first $k-1$ columns, so the columns that have already been eliminated are not disturbed. □

Applying Theorem 9 for $k = 1, \dots, m-1$ (assuming that each A_{kk} is nonzero when we reach it) to both sides of $A\vec{x} = \vec{b}$ gives

$$\underbrace{L_{m-1}\cdots L_1 A}_{\text{upper triangular}}\, \vec{x} = L_{m-1}\cdots L_1 \vec{b}. \tag{2.13}$$

With $U := L_{m-1}\cdots L_1 A$ and $L := (L_{m-1}\cdots L_1)^{-1}$, we obtain $A = LU$ with U upper triangular. We stop here with two questions: *is L lower triangular, and can we compute it?* Both are answered in Lecture 3.

2.9 (SUPPLEMENT) Further Topics

2.9.1 Leading Dimension

In column-major order, the entry A_{ij} lives at

$$\mathrm{addr}(A_{ij}) = \text{base} + (j-1)\cdot \mathrm{lda} + (i-1), \tag{2.14}$$

where the *leading dimension* $\mathrm{lda} \geq m$ is the distance between consecutive columns. Allowing $\mathrm{lda} \neq m$ lets us describe a *submatrix* of a larger matrix by the quadruple (start address, m, n, lda) without copying any data. This is what Julia's `view` and `@views` do, whereas `A[1:100, 1:100]` makes a copy.

2.9.2 Computational Intensity

We can quantify this [2, §2.6]. Consider two levels of memory, a fast memory of size M and a slow memory. Let one flop take time t_f and one slow-memory access take time $t_m \gg t_f$.

Definition 10 (Computational Intensity)

If an algorithm performs f flops and m slow-memory accesses, its *computational intensity* is $q \coloneqq f/m$.

The total time is roughly

$$T = ft_f + mt_m = ft_f \left(1 + \frac{t_m}{t_f} \cdot \frac{1}{q}\right). \tag{2.15}$$

Running near the peak rate ft_f requires $q \gg t_m/t_f$, and on modern machines t_m/t_f is in the tens to hundreds. The three levels of the BLAS are distinguished precisely by q (here n is the vector length or matrix order):

Level	Typical operation	Flops f	Accesses m	q
Level 1	`axpy` $\vec{y} \leftarrow \alpha\vec{x} + \vec{y}$	$2n$	$3n$	$2/3$
Level 2	`gemv` $\vec{y} \leftarrow \alpha A\vec{x} + \beta\vec{y}$	$2n^2$	n^2	2
Level 3	`gemm` $C \leftarrow \alpha AB + \beta C$	$2n^3$	$4n^2$	$n/2$

Only Level 3 has an intensity that grows with the problem size, matching the observation that matrix multiplication performs m^3 multiply–adds on m^2 data. A `dot` has $q \approx 1$, and a triangular solve (n^2 flops on about $n^2/2$ entries) has $q \approx 2$; both are limited by bandwidth.

For naive $C \mathrel{+}= AB$ with $n \times n$ matrices, B is read n times (n^3 accesses) while A and C cost $3n^2$, so

$$q = \frac{2n^3}{n^3 + 3n^2} \approx 2, \tag{2.16}$$

no better than Level 2. For the tiled version (Section 2.5) the slow-memory traffic is $m = 2n^2 + 2Nn^2$, so

$$q \approx \frac{2n^3}{2Nn^2} = \frac{n}{N} = b. \tag{2.17}$$

Three blocks must fit in fast memory, so $3b^2 \le M$ and $q \approx b \le \sqrt{M/3}$. A lower bound of Hong and Kung shows that any matrix multiplication that only reorders the sums has $q = O(\sqrt{M})$, so tiling is asymptotically optimal. Real implementations tile at *several* levels (registers, L1, L2, and across nodes), each with its own b.

2.9.3 Blocked LU

The in-place LU algorithm of Lecture 3 spends its time in rank-one updates $A \leftarrow A - \vec{x}\vec{y}^*$. These read and write all of A for 2 flops per entry ($q \approx 1$), so naive Gaussian elimination is bandwidth-bound. The tiling idea fixes this [2, §2.6.3]. Choose a block width b and write

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \qquad A_{11} \in \mathbb{K}^{b \times b}. \tag{2.18}$$

1. Factor the $m \times b$ panel $\begin{bmatrix} A_{11} \\ A_{21} \end{bmatrix}$ by ordinary elimination, obtaining L_{11}, L_{21}, U_{11} (Level 2, but only an $O(b/m)$ fraction of the work).
2. Compute $U_{12} = L_{11}^{-1} A_{12}$, a triangular solve with many right-hand sides (Level 3).
3. Compute $\tilde{A}_{22} = A_{22} - L_{21}U_{12}$, a matrix multiplication (Level 3) that contains most of the flops.
4. Recurse on $\tilde{A}_{22}$.

Almost all the work is matrix multiplication with $q \approx b$. This is how LAPACK's `getrf` beats the older LINPACK: rewrite everything as matrix multiplication.

Lecture 3

LU Factorization and Its Stability

Supplementary reading:

- [1] Lectures 20 and 21.
- [2] Section 2.3.

3.1 Recap: From Triangular Systems to Elimination

We begin with a recap of Lecture 2: matrices are stored column by column, memory access costs far more than arithmetic, and the loop order and tiling of an algorithm should match the storage. We then returned to linear systems. Triangular systems are solved by forward or back substitution in $O(m^2)$ flops, and Gaussian elimination reduces a general system to a triangular one: by Theorem 9, applying elementary lower triangular matrices $L_k = I - \vec{\ell}_k \vec{e}_k^*$ for $k = 1, \ldots, m-1$ gives

$$U \coloneqq L_{m-1} \cdots L_1 A \text{ upper triangular}, \qquad L \coloneqq (L_{m-1} \cdots L_1)^{-1}, \qquad A = LU. \tag{3.1}$$

This lecture answers the two questions left open: *is L lower triangular, and can we compute it cheaply?* Answering them requires a little algebra of lower triangular matrices.

3.2 The Algebra of Lower Triangular Matrices

Proposition 11 (Closure Properties)

The lower triangular matrices in $\mathbb{K}^{m \times m}$ are closed under

1. addition,
2. scalar multiplication,
3. matrix multiplication, and
4. inversion (of invertible ones).

Proof. Items 1 and 2 are clear. For item 3, let L and $\tilde{L}$ be lower triangular, so $(L\tilde{L})_{ij} = \sum_k L_{ik}\tilde{L}_{kj}$. A term $L_{ik}\tilde{L}_{kj}$ can be nonzero only if $i \geq k$ and $k \geq j$, hence $i \geq j$. So every term vanishes when $i < j$, and $L\tilde{L}$ is

lower triangular. Item 4 is proved below for the matrices arising in LU, using Theorem 12; the general case is in Section 3.8. □

Theorem 12 (Inverses and Products of Elementary Lower Triangular Matrices)

Let $L_k = I - \vec{\ell}_k \vec{e}_k^*$ be as in Theorem 9. Then:

1. $L_k^{-1} = I + \vec{\ell}_k \vec{e}_k^*$, i.e., inversion just *flips the signs* of the multipliers;
2. the product of the inverses, *in this order*, places every multiplier in its own position:

$$L_1^{-1} L_2^{-1} \cdots L_{m-1}^{-1} = \begin{bmatrix} 1 & & & \\ \ell_{21} & 1 & & \\ \vdots & \ddots & \ddots & \\ \ell_{m1} & \cdots & \ell_{m,m-1} & 1 \end{bmatrix}. \tag{3.2}$$

Trefethen and Bau call these two facts the "two strokes of luck" of Gaussian elimination [1, Lecture 20]. The order in item 2 matters. In the reverse order the multipliers mix: for example,

$$L_2^{-1} L_1^{-1} = (I + \vec{\ell}_2 \vec{e}_2^*)(I + \vec{\ell}_1 \vec{e}_1^*) = I + \vec{\ell}_1 \vec{e}_1^* + \vec{\ell}_2 \vec{e}_2^* + \ell_{21}\, \vec{\ell}_2 \vec{e}_1^*, \tag{3.3}$$

and the last term adds $\ell_{21}\ell_{i2}$ to the entries of column 1 below row 2.

Both items rest on one simple fact: $\vec{\ell}_k$ is zero in its first k entries. The number $\vec{e}_p^* \vec{\ell}_k$ is entry p of $\vec{\ell}_k$, so it is 0 for every $p \le k$. This is what makes the extra terms vanish in the proof below.

Proof. For item 1, since $\vec{e}_k^* \vec{\ell}_k = 0$,

$$(I + \vec{\ell}_k \vec{e}_k^*)(I - \vec{\ell}_k \vec{e}_k^*) = I + \vec{\ell}_k \vec{e}_k^* - \vec{\ell}_k \vec{e}_k^* - \vec{\ell}_k (\vec{e}_k^* \vec{\ell}_k) \vec{e}_k^* = I. \tag{3.4}$$

For item 2, we compute the product column by column. Column j of $M := L_1^{-1} \cdots L_{m-1}^{-1}$ is

$$M \vec{e}_j = (I + \vec{\ell}_1 \vec{e}_1^*) \cdots (I + \vec{\ell}_{m-1} \vec{e}_{m-1}^*)\, \vec{e}_j. \tag{3.5}$$

Here j is fixed: it is the column we compute. The factors are $L_p^{-1} = I + \vec{\ell}_p \vec{e}_p^*$ for $p = 1, \ldots, m-1$. We apply them to $\vec{e}_j$ from right to left, using $(I + \vec{\ell}_p \vec{e}_p^*)\vec{x} = \vec{x} + \vec{\ell}_p\, (\vec{e}_p^* \vec{x})$, where $\vec{e}_p^* \vec{x}$ is entry p of $\vec{x}$.

- *Factors with $p > j$*: entry p of $\vec{e}_j$ is 0, so these factors leave $\vec{e}_j$ unchanged.
- *The factor $p = j$*: entry j of $\vec{e}_j$ is 1, so it gives $\vec{e}_j + \vec{\ell}_j$.
- *Factors with $p < j$*: entry p of $\vec{e}_j + \vec{\ell}_j$ is 0, since $\vec{e}_j$ has its 1 in position j and $\vec{\ell}_j$ is zero in its first j entries. So these factors leave $\vec{e}_j + \vec{\ell}_j$ unchanged.

Hence column j of M is

$$M \vec{e}_j = \vec{e}_j + \vec{\ell}_j = \begin{bmatrix} 0 & \cdots & 0 & 1 & \ell_{j+1,j} & \cdots & \ell_{mj} \end{bmatrix}^\top, \tag{3.6}$$

with the 1 in position j. These are exactly the columns of the matrix in item 2. □

3.3 Answering the Two Questions

Now we can answer the two questions from the end of Lecture 2. Since the inverse of a product reverses the order of the factors, $L = (L_{m-1} \cdots L_1)^{-1} = L_1^{-1} L_2^{-1} \cdots L_{m-1}^{-1}$.

- *Is L lower triangular? Yes.* Each L_k^{-1} is lower triangular (item 1 of Theorem 12), and so is their product (item 3 of Proposition 11). Its diagonal is all ones: L is *unit* lower triangular.
- *Can we compute L cheaply? Yes, for free.* By item 2 of Theorem 12, L is the matrix (3.2), whose entries below the diagonal are the multipliers ℓ_{ik} that elimination computes anyway.

3.4 The In-Place Algorithm

Idea: transform A into U with Gaussian elimination while recording the multipliers. Both L and U are stored in place, in the memory of A:

- U goes on and above the diagonal;
- the multipliers ℓ_{ik} go below the diagonal, where elimination has just created zeros;
- the diagonal of L is all ones, so it need not be stored.

Hence we can overwrite A with L and U, without any extra memory.

Algorithm 5 LU factorization without pivoting, in place.

```
for k = 1, ..., m do
    A_{k+1:m, k} ← A_{k+1:m, k} / A_{kk}          ▷ multipliers ℓ_ik, stored in place
    for j = k + 1, ..., m do
        for i = k + 1, ..., m do
            A_ij ← A_ij − A_ik A_kj               ▷ = A_ij − ℓ_ik u_kj
```

When the algorithm finishes, A holds both factors: U on and above the diagonal, and the multipliers of L below it. The inner loop runs over i, i.e., down a column of A, which is the fast direction in column-major storage (Lecture 2).

Vector form. Step k computes the multipliers $\vec{\ell}_k$ and subtracts an outer product from the trailing block:

$$\vec{\ell}_k = A_{k+1:m,\,k} / A_{kk}, \qquad A_{k+1:m,\,k+1:m} \leftarrow A_{k+1:m,\,k+1:m} - \vec{\ell}_k \, A_{k,\,k+1:m}. \tag{3.7}$$

A column vector times a row vector has rank one, so each step is a *rank-one update* of the trailing block.

Block form. Write $A = \begin{bmatrix} a_{11} & \vec{w}^* \\ \vec{v} & B \end{bmatrix}$ with a scalar a_{11}. The first step of elimination is the factorization

$$A = \begin{bmatrix} 1 & 0 \\ \vec{v}/a_{11} & I \end{bmatrix} \begin{bmatrix} a_{11} & \vec{w}^* \\ 0 & S \end{bmatrix}, \qquad S = B - \frac{\vec{v}\vec{w}^*}{a_{11}}. \tag{3.8}$$

S is the *Schur complement* (Lecture 9). The algorithm then factors S in the same way, so LU is this one step applied recursively.

3.5 Zero Pivots

The algorithm divides by A_{kk}, the k-th *pivot*. If a pivot is zero, it cannot proceed.

Example 13 (An Invertible Matrix with No LU Factorization)**.** The matrix

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \tag{3.9}$$

is invertible, but $A_{11} = 0$ is a zero pivot, so the very first step divides by zero and $A = LU$ does not exist. *Invertibility does not imply the existence of an LU factorization.*

Theorem 14 (Zero Pivots and Leading Principal Submatrices)

If the k-th pivot of the LU factorization of A is zero, then the leading principal submatrix $A[1{:}k, 1{:}k]$ is singular. All pivots of the LU factorization are nonzero if and only if every leading principal submatrix of A is nonsingular.

Proof. Step 1: the top-left block. Take the top-left $k \times k$ block of $A = LU$. Entry (i, j) with $i, j \le k$ is $\sum_p L_{ip}U_{pj}$, and $L_{ip} = 0$ for $p > i$, so only $p \le k$ contributes. Hence

$$A[1{:}k, 1{:}k] = L[1{:}k, 1{:}k]\, U[1{:}k, 1{:}k]. \tag{3.10}$$

Step 2: determinants. $L[1{:}k, 1{:}k]$ is unit lower triangular, so its determinant is 1. $U[1{:}k, 1{:}k]$ is upper triangular, so its determinant is the product of its diagonal, i.e., of the first k pivots:

$$\det A[1{:}k, 1{:}k] = u_{11}u_{22}\cdots u_{kk}. \tag{3.11}$$

So $A[1{:}k, 1{:}k]$ is nonsingular if and only if the first k pivots are nonzero.

Step 3: a zero pivot stops the algorithm early. If $u_{kk} = 0$, the algorithm cannot continue past step k. But the first k rows and columns of L and U have already been computed, and Step 1 uses only those. So the argument still applies. □

3.6 Small Pivots Are Also Dangerous

Exactly zero pivots are unlikely in practice. Is it then enough that no pivot is exactly zero? Consider a small number $\varepsilon > 0$, much smaller than the machine epsilon (e.g., $\varepsilon = 10^{-20}$; here ε is *not* the machine epsilon):

$$A = \begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix}, \qquad L = \begin{bmatrix} 1 & 0 \\ \varepsilon^{-1} & 1 \end{bmatrix}, \qquad U = \begin{bmatrix} \varepsilon & 1 \\ 0 & 1 - \varepsilon^{-1} \end{bmatrix}. \tag{3.12}$$

This is mathematically correct. In floating-point arithmetic, however, ε^{-1} is huge and $1 - \varepsilon^{-1}$ rounds to $-\varepsilon^{-1}$. The computed factors then give

$$\tilde{L}\tilde{U} = \begin{bmatrix} \varepsilon & 1 \\ 1 & \varepsilon^{-1} - \varepsilon^{-1} \end{bmatrix} = \begin{bmatrix} \varepsilon & 1 \\ 1 & 0 \end{bmatrix} \approx \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \tag{3.13}$$

which is nothing like A. The small pivot produces a huge multiplier, and rounding wipes out all information in A_{22}.

3.7 Error Analysis of the LU Factorization

Theorem 15 (Backward Error of LU)

Let $\varepsilon_{\text{mach}}$ be the machine epsilon. If the LU factorization runs to completion in floating-point arithmetic without encountering a zero pivot, the computed factors L, U satisfy, to first order in $\varepsilon_{\text{mach}}$,

$$A = LU + E, \qquad |E| \leq \varepsilon_{\text{mach}}\, m\, |L|\, |U|\,, \tag{3.14}$$

where $|\cdot|$ takes entrywise absolute values and the inequality holds entrywise.

The proof is given in Lecture 4. The theorem says that the computed factorization is the exact factorization of a nearby matrix, with a perturbation controlled by $|L|\,|U|$. If $|L|\,|U|$ is comparable to $|A|$, the error is at the level of machine precision. If L or U contains huge entries, the bound is useless. In the example above, L contains ε^{-1} and U contains $1 - \varepsilon^{-1}$, so $|L|\,|U|$ has entries of size ε^{-1} and the error can be as large as A itself.

3.8 (SUPPLEMENT) Further Topics

3.8.1 Lower Triangular Matrices

Inverses of general lower triangular matrices. Above, we proved item 4 of Proposition 11 only for matrices of the form $(L_{m-1} \cdots L_1)^{-1}$. It holds for every invertible lower triangular T:

1. $\det T$ is the product of the diagonal entries, so they are all nonzero. Let $D = \operatorname{diag}(T)$.
2. $N \coloneqq D^{-1}T$ divides row i of T by T_{ii}, so N is *unit* lower triangular.
3. By item 2 of Theorem 12, $N = L_1^{-1} \cdots L_{m-1}^{-1}$ if we take the entries of N below the diagonal as the multipliers. So $N^{-1} = L_{m-1} \cdots L_1$ is a product of lower triangular matrices, hence lower triangular.
4. $T = DN$, so $T^{-1} = N^{-1}D^{-1}$ is lower triangular as well.

A shorter proof of item 2 of Theorem 12. Following [1, Lecture 20]: for $k < j$ we have $\vec{e}_k^{\,*}\vec{\ell}_j = 0$, so multiplying two of the factors just adds their multipliers:

$$(I + \vec{\ell}_k\vec{e}_k^{\,*})(I + \vec{\ell}_j\vec{e}_j^{\,*}) = I + \vec{\ell}_k\vec{e}_k^{\,*} + \vec{\ell}_j\vec{e}_j^{\,*}. \tag{3.15}$$

Multiplying in the factors one at a time from the right, the same cancellation happens at every step, and we get $L_1^{-1} \cdots L_{m-1}^{-1} = I + \sum_{k=1}^{m-1} \vec{\ell}_k\vec{e}_k^{\,*}$.

3.8.2 Cost of LU and Solving Linear Systems

Cost of LU. Step k of Algorithm 5 updates the trailing $(m-k) \times (m-k)$ block, with one multiplication and one subtraction per entry. Adding up over all steps,

$$\sum_{k=1}^{m-1} 2(m-k)^2 = 2\sum_{j=1}^{m-1} j^2 \approx \frac{2}{3}m^3 \text{ flops.} \tag{3.16}$$

The divisions for the multipliers add only $O(m^2)$. The update is a rank-one update, a Level 2 operation with low computational intensity; Section 2.9.3 shows how blocking turns most of it into matrix multiplication.

Solving $A\vec{x} = \vec{b}$ with LU. We split the solve into three steps:

Step	Operation	Cost
1. factor	$A = LU$	$\frac{2}{3}m^3$
2. forward substitution	$L\vec{y} = \vec{b}$	m^2
3. back substitution	$U\vec{x} = \vec{y}$	m^2

The factorization is done once. Every further right-hand side costs only $2m^2$, the same as a multiplication by A^{-1}. Forming A^{-1} would cost about $2m^3$ up front, three times as much as LU, and is less accurate. So we never compute A^{-1} to solve a linear system.

3.8.3 Pivots, Minors, and Uniqueness

The pivots as ratios of minors. Following [2, §2.3]: Step 2 of the proof of Theorem 14 gives $\det A[1{:}k, 1{:}k] = u_{11} \cdots u_{kk}$ for each k. Dividing two consecutive ones,

$$u_{kk} = \frac{\det A[1{:}k, 1{:}k]}{\det A[1{:}k{-}1, 1{:}k{-}1]}. \tag{3.17}$$

So each pivot is determined by A alone, before any elimination is done. For example, for the matrix of the small-pivot example, $u_{11} = \varepsilon$ and $u_{22} = \det A/\varepsilon = 1 - \varepsilon^{-1}$, which matches the U computed there. Moreover, $A = LU$ with L unit lower triangular exists and is unique if and only if $A[1{:}k, 1{:}k]$ is nonsingular for every $k = 1, \ldots, m-1$.

Why the factorization is unique. Let A be nonsingular and suppose $A = L_1U_1 = L_2U_2$, with L_1, L_2 unit lower triangular and U_1, U_2 upper triangular. All four are invertible, and

$$L_2^{-1}L_1 = U_2U_1^{-1}. \tag{3.18}$$

The left side is unit lower triangular and the right side is upper triangular (Proposition 11). A matrix that is both must be I, so $L_1 = L_2$ and $U_1 = U_2$.

3.8.4 Pivoting Preview

Swapping rows. In the small-pivot example, swap the two rows first:

$$PA = \begin{bmatrix} 1 & 1 \\ \varepsilon & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \varepsilon & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1-\varepsilon \end{bmatrix}. \tag{3.19}$$

Now the only multiplier is ε, and every entry of L and U is at most 1 in size. So $|L|\,|U|$ is about as large as $|A|$, and by Theorem 15 the error is at the level of machine precision. This is the idea of *partial pivoting*: at each step, swap rows so that the entry of largest absolute value in the current column becomes the pivot. Then every multiplier satisfies $|\ell_{ik}| \le 1$, and we get $PA = LU$ for a permutation matrix P. This is the subject of Lecture 4.

In Julia. `lu(A)` computes $PA = LU$ with partial pivoting; `lu(A, NoPivot())` runs Algorithm 5 as written. For $A = \left[\begin{smallmatrix} 10^{-20} & 1 \\ 1 & 1 \end{smallmatrix}\right]$ the unpivoted factors satisfy $LU - A = \left[\begin{smallmatrix} 0 & 0 \\ 0 & -1 \end{smallmatrix}\right]$, exactly the failure of the small-pivot example, while the pivoted ones are exact.

```
F = lu(A)                 # F.L, F.U, F.p (row permutation)
x = F \ b                 # reuses the factorization: 2m^2 flops per b
G = lu(A, NoPivot())      # G.L * G.U - A == [0 0; 0 -1] for this A
```

Lecture 4

LU Factorization with Pivoting

Supplementary reading:

- [1] Lectures 21 and 22.
- [2] Sections 2.3 and 2.4.
- [7] Chapter 9.

4.1 Backward Error of the LU Factorization

Recall from Lecture 3 that Gaussian elimination computes $A = LU$ in place, with the multipliers stored below the diagonal and U on and above it, and that it fails on a zero pivot even when A is nonsingular (for example $\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$). Lecture 3 ended with an example in which a tiny pivot destroys the LU factorization of the well-conditioned matrix

$$A = \begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix}, \qquad \kappa(A) \approx 2.6 \text{ as } \varepsilon \to 0. \tag{4.1}$$

Here ε is just a small number, much smaller than $\varepsilon_{\text{mach}}$ (e.g., 10^{-20}); it is not the machine epsilon. The problem is well-conditioned, so the failure lies with the algorithm. The following theorem explains exactly where it comes from.

Theorem 16 (Backward Error of LU)

Suppose the LU factorization of $A \in \mathbb{K}^{m\times m}$ runs to completion in floating-point arithmetic without encountering a zero pivot, producing $\tilde{L}$ and $\tilde{U}$. Then, to first order in $\varepsilon_{\text{mach}}$,

$$A = \tilde{L}\tilde{U} + E, \qquad |E| \le m\varepsilon_{\text{mach}} \left|\tilde{L}\right| \left|\tilde{U}\right|. \tag{4.2}$$

So $\tilde{L}$ and $\tilde{U}$ are the exact LU factors of a nearby matrix, $A - E$. If $\left|\tilde{L}\right| \left|\tilde{U}\right|$ is about as large as $|A|$, then E is tiny compared with A and the algorithm is backward stable. If $\left|\tilde{L}\right| \left|\tilde{U}\right|$ is much larger than $|A|$, then E can be as large as A. ("To first order" means we drop terms of size $\varepsilon_{\text{mach}}^2$.)

4.1.1 Proof of Theorem 16

Formulas for L and U. From $A = LU$ and $L_{ii} = 1$, solving for the entries of L and U gives (these are the numbers elimination computes, in a different order)

$$U_{ik} = A_{ik} - \sum_{j=1}^{i-1} L_{ij}U_{jk}, \qquad i \le k, \tag{4.3}$$

$$L_{ik} = \frac{1}{U_{kk}}\left(A_{ik} - \sum_{j=1}^{k-1} L_{ij}U_{jk}\right), \qquad i > k. \tag{4.4}$$

Rounding in inner products (Homework 2, Proposition 111): in floating point, every term of a sum gets its own small relative error,

$$\mathrm{fl}\Big(\sum_{i=1}^{m} x_i y_i\Big) = \sum_{i=1}^{m} x_i y_i(1+\delta_i), \qquad |\delta_i| \lesssim m\varepsilon_{\mathrm{mach}}. \tag{4.5}$$

Proof. The U part ($i \le k$). In floating point, (4.3) becomes

$$\tilde{U}_{ik} = \Big(A_{ik} - \sum_{j=1}^{i-1} \tilde{L}_{ij}\tilde{U}_{jk}(1+\delta_j)\Big)(1+\delta'), \tag{4.6}$$

where δ' is the error of the last subtraction. Solve for A_{ik}, use $\tilde{L}_{ii} = 1$, and write $1/(1+\delta') = 1+\delta_i$:

$$A_{ik} = \sum_{j=1}^{i} \tilde{L}_{ij}\tilde{U}_{jk}(1+\delta_j) = \sum_{j=1}^{i} \tilde{L}_{ij}\tilde{U}_{jk} + \underbrace{\sum_{j=1}^{i} \tilde{L}_{ij}\tilde{U}_{jk}\delta_j}_{=:E_{ik}}. \tag{4.7}$$

Every $|\delta_j| \lesssim m\varepsilon_{\mathrm{mach}}$, so $|E_{ik}| \le m\varepsilon_{\mathrm{mach}} \sum_j \left|\tilde{L}_{ij}\right|\left|\tilde{U}_{jk}\right| = m\varepsilon_{\mathrm{mach}}(\left|\tilde{L}\right|\left|\tilde{U}\right|)_{ik}$. (The terms with $j > i$ are zero because $\tilde{L}$ is lower triangular.)

The L part ($i > k$). The same argument applied to (4.4). The division by $\tilde{U}_{kk}$ adds one more rounding error, which is merged into δ_k in the same way, and the bound is the same. □

4.1.2 What the Bound Says

If $\left|\tilde{L}\right|\left|\tilde{U}\right|$ is of the same order as $|A|$, then E is a relative perturbation of size $O(m\varepsilon_{\mathrm{mach}})$ and the algorithm is backward stable. But $\left|\tilde{L}\right|\left|\tilde{U}\right|$ can be far larger than $|A|$. In the example above,

$$|L|\,|U| = \begin{bmatrix} 1 & 0 \\ \varepsilon^{-1} & 1 \end{bmatrix}\begin{bmatrix} \varepsilon & 1 \\ 0 & \varepsilon^{-1}-1 \end{bmatrix} = \begin{bmatrix} \varepsilon & 1 \\ 1 & 2\varepsilon^{-1}-1 \end{bmatrix}. \tag{4.8}$$

The bottom-right entry is about $2\varepsilon^{-1}$; multiplied by $\varepsilon_{\mathrm{mach}}$, the error can be $O(1)$ or larger, exactly as observed. The question becomes: *how can we keep $|L|\,|U|$ small?*

4.2 Pivots and the Growth of Products

Look at the storage after $k-1$ steps of elimination: the first $k-1$ rows already hold U, the first $k-1$ columns below the diagonal hold L, and the trailing block $\tilde{A}$ is still being processed.

$$\begin{bmatrix} U_{11} & \cdots & & & & \\ & \ddots & & & & \\ & & U_{k-1,k-1} & U_{k-1,k} & \cdots & U_{k-1,m} \\ & L & L_{k,k-1} & \tilde{A}_{kk} & \cdots & \tilde{A}_{km} \\ & & \vdots & \vdots & & \vdots \\ & & L_{m,k-1} & \tilde{A}_{mk} & \cdots & \tilde{A}_{mm} \end{bmatrix} \tag{4.9}$$

Step k turns row k into a row of U and column k below the diagonal into a column of L, and updates the trailing $(m-k)\times(m-k)$ block:

$$L_{ik} = \frac{\tilde{A}_{ik}}{\tilde{A}_{kk}}, \qquad U_{kj} = \tilde{A}_{kj}, \qquad \tilde{A}_{ij} \leftarrow \tilde{A}_{ij} - L_{ik}U_{kj}. \tag{4.10}$$

The entry $\tilde{A}_{kk}$ is the k-th *pivot*. The largest product created in step k, which enters both the update and $|L|\,|U|$, is

$$\max_{i,j} |L_{ik}U_{kj}| = \max_{i,j} \frac{\left|\tilde{A}_{ik}\right|\left|\tilde{A}_{kj}\right|}{\left|\tilde{A}_{kk}\right|}. \tag{4.11}$$

The numerator is determined by column k and row k. To keep this quantity small, we want the denominator, the pivot, to be as large as possible.

4.3 Partial Pivoting

We must pivot when $\tilde{A}_{kk} = 0$, since otherwise the algorithm cannot continue. Even when $\tilde{A}_{kk} \neq 0$, we want to avoid *small* pivots, which make the products $L_{ik}U_{kj}$ explode.

Definition 17 (Partial (Row) Pivoting)

At step k, *partial pivoting* chooses the pivot to be the entry of largest absolute value in column k, on or below the diagonal:

$$r = \operatorname*{argmax}_{i\geq k} \left|\tilde{A}_{ik}\right|. \tag{4.12}$$

It swaps rows r and k with a permutation matrix P_k and then eliminates with L_k. The sequence $A \xrightarrow{L_k} \cdots$ becomes $A \xrightarrow{P_k} \cdot \xrightarrow{L_k} \cdots$.

Because the pivot is the largest entry in its column, every multiplier satisfies

$$|L_{ik}| = \frac{\left|\tilde{A}_{ik}\right|}{\left|\tilde{A}_{kk}\right|} \leq 1, \tag{4.13}$$

so $|L_{ik}U_{kj}| \leq |U_{kj}|$: no update exceeds the size of the current pivot row.

4.3.1 Moving the Permutations Together

With row swaps, elimination becomes

$$L_{m-1}P_{m-1}\cdots L_2P_2L_1P_1A = U, \tag{4.14}$$

where L's and P's alternate, so we cannot directly write $PA = LU$. We need to move all permutations to the right.

Proposition 18 (Commuting a Permutation Past an Elimination)

Let $\ell > k$ and let P_ℓ swap rows ℓ and i for some $i \geq \ell$. Swapping twice undoes the swap, so $P_\ell^{-1} = P_\ell$. Then

$$P_\ell L_k = (P_\ell L_k P_\ell)P_\ell, \qquad P_\ell L_k P_\ell = I - (P_\ell \vec{\ell}_k)\vec{e}_k^*, \tag{4.15}$$

so $P_\ell L_k P_\ell$ is again an elementary lower triangular matrix, obtained from L_k by swapping two of its multipliers.

Proof. The first identity holds because $P_\ell P_\ell = I$. For the second, substitute $L_k = I - \vec{\ell}_k\vec{e}_k^*$ and expand:

$$P_\ell(I - \vec{\ell}_k\vec{e}_k^*)P_\ell = P_\ell P_\ell - (P_\ell\vec{\ell}_k)(\vec{e}_k^*P_\ell) = I - (P_\ell\vec{\ell}_k)(\vec{e}_k^*P_\ell). \tag{4.16}$$

Multiplying the row vector $\vec{e}_k^*$ on the right by P_ℓ swaps its entries ℓ and i. Both entries are 0, since $\ell, i > k$, so nothing changes: $\vec{e}_k^*P_\ell = \vec{e}_k^*$. This gives $P_\ell L_k P_\ell = I - (P_\ell\vec{\ell}_k)\vec{e}_k^*$.

Finally, $P_\ell\vec{\ell}_k$ is $\vec{\ell}_k$ with entries ℓ and i swapped. Its first k entries are still 0, so $P_\ell L_k P_\ell$ is again elementary lower triangular. □

Using Proposition 18 repeatedly to push every P to the right,

$$L_{m-1}P_{m-1}\cdots L_1P_1 = L'_{m-1}\cdots L'_1\,P_{m-1}\cdots P_1, \qquad L'_k \coloneqq P_{m-1}\cdots P_{k+1}L_kP_{k+1}\cdots P_{m-1}, \tag{4.17}$$

and each L'_k is still elementary lower triangular. With $P \coloneqq P_{m-1}\cdots P_1$ and $L \coloneqq (L'_{m-1}\cdots L'_1)^{-1}$, we obtain

$$PA = LU. \tag{4.18}$$

4.4 The In-Place Algorithm with Partial Pivoting

How do we compute this in place, up to the permutation? Suppose k steps are done:

$$L_k\cdots L_1PA = U_k, \tag{4.19}$$

where the L_j are already the "permutation-adjusted" versions and U_k is the current working matrix. In storage, U_k occupies the upper triangle and the trailing block, and the multipliers of $L_1, \ldots, L_k$ sit in the strictly lower part of the first k columns. Step $k+1$ swaps rows and then eliminates:

$$L_{k+1}P_{k+1}L_k\cdots L_1PA = L_{k+1}P_{k+1}U_k. \tag{4.20}$$

Moving P_{k+1} to the right,

$$L_{k+1}\underbrace{L'_k\cdots L'_1}_{\text{multipliers permuted by } P_{k+1}}\underbrace{P_{k+1}P}_{P'}A = L_{k+1}\underbrace{P_{k+1}U_k}_{U'_k} = U_{k+1}. \tag{4.21}$$

Reading term by term:

- $L'_j = P_{k+1} L_j P_{k+1}$ only permutes the stored multipliers;
- $P' = P_{k+1} P$ swaps the same two rows of the accumulated permutation;
- $U'_k = P_{k+1} U_k$ swaps the same two rows of the working matrix.

Since the multipliers and U_k live in the same array, *swapping entire rows (columns* 1 *through* m*)* performs all three updates at once, after which we eliminate as usual.

Algorithm 6 LU factorization with partial pivoting, in place.

$P \leftarrow I$
for $k = 1, \dots, m$ **do**
  $r \leftarrow \operatorname{argmax}_{i \geq k} |A_{ik}|$
  swap rows r and k of A (all columns $1, \dots, m$) and of P
  for $i = k+1, \dots, m$ **do**
    $A_{ik} \leftarrow A_{ik} / A_{kk}$
  for $j = k+1, \dots, m$ **do**
    for $i = k+1, \dots, m$ **do**
      $A_{ij} \leftarrow A_{ij} - A_{ik} A_{kj}$

Some remarks:

- In Julia, `argmax(abs.(A[k:end, k]))` returns an index into the subvector, so one adds $k-1$ to recover the global row index.
- The row swap runs over all columns $1, \dots, m$, not just $k, \dots, m$: the first $k-1$ columns hold multipliers, and swapping them is exactly $L'_j = P_{k+1} L_j P_{k+1}$.
- On exit the strictly lower part of A holds L, the upper part holds U, and $PA = LU$.

4.5 (SUPPLEMENT) Further Topics

4.5.1 Storing P and the Cost of Pivoting

In practice P is not stored as a matrix but as a permutation vector of length m (LAPACK's `ipiv`), so each swap exchanges two integers. The search for pivots costs only $O(m^2)$ comparisons on top of the $\frac{2}{3}m^3$ flops. In column-major storage, however, a row swap is a strided access, and its cost is not negligible.

4.5.2 Growth Factor

Definition 19 (Growth Factor)

The *growth factor* of Gaussian elimination on A is

$$\rho := \frac{\max_{i,j} |U_{ij}|}{\max_{i,j} |A_{ij}|}. \tag{4.22}$$

Why it matters. With partial pivoting $|L_{ik}| \le 1$, so each entry of $|L|\,|U|$ is a sum of at most m terms, each at most $\max|U_{ij}| = \rho \max|A_{ij}|$:

$$(|L|\,|U|)_{ij} \le m\rho \max_{i,j} |A_{ij}| \qquad \Longrightarrow \qquad |E_{ij}| \le m^2 \rho\, \varepsilon_{\text{mach}} \max_{i,j} |A_{ij}| \tag{4.23}$$

by Theorem 16. So partial pivoting is backward stable exactly when ρ stays small.

How large can ρ be? Each step updates $\tilde{A}_{ij} \leftarrow \tilde{A}_{ij} - L_{ik}U_{kj}$ with $|L_{ik}| \le 1$, so the largest entry can at most double. After $m-1$ steps,

$$\rho \le 2^{m-1}. \tag{4.24}$$

This bound is attained by the matrix with 1 on the diagonal, -1 below it, and 1 in the last column (Example 20, Homework 2): no rows are swapped, and the last column of U is $1, 2, 4, \ldots, 2^{m-1}$. For $m = 8$, Julia's `lu` gives $\rho = 128 = 2^7$.

In practice. Such matrices are extremely rare. For random matrices ρ grows slowly; with `randn` (median of 5 runs) we measured $\rho \approx 2, 4, 8, 19$ for $m = 16, 64, 256, 1024$. This is why partial pivoting is used as a stable algorithm in practice, although in theory it is not backward stable.

4.5.3 Complete Pivoting

The idea. At step k, *complete pivoting* chooses the pivot to be the entry of largest absolute value in the whole trailing block, $\max_{i,j\ge k} \left|\tilde{A}_{ij}\right|$, and moves it to position (k,k) by swapping two rows *and* two columns. This gives

$$PAQ = LU, \tag{4.25}$$

with a row permutation P and a column permutation Q. Swapping columns reorders the unknowns, so $A\vec{x} = \vec{b}$ is solved as $LU(Q^\top \vec{x}) = P\vec{b}$.

Benefit. Every entry of the trailing block is now at most the pivot, so the growth is much better controlled. Wilkinson showed a bound on ρ that grows much more slowly than 2^{m-1}, roughly like $m^{(\ln m)/4}$ [7, Chapter 9].

Cost. Step k searches all $(m-k+1)^2$ entries of the trailing block, about $m^3/3$ comparisons in total. That is the same order as the $\frac{2}{3}m^3$ flops, and the column swaps add strided memory access. Since partial pivoting is almost always good enough, complete pivoting is rarely used.

Example. We ran complete pivoting (a short Julia implementation) on the worst-case matrix above and on random matrices, and compared with partial pivoting:

Matrix	ρ, partial pivoting	ρ, complete pivoting
worst case, $m = 64$	$2^{63} \approx 9 \cdot 10^{18}$	2
`randn`, $m = 64$	4.2	1.9
`randn`, $m = 256$	7.8	3.6

On the worst-case matrix, complete pivoting removes the growth entirely. On random matrices both are small, which is why the extra cost of complete pivoting is rarely worth it.

A compromise: rook pivoting. Search alternately along a column and a row until the entry is the largest in both. This is cheap and controls growth much better [7, Chapter 9].

Lecture 5

Least Squares Problems, Gram–Schmidt, and QR

Supplementary reading:

- [1] Lectures 6, 7, 8, 11, and 22.
- [2] Sections 3.1–3.3.

5.1 Is Partial Pivoting Enough?

Lecture 4 bounded the backward error of LU by $m\varepsilon_{\text{mach}} \left|\tilde{L}\right| \left|\tilde{U}\right|$ and introduced partial pivoting to keep the multipliers bounded, $|L_{ij}| \leq 1$; commuting the permutations past the elimination matrices then gave $PA = LU$ and an in-place algorithm. Rows and columns can also be pivoted simultaneously (*complete pivoting*), giving $PA\tilde{P} = LU$ with a column permutation $\tilde{P}$. In practice, partial pivoting is usually enough. In theory, however, it can be *unstable* when m is large.

Example 20 (Exponential Growth Under Partial Pivoting)**.** Consider

$$A = \begin{bmatrix} 1 & & & 1 \\ -1 & 1 & & 1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & 1 \end{bmatrix}. \tag{5.1}$$

In every column the candidate pivots have the same absolute value, $|1| = |-1|$, and `argmax` returns the first, so partial pivoting never swaps rows. Tracking the in-place storage (multipliers below the diagonal),

$$A \rightsquigarrow \begin{bmatrix} 1 & & & 1 \\ -1 & 1 & & 2 \\ -1 & -1 & 1 & 2 \\ -1 & -1 & -1 & 2 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 1 & & & 1 \\ -1 & 1 & & 2 \\ -1 & -1 & 1 & 4 \\ -1 & -1 & -1 & 4 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 1 & & & 1 \\ -1 & 1 & & 2 \\ -1 & -1 & 1 & 4 \\ -1 & -1 & -1 & 8 \end{bmatrix}. \tag{5.2}$$

The multiplier at step k is -1, i.e., row k is added to every row below it. Row k is zero in columns $k+1$

through $m-1$, so the middle part never changes, but the last column doubles at every step. The result is

$$L = \begin{bmatrix} 1 & & & \\ -1 & 1 & & \\ -1 & -1 & 1 & \\ -1 & -1 & -1 & 1 \end{bmatrix}, \qquad U = \begin{bmatrix} 1 & & & 1 \\ & 1 & & 2 \\ & & 1 & 4 \\ & & & 8 \end{bmatrix}. \tag{5.3}$$

For general m, $U_{mm} = 2^{m-1}$, so $|L|\,|U| \gtrsim 2^m$ and the error bound $m\varepsilon_{\text{mach}}\,|L|\,|U|$ can reach $m2^m\varepsilon_{\text{mach}}$. In double precision, $2^{m-1}\varepsilon_{\text{mach}} \approx 1$ already for $m \approx 53$, so all accuracy can be lost.

This is the worst case $\rho = 2^{m-1}$ of the growth factor mentioned in Lecture 4. Fortunately, such matrices are extremely rare in practice.

5.2 Non-Square Systems

So far we have only considered $A \in \mathbb{K}^{m\times n}$ with $m = n$. If $m \neq n$, there are two cases.

Overdetermined systems ($m > n$). A is tall: there are more equations than unknowns. Then $\mathcal{R}(A) \neq \mathbb{K}^m$, since n columns span at most an n-dimensional subspace. Hence for a general $\vec{b}$ the equation $A\vec{x} = \vec{b}$ *does not have a solution*. Such a system is called an *overdetermined linear system*.

Underdetermined systems ($m < n$). A is wide: there are more unknowns than equations. Then $\mathcal{N}(A) \neq \{\vec{0}\}$, since the rank is at most $m < n$. Hence even when a solution exists, it is *never unique*: adding any vector of the null space to a solution gives another solution. Such a system is called an *underdetermined linear system*.

5.3 Least Squares Problems

Overdetermined systems usually have no solution, and underdetermined systems have many. In both cases we replace the equation by a minimization problem:

$$\text{overdetermined:} \quad \min_{\vec{x}} \left\| A\vec{x} - \vec{b} \right\|, \qquad \text{underdetermined:} \quad \min_{\vec{x}\,:\,A\vec{x}=\vec{b}} \|\vec{x}\|. \tag{5.4}$$

In the overdetermined case we take the $\vec{x}$ that comes closest to satisfying the equations; in the underdetermined case we take the smallest of all solutions.

Which norm should we use? There are many choices, depending on the application. In linear regression, for example, the norm in the overdetermined case measures the *fitting error*, and in the underdetermined case it measures how *complex* the model is.

The key fact: the solution depends linearly on $\vec{b}$ *only if the norm comes from an inner product*, $\|\cdot\| = \sqrt{\langle\cdot,\cdot\rangle}$. This case is the *least squares problem*, and we can solve it with linear algebra.

Example 21 (Mean Versus Median)**.** Let $A = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}^\top$. Then $\min_x \left\| Ax - \vec{b} \right\|$ looks for the single number x closest to all of b_1, b_2, b_3.

- In the 2-norm, the best x is the mean $\frac{1}{3}(b_1 + b_2 + b_3)$, which is linear in $\vec{b}$.
- In the 1-norm, the best x is the median, which is not linear in $\vec{b}$.

5.4 The Geometry of Least Squares

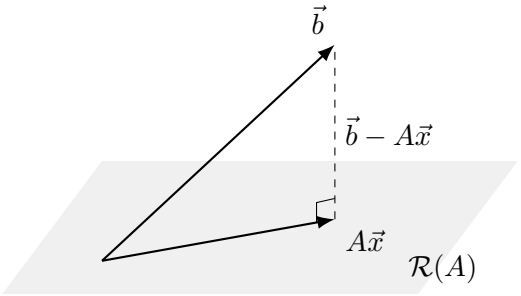
For $m = n$, solving $A\vec{x} = \vec{b}$ means finding a linear combination of the columns of A that equals $\vec{b}$ exactly:

$$A\vec{x} = x_1\vec{a}_1 + x_2\vec{a}_2 + \cdots + x_n\vec{a}_n = \vec{b}. \tag{5.5}$$

For $m > n$, consider

$$\min_{\vec{x}} \left\| A\vec{x} - \vec{b} \right\|_2^2. \tag{5.6}$$

The columns of A span only an n-dimensional subspace $\mathcal{R}(A)$ of $\mathbb{K}^m$, and $\vec{b}$ generally does not lie in it. As $\vec{x}$ varies, $A\vec{x}$ ranges over this subspace, and the point closest to $\vec{b}$ is the *orthogonal projection* of $\vec{b}$ onto $\mathcal{R}(A)$: dropping a perpendicular from $\vec{b}$ to the subspace, its foot is the optimal $A\vec{x}$, and the residual $A\vec{x} - \vec{b}$ is orthogonal to the whole subspace.



Writing "the residual is orthogonal to every column of A" as an equation gives the *normal equations*:

$$A^*(A\vec{x} - \vec{b}) = \vec{0} \quad \Longleftrightarrow \quad A^*A\vec{x} = A^*\vec{b}. \tag{5.7}$$

Orthonormal columns. If the columns of A are orthonormal, then $A^*A = I$ and the normal equations give the solution directly:

$$\vec{x} = A^*\vec{b}, \qquad \text{i.e.,} \qquad x_i = \left\langle \vec{a}_i, \vec{b} \right\rangle. \tag{5.8}$$

Since $\vec{a}_i$ is a unit vector, x_i is how much of $\vec{b}$ points along the i-th column. The columns are orthogonal, so each x_i is computed on its own, and $A\vec{x} = \sum_i x_i\vec{a}_i$ is the projection of $\vec{b}$ onto $\mathcal{R}(A)$. Without orthogonality the coefficients depend on each other, and we must solve $A^*A\vec{x} = A^*\vec{b}$.

Example 22 (Projection onto a Coordinate Plane)**.** For

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \tag{5.9}$$

we get $\vec{x} = (b_1, b_2)^\top$ and $A\vec{x} = (b_1, b_2, 0)^\top$, the closest point of the xy-plane to $\vec{b}$. The residual $(0, 0, b_3)^\top$ is orthogonal to the plane.

This motivates the next section: since orthonormal columns make least squares easy, we first *make* the columns orthonormal.

5.5 Orthogonalization: The QR Factorization

What if the columns of A are not orthonormal? The idea is to find an *orthonormal basis* of the space spanned by the columns of A and to transfer the problem to that basis. In matrix form,

$$A = QR, \tag{5.10}$$

where $Q \in \mathbb{K}^{m\times n}$ has orthonormal columns and $R \in \mathbb{K}^{n\times n}$ is invertible (it will turn out to be upper triangular). The least squares solution is then

$$\vec{x} = R^{-1}Q^*\vec{b}. \tag{5.11}$$

Both steps have a clear meaning:

- $Q^*\vec{b}$ expresses $\vec{b}$ in terms of the columns of Q (the coordinates of the projection, exactly as in the orthonormal case);
- $A = QR$ says that R expresses the columns of A as combinations of the columns of Q, so R^{-1} converts coefficients with respect to Q back into coefficients with respect to A.

Algebraically, in the normal equations (5.7) we have $A^*A = R^*Q^*QR = R^*R$ and $A^*\vec{b} = R^*Q^*\vec{b}$; cancelling the invertible R^* leaves $R\vec{x} = Q^*\vec{b}$.

5.6 Classical Gram–Schmidt

The idea. Process the columns of A one at a time. When we reach $\vec{a}_j$, the vectors $\vec{q}_1, \dots, \vec{q}_{j-1}$ are already orthonormal. To get $\vec{q}_j$:

1. subtract from $\vec{a}_j$ its components along $\vec{q}_1, \dots, \vec{q}_{j-1}$;
2. normalize what is left.

We first make $\vec{v}_j$ orthogonal to the previous vectors and normalize afterwards. The component of $\vec{a}_j$ along $\vec{v}_i$ is $\frac{\langle \vec{v}_i, \vec{a}_j\rangle}{\langle \vec{v}_i, \vec{v}_i\rangle}\vec{v}_i$, so

$$\begin{aligned}
\vec{v}_1 &= \vec{a}_1, \\
\vec{v}_2 &= \vec{a}_2 - \frac{\langle \vec{v}_1, \vec{a}_2\rangle}{\langle \vec{v}_1, \vec{v}_1\rangle}\vec{v}_1, \\
\vec{v}_3 &= \vec{a}_3 - \frac{\langle \vec{v}_1, \vec{a}_3\rangle}{\langle \vec{v}_1, \vec{v}_1\rangle}\vec{v}_1 - \frac{\langle \vec{v}_2, \vec{a}_3\rangle}{\langle \vec{v}_2, \vec{v}_2\rangle}\vec{v}_2, \\
&\vdots \\
\vec{v}_j &= \vec{a}_j - \sum_{i=1}^{j-1} \frac{\langle \vec{v}_i, \vec{a}_j\rangle}{\langle \vec{v}_i, \vec{v}_i\rangle}\vec{v}_i,
\end{aligned} \tag{5.12}$$

and finally $\vec{q}_j = \vec{v}_j / \|\vec{v}_j\|_2$. Since $\vec{q}_i = \vec{v}_i / \|\vec{v}_i\|_2$, each term equals $\langle \vec{q}_i, \vec{a}_j\rangle \vec{q}_i$, the component of $\vec{a}_j$ along $\vec{q}_i$.

Where R comes from. Write each term of (5.12) as $\langle \vec{q}_i, \vec{a}_j\rangle \vec{q}_i$ and name the coefficients:

$$r_{ij} = \langle \vec{q}_i, \vec{a}_j\rangle \quad (i < j), \qquad r_{jj} = \|\vec{v}_j\|_2, \qquad r_{ij} = 0 \quad (i > j). \tag{5.13}$$

Since $\vec{v}_j = r_{jj}\vec{q}_j$, solving (5.12) for $\vec{a}_j$ gives

$$\vec{a}_j = r_{1j}\vec{q}_1 + \cdots + r_{j-1,j}\vec{q}_{j-1} + r_{jj}\vec{q}_j = \sum_{i=1}^{j} r_{ij}\vec{q}_i, \tag{5.14}$$

that is, column j of A is Q times column j of R. For all columns at once,

$$\underbrace{\begin{bmatrix}\vec{a}_1 & \vec{a}_2 & \cdots & \vec{a}_n\end{bmatrix}}_{A} = \underbrace{\begin{bmatrix}\vec{q}_1 & \vec{q}_2 & \cdots & \vec{q}_n\end{bmatrix}}_{Q} \underbrace{\begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ & r_{22} & \cdots & r_{2n} \\ & & \ddots & \vdots \\ & & & r_{nn}\end{bmatrix}}_{R}. \tag{5.15}$$

R is upper triangular because $\vec{a}_j$ involves only $\vec{q}_1, \ldots, \vec{q}_j$, and invertible because $r_{jj} > 0$. Entry r_{ij} $(i < j)$ is the amount of $\vec{q}_i$ in $\vec{a}_j$; r_{jj} is the length of the new part of $\vec{a}_j$.

The algorithm. Computing the sum in (5.12) one term at a time:

Algorithm 7 Classical Gram–Schmidt (CGS).

for $j = 1, \ldots, n$ **do**
  $\vec{v}_j \leftarrow \vec{a}_j$
  for $i = 1, \ldots, j-1$ **do**
    $r_{ij} \leftarrow \langle \vec{q}_i, \vec{a}_j \rangle$ ▷ inner product with the *original* column
    $\vec{v}_j \leftarrow \vec{v}_j - r_{ij}\vec{q}_i$
  $r_{jj} \leftarrow \|\vec{v}_j\|_2$
  $\vec{q}_j \leftarrow \vec{v}_j / r_{jj}$

Theorem 23 (Gram–Schmidt)

Let $\vec{a}_1, \ldots, \vec{a}_n \in \mathbb{K}^m$ $(m \geq n)$ be linearly independent. Then the vectors $\vec{q}_1, \ldots, \vec{q}_n$ produced by Algorithm 7 form an orthonormal basis of $\operatorname{span}(\vec{a}_1, \ldots, \vec{a}_n)$, and $\operatorname{span}(\vec{q}_1, \ldots, \vec{q}_j) = \operatorname{span}(\vec{a}_1, \ldots, \vec{a}_j)$ for every j.

Proof. By induction on j: $\vec{q}_1, \ldots, \vec{q}_j$ are orthonormal and span $\operatorname{span}(\vec{a}_1, \ldots, \vec{a}_j)$. *Base case:* $\vec{a}_1 \neq \vec{0}$ by linear independence, so $\vec{q}_1 = \vec{a}_1 / \|\vec{a}_1\|_2$ is a unit vector spanning the same line.

Inductive step. Assume the claim for j, and let $\vec{v}_{j+1} = \vec{a}_{j+1} - \sum_{i=1}^{j} \langle \vec{q}_i, \vec{a}_{j+1} \rangle \vec{q}_i$.

1. *Orthogonality.* For every $k \leq j$,

$$\langle \vec{q}_k, \vec{v}_{j+1} \rangle = \langle \vec{q}_k, \vec{a}_{j+1} \rangle - \sum_{i=1}^{j} \langle \vec{q}_i, \vec{a}_{j+1} \rangle \underbrace{\langle \vec{q}_k, \vec{q}_i \rangle}_{\delta_{ki}} = \langle \vec{q}_k, \vec{a}_{j+1} \rangle - \langle \vec{q}_k, \vec{a}_{j+1} \rangle = 0. \tag{5.16}$$

2. *Nonzero.* If $\vec{v}_{j+1} = \vec{0}$, then $\vec{a}_{j+1} \in \operatorname{span}(\vec{q}_1, \ldots, \vec{q}_j) = \operatorname{span}(\vec{a}_1, \ldots, \vec{a}_j)$, contradicting linear independence. So $\vec{q}_{j+1} = \vec{v}_{j+1} / \|\vec{v}_{j+1}\|_2$ is defined, and $\vec{q}_1, \ldots, \vec{q}_{j+1}$ are orthonormal.

3. *Span.* $\vec{q}_{j+1} \in \operatorname{span}(\vec{a}_1, \ldots, \vec{a}_{j+1})$, and $\vec{q}_1, \ldots, \vec{q}_{j+1}$ are $j+1$ orthonormal, hence independent, vectors in this $(j+1)$-dimensional space, so they span it. □

Example 24 (CGS Loses Orthogonality)**.** Consider the matrix

$$A = \begin{bmatrix} 1 & 1 & 1 \\ \varepsilon & 0 & 0 \\ 0 & \varepsilon & 0 \\ 0 & 0 & \varepsilon \end{bmatrix}, \tag{5.17}$$

with ε small enough that $1 + \varepsilon^2 = 1$ in floating point, i.e., $\varepsilon < \sqrt{u} \approx 10^{-8}$, for example $\varepsilon = 10^{-10}$.

Step 1. $\vec{v}_1 = \vec{a}_1 = (1, \varepsilon, 0, 0)^\top$ and $\vec{q}_1 = (1 + \varepsilon^2)^{-1/2}(1, \varepsilon, 0, 0)^\top$, which in floating point is $(1, \varepsilon, 0, 0)^\top$.

Step 2. $r_{12} = \langle \vec{q}_1, \vec{a}_2 \rangle = 1$, so

$$\vec{v}_2 = \vec{a}_2 - r_{12}\vec{q}_1 = \begin{bmatrix} 1 \\ 0 \\ \varepsilon \\ 0 \end{bmatrix} - \begin{bmatrix} 1 \\ \varepsilon \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ -\varepsilon \\ \varepsilon \\ 0 \end{bmatrix}, \qquad \vec{q}_2 = \frac{\vec{v}_2}{\sqrt{2}\,\varepsilon} = \begin{bmatrix} 0 \\ -1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{bmatrix}. \tag{5.18}$$

Step 3. $r_{13} = \langle \vec{q}_1, \vec{a}_3 \rangle = 1$ and $r_{23} = \langle \vec{q}_2, \vec{a}_3 \rangle = 0$, so

$$\vec{v}_3 = \vec{a}_3 - r_{13}\vec{q}_1 - r_{23}\vec{q}_2 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \varepsilon \end{bmatrix} - \begin{bmatrix} 1 \\ \varepsilon \\ 0 \\ 0 \end{bmatrix} - 0 \cdot \vec{q}_2 = \begin{bmatrix} 0 \\ -\varepsilon \\ 0 \\ \varepsilon \end{bmatrix}, \qquad \vec{q}_3 = \frac{\vec{v}_3}{\sqrt{2}\,\varepsilon} = \begin{bmatrix} 0 \\ -1/\sqrt{2} \\ 0 \\ 1/\sqrt{2} \end{bmatrix}. \tag{5.19}$$

The result

$$Q = \begin{bmatrix} 1 & 0 & 0 \\ \varepsilon & -1/\sqrt{2} & -1/\sqrt{2} \\ 0 & 1/\sqrt{2} & 0 \\ 0 & 0 & 1/\sqrt{2} \end{bmatrix} \tag{5.20}$$

is *far from orthogonal*: $\langle \vec{q}_2, \vec{q}_3 \rangle = \frac{1}{2}$, so the angle between them is $60°$ rather than $90°$.

5.7 Modified Gram–Schmidt

What went wrong. Look at step 3 of Example 24.

- First CGS removes the $\vec{q}_1$-component: $\vec{a}_3 - r_{13}\vec{q}_1 = (0, -\varepsilon, 0, \varepsilon)^\top$. Two vectors of size 1 cancel, and only a vector of size ε is left. So any error of size ε is now as large as what is left.
- Such an error is there: the computed $\vec{q}_1$ and $\vec{q}_2$ are not exactly orthogonal, $\langle \vec{q}_1, \vec{q}_2 \rangle = -\varepsilon/\sqrt{2}$.
- Next CGS removes the $\vec{q}_2$-component, but it measures this component in the *original* column: $r_{23} = \langle \vec{q}_2, \vec{a}_3 \rangle = 0$. What we actually need to remove is the $\vec{q}_2$-component of *what is left*:

$$r_{23} = \langle \vec{q}_2, \vec{a}_3 - r_{13}\vec{q}_1 \rangle = \frac{\varepsilon}{\sqrt{2}} \neq 0. \tag{5.21}$$

The two values differ by $-r_{13}\langle \vec{q}_2, \vec{q}_1 \rangle$, which is exactly the orthogonality error. In exact arithmetic it is 0 and the two agree; in floating point it is not.

Notation. To write the fix, we name the intermediate values of $\vec{v}_j$ in Algorithm 7: $\vec{v}_j^{(i)}$ is $\vec{v}_j$ after i subtractions,

$$\vec{v}_j^{(0)} = \vec{a}_j, \qquad \vec{v}_j^{(i)} = \vec{v}_j^{(i-1)} - r_{ij}\vec{q}_i, \qquad \vec{v}_j^{(j-1)} = \vec{v}_j. \tag{5.22}$$

For example, $\vec{v}_3^{(1)} = \vec{a}_3 - r_{13}\vec{q}_1$ is the "what is left" above.

The fix. Change a single line: compute r_{ij} from the *partially orthogonalized* vector instead of the original column.

$$\text{CGS:} \quad r_{ij} = \left\langle \vec{q}_i, \vec{v}_j^{(0)} \right\rangle = \langle \vec{q}_i, \vec{a}_j \rangle, \qquad \text{MGS:} \quad r_{ij} = \left\langle \vec{q}_i, \vec{v}_j^{(i-1)} \right\rangle. \tag{5.23}$$

Everything else, including the update $\vec{v}_j^{(i)} = \vec{v}_j^{(i-1)} - r_{ij}\vec{q}_i$, stays the same.

To compare with CGS directly, we write MGS in the same unnormalized form. MGS subtracts one direction at a time, and each inner product uses the vector *as it is now*:

$$\begin{aligned}
\vec{v}_1 &= \vec{a}_1, \\
\vec{v}_2 &= \vec{a}_2 - \frac{\langle \vec{v}_1, \vec{a}_2 \rangle}{\langle \vec{v}_1, \vec{v}_1 \rangle}\vec{v}_1, \\
\vec{v}_3^{(1)} &= \vec{a}_3 - \frac{\langle \vec{v}_1, \vec{a}_3 \rangle}{\langle \vec{v}_1, \vec{v}_1 \rangle}\vec{v}_1, \\
\vec{v}_3 &= \vec{v}_3^{(1)} - \frac{\langle \vec{v}_2, \vec{v}_3^{(1)} \rangle}{\langle \vec{v}_2, \vec{v}_2 \rangle}\vec{v}_2, \\
&\vdots \\
\vec{v}_j^{(i)} &= \vec{v}_j^{(i-1)} - \frac{\langle \vec{v}_i, \vec{v}_j^{(i-1)} \rangle}{\langle \vec{v}_i, \vec{v}_i \rangle}\vec{v}_i, \qquad i = 1, \dots, j-1,
\end{aligned} \tag{5.24}$$

In terms of the normalized $\vec{q}_i$, this gives the following algorithm. It differs from Algorithm 7 only in line 4.

Algorithm 8 Modified Gram–Schmidt (MGS).

```
1: for j = 1, ..., n do
2:     v_j ← a_j
3:     for i = 1, ..., j − 1 do
4:         r_ij ← ⟨q_i, v_j⟩                ▷ inner product with the updated vector
5:         v_j ← v_j − r_ij q_i
6:     r_jj ← ‖v_j‖_2
7:     q_j ← v_j / r_jj
```

Same in exact arithmetic. Expanding (5.22),

$$\left\langle \vec{q}_i, \vec{v}_j^{(i-1)} \right\rangle = \langle \vec{q}_i, \vec{a}_j \rangle - \sum_{k=1}^{i-1} r_{kj} \langle \vec{q}_i, \vec{q}_k \rangle, \tag{5.25}$$

and the sum vanishes when the $\vec{q}$'s are orthonormal, so CGS and MGS compute the same r_{ij}. The point is that CGS *relies* on $\langle \vec{q}_i, \vec{q}_k \rangle = 0$, which holds only approximately in floating point; as soon as cancellation occurs, this small error becomes large. MGS does not rely on it: each r_{ij} is the $\vec{q}_i$-component of what is actually left, so errors made in earlier steps are projected out by later ones.

Example 25 (MGS on the Same Matrix)**.** For the matrix of Example 24, $\vec{q}_1$ and $\vec{q}_2$ are computed exactly as in CGS (with one previous vector there is nothing to modify). Step 3 now projects in two stages:

$$\vec{v}_3^{(1)} = \vec{a}_3 - r_{13}\vec{q}_1 = \begin{bmatrix}1\\0\\0\\\varepsilon\end{bmatrix} - \begin{bmatrix}1\\\varepsilon\\0\\0\end{bmatrix} = \begin{bmatrix}0\\-\varepsilon\\0\\\varepsilon\end{bmatrix}. \tag{5.26}$$

Now $r_{23} = \left\langle \vec{q}_2, \vec{v}_3^{(1)} \right\rangle = \varepsilon/\sqrt{2}$ is no longer zero, and

$$\vec{v}_3^{(2)} = \vec{v}_3^{(1)} - r_{23}\vec{q}_2 = \begin{bmatrix}0\\-\varepsilon\\0\\\varepsilon\end{bmatrix} - \frac{\varepsilon}{\sqrt{2}}\begin{bmatrix}0\\-1/\sqrt{2}\\1/\sqrt{2}\\0\end{bmatrix} = \begin{bmatrix}0\\-\varepsilon/2\\-\varepsilon/2\\\varepsilon\end{bmatrix}, \qquad \vec{q}_3 = \frac{\vec{v}_3^{(2)}}{\varepsilon\sqrt{3/2}} = \begin{bmatrix}0\\-1/\sqrt{6}\\-1/\sqrt{6}\\\sqrt{2/3}\end{bmatrix}. \tag{5.27}$$

The result is

$$Q = \begin{bmatrix}1 & 0 & 0\\ \varepsilon & -1/\sqrt{2} & -1/\sqrt{6}\\ 0 & 1/\sqrt{2} & -1/\sqrt{6}\\ 0 & 0 & \sqrt{2/3}\end{bmatrix}. \tag{5.28}$$

Now $\langle \vec{q}_2, \vec{q}_3\rangle = \frac{1}{\sqrt{12}} - \frac{1}{\sqrt{12}} = 0$, and the remaining inner products $\langle \vec{q}_1, \vec{q}_2\rangle = -\varepsilon/\sqrt{2}$ and $\langle \vec{q}_1, \vec{q}_3\rangle = -\varepsilon/\sqrt{6}$ are $O(\varepsilon)$: the columns are *nearly orthogonal*.

5.8 (SUPPLEMENT) Further Topics

5.8.1 The Projector View

Trefethen and Bau [1, Lecture 8] write both algorithms with projectors, which shows the difference in one line.

Two kinds of projectors. Let $Q_{j-1} = \begin{bmatrix}\vec{q}_1 & \cdots & \vec{q}_{j-1}\end{bmatrix}$.

- $P_i\vec{x} := \vec{x} - \langle \vec{q}_i, \vec{x}\rangle\, \vec{q}_i$ removes the $\vec{q}_i$-component of $\vec{x}$: *one* direction.
- $(I - Q_{j-1}Q_{j-1}^*)\vec{x} = \vec{x} - \sum_{i<j} \langle \vec{q}_i, \vec{x}\rangle\, \vec{q}_i$ removes the components along $\vec{q}_1, \ldots, \vec{q}_{j-1}$: *all* of them at once.

Then the two algorithms compute

$$\text{CGS:}\quad \vec{v}_j = (I - Q_{j-1}Q_{j-1}^*)\,\vec{a}_j, \qquad \text{all directions at once, using } \vec{a}_j, \tag{5.29}$$

$$\text{MGS:}\quad \vec{v}_j = P_{j-1}\cdots P_2P_1\,\vec{a}_j, \qquad \text{one direction at a time, using the current vector.} \tag{5.30}$$

If the $\vec{q}_i$ are exactly orthonormal, both give the same result.

Where they differ. Take $j = 3$ and apply P_1, then P_2:

$$P_1\vec{a}_3 = \vec{a}_3 - \langle \vec{q}_1, \vec{a}_3\rangle\, \vec{q}_1, \tag{5.31}$$

$$P_2P_1\vec{a}_3 = \vec{a}_3 - \langle \vec{q}_1, \vec{a}_3\rangle\, \vec{q}_1 - \langle \vec{q}_2, \vec{a}_3 - \langle \vec{q}_1, \vec{a}_3\rangle\, \vec{q}_1\rangle\, \vec{q}_2 \tag{5.32}$$

$$= \underbrace{\vec{a}_3 - \langle \vec{q}_1, \vec{a}_3\rangle\, \vec{q}_1 - \langle \vec{q}_2, \vec{a}_3\rangle\, \vec{q}_2}_{\text{CGS}} + \langle \vec{q}_2, \vec{q}_1\rangle\, \langle \vec{q}_1, \vec{a}_3\rangle\, \vec{q}_2. \tag{5.33}$$

So MGS equals CGS plus one extra term. That term is zero when $\vec{q}_1 \perp \vec{q}_2$. When rounding has made $\vec{q}_1$ and $\vec{q}_2$ slightly non-orthogonal, the extra term is exactly the correction that CGS misses.

Check on the example. In Example 24, $\langle \vec{q}_2, \vec{q}_1 \rangle = -\varepsilon/\sqrt{2}$ and $\langle \vec{q}_1, \vec{a}_3 \rangle = 1$, so the MGS vector should be the CGS vector plus $-\frac{\varepsilon}{\sqrt{2}}\vec{q}_2$:

$$\underbrace{\begin{bmatrix} 0 \\ -\varepsilon \\ 0 \\ \varepsilon \end{bmatrix}}_{\text{CGS}} - \frac{\varepsilon}{\sqrt{2}} \begin{bmatrix} 0 \\ -1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{bmatrix} = \underbrace{\begin{bmatrix} 0 \\ -\varepsilon/2 \\ -\varepsilon/2 \\ \varepsilon \end{bmatrix}}_{\text{MGS}}, \tag{5.34}$$

which matches Example 25.

5.8.2 How Much Orthogonality Is Lost?

The quantity $\|I - Q^*Q\|$ measures how far the computed Q is from having orthonormal columns. A result of Björck states:

- MGS: $\|I - Q^*Q\| \approx \varepsilon_{\text{mach}}\, \kappa(A)$;
- CGS: the loss can be as bad as $\varepsilon_{\text{mach}}\, \kappa(A)^2$.

The matrix of Example 24 has $\kappa(A) \approx \sqrt{3}/\varepsilon$. With $\varepsilon = 10^{-10}$, $\varepsilon_{\text{mach}}\kappa(A)^2 \approx 7 \cdot 10^4 \gg 1$, so CGS may lose orthogonality completely, while for MGS $\varepsilon_{\text{mach}}\kappa(A) \approx 4 \cdot 10^{-6}$. In Julia we measured $\|I - Q^*Q\| \approx 0.71$ for CGS (with $\langle \vec{q}_2, \vec{q}_3 \rangle = 0.5$) and $\approx 1.2 \cdot 10^{-10}$ for MGS.

5.8.3 Why Not Solve the Normal Equations?

Solving $A^*A\vec{x} = A^*\vec{b}$ looks most direct, but forming A^*A squares the condition number, $\kappa(A^*A) = \kappa(A)^2$ [1, Lecture 11]. For the matrix of Example 24,

$$A^*A = \begin{bmatrix} 1+\varepsilon^2 & 1 & 1 \\ 1 & 1+\varepsilon^2 & 1 \\ 1 & 1 & 1+\varepsilon^2 \end{bmatrix}, \tag{5.35}$$

and in floating point $1 + \varepsilon^2 = 1$. So the computed A^*A is the all-ones matrix, which has rank 1: the information is lost before we even start solving. QR works on A directly and avoids this [2, §3.3].

5.8.4 Householder QR and Reorthogonalization

Even with MGS, how orthogonal Q is depends on $\kappa(A)$. Libraries therefore use *Householder QR* (Lecture 6), whose Q is orthogonal to working precision whatever $\kappa(A)$ is. Another remedy is to *run CGS twice* ("twice is enough"); it parallelizes better than MGS, since the inner products of CGS form one matrix–vector product $Q_{j-1}^*\vec{a}_j$.

Lecture 6

Product View on Error Bounds, Householder QR

Supplementary reading:

- [1] Lectures 8, 10, and 16.
- [2] Section 3.4.

6.1 Why Write Algorithms as Matrix Products?

So far we described LU (Lecture 3) and Gram–Schmidt (Lecture 5) step by step, one row or one vector at a time, and we analyzed the error of LU entry by entry (Lecture 4). This has two drawbacks:

- the analysis has to be redone for every new algorithm;
- it does not show *why* some algorithms are stable and others are not.

In this lecture we write every factorization algorithm as a *product of matrices*: each step multiplies by one matrix. Then we only need to look at that matrix. It turns out that stability depends on one number: the condition number of the matrix used at each step.

6.2 The Product Form of Gram–Schmidt

The analogy with LU. LU *left*-multiplies by a sequence of lower triangular matrices in order to reach an *upper triangular* matrix:

$$L_{m-1} \cdots L_1 A = U. \tag{6.1}$$

Each L_k performs row operations that zero column k below the diagonal. Gram–Schmidt does the opposite: it *right*-multiplies by a sequence of upper triangular matrices in order to reach *orthonormal columns*:

$$A R_1 R_2 \cdots R_n = \hat{Q}. \tag{6.2}$$

Right multiplication performs column operations, and column operations are exactly what Gram–Schmidt does: normalize one column, then subtract its component from the other columns.

The step matrices. We first fix the notation.

- $\vec{a}_j^{(k-1)}$ is the j-th column after $k-1$ steps, i.e., $\vec{a}_j$ with its components along $\vec{q}_1, \dots, \vec{q}_{k-1}$ removed.
- $\vec{a}_j^{(0)} = \vec{a}_j$.

At the start of step k the matrix is

$$\begin{bmatrix} \vec{q}_1 & \cdots & \vec{q}_{k-1} & \vec{a}_k^{(k-1)} & \cdots & \vec{a}_n^{(k-1)} \end{bmatrix}. \tag{6.3}$$

- The first $k-1$ columns are orthonormal.
- The remaining columns are orthogonal to them, but not yet to each other.

Step k uses the numbers

$$r_{kk} \coloneqq \left\| \vec{a}_k^{(k-1)} \right\|, \qquad \vec{q}_k \coloneqq \vec{a}_k^{(k-1)} / r_{kk}, \qquad r_{kj} \coloneqq \left\langle \vec{q}_k, \vec{a}_j^{(k-1)} \right\rangle \quad (j > k). \tag{6.4}$$

The step matrix R_k equals the identity except in row k:

$$R_k = \begin{bmatrix} 1 & & & & & & \\ & \ddots & & & & & \\ & & \frac{1}{r_{kk}} & -\frac{r_{k,k+1}}{r_{kk}} & \cdots & -\frac{r_{kn}}{r_{kk}} & \\ & & & 1 & & & \\ & & & & \ddots & & \\ & & & & & 1 & \end{bmatrix}. \tag{6.5}$$

Right multiplication by R_k performs step k:

$$\begin{bmatrix} \vec{q}_1 & \cdots & \vec{q}_{k-1} & \vec{a}_k^{(k-1)} & \cdots & \vec{a}_n^{(k-1)} \end{bmatrix} R_k = \begin{bmatrix} \vec{q}_1 & \cdots & \vec{q}_k & \vec{a}_{k+1}^{(k)} & \cdots & \vec{a}_n^{(k)} \end{bmatrix}. \tag{6.6}$$

To check this, look at the columns of the product.

- Columns $1, \dots, k-1$ are unchanged.
- Column k is
$$\vec{a}_k^{(k-1)} / r_{kk} = \vec{q}_k.$$
- Column $j > k$ is
$$\vec{a}_j^{(k-1)} - \frac{r_{kj}}{r_{kk}} \vec{a}_k^{(k-1)} = \vec{a}_j^{(k-1)} - r_{kj} \vec{q}_k = \vec{a}_j^{(k)}.$$

Note the factor $1/r_{kk}$ in the off-diagonal entries. It is there because the coefficient multiplies the *unnormalized* column $\vec{a}_k^{(k-1)}$, not $\vec{q}_k$. For example, for $k = 1$,

$$(R_1)_{1j} = -\frac{\langle \vec{q}_1, \vec{a}_j \rangle}{\|\vec{a}_1\|} = -\frac{\langle \vec{a}_1, \vec{a}_j \rangle}{\|\vec{a}_1\|^2}. \tag{6.7}$$

The product form is modified Gram–Schmidt. CGS and MGS differ only in which column the coefficient r_{kj} uses:

- CGS uses the original column: $r_{kj} = \langle \vec{q}_k, \vec{a}_j \rangle$.
- MGS uses the updated column: $r_{kj} = \left\langle \vec{q}_k, \vec{a}_j^{(k-1)} \right\rangle$.

The product form can only be MGS. R_k multiplies the *current* matrix, whose j-th column is already $\vec{a}_j^{(k-1)}$; the original $\vec{a}_j$ has been overwritten. So each step computes $\vec{q}_k$ and right away removes it from all later columns, which is exactly MGS.

The reduced QR factorization. Each R_k is upper triangular, and so is their product:

$$R_1 R_2 \cdots R_n = \hat{R}^{-1} \quad \Longrightarrow \quad A = \hat{Q}\hat{R}. \tag{6.8}$$

Here $\hat{Q} \in \mathbb{K}^{m \times n}$ is the factor of the *reduced QR factorization*: it has only n orthonormal columns, $\hat{Q}^* \hat{Q} = I_n$, but $\hat{Q}\hat{Q}^* \neq I_m$ (it is the orthogonal projector onto $\mathcal{R}(A)$). The factor $\hat{R} \in \mathbb{K}^{n \times n}$ is upper triangular and invertible.

Like R_k, the inverse R_k^{-1} differs from the identity only in row k, and that row has no reciprocals and no minus signs:

$$R_k^{-1} = \begin{bmatrix} 1 & & & & & & \\ & \ddots & & & & & \\ & & r_{kk} & r_{k,k+1} & \cdots & r_{kn} & \\ & & & 1 & & & \\ & & & & \ddots & & \\ & & & & & 1 & \end{bmatrix}. \tag{6.9}$$

The same happens in LU, where L_k^{-1} is L_k with the minus signs removed.

Consequently

$$\hat{R} = R_n^{-1} \cdots R_1^{-1} = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ & r_{22} & \cdots & r_{2n} \\ & & \ddots & \vdots \\ & & & r_{nn} \end{bmatrix} \tag{6.10}$$

That is, row k of $\hat{R}$ is simply row k of R_k^{-1}. LU works the same way: in $L = L_1^{-1} \cdots L_{m-1}^{-1}$, column k of L is simply column k of L_k^{-1}, i.e., the multipliers of step k (Theorem 12).

6.3 Error Amplification in Products

Consider algorithms of the form

$$A_k = X_k A_{k-1}, \qquad A_0 = A. \tag{6.11}$$

LU ($X_k = L_k$) and Householder QR (X_k orthogonal, below) are of this type; Gram–Schmidt multiplies from the right and is analyzed symmetrically. Every step incurs rounding errors. How stable is such an algorithm?

Forward error of one step. One can show that in floating-point arithmetic

$$\mathrm{fl}(XA) = XA + E + O(\varepsilon_{\mathrm{mach}}^2), \qquad \text{for } \|E\|_2 \le O(\varepsilon_{\mathrm{mach}})\,\|X\|_2\,\|A\|_2\,. \tag{6.12}$$

Note that E is measured against $\|X\|\,\|A\|$, not $\|XA\|$. If XA is small because of cancellation, E can be large relative to the result. This is the same as in LU, where the error is governed by $|L|\,|U|$ rather than $|A|$.

The $O(\varepsilon_{\mathrm{mach}})$ hides a dimension-dependent constant, and $O(\varepsilon_{\mathrm{mach}}^2)$ means we only keep first-order terms.

Rewriting as a backward error. Thus, with $F := X^{-1}E$,

$$\mathrm{fl}(XA) = X(A+F) + O(\varepsilon_{\mathrm{mach}}^2). \tag{6.13}$$

So the computed result is the *exact* product with a perturbed input $A + F$.

The perturbation satisfies

$$\|F\|_2 = \|X^{-1}E\|_2 \le \|X^{-1}\|_2\|E\|_2 \le O(\varepsilon_{\mathrm{mach}})\,\kappa(X)\,\|A\|_2. \tag{6.14}$$

Dividing by $\|A\|_2$ gives the relative backward error:

$$\frac{\|F\|_2}{\|A\|_2} \le O(\varepsilon_{\mathrm{mach}})\,\kappa(X).$$

Theorem 26 (The Transformation Determines the Backward Error)

For one step $A \mapsto XA$ of a product-form algorithm with invertible X, the computed result equals $X(A+F)$ with

$$\frac{\|F\|_2}{\|A\|_2} \le O(\varepsilon_{\mathrm{mach}})\,\kappa(X). \tag{6.15}$$

Two condition numbers must be kept apart.

- $\kappa(X)$ belongs to the *algorithm*. It determines the backward error, i.e., whether the algorithm is stable, and the algorithm designer controls it.
- $\kappa(A)$ belongs to the *problem*. It is fixed by the input and turns backward error into forward error: forward error $\lesssim \kappa(A)\times$ backward error.

Design principle: choose transformations with small condition numbers. Orthogonal matrices have $\kappa = 1$, the best possible.

6.4 Lower Triangularization: The Case of LU

Apply the framework to LU. After k steps of elimination the accumulated transformation is $X = L_k \cdots L_1$, and its inverse is the unit lower triangular matrix of multipliers,

$$L = X^{-1} = \begin{bmatrix} 1 & & & \\ \ell_{21} & 1 & & \\ \vdots & \ddots & \ddots & \\ \ell_{m1} & \cdots & \ell_{m,m-1} & 1 \end{bmatrix}. \tag{6.16}$$

Since $\kappa(X) = \kappa(X^{-1}) = \kappa(L)$, Theorem 26 gives a backward error of about $O(\varepsilon_{\text{mach}})\kappa(L)\,\|A\|_2$. The stability of LU thus comes down to the condition number of L. Partial pivoting chooses the largest entry of each column as pivot and so guarantees $|\ell_{ij}| \le 1$.

$\|L\|$ is under control. Since $|\ell_{ij}| \le 1$,

$$\|L\|_2 \le \|L\|_F \le \sqrt{\frac{m(m+1)}{2}} \le m. \tag{6.17}$$

The middle step holds because L has m ones on the diagonal and $m(m-1)/2$ entries of absolute value at most 1 below it, so $\|L\|_F^2 \le m(m+1)/2$. Here $\|\cdot\|_F$ is the Frobenius norm (Section B.4).

The bound is loose. What matters is that it is polynomial in m, not exponential.

$\|L^{-1}\|$ is not. What about $\|L^{-1}\|_2$? To control $\kappa(L) = \|L\|_2 \|L^{-1}\|_2$, we also need $\|L^{-1}\|$ to be moderate. But $|\ell_{ij}| \le 1$ does *not* guarantee this.

The reason is forward substitution: each row uses the results of all previous rows. So values pile up, and the entries of L^{-1} can grow exponentially.

Example 27 (Exponential Growth of L^{-1})**.** The factor L of Example 20 has -1 everywhere below the diagonal, so $|\ell_{ij}| \le 1$, yet

$$L = \begin{bmatrix} 1 & & & \\ -1 & 1 & & \\ -1 & -1 & 1 & \\ -1 & -1 & -1 & 1 \end{bmatrix}, \qquad L^{-1} = \begin{bmatrix} 1 & & & \\ 1 & 1 & & \\ 2 & 1 & 1 & \\ 4 & 2 & 1 & 1 \end{bmatrix}. \tag{6.18}$$

For general m, $(L^{-1})_{ij} = 2^{i-j-1}$ for $i > j$, and the bottom-left entry is 2^{m-2}, so

$$\left\|L^{-1}\right\|_2 \ge 2^{m-2}, \qquad \kappa(L) \ge 2^{m-2}. \tag{6.19}$$

To see where the doubling comes from, note that computing L^{-1} means solving $L\vec{x} = \vec{b}$. By forward substitution, the i-th equation $x_i - x_1 - \cdots - x_{i-1} = b_i$ gives

$$x_i = b_i + (x_1 + \cdots + x_{i-1}). \tag{6.20}$$

For the first column, $\vec{b} = \vec{e}_1$: $x_1 = 1$, $x_2 = 1$, $x_3 = 1 + 1 = 2$, $x_4 = 1 + 1 + 2 = 4$, and so on. Each entry is the sum of all previous ones, so from the second entry on they double, $x_i = 2^{i-2}$. The other columns are the same pattern shifted down.

Large L^{-1} causes large U. From $A = LU$,

$$U = L^{-1}A, \tag{6.21}$$

so a large $\|L^{-1}\|$ can produce a large U: *large U comes from large L^{-1}*. This connects the condition-number viewpoint with the growth factor of Lectures 4 and 5; they describe the same thing. In Example 20 the last column of A is all ones, so the last column of U is L^{-1} times the all-ones vector, i.e., the row sums of L^{-1}:

$$1 + \sum_{j=1}^{i-1} 2^{i-j-1} = 1 + (2^{i-1} - 1) = 2^{i-1}, \tag{6.22}$$

which is exactly the last column $1, 2, 4, 8, \dots$ of U.

Partial pivoting thus solves only half of the problem: it controls $\|L\|$ but not $\|L^{-1}\|$, and in the worst case LU with partial pivoting can still be unstable. Such matrices are extremely rare, so in practice it is used as a stable algorithm; but to *guarantee* well-conditioned transformations we must switch to orthogonal ones.

6.5 Gram–Schmidt Is Also a Lower Triangularization

The upper triangular R_k is nontrivial only in row k, so its transpose $L_k^\top := R_k$ has L_k nontrivial only in column k: an elementary lower triangular matrix (with diagonal entry $1/r_{kk}$ instead of 1). Then $AR_1 \cdots R_n = \hat{Q}$ reads $AL_1^\top \cdots L_n^\top = \hat{Q}$, and transposing,

$$L_n \cdots L_1 A^\top = \hat{Q}^\top. \tag{6.23}$$

Gram–Schmidt on A is therefore the same as left-multiplying $A^\top$ by a sequence of lower triangular matrices. It too falls into the error-amplification framework, and its stability is likewise governed by the condition numbers of these triangular factors.

6.6 Householder Triangularization

Idea: multiply by orthogonal matrices. LU and Gram–Schmidt have the same defect: each step applies a triangular matrix, whose condition number is not under control. Following the design principle of Section 6.3, we make every step orthogonal instead.

We are looking for unitary matrices Q_k such that each step zeros one column below the diagonal:

$$\begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{Q_1} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{Q_2} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \\ 0 & 0 & \times \end{bmatrix} \xrightarrow{Q_3} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}. \tag{6.24}$$

After n steps,

$$Q_n \cdots Q_1 A = R \quad \Longleftrightarrow \quad A = \underbrace{(Q_n \cdots Q_1)^{-1}}_{Q} R = Q_1^* \cdots Q_n^* R. \tag{6.25}$$

This is the *full QR factorization*: $Q \in \mathbb{K}^{m\times m}$ is a square unitary matrix, and $R \in \mathbb{K}^{m\times n}$ is upper triangular, with $m - n$ zero rows at the bottom. Taking the first n columns of Q and the first n rows of R recovers the reduced QR factorization.

Compare with the two earlier algorithms:

- like LU, Householder multiplies from the *left* and aims for an upper triangular matrix, but with unitary instead of triangular matrices;
- Gram–Schmidt uses triangular matrices to *produce* an orthogonal matrix; Householder uses orthogonal matrices to *produce* a triangular one.

How do we choose the Q_k?

The Householder reflector. Let us begin with the first step. Q_1 must map the first column $\vec{a}_1$ to a multiple of $\vec{e}_1$. Unitary maps preserve the 2-norm, so there are only two possible targets:

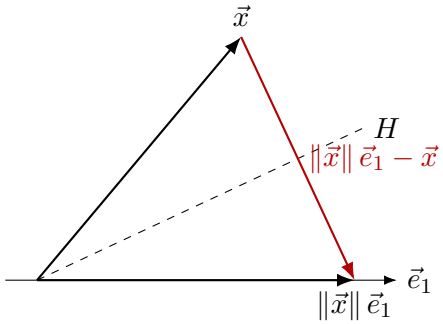
$$Q_1 \vec{a}_1 = \pm \|\vec{a}_1\| \, \vec{e}_1. \tag{6.26}$$

Take the $+$ sign first.

Idea: use a reflection. Since $\vec{a}_1$ and $\|\vec{a}_1\| \, \vec{e}_1$ have the same length, they are mirror images of each other. The mirror is the hyperplane through the origin with normal vector

$$\vec{v} \coloneqq \vec{a}_1 - \|\vec{a}_1\| \, \vec{e}_1. \tag{6.27}$$

Reflecting across it sends $\vec{a}_1$ to $\|\vec{a}_1\| \, \vec{e}_1$.



Definition 28 (Householder Reflector)

For a nonzero $\vec{v} \in \mathbb{K}^m$, the *Householder reflector* is

$$F = I - 2\frac{\vec{v}\vec{v}^*}{\vec{v}^*\vec{v}}. \tag{6.28}$$

F flips the component along $\vec{v}$ and keeps the rest:

- $\vec{u} \perp \vec{v} \implies F\vec{u} = \vec{u}$: vectors in the hyperplane stay fixed;
- $\vec{u} \parallel \vec{v} \implies F\vec{u} = -\vec{u}$: the normal direction is flipped.

F is Hermitian, and reflecting twice changes nothing, so $F^2 = I$. Hence $F^*F = F^2 = I$: F is unitary and is its own inverse.

We now check that $Q_1 = I - 2\vec{v}\vec{v}^*/(\vec{v}^*\vec{v})$ does what we want (real case). First,

$$\vec{v}^*\vec{v} = 2\|\vec{a}_1\|^2 - 2\|\vec{a}_1\| a_{11}, \qquad \vec{v}^*\vec{a}_1 = \|\vec{a}_1\|^2 - \|\vec{a}_1\| a_{11} = \tfrac{1}{2}\vec{v}^*\vec{v}, \tag{6.29}$$

so

$$Q_1\vec{a}_1 = \vec{a}_1 - 2\vec{v}\frac{\vec{v}^*\vec{a}_1}{\vec{v}^*\vec{v}} = \vec{a}_1 - \vec{v} = \|\vec{a}_1\|\vec{e}_1. \tag{6.30}$$

Choosing the sign. There are two possible reflections, onto $+\|\vec{a}_1\|\vec{e}_1$ and onto $-\|\vec{a}_1\|\vec{e}_1$. Which one should we choose? Mathematically, there is no preference. In finite precision, we choose the target *farther from* $\vec{a}_1$:

$$Q_1\vec{a}_1 = -\operatorname{sign}(a_{11})\|\vec{a}_1\|\vec{e}_1 \quad \Longrightarrow \quad \vec{v} \propto \operatorname{sign}(a_{11})\|\vec{a}_1\|\vec{e}_1 + \vec{a}_1. \tag{6.31}$$

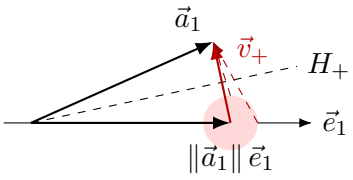
We take $\operatorname{sign}(0) = 1$.

Why? Suppose $\vec{a}_1$ is nearly parallel to $\vec{e}_1$, with $a_{11} > 0$. With the $+$ sign,

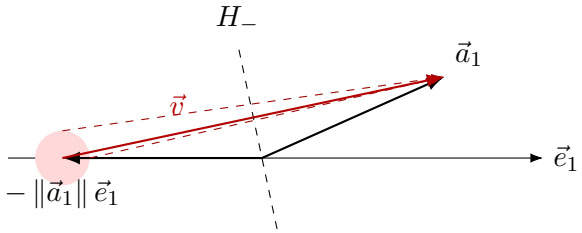
$$\|\vec{v}_+\| = \|\|\vec{a}_1\|\vec{e}_1 - \vec{a}_1\| \ll \|\vec{a}_1\|. \tag{6.32}$$

So $\vec{v}_+$ is the difference of two nearly equal vectors, and its first entry $\|\vec{a}_1\| - a_{11}$ suffers cancellation. The computed $\|\vec{a}_1\|$ has an absolute error of about $\varepsilon_{\text{mach}}\|\vec{a}_1\|$, which is large compared with the short $\vec{v}_+$. So the direction of $\vec{v}_+$, and with it the reflection, is inaccurate.

With $-\operatorname{sign}(a_{11})$, the first entry of $\vec{v}$ is $a_{11} + \operatorname{sign}(a_{11})\|\vec{a}_1\|$, a sum of two numbers of the same sign. There is no cancellation, and $|v_1| \geq \|\vec{a}_1\|$, so $\vec{v}$ is never short.



Reflecting onto $+\|\vec{a}_1\|\vec{e}_1$: $\vec{v}_+$ is short, so its direction is unreliable.



Reflecting onto $-\|\vec{a}_1\|\vec{e}_1$: $\vec{v}$ is long, so its direction is reliable.

Step k: the block form. What about Q_k for $k > 1$? The first $k-1$ rows and columns are finished and must not change. So we use

$$Q_k = \begin{bmatrix} I_{k-1} & 0 \\ 0 & F_k \end{bmatrix}, \tag{6.33}$$

where F_k is the $(m-k+1)\times(m-k+1)$ Householder reflector built as in step 1 from $\vec{x} = A[k{:}m, k]$, the first column of the trailing submatrix $A[k{:}m, k{:}n]$.

Earlier results are preserved:

- the block I_{k-1} leaves the first $k-1$ rows untouched;
- the first $k-1$ columns are already zero from row k down, and F_k maps zero to zero.

Q_k is itself a Householder reflector (with $\vec{v}$ padded by $k-1$ leading zeros), so it is unitary.

Algorithm 9 Householder QR, in place.

for $k = 1, \dots, n$ **do**
  $\vec{x} \leftarrow A[k{:}m, k]$
  $\vec{v} \leftarrow \operatorname{sign}(x_1) \|\vec{x}\|_2 \vec{e}_1 + \vec{x}$
  $\vec{v} \leftarrow \vec{v}/v_1$ ▷ scale so that $v_1 = 1$
  $A[k{:}m, k{:}n] \leftarrow A[k{:}m, k{:}n] - 2\vec{v}\left(\vec{v}^* A[k{:}m, k{:}n]\right)/(\vec{v}^*\vec{v})$
  $A[k{+}1{:}m, k] \leftarrow \vec{v}[2{:}\text{end}]$ ▷ store $\vec{v}$ below the diagonal
return A ▷ upper triangle: R; strictly lower triangle: the vectors $\vec{v}$

Some remarks on the implementation (see also [1, Lecture 10]):

- *F_k is never formed.* Following the parentheses, first compute the row vector $\vec{v}^* A[k{:}m, k{:}n]$, then perform a rank-one update. Each step costs $O((m-k)(n-k))$; forming $\vec{v}\vec{v}^*$ first would cost an order of magnitude more.
- *Scaling $\vec{v}$ by $1/v_1$* makes its first entry 1. The reflector depends only on the direction of $\vec{v}$, so scaling does not change F_k. Since $|v_1| \geq \|\vec{x}\| > 0$ (for $\vec{x} \neq \vec{0}$), this never divides by zero.
- *Storage.* After the update, $A[k{+}1{:}m, k]$ should be zero, which frees exactly enough room for the remaining $m-k$ entries of $\vec{v}$; the leading $v_1 = 1$ is implicit.
- On exit the upper triangle of A is R, with diagonal entries $r_{kk} = -\operatorname{sign}(x_1)\|\vec{x}\|$. The matrix Q is never formed explicitly; it is stored implicitly through the vectors $\vec{v}$ (see Lecture 7).
- $\operatorname{sign}(0)$ must be taken as 1. Otherwise $x_1 = 0$ gives $\vec{v} = \vec{x}$, and the reflection maps $\vec{x}$ to $-\vec{x}$ instead of a multiple of $\vec{e}_1$.

Back into the framework. Each $X_k = Q_k$ is unitary, so $\|Q_k\|_2 = \left\|Q_k^{-1}\right\|_2 = 1$ and $\kappa(Q_k) = 1$ (Appendix B). By Theorem 26,

$$\|F\|_2 \leq O(\varepsilon_{\text{mach}})\,\kappa(Q_k)\,\|A\|_2 = O(\varepsilon_{\text{mach}})\,\|A\|_2\,. \tag{6.34}$$

- The transformation does not amplify the error at all.
- Unitary matrices preserve the 2-norm, so every intermediate A_k has norm $\|A\|_2$. There is no element growth as in LU.
- Over all steps, the backward error grows only linearly with the number of steps (Section 6.7). So Householder QR is backward stable.

This presumes that each Q_k is computed accurately, i.e., that the direction of $\vec{v}$ is accurate. This is exactly what the sign choice ensures.

6.7 (SUPPLEMENT) Further Topics

6.7.1 Three Factorizations as Products

	Side	Transformation	Target	Condition of transformation
LU	left	lower triangular L_k	upper triangular U	can be large
Gram–Schmidt	right	upper triangular R_k	orthonormal $\hat{Q}$	depends on A
Householder	left	unitary Q_k	upper triangular R	1

Trefethen and Bau call Gram–Schmidt *triangular orthogonalization* and Householder *orthogonal triangularization* [1, Lectures 8 and 10].

6.7.2 The Condition of the Gram–Schmidt Transformation

The accumulated transformation of Gram–Schmidt is

$$R_1 \cdots R_n = \hat{R}^{-1}, \qquad \text{so} \qquad \kappa(R_1 \cdots R_n) = \kappa(\hat{R}).$$

This equals $\kappa(A)$. Since $A = \hat{Q}\hat{R}$ and $\hat{Q}$ has orthonormal columns,

$$\|A\vec{x}\|_2 = \left\|\hat{Q}\hat{R}\vec{x}\right\|_2 = \left\|\hat{R}\vec{x}\right\|_2 \quad \text{for all } \vec{x},$$

so A and $\hat{R}$ stretch every vector by the same factor. Their largest and smallest stretches agree, and hence $\kappa(\hat{R}) = \kappa(A)$.

So the triangular transformations of Gram–Schmidt are exactly as ill-conditioned as the problem: the more ill-conditioned A, the more the error is amplified. This matches the loss of orthogonality $\approx \varepsilon_{\text{mach}}\kappa(A)$ of MGS in Lecture 5.

6.7.3 Backward Error Over Many Steps

Theorem 26 covers one step. For many steps, write step k as $\tilde{A}_k = X_k(\tilde{A}_{k-1} + F_k)$. After two steps,

$$\tilde{A}_2 = X_2(X_1(A + F_1) + F_2) = X_2X_1(A + F_1 + X_1^{-1}F_2).$$

Continuing in the same way, every perturbation is pulled back to the original input:

$$\tilde{A}_n = X_n \cdots X_1 \left(A + \sum_{k=1}^{n} (X_{k-1} \cdots X_1)^{-1} F_k \right) + O(\varepsilon_{\text{mach}}^2). \tag{6.35}$$

For general X_k, the factors $(X_{k-1} \cdots X_1)^{-1}$ can amplify the F_k. When every X_k is unitary:

- $(X_{k-1} \cdots X_1)^{-1}$ is unitary, so it does not change the norm of F_k;
- $\left\|\tilde{A}_{k-1}\right\|_2 = \|A\|_2$ to first order, so $\|F_k\|_2 \le O(\varepsilon_{\text{mach}}) \|A\|_2$.

Adding up the n terms,

$$\|F_{\text{total}}\|_2 \lesssim O(n\varepsilon_{\text{mach}}) \|A\|_2 . \tag{6.36}$$

This is the argument behind Demmel's Lemma 3.1 and Theorem 3.5 [2, §3.4.3]. Trefethen and Bau state the result as $\tilde{Q}\tilde{R} = A + \delta A$ with $\|\delta A\| / \|A\| = O(\varepsilon_{\text{mach}})$ [1, Theorem 16.1], where $\tilde{Q}$ is the *exactly* unitary product of the reflectors defined by the computed vectors $\tilde{\vec{v}}_k$.

6.7.4 Orthogonality of Q

We measure the loss of orthogonality by $\|Q^*Q - I\|_2$.

- MGS computes $\hat{Q}$ by triangular column operations on A. Their condition is $\kappa(A)$, so $\left\|\hat{Q}^*\hat{Q} - I\right\|_2 \approx \varepsilon_{\text{mach}}\kappa(A)$.
- Householder never builds Q from A. Q is a product of reflectors, each unitary up to rounding in $\vec{v}_k$, so $\|Q^*Q - I\|_2 = O(\varepsilon_{\text{mach}})$, independently of $\kappa(A)$.

When an orthonormal basis of $\mathcal{R}(A)$ is needed explicitly, Householder is the safer choice.

6.7.5 Operation Counts

Step k of Algorithm 9 updates the $(m-k+1)\times(n-k+1)$ block $A[k{:}m, k{:}n]$. For each column $\vec{a}$ of this block, the inner product $\vec{v}^*\vec{a}$ and the update $\vec{a} - (2\vec{v}^*\vec{a}/\vec{v}^*\vec{v})\,\vec{v}$ each cost about $2(m-k+1)$ flops. So step k costs about $4(m-k+1)(n-k+1)$ flops, and in total

$$\sum_{k=1}^{n} 4(m-k+1)(n-k+1) \approx 4\int_0^n (m-t)(n-t)\,\mathrm{d}t = 2mn^2 - \frac{2}{3}n^3. \tag{6.37}$$

- For $m \gg n$ this is about $2mn^2$, the same as MGS.
- For $m = n$ it is $\frac{4}{3}n^3$, twice as much as LU ($\frac{2}{3}n^3$). This is the price of stability.

6.7.6 Using Q Implicitly for Least Squares

To solve $\min_{\vec{x}} \left\|A\vec{x} - \vec{b}\right\|_2$ we never form Q. Instead we apply the stored reflectors to $\vec{b}$ one by one, exactly as in step 5 of Algorithm 9:

$$\vec{c} \coloneqq Q^*\vec{b} = Q_n \cdots Q_1\vec{b}, \qquad \text{cost } O(mn).$$

Split $\vec{c}$ into $\vec{c}_1 = \vec{c}[1{:}n]$ and $\vec{c}_2 = \vec{c}[n{+}1{:}m]$. Since Q^* preserves the 2-norm,

$$\left\|A\vec{x} - \vec{b}\right\|_2^2 = \|R\vec{x} - \vec{c}\|_2^2 = \left\|\hat{R}\vec{x} - \vec{c}_1\right\|_2^2 + \|\vec{c}_2\|_2^2\,.$$

The first term vanishes when $\hat{R}\vec{x} = \vec{c}_1$, which we solve by back substitution. The second term does not depend on $\vec{x}$, so $\|\vec{c}_2\|_2$ is the minimal residual. Details are in Lecture 7.

6.7.7 The Complex Case

For complex data the real sign does not work: with the target $+\|\vec{x}\|\,\vec{e}_1$, the number $\vec{v}^*\vec{x}$ is in general not real, and the verification of $F\vec{x} = \|\vec{x}\|\,\vec{e}_1$ fails. Instead we take

$$\operatorname{sign}(x_1) \coloneqq x_1/\,|x_1| \quad (\text{and } 1 \text{ if } x_1 = 0), \qquad \vec{v} = \operatorname{sign}(x_1)\,\|\vec{x}\|\,\vec{e}_1 + \vec{x}.$$

- The first entry $v_1 = \operatorname{sign}(x_1)(\,\|\vec{x}\| + |x_1|\,)$ adds two numbers with the same phase, so there is again no cancellation.
- Since $\overline{\operatorname{sign}(x_1)}\,x_1 = |x_1|$,

$$\vec{v}^*\vec{x} = \|\vec{x}\|^2 + \|\vec{x}\|\,|x_1|\,, \qquad \vec{v}^*\vec{v} = 2\,\|\vec{x}\|^2 + 2\,\|\vec{x}\|\,|x_1| = 2\,\vec{v}^*\vec{x}.$$

- Hence, as in the real case, $F\vec{x} = \vec{x} - \vec{v} = -\operatorname{sign}(x_1)\,\|\vec{x}\|\,\vec{e}_1$.

Lecture 7

Householder QR in Practice, Givens Rotations, Existence and Uniqueness of QR

Supplementary reading:

- [1] Lectures 7 and 10.
- [2] Sections 3.2.2 and 3.4.

7.1 Picking Up from Householder QR

At the end of Householder QR (Algorithm 9), the upper triangle of A holds R and the strictly lower triangle holds the Householder vectors $\vec{v}_k$; Q is never computed explicitly. This raises two questions:

- Applications need $Q^*\vec{b}$ (for least squares) or $Q\vec{x}$. How do we compute them from the $\vec{v}_k$ alone?
- Every Householder step processes a whole column. If A is already close to upper triangular and only a few entries need to be eliminated, is this wasteful?

The first leads to the implicit use of Q, the second to Givens rotations. Finally we return to theory: does a QR factorization always exist, and is it unique?

7.2 Using Q

Implicit storage. Householder QR does not store Q as a matrix, only the vectors $\vec{v}_k$ defining the reflectors

$$Q_k = I - 2\frac{\vec{v}_k\vec{v}_k^*}{\vec{v}_k^*\vec{v}_k}. \tag{7.1}$$

Here $\vec{v}_k$ is regarded as a vector of length m whose first $k-1$ entries are zero, so Q_k acts only on rows k through m; this is the same as the block form $Q_k = \mathrm{diag}(I_{k-1}, F_k)$ of Lecture 6. From $Q_n \cdots Q_1 A = R$, and since each Q_k is Hermitian and its own inverse ($Q_k^* = Q_k = Q_k^{-1}$),

$$Q = (Q_n \cdots Q_1)^{-1} = Q_1 Q_2 \cdots Q_n, \qquad Q^* = Q_n \cdots Q_1. \tag{7.2}$$

Two algorithms. A product of matrices acts on a vector from right to left. In $Q^*\vec{b} = Q_n \cdots Q_1 \vec{b}$, the factor Q_1 acts first, so the loop runs $k = 1, \ldots, n$. In $Q\vec{x} = Q_1 \cdots Q_n \vec{x}$, the factor Q_n acts first, so the loop runs $k = n, \ldots, 1$.

Algorithm 10 Implicit computation of $Q^*\vec{b}$.

1: **for** $k = 1, \ldots, n$ **do**
2: $\quad \vec{b}[k{:}m] \leftarrow \vec{b}[k{:}m] - 2\vec{v}_k \left(\vec{v}_k^* \vec{b}[k{:}m]\right) / (\vec{v}_k^* \vec{v}_k)$

Algorithm 11 Implicit computation of $Q\vec{x}$.

1: **for** $k = n, n-1, \ldots, 1$ **do**
2: $\quad \vec{x}[k{:}m] \leftarrow \vec{x}[k{:}m] - 2\vec{v}_k \left(\vec{v}_k^* \vec{x}[k{:}m]\right) / (\vec{v}_k^* \vec{v}_k)$

As in Lecture 6, we follow the parentheses: first compute the scalar $\vec{v}_k^* \vec{b}[k{:}m]$, then update the vector. Each step costs $O(m-k)$, and the total is about $4mn - 2n^2$ flops, the same order as a matrix–vector product. We never form $\vec{v}_k \vec{v}_k^*$.

Each step is the same in both algorithms; only the loop runs in opposite directions. This is just like forward and back substitution.

Consequences of implicit storage.

In place. We scale $\vec{v}_k$ so that its k-th entry is 1, so this entry need not be stored. Scaling does not change the reflector, which only depends on the direction of $\vec{v}_k$. The other $m-k$ entries fit exactly into column k below the diagonal, which was just set to zero. So the factorization simply overwrites A. For $m = 4$, $n = 3$, writing v_{jk} for entry j of $\vec{v}_k$:

$$A \;\to\; \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ v_{21} & r_{22} & r_{23} \\ v_{31} & v_{32} & r_{33} \\ v_{41} & v_{42} & v_{43} \end{bmatrix}. \tag{7.3}$$

A full QR factorization from only n vectors. An explicit Q takes m^2 numbers. The $\vec{v}_k$ take only about $mn - \frac{n^2}{2}$. This is a big saving when $m \gg n$. Applying Q is cheaper too: $4mn - 2n^2$ flops instead of $2m^2$ with an explicit Q. If we really need $\hat{Q}$ (the first n columns of Q), we compute $Q\vec{e}_1, \ldots, Q\vec{e}_n$ with Algorithm 11.

Q is orthogonal by definition. This point is important. The computed $\vec{v}_k$ have rounding errors. But $I - 2\vec{v}\vec{v}^*/(\vec{v}^*\vec{v})$ is an exact reflector for *every* nonzero $\vec{v}$, so Q is still exactly unitary. Rounding errors only change *which* unitary matrix we get. They show up as a small backward error in A: $Q\tilde{R} = A + \delta A$ with $\|\delta A\| \le O(\varepsilon_{\text{mach}})\,\|A\|$.

Gram–Schmidt is different. It computes the columns of $\hat{Q}$ explicitly, so rounding errors make them lose orthogonality: by about $\varepsilon_{\text{mach}}\kappa(A)$ for MGS and $\varepsilon_{\text{mach}}\kappa(A)^2$ for CGS. For example, with $\kappa(A) = 10^8$, MGS keeps only about 8 digits of orthogonality, and CGS keeps none. Householder stays orthogonal to $O(\varepsilon_{\text{mach}})$ for any $\kappa(A)$.

When we apply the Householder Q (for example to compute $Q^*\vec{b}$), we only apply exact reflectors in floating point. This is backward stable, which is why least squares via Householder QR is reliable.

7.3 Givens Rotations

Motivation: Householder can be overkill. Householder works on a whole column at a time. Step k always updates the whole block $A[k{:}m, k{:}n]$, even if column k has only one nonzero below the diagonal. This is wasteful when A is already close to upper triangular. A typical example is an *upper Hessenberg* matrix, which is zero below the first subdiagonal:

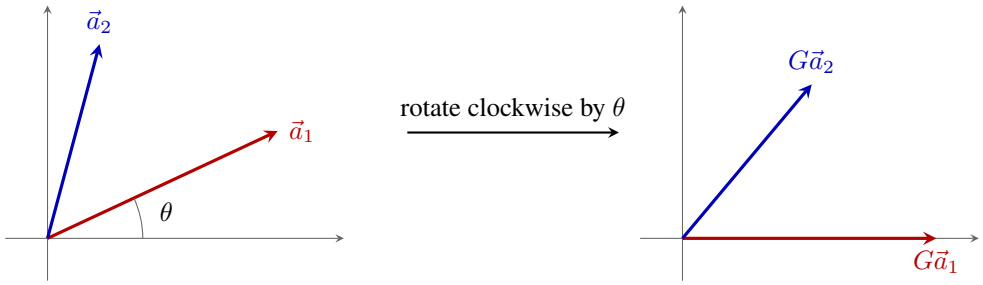
$$\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix}. \tag{7.4}$$

Here each column has just one entry to eliminate. Such matrices appear in the QR algorithm for eigenvalues (Lecture 14). So we want an orthogonal transformation that zeros *one entry at a time* and touches only the two rows involved. Givens rotations do exactly this.

The 2×2 case. Consider the leading 2×2 block

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}. \tag{7.5}$$

We want to eliminate A_{21}. Geometrically, the first column $\vec{a}_1 = (A_{11}, A_{21})^\top$ is a vector in the plane. We *rotate* the plane so that $\vec{a}_1$ lands on the horizontal axis. The second column $\vec{a}_2$ rotates along with it.



Definition 29 (Givens Rotation)

With $\|\vec{a}_1\| = \sqrt{A_{11}^2 + A_{21}^2}$, the *Givens rotation* for $\vec{a}_1$ is

$$G = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}, \qquad c = \frac{A_{11}}{\|\vec{a}_1\|}, \qquad s = \frac{A_{21}}{\|\vec{a}_1\|}. \tag{7.6}$$

Applying G to $\vec{a}_1$ indeed puts it on the horizontal axis:

$$G \begin{bmatrix} A_{11} \\ A_{21} \end{bmatrix} = \frac{1}{\|\vec{a}_1\|} \begin{bmatrix} A_{11}^2 + A_{21}^2 \\ -A_{21}A_{11} + A_{11}A_{21} \end{bmatrix} = \begin{bmatrix} \|\vec{a}_1\| \\ 0 \end{bmatrix}. \tag{7.7}$$

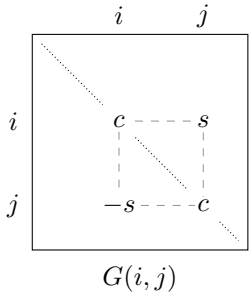
G is a rotation:

- $c^2 + s^2 = 1$, so $G^\top G = I$ and $\det G = 1$.
- If $\vec{a}_1$ makes the angle θ with the horizontal axis, then $c = \cos\theta$ and $s = \sin\theta$, and G rotates clockwise by θ.

Compare with Householder:

- A Householder reflector is a *reflection* ($\det = -1$). It maps a whole vector onto the direction of $\vec{e}_1$ at once.
- A Givens rotation is a *rotation* ($\det = 1$). It acts in one two-dimensional plane and zeros one entry.

QR by Givens rotations. In the $m \times m$ setting, the Givens rotation acting on rows i and j is the identity except in four positions:



$G(i,j)$

with c in positions (i,i) and (j,j), s in (i,j), and $-s$ in (j,i). The dotted diagonal is all 1, and all other entries are 0. Left multiplication by $G(i,j)$ changes only rows i and j: writing $\vec{r}_i^{\,\top}$ for the i-th row of A,

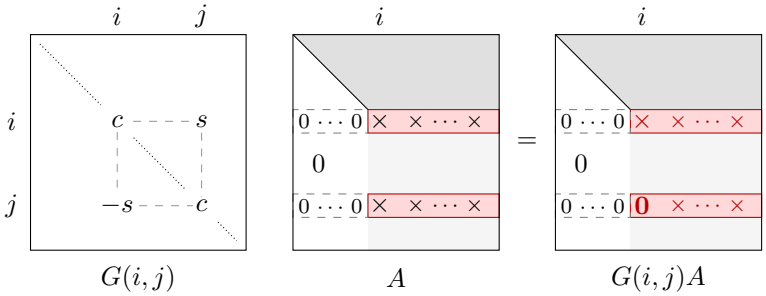
$$\vec{r}_i^{\,\top} \leftarrow c\,\vec{r}_i^{\,\top} + s\,\vec{r}_j^{\,\top}, \qquad \vec{r}_j^{\,\top} \leftarrow -s\,\vec{r}_i^{\,\top} + c\,\vec{r}_j^{\,\top}, \tag{7.8}$$

and every other row is left alone (see Section 7.7.4 for why).

To eliminate A_{ji} ($j > i$), take

$$c = \frac{A_{ii}}{\sqrt{A_{ii}^2 + A_{ji}^2}}, \qquad s = \frac{A_{ji}}{\sqrt{A_{ii}^2 + A_{ji}^2}}, \tag{7.9}$$

which is the 2×2 case transplanted to rows i and j. Suppose the first $i-1$ columns have already been reduced. Then one step gives the following picture.



$G(i,j)$ $\quad A$ $\quad G(i,j)A$

The dark gray rows are finished rows of R, and the light gray block is still to be reduced. Only rows i and j (outlined) change; in the first $i-1$ columns both are zero, and in column i the entry A_{ji} is rotated into A_{ii}.

Existing zeros are preserved:

- In the first $i-1$ columns, rows i and j are both zero. A linear combination of two zeros is zero.
- In column i, row i becomes $\sqrt{A_{ii}^2 + A_{ji}^2}$, row j becomes 0, and no other row changes.

So we sweep through the subdiagonal nonzeros column by column, from left to right. Within column i, we pair row i with $j = i+1, \dots, m$ in turn. For a 4×3 matrix this takes six rotations, one per subdiagonal entry.

In the picture below, $\times$ is a generic nonzero, red marks the entries that the current rotation changes, and the bold $\mathbf{0}$ is the entry it eliminates.

$$A = \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{G(1,2)} \begin{bmatrix} \times & \times & \times \\ \mathbf{0} & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{G(1,3)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ \mathbf{0} & \times & \times \\ \times & \times & \times \end{bmatrix} \xrightarrow{G(1,4)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \times & \times \\ \mathbf{0} & \times & \times \end{bmatrix}$$

$$\xrightarrow{G(2,3)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & \mathbf{0} & \times \\ 0 & \times & \times \end{bmatrix} \xrightarrow{G(2,4)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & \mathbf{0} & \times \end{bmatrix} \xrightarrow{G(3,4)} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & \mathbf{0} \end{bmatrix} = R.$$

Notice that $G(1,3)$ does not touch row 2, so the zero created by $G(1,2)$ survives. From $G(2,3)$ on, the zeros in column 1 are carried along as combinations of zeros.

Labeling the rotations in the order they are applied, we get

$$G_N \cdots G_1 A = R, \qquad Q = G_1^* \cdots G_N^*, \tag{7.10}$$

where N is the number of subdiagonal nonzeros. For a dense $m \times n$ matrix,

$$N = \sum_{i=1}^{n} (m-i) = mn - \frac{n(n+1)}{2}.$$

As with Householder, Q is not formed explicitly. We store the pairs (c, s) and apply the rotations in sequence when needed, just as in Algorithms 10 and 11. Every G is orthogonal with $\kappa = 1$, so by the error-amplification framework of Lecture 6, Givens QR is backward stable as well.

Efficiency.

Dense matrices: slower than Householder. Each rotation acts on two rows, at 6 flops per column (4 multiplications and 2 additions). Eliminating column i takes $m-i$ rotations, each acting on rows of length about $n-i$, for a total of about

$$\sum_{i=1}^{n} 6(m-i)(n-i) \approx 3mn^2 - n^3 \text{ flops}, \tag{7.11}$$

about 1.5 times the cost of Householder ($2mn^2 - \frac{2}{3}n^3$).

Sparse or nearly triangular matrices: possibly faster. For an $n \times n$ upper Hessenberg matrix, only $n-1$ rotations are needed (one per subdiagonal entry), each costing $O(n)$, for a total of about $3n^2$ flops. Givens touches only the entries that must be eliminated and so exploits sparsity naturally.

7.4 Full and Reduced QR

Let $A \in \mathbb{K}^{m\times n}$ with $m \geq n$.

Definition 30 (Reduced and Full QR Factorizations)

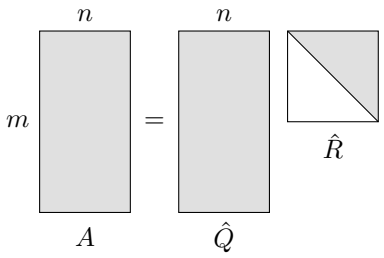
A *reduced QR factorization* of A is

$$A = \hat{Q}\hat{R}, \qquad \hat{Q} \in \mathbb{K}^{m\times n} \text{ with orthonormal columns}, \quad \hat{R} \in \mathbb{K}^{n\times n} \text{ upper triangular.} \tag{7.12}$$

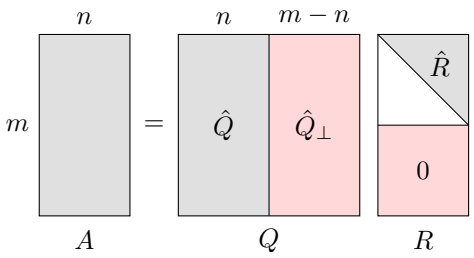
A *full QR factorization* of A is

$$A = QR, \qquad Q \in \mathbb{K}^{m\times m} \text{ unitary}, \quad R = \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix} \in \mathbb{K}^{m\times n}, \ \hat{R} \text{ upper triangular.} \tag{7.13}$$

Gram–Schmidt produces a reduced factorization; Householder and Givens produce a full one. Drawing each factor to scale, this gives the following picture.



Reduced QR (Gram–Schmidt): $\hat{Q}$ has the same shape as A, and $\hat{R}$ is $n \times n$.



Full QR (Householder, Givens): Q is $m \times m$, and R is $\hat{R}$ padded with $m - n$ zero rows.

Splitting $Q = \begin{bmatrix} \hat{Q} & \hat{Q}_\perp \end{bmatrix}$ by columns,

$$QR = \begin{bmatrix} \hat{Q} & \hat{Q}_\perp \end{bmatrix} \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix} = \hat{Q}\hat{R}. \tag{7.14}$$

In the picture, the extra $m - n$ columns $\hat{Q}_\perp$ (red) multiply only the zero block of R (also red) and do not contribute to the product. Their role is to complete $\hat{Q}$ to an orthonormal basis of $\mathbb{K}^m$; when A has full rank, the columns of $\hat{Q}_\perp$ span $\mathcal{R}(A)^\perp$.

7.5 Existence

Theorem 31 (Existence of QR)

Every $A \in \mathbb{K}^{m\times n}$ with $m \geq n$ has a full QR factorization and a reduced QR factorization.

The proof is constructive: we run Gram–Schmidt, look at where it can fail, and fix those cases.

Proof. Case 1: A has full rank, reduced QR. Gram–Schmidt can only fail at the normalization $\vec{q}_k = \hat{\vec{q}}_k / \left\|\hat{\vec{q}}_k\right\|$, if $\hat{\vec{q}}_k = \vec{0}$. This happens only if

$$\left\|\hat{\vec{q}}_k\right\| = 0 \iff \vec{a}_k \in \operatorname{span}(\vec{q}_1, \ldots, \vec{q}_{k-1}) = \operatorname{span}(\vec{a}_1, \ldots, \vec{a}_{k-1}). \tag{7.15}$$

Then $\vec{a}_k$ would be a combination of the earlier columns, which contradicts full rank. So Gram–Schmidt never fails, and every $r_{kk} = \left\|\hat{\vec{q}}_k\right\| > 0$.

Case 2: A is rank deficient, reduced QR. We run Gram–Schmidt as usual. When $\hat{\vec{q}}_k = \vec{0}$, we set $r_{kk} = 0$ and pick any unit vector $\vec{q}_k$ orthogonal to $\vec{q}_1, \ldots, \vec{q}_{k-1}$. Such a vector exists because $k - 1 < n \leq m$. We still have

$$\vec{a}_k = \sum_{i<k} r_{ik}\vec{q}_i + 0 \cdot \vec{q}_k, \tag{7.16}$$

so $A = \hat{Q}\hat{R}$ holds. The only difference is that $\hat{R}$ has some zeros on its diagonal.

Case 3: full QR. We first compute a reduced QR factorization. Then we add $m - n$ orthonormal vectors to $\vec{q}_1, \ldots, \vec{q}_n$ to get an orthonormal basis of $\mathbb{K}^m$, and we add $m - n$ zero rows to $\hat{R}$. □

7.6 Uniqueness

When QR is not unique.

Full QR with $m > n$ is not unique. In $QR = \hat{Q}\hat{R} + \hat{Q}_\perp \cdot 0$, the columns of $\hat{Q}_\perp$ only multiply the zero block of R, so they do not affect the product. Any orthonormal basis of $\mathcal{R}(\hat{Q})^\perp$ works. For example, we can replace $\hat{Q}_\perp$ by $\hat{Q}_\perp U$ for any unitary $U \in \mathbb{K}^{(m-n)\times(m-n)}$.

Rank-deficient A. If $\hat{\vec{q}}_k = \vec{0}$, then $r_{kk} = 0$, so $\vec{q}_k$ is multiplied by zero in $\vec{a}_k = \sum_{i<k} r_{ik}\vec{q}_i + 0 \cdot \vec{q}_k$. Any unit vector orthogonal to $\vec{q}_1, \ldots, \vec{q}_{k-1}$ works.

Example 32 (Non-Uniqueness for a Rank-Deficient Matrix)**.** Let

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \qquad \vec{q}_1 = \vec{e}_1, \qquad \hat{\vec{q}}_2 = \vec{a}_2 - \vec{e}_1 = \vec{0}. \tag{7.17}$$

Both $\vec{q}_2 = \vec{e}_2$ and $\vec{q}_2 = \vec{e}_3$ work, with the same $\hat{R} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$.

Phases. Even for full-rank A, some freedom is left. For any d_k with $|d_k| = 1$, we can multiply $\vec{q}_k$ by d_k and row k of $\hat{R}$ by $\bar{d}_k$. The product does not change, because $\bar{d}_k d_k = 1$. In the real case, this means each column can flip its sign. So for uniqueness we need a normalization, such as $r_{kk} > 0$.

Theorem 33 (Uniqueness of Reduced QR)

Let $A \in \mathbb{K}^{m\times n}$, $m \geq n$, have full column rank. Then the reduced QR factorization $A = \hat{Q}\hat{R}$ with $r_{jj} > 0$ for all j is unique.

Proof. Let $A = \hat{Q}\hat{R} = \tilde{Q}\tilde{R}$ be two such factorizations. We show that the columns agree, $\tilde{\vec{q}}_k = \vec{q}_k$, one k at a time. Fix k and write $\vec{q} = \vec{q}_k$ and $\tilde{\vec{q}} = \tilde{\vec{q}}_k$. We use the inner product $\langle \vec{x}, \vec{y} \rangle = \vec{x}^* \vec{y}$.

Step 1: what we know about $\vec{q}$ and $\tilde{\vec{q}}$. Since A has full rank, all $r_{jj} \neq 0$, so $\hat{R}$ is upper triangular and invertible. From $\hat{Q} = A\hat{R}^{-1}$, the first k columns of $\hat{Q}$ span the same space as the first k columns of A:

$$\operatorname{span}(\vec{q}_1, \ldots, \vec{q}_k) = \operatorname{span}(\vec{a}_1, \ldots, \vec{a}_k) \qquad \text{for every } k. \tag{7.18}$$

So $\vec{q}$ lies in $\operatorname{span}(\vec{a}_1, \ldots, \vec{a}_k)$. It is also orthogonal to $\vec{q}_1, \ldots, \vec{q}_{k-1}$, hence to $\vec{a}_1, \ldots, \vec{a}_{k-1}$. The same holds for $\tilde{\vec{q}}$.

Step 2: $\tilde{\vec{q}}$ is a multiple of $\vec{q}$. Since $\operatorname{span}(\vec{a}_1, \ldots, \vec{a}_k) = \operatorname{span}(\vec{a}_1, \ldots, \vec{a}_{k-1}, \vec{q})$, we can write

$$\tilde{\vec{q}} = \alpha \vec{q} + \sum_{j=1}^{k-1} \beta_j \vec{a}_j. \tag{7.19}$$

We take inner products with $\vec{q}$ and with $\tilde{\vec{q}}$. The $\vec{a}_j$ terms drop out, because both $\vec{q}$ and $\tilde{\vec{q}}$ are orthogonal to every $\vec{a}_j$ with $j < k$:

$$\left\langle \vec{q}, \tilde{\vec{q}} \right\rangle = \alpha, \qquad 1 = \left\langle \tilde{\vec{q}}, \tilde{\vec{q}} \right\rangle = \alpha \left\langle \tilde{\vec{q}}, \vec{q} \right\rangle = \alpha \bar{\alpha} = |\alpha|^2 . \tag{7.20}$$

With these two facts,

$$\left\| \tilde{\vec{q}} - \alpha \vec{q} \right\|^2 = \left\langle \tilde{\vec{q}}, \tilde{\vec{q}} \right\rangle - \alpha \left\langle \tilde{\vec{q}}, \vec{q} \right\rangle - \bar{\alpha} \left\langle \vec{q}, \tilde{\vec{q}} \right\rangle + |\alpha|^2 = 1 - 1 - 1 + 1 = 0. \tag{7.21}$$

So $\tilde{\vec{q}} = \alpha \vec{q}$ with $|\alpha| = 1$.

Step 3: $r_{kk} > 0$ forces $\alpha = 1$. From $A = \hat{Q}\hat{R}$ we get $r_{kk} = \langle \vec{q}_k, \vec{a}_k \rangle$. In the same way,

$$\tilde{r}_{kk} = \left\langle \tilde{\vec{q}}_k, \vec{a}_k \right\rangle = \langle \alpha \vec{q}_k, \vec{a}_k \rangle = \bar{\alpha} r_{kk}.$$

Both r_{kk} and $\tilde{r}_{kk}$ are positive, so $\bar{\alpha} = \tilde{r}_{kk}/r_{kk} > 0$. A positive number with $|\alpha| = 1$ is 1, so $\alpha = 1$.

So $\tilde{\vec{q}}_k = \vec{q}_k$ for every k, i.e., $\tilde{Q} = \hat{Q}$. Then $\tilde{R} = \hat{Q}^* A = \hat{R}$. □

In short: inside $\operatorname{span}(\vec{a}_1, \ldots, \vec{a}_k)$, the directions orthogonal to $\operatorname{span}(\vec{a}_1, \ldots, \vec{a}_{k-1})$ form a line. So A fixes $\vec{q}_k$ up to a factor of absolute value 1, and $r_{kk} > 0$ fixes that factor.

7.7 (SUPPLEMENT) Further Topics

7.7.1 Three QR Algorithms

	Factorization	Form of Q	Orthogonality of Q	Dense flops
MGS	reduced	explicit $\hat{Q}$	loses $\sim \varepsilon_{\text{mach}} \kappa(A)$	$2mn^2$
Householder	full	implicit (n vectors $\vec{v}_k$)	orthogonal by definition	$2mn^2 - \frac{2}{3}n^3$
Givens	full	implicit (pairs (c, s))	orthogonal by definition	$3mn^2 - n^3$

7.7.2 Householder as an Existence Proof

The Householder algorithm never breaks down. The only special case is $\vec{x} = A[k{:}m, k] = \vec{0}$; then we simply take $F_k = I$. So Householder gives a full QR factorization of *any* A, rank-deficient or not. This is a shorter existence proof than the one based on Gram–Schmidt. Trefethen and Bau prove existence with Gram–Schmidt in Lecture 7; their Lecture 10 implicitly gives the Householder proof [1].

7.7.3 Uniqueness via Cholesky

If $A = \hat{Q}\hat{R}$, then

$$A^*A = \hat{R}^*\hat{Q}^*\hat{Q}\hat{R} = \hat{R}^*\hat{R}. \tag{7.22}$$

So $\hat{R}^*$ is a Cholesky factor of A^*A. If A has full column rank, A^*A is Hermitian positive definite. Its Cholesky factorization with positive diagonal is unique. So $\hat{R}$ is unique, and then $\hat{Q} = A\hat{R}^{-1}$ is unique too.

This is a proof, not an algorithm. We should not *compute* QR this way, because forming A^*A squares the condition number (Lecture 5).

7.7.4 Why a Givens Rotation Changes Only Two Rows

Think of each column of A as a point in $\mathbb{K}^m$. Row k of A collects the k-th coordinates of all these points. $G(i, j)$ rotates in the plane spanned by $\vec{e}_i$ and $\vec{e}_j$. This is like a rotation about the z-axis in 3D: every point moves, but only its x and y coordinates change, and its z coordinate stays the same. In the same way, every column of A moves, but only its entries i and j change. So only rows i and j of A change.

7.7.5 Computing c and s Without Overflow

Computing $\sqrt{A_{ii}^2 + A_{ji}^2}$ directly can go wrong. The squares can overflow for large entries and underflow for small ones. So we first divide by the larger entry. If $|A_{ji}| > |A_{ii}|$, we set

$$\tau = A_{ii}/A_{ji}, \qquad s = 1/\sqrt{1+\tau^2}, \qquad c = s\tau,$$

and we swap the roles of A_{ii} and A_{ji} otherwise. Now $\tau^2 \leq 1$, so nothing overflows. This gives (c, s) up to a common sign, which still zeros the entry. The function `hypot` in many languages works this way. Demmel also shows how to store a rotation in a single number, so that it fits into the zeroed entry [2, §3.4.2].

7.7.6 Other Advantages of Givens Rotations

- *Parallelism.* Rotations on different pairs of rows do not interact. We can apply them at the same time.
- *Updating.* Suppose we already have the QR factorization of A, and a new row is added. We can eliminate the new row with n Givens rotations, instead of factoring again. This is common in recursive least squares.
- *Hessenberg matrices.* QR of a Hessenberg matrix is the core step of the QR algorithm for eigenvalues [1, Lectures 28–29].

7.7.7 Correspondence with Trefethen and Bau

- Full and reduced QR, existence and uniqueness: Theorems 7.1 and 7.2 of [1].
- The two implicit algorithms for $Q^*\vec{b}$ and $Q\vec{x}$: Algorithms 10.2 and 10.3.
- Givens rotations: Exercise 10.4.

Lecture 8

Overdetermined Least Squares and the Normal Equations

Supplementary reading:

- [1] Lectures 2, 11, and 23.
- [2] Sections 2.7.1 and 3.2.

8.1 Recap and Motivation

Last lecture we finished the QR factorization.

- *Using Q.* Householder QR never forms Q. To apply Q^* to a vector, we apply the stored reflectors in the order $\vec{v}_1, \dots, \vec{v}_n$. To apply Q, we use the reverse order.
- *Givens rotations.* A rotation in the (i, j) plane with $c = A_{ii}/\sqrt{A_{ii}^2 + A_{ji}^2}$ and $s = A_{ji}/\sqrt{A_{ii}^2 + A_{ji}^2}$ sets one entry to zero. Repeating this gives Givens QR.
- *Full and reduced QR*, and the *existence and uniqueness* of QR.

Solving overdetermined least squares. We now come back to the problem that led us to QR: given $A \in \mathbb{K}^{m \times n}$ with $m > n$ and $\vec{b} \in \mathbb{K}^m$, find the $\vec{x}$ that minimizes

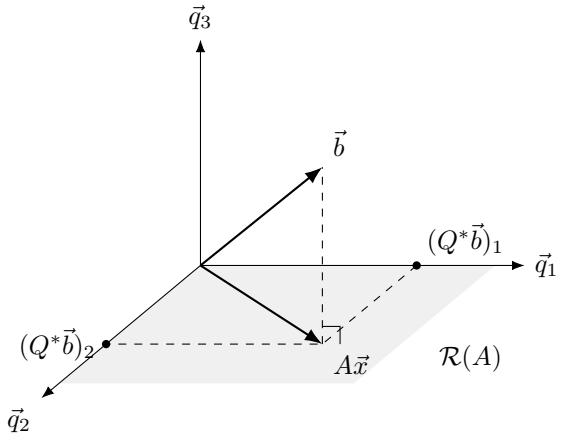
$$\left\| A\vec{x} - \vec{b} \right\|^2 . \tag{8.1}$$

There are more equations than unknowns, so $A\vec{x} = \vec{b}$ usually has no solution. We cannot solve it as in the square case.

We already saw an informal reason why $\vec{x} = \hat{R}^{-1}\hat{Q}^*\vec{b}$ solves this problem, for a reduced QR factorization $A = \hat{Q}\hat{R}$. Let us recall it. We extend $\vec{q}_1, \dots, \vec{q}_n$ to an orthonormal basis $\vec{q}_1, \dots, \vec{q}_m$ of $\mathbb{K}^m$. In this basis,

- $\vec{b}$ has the coordinates $Q^*\vec{b}$;
- $A\vec{x} = \hat{Q}(\hat{R}\vec{x})$ has the coordinates $\hat{R}\vec{x}$ in the first n directions, and 0 in the other $m - n$.

Since $\hat{R}$ is invertible, we can make the first n coordinates of $A\vec{x}$ equal to those of $\vec{b}$, by solving $\hat{R}\vec{x} = \hat{Q}^*\vec{b}$. The other $m - n$ coordinates of $A\vec{x}$ are always 0, so we cannot reduce the error there. For $m = 3$ and $n = 2$, we get the following picture.



In the basis $\vec{q}_1, \vec{q}_2, \vec{q}_3$, $A\vec{x}$ can match the first two coordinates of $\vec{b}$. The third coordinate, $(Q^*\vec{b})_3$, is the error that no $\vec{x}$ can remove.

This argument is convincing, but it relies on the picture. Why is matching the coordinates the best choice? Is it the only best choice? *How do we prove this?* One way uses that Q^* preserves norms (Section B.2). In this lecture we take a different way, which needs no factorization at all.

First we answer a more basic question: *what condition does a minimizer satisfy?* Such a condition helps in two ways.

- It lets us *check* that the $\vec{x}$ from QR really is a minimizer.
- It turns a minimization problem into a *system of equations*.

The main result of this lecture is

$$\vec{x} \text{ minimizes } \left\|A\vec{x} - \vec{b}\right\|^2 \quad \Longleftrightarrow \quad A\vec{x} - \vec{b} \perp \mathcal{R}(A) \quad \Longleftrightarrow \quad A^*A\vec{x} = A^*\vec{b}. \tag{8.2}$$

In Lecture 5 we saw this geometrically; now we prove it. Then we study the matrix A^*A of the last system. This leads to Gram matrices and positive semidefinite matrices.

8.2 Conventions

We use the inner product $\langle \vec{x}, \vec{y} \rangle = \vec{x}^*\vec{y}$, as in Lectures 5 and 7. It is linear in the second argument and conjugate linear in the first. Swapping the two arguments conjugates it: $\langle \vec{y}, \vec{x} \rangle = \overline{\langle \vec{x}, \vec{y} \rangle}$. So

$$\langle \vec{x}, \vec{y} \rangle + \langle \vec{y}, \vec{x} \rangle = 2 \operatorname{Re} \langle \vec{x}, \vec{y} \rangle . \tag{8.3}$$

We write $\vec{r} \coloneqq A\vec{x} - \vec{b}$ for the residual, and $\vec{d}$ for a perturbation direction.

We will use the following identity many times in this lecture.

Lemma 34 (Moving a Matrix Across the Inner Product)

For any $M \in \mathbb{K}^{m\times n}$ (not necessarily square or unitary), $\vec{u} \in \mathbb{K}^n$, and $\vec{v} \in \mathbb{K}^m$,

$$\langle M\vec{u}, \vec{v}\rangle = \langle \vec{u}, M^*\vec{v}\rangle \,. \tag{8.4}$$

Proof. This follows from associativity: $\langle M\vec{u}, \vec{v}\rangle = (M\vec{u})^*\vec{v} = \vec{u}^*(M^*\vec{v}) = \langle \vec{u}, M^*\vec{v}\rangle$. □

In words: we can move a matrix from one side of the inner product to the other, if we replace M by M^*. Two remarks.

- This is *not* the same as multiplying both sides by M^*. For example, the $m \times n$ factor $\hat{Q}$ of a reduced QR factorization has $\hat{Q}^*\hat{Q} = I$ but $\hat{Q}\hat{Q}^* \neq I$. In general $\left\langle \hat{Q}^*\vec{y}, \hat{Q}^*\vec{z}\right\rangle \neq \langle \vec{y}, \vec{z}\rangle$ (compare Section B.2).
- In Axler's *Linear Algebra Done Right* (Section 7.A), Lemma 34 is used as the definition of the adjoint.

8.3 The Optimality Condition

We want to find $\vec{x}$ that minimizes $\left\|A\vec{x} - \vec{b}\right\|^2$. The idea: if $\vec{x}$ is a minimizer, moving it cannot make the value smaller. So for every $\vec{\varepsilon} \in \mathbb{K}^n$,

$$\left\|A(\vec{x} + \vec{\varepsilon}) - \vec{b}\right\|^2 \geq \left\|A\vec{x} - \vec{b}\right\|^2 . \tag{8.5}$$

So we first compute how much $\left\|A\vec{x} - \vec{b}\right\|^2$ changes when we move $\vec{x}$ by $\vec{\varepsilon}$.

Lemma 35 (How the Squared Residual Changes)

Let $A \in \mathbb{K}^{m\times n}$ and $\vec{b} \in \mathbb{K}^m$. For all $\vec{x}, \vec{\varepsilon} \in \mathbb{K}^n$, with $\vec{r} = A\vec{x} - \vec{b}$,

$$\left\|A(\vec{x} + \vec{\varepsilon}) - \vec{b}\right\|^2 - \left\|A\vec{x} - \vec{b}\right\|^2 = \|A\vec{\varepsilon}\|^2 + 2\,\mathrm{Re}\,\langle A\vec{\varepsilon}, \vec{r}\rangle \,. \tag{8.6}$$

Proof. Since $A(\vec{x} + \vec{\varepsilon}) - \vec{b} = A\vec{\varepsilon} + \vec{r}$, we have

$$\begin{aligned} \|A\vec{\varepsilon} + \vec{r}\|^2 - \|\vec{r}\|^2 &= \langle A\vec{\varepsilon} + \vec{r}, A\vec{\varepsilon} + \vec{r}\rangle - \langle \vec{r}, \vec{r}\rangle \\ &= \langle A\vec{\varepsilon}, A\vec{\varepsilon}\rangle + \langle A\vec{\varepsilon}, \vec{r}\rangle + \underbrace{\langle \vec{r}, A\vec{\varepsilon}\rangle}_{=\overline{\langle A\vec{\varepsilon}, \vec{r}\rangle}} \\ &= \|A\vec{\varepsilon}\|^2 + 2\,\mathrm{Re}\,\langle A\vec{\varepsilon}, \vec{r}\rangle \,, \end{aligned} \tag{8.7}$$

In the last step we used $z + \bar{z} = 2\,\mathrm{Re}\,z$. □

This holds for any $\vec{x}$. Look at the right-hand side:

- The term $\|A\vec{\varepsilon}\|^2$ is always ≥ 0, so it never makes the value smaller.
- Only the mixed term $2\,\mathrm{Re}\,\langle A\vec{\varepsilon}, \vec{r}\rangle$ can be negative.

So we guess: $\vec{x}$ is a minimizer exactly when the mixed term is zero for every $\vec{\varepsilon}$. The next theorem shows that this is true.

Theorem 36 (Optimality Condition for Least Squares)

Let $A \in \mathbb{K}^{m\times n}$ and $\vec{b} \in \mathbb{K}^m$. Then $\vec{x} \in \mathbb{K}^n$ minimizes $\left\|A\vec{x} - \vec{b}\right\|^2$ if and only if the residual is orthogonal to $\mathcal{R}(A)$:

$$\left\langle A\vec{d}, A\vec{x} - \vec{b} \right\rangle = 0 \qquad \text{for all } \vec{d} \in \mathbb{K}^n. \tag{8.8}$$

Also, if $\vec{x}$ is a minimizer, then the minimizers are exactly the vectors in $\vec{x} + \mathcal{N}(A)$. So the minimizer is unique if and only if A has full column rank.

Proof. Write $\vec{r} = A\vec{x} - \vec{b}$.

If $\vec{x}$ is a minimizer, the residual is orthogonal. Let $\vec{x}$ be a minimizer. Take any $\vec{d} \in \mathbb{K}^n$ and $\delta > 0$. By Lemma 35 with $\vec{\varepsilon} = \delta\vec{d}$,

$$0 \le \left\|A(\vec{x} + \delta\vec{d}) - \vec{b}\right\|^2 - \left\|A\vec{x} - \vec{b}\right\|^2 = \delta^2 \left\|A\vec{d}\right\|^2 + 2\delta \operatorname{Re}\left\langle A\vec{d}, \vec{r}\right\rangle. \tag{8.9}$$

We divide by δ and let $\delta \to 0^+$:

$$\operatorname{Re}\left\langle A\vec{d}, \vec{r}\right\rangle \ge 0 \qquad \text{for all } \vec{d} \in \mathbb{K}^n.$$

We show that both the real and the imaginary part of $\left\langle A\vec{d}, \vec{r}\right\rangle$ are zero.

Real part. The inequality holds for every vector in $\mathbb{K}^n$, so it also holds for $-\vec{d}$. The inner product is conjugate-linear in its first argument, and $\overline{-1} = -1$, so

$$0 \le \operatorname{Re}\left\langle A(-\vec{d}), \vec{r}\right\rangle = \operatorname{Re}(-\left\langle A\vec{d}, \vec{r}\right\rangle) = -\operatorname{Re}\left\langle A\vec{d}, \vec{r}\right\rangle.$$

So $\operatorname{Re}\left\langle A\vec{d}, \vec{r}\right\rangle \le 0$. Together with $\operatorname{Re}\left\langle A\vec{d}, \vec{r}\right\rangle \ge 0$ this gives $\operatorname{Re}\left\langle A\vec{d}, \vec{r}\right\rangle = 0$.

Imaginary part. If $\mathbb{K} = \mathbb{R}$, then $\left\langle A\vec{d}, \vec{r}\right\rangle$ is real and we are done. If $\mathbb{K} = \mathbb{C}$, we use the vector $i\vec{d}$ instead. The real-part argument above works for every vector, so it gives $\operatorname{Re}\left\langle A(i\vec{d}), \vec{r}\right\rangle = 0$. Since $\bar{i} = -i$,

$$\left\langle A(i\vec{d}), \vec{r}\right\rangle = \bar{i}\left\langle A\vec{d}, \vec{r}\right\rangle = -i\left\langle A\vec{d}, \vec{r}\right\rangle.$$

For any complex number $a + bi$ with a, b real, $-i(a + bi) = b - ai$, so $\operatorname{Re}(-i(a + bi)) = b$. That is, multiplying by $-i$ turns the imaginary part into the real part. Hence

$$0 = \operatorname{Re}\left\langle A(i\vec{d}), \vec{r}\right\rangle = \operatorname{Re}(-i\left\langle A\vec{d}, \vec{r}\right\rangle) = \operatorname{Im}\left\langle A\vec{d}, \vec{r}\right\rangle.$$

So $\left\langle A\vec{d}, \vec{r}\right\rangle = 0$ for all $\vec{d} \in \mathbb{K}^n$.

If the residual is orthogonal, $\vec{x}$ is a minimizer. Suppose (8.8) holds. Take any $\vec{y} \in \mathbb{K}^n$. We use Lemma 35 with $\vec{\varepsilon} = \vec{y} - \vec{x}$. The mixed term is zero by (8.8), so

$$\left\|A\vec{y} - \vec{b}\right\|^2 = \left\|A\vec{x} - \vec{b}\right\|^2 + \|A(\vec{y} - \vec{x})\|^2 \ge \left\|A\vec{x} - \vec{b}\right\|^2. \tag{8.10}$$

All minimizers. By (8.10), $\vec{y}$ is also a minimizer if and only if $A(\vec{y} - \vec{x}) = \vec{0}$, i.e., $\vec{y} - \vec{x} \in \mathcal{N}(A)$. So the minimizer is unique if and only if $\mathcal{N}(A) = \{\vec{0}\}$, i.e., A has full column rank. □

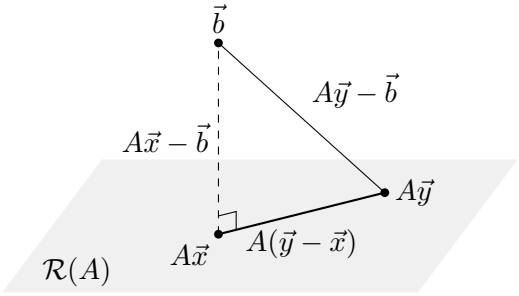
Thus, if $\vec{x}$ minimizes the squared residual, then

$$\left\langle A\vec{d}, A\vec{x} \right\rangle = \left\langle A\vec{d}, \vec{b} \right\rangle \qquad \text{for all } \vec{d} \in \mathbb{K}^n.$$

Both directions of the theorem are useful:

- Every minimizer satisfies (8.8). So solving (8.8) finds all minimizers.
- To show that a given $\vec{x}$ is a minimizer, it is enough to check (8.8). We do this for the QR solution in Section 8.4.

Geometrically, (8.10) is the Pythagorean theorem.



When the residual $A\vec{x} - \vec{b}$ is orthogonal to $\mathcal{R}(A)$, (8.10) is the Pythagorean theorem. Every other point $A\vec{y}$ in $\mathcal{R}(A)$ is at least as far from $\vec{b}$.

Geometric meaning. The vectors $A\vec{d}$, for all $\vec{d} \in \mathbb{K}^n$, are exactly the vectors in $\mathcal{R}(A)$. So the condition says: the residual is orthogonal to every vector in $\mathcal{R}(A)$. In other words, $A\vec{x}$ is the point of $\mathcal{R}(A)$ closest to $\vec{b}$. It is the orthogonal projection of $\vec{b}$ onto $\mathcal{R}(A)$. This is the picture of Lecture 5, now with a proof.

8.4 What the Optimality Condition Says

What does the optimality condition mean in concrete cases? The condition only tests the residual against vectors of the form $A\vec{d}$. How much this tells us depends on how many vectors have this form.

Invertible square matrices: solving a linear system. Let $m = n$ and let A be invertible. Let $\vec{x}$ be a minimizer. By Theorem 36, the residual is orthogonal to every $A\vec{d}$. Since A is invertible, we can choose $\vec{d} = A^{-1}(A\vec{x} - \vec{b})$, so that $A\vec{d} = A\vec{x} - \vec{b}$. Then

$$0 = \left\langle A\vec{d}, A\vec{x} - \vec{b} \right\rangle = \left\langle A\vec{x} - \vec{b}, A\vec{x} - \vec{b} \right\rangle = \left\| A\vec{x} - \vec{b} \right\|^2, \tag{8.11}$$

so $A\vec{x} = \vec{b}$. So for invertible square matrices, least squares is just solving the linear system, and the smallest residual is zero.

Singular or tall matrices: only some directions are tested. The trick above needs $A\vec{d} = \vec{r}$, where $\vec{r} = A\vec{x} - \vec{b}$ is the residual. If A is singular or $m > n$, then $\mathcal{R}(A)$ is smaller than $\mathbb{K}^m$. If also $\vec{b} \notin \mathcal{R}(A)$, then $\vec{r} \notin \mathcal{R}(A)$, and no such $\vec{d}$ exists. So we only get $\vec{r} \perp \mathcal{R}(A)$, not $\vec{r} = \vec{0}$. For example, $A = \left(\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right)$ and $\vec{b} = \left(\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right)$ give $x = 1$ and $\vec{r} = \left(\begin{smallmatrix} 0 \\ -1 \end{smallmatrix}\right) \neq \vec{0}$.

Checking the QR solution. Now we can prove the informal QR argument from the start of this lecture. Let $A = \hat{Q}\hat{R}$ be a reduced QR factorization, with $\hat{Q}^*\hat{Q} = I$. Let A have full column rank, so $\hat{R}$ is upper triangular and invertible. Take $\vec{x} = \hat{R}^{-1}\hat{Q}^*\vec{b}$. We check the orthogonality condition, moving $\hat{Q}$ to the right with Lemma 34:

$$\begin{aligned}\left\langle A\vec{d}, A\vec{x} - \vec{b}\right\rangle &= \left\langle \hat{Q}\hat{R}\vec{d}, \hat{Q}\hat{R}\hat{R}^{-1}\hat{Q}^*\vec{b} - \vec{b}\right\rangle \\ &= \left\langle \hat{R}\vec{d}, \hat{Q}^*\hat{Q}\hat{R}\hat{R}^{-1}\hat{Q}^*\vec{b} - \hat{Q}^*\vec{b}\right\rangle \\ &= \left\langle \hat{R}\vec{d}, \hat{Q}^*\vec{b} - \hat{Q}^*\vec{b}\right\rangle = 0.\end{aligned} \tag{8.12}$$

So $\vec{x} = \hat{R}^{-1}\hat{Q}^*\vec{b}$ satisfies the orthogonality condition. By Theorem 36, it is a least squares solution, and since A has full column rank, it is the only one. In practice we never form $\hat{R}^{-1}$. We solve the upper triangular system $\hat{R}\vec{x} = \hat{Q}^*\vec{b}$ by back substitution.

8.5 The Normal Equations

The optimality condition holds "for all $\vec{d}$". We now turn it into one linear system. The trick is to use the transpose: we apply Lemma 34 to both sides and move A to the right.

$$\left\langle A\vec{d}, A\vec{x}\right\rangle = \left\langle A\vec{d}, \vec{b}\right\rangle \ \ \forall \vec{d} \in \mathbb{K}^n \quad \Longleftrightarrow \quad \left\langle \vec{d}, A^*A\vec{x}\right\rangle = \left\langle \vec{d}, A^*\vec{b}\right\rangle \ \ \forall \vec{d} \in \mathbb{K}^n \quad \Longleftrightarrow \quad A^*A\vec{x} = A^*\vec{b}. \tag{8.13}$$

The first step is Lemma 34 applied to each side. For the last step, let $\vec{w} = A^*A\vec{x} - A^*\vec{b}$. The inner product is linear in its second argument, so subtracting the two sides turns the middle condition into $\left\langle \vec{d}, \vec{w}\right\rangle = 0$ for every $\vec{d} \in \mathbb{K}^n$. ($\Rightarrow$) Then $\vec{w}$ is orthogonal to every vector, including itself. Taking $\vec{d} = \vec{w}$ gives $\|\vec{w}\|^2 = 0$, so $\vec{w} = \vec{0}$, that is, $A^*A\vec{x} = A^*\vec{b}$. ($\Leftarrow$) If $\vec{w} = \vec{0}$, then $\left\langle \vec{d}, \vec{w}\right\rangle = 0$ for every $\vec{d}$.

So the orthogonality condition is the same as the normal equations. Together with Theorem 36, we get the following theorem.

Theorem 37 (The Normal Equations)

$\vec{x}$ minimizes $\left\|A\vec{x} - \vec{b}\right\|^2$ if and only if it solves the *normal equations*

$$A^*A\vec{x} = A^*\vec{b}. \tag{8.14}$$

We have turned our system into a square one! The $m \times n$ system $A\vec{x} = \vec{b}$ has become an $n \times n$ system. "Normal" means perpendicular: the equation $A^*(A\vec{x} - \vec{b}) = \vec{0}$ says that the residual is perpendicular to the column space, as in (5.7).

The normal equations have advantages and one serious drawback.

- *Pro:* the system is square, and its matrix A^*A is Hermitian (symmetric in the real case). In fact it is positive semidefinite (Theorem 42). We can solve such systems with the Cholesky factorization of Lecture 9, at half the cost of LU.
- *Con:* the condition number is squared, $\kappa_2(A^*A) = \kappa_2(A)^2$. So forming A^*A loses precision when A is ill-conditioned (see the supplement at the end of this lecture).

8.6 Gram Matrices and Positive Semidefinite Matrices

Solving the normal equations means solving a system with a matrix of the form B^*B. (We use the letter B here, so that A is free for the matrix of the system.) So we study such matrices. We start with a larger class.

Definition 38 (Hermitian and Anti-Hermitian Matrices)

A matrix $A \in \mathbb{K}^{m\times m}$ is *Hermitian* if $A^* = A$, and *anti-Hermitian* (or skew-Hermitian) if $A^* = -A$. For $\mathbb{K} = \mathbb{R}$, they are called *symmetric* and *antisymmetric* (or skew-symmetric).

Proposition 39 (Eigenvalues of (Anti-)Hermitian Matrices)

All eigenvalues of a Hermitian matrix are real. All eigenvalues of an anti-Hermitian matrix are purely imaginary.

Proof. Let $A\vec{v} = \lambda\vec{v}$ with $\vec{v} \neq \vec{0}$. Then $\vec{v}^*A\vec{v} = \lambda \|\vec{v}\|^2$. We take the complex conjugate of the number $\vec{v}^*A\vec{v}$:

$$\overline{\vec{v}^*A\vec{v}} = (\vec{v}^*A\vec{v})^* = \vec{v}^*A^*\vec{v} = \pm\vec{v}^*A\vec{v}, \tag{8.15}$$

with $+$ for Hermitian and $-$ for anti-Hermitian A. So $\vec{v}^*A\vec{v}$ is real in the first case and purely imaginary in the second. The same holds for $\lambda = \vec{v}^*A\vec{v}/\|\vec{v}\|^2$. □

These matrices behave a lot like complex numbers.

- Every square matrix splits into a Hermitian and an anti-Hermitian part, $A = \frac{1}{2}(A + A^*) + \frac{1}{2}(A - A^*)$. This is like splitting a complex number into its real and imaginary parts.
- A second analogy is $z = e^{i\theta}r$ with $r \geq 0$. Its matrix version is the polar decomposition below. It uses the following notion, which we study in detail later in this section: a Hermitian matrix S is *positive semidefinite* if $\vec{v}^*S\vec{v} \geq 0$ for all $\vec{v}$.

Theorem 40 (Polar Decomposition)

For every $A \in \mathbb{K}^{m\times m}$, there are a unitary U and a Hermitian positive semidefinite S such that

$$A = US. \tag{8.16}$$

Proof. Take an SVD $A = W\Sigma V^*$ and insert $V^*V = I$:

$$A = W\Sigma V^* = \underbrace{(WV^*)}_{U}\underbrace{(V\Sigma V^*)}_{S}. \tag{8.17}$$

We check the two factors.

- $U = WV^*$ is a product of unitary matrices, so it is unitary.
- $S = V\Sigma V^*$ is Hermitian, since Σ is real and diagonal. Also $\vec{v}^*S\vec{v} = \sum_i \sigma_i |(V^*\vec{v})_i|^2 \geq 0$. □

Geometrically, S stretches space along the orthogonal directions given by the columns of V. Then U rotates (or reflects) the result. The factor S is always determined by A: since $A^*A = SU^*US = S^2$, S is the positive semidefinite square root of A^*A. If A is invertible, then so is S, and $U = AS^{-1}$ is determined too.

Gram matrices. A matrix of the form B^*B is called a *Gram matrix* (or inner product matrix). Its entries are the inner products of the columns $B_{:,1}, \dots, B_{:,n}$ of B with each other. Using $B\vec{e}_j = B_{:,j}$,

$$(B^*B)_{ij} = \vec{e}_i^* B^* B \vec{e}_j = \langle B\vec{e}_i, B\vec{e}_j \rangle = \langle B_{:,i}, B_{:,j} \rangle . \tag{8.18}$$

We see right away:

- The diagonal entries $(B^*B)_{ii} = \|B_{:,i}\|^2$ are ≥ 0.
- $B^*B = I$ exactly when the columns of B are orthonormal.

Definition 41 (Positive (Semi)definite Matrices)

A Hermitian matrix $A \in \mathbb{K}^{n\times n}$ is

- *positive semidefinite* if $\vec{x}^* A \vec{x} \geq 0$ for all $\vec{x} \in \mathbb{K}^n$;
- *positive definite* if $\vec{x}^* A \vec{x} > 0$ for all $\vec{x} \neq \vec{0}$.

The main result of this section: Gram matrices and positive semidefinite matrices are the same thing.

Theorem 42 (Gram Matrices Are Exactly the Positive Semidefinite Matrices)

1. $A \in \mathbb{K}^{n\times n}$ is Hermitian positive semidefinite if and only if it is a Gram matrix, i.e., $A = B^*B$ for some matrix B.
2. For $B \in \mathbb{K}^{m\times n}$, the Gram matrix B^*B is positive definite if and only if B has full column rank.

Item 1 is false for general Hermitian matrices. For example, $\mathrm{diag}(1, -1)$ is symmetric but not a Gram matrix: its second diagonal entry would have to be $\|B_{:,2}\|^2 \geq 0$. In item 2, the rank condition is on B, not on the square matrix B^*B.

Proof of item 1. Gram matrices are Hermitian positive semidefinite. We have

$$(B^*B)^* = B^*(B^*)^* = B^*B, \qquad \vec{x}^* B^* B \vec{x} = \|B\vec{x}\|^2 \geq 0. \tag{8.19}$$

Hermitian positive semidefinite matrices are Gram matrices. We take a square root of A in three steps.

1. *Diagonalize.* By the spectral theorem, $A = U\Lambda U^*$, where U is unitary and Λ is diagonal with the eigenvalues of A on its diagonal.
2. *The eigenvalues are* ≥ 0. They are real by Proposition 39. If $A\vec{v} = \lambda\vec{v}$ with $\vec{v} \neq \vec{0}$, then $\lambda \|\vec{v}\|^2 = \vec{v}^* A \vec{v} \geq 0$, so $\lambda \geq 0$. (If A is positive definite, the same argument gives $\lambda > 0$.)
3. *Take square roots.* So we can define $\sqrt{\Lambda} = \mathrm{diag}(\sqrt{\lambda_1}, \dots, \sqrt{\lambda_n})$. We insert $U^*U = I$ in the middle:

$$A = U\sqrt{\Lambda}\, U^* U \sqrt{\Lambda}\, U^* = \left(U\sqrt{\Lambda}\, U^*\right)^* \left(U\sqrt{\Lambda}\, U^*\right). \tag{8.20}$$

   In the last step we used that $\sqrt{\Lambda}$ is real and diagonal, so it equals its conjugate transpose.

So $B = U\sqrt{\Lambda}\, U^*$ gives $A = B^*B$. □

The matrix $B = U\sqrt{\Lambda}U^*$ is the *matrix square root* $A^{1/2}$. It is not the only choice: for any unitary V, $(VB)^*(VB) = B^*B$. For example, a QR factorization $B = QR$ (Theorem 31) gives $A = R^*Q^*QR = R^*R$ with R upper triangular. This is the *Cholesky factorization* of A, the topic of the next lecture.

Proof of item 2. We show the equivalent statement with both sides negated:

$$\begin{aligned} B \text{ is column rank deficient} &\iff \exists\, \vec{x} \in \mathbb{K}^n \setminus \{\vec{0}\} : B\vec{x} = \vec{0} \\ &\iff \exists\, \vec{x} \in \mathbb{K}^n \setminus \{\vec{0}\} : \|B\vec{x}\|^2 = 0 \\ &\iff \exists\, \vec{x} \in \mathbb{K}^n \setminus \{\vec{0}\} : \vec{x}^*B^*B\vec{x} = 0. \end{aligned} \tag{8.21}$$

By item 1, $\vec{x}^*B^*B\vec{x} \geq 0$ always. So B^*B is not positive definite exactly when $\vec{x}^*B^*B\vec{x} = 0$ for some $\vec{x} \neq \vec{0}$. This is the last line above. □

What this means for least squares. Putting Theorems 37 and 42 together,

$$\begin{aligned} A \text{ has full column rank} &\iff A^*A \text{ is positive definite} \\ &\iff \text{the normal equations have the unique solution } \vec{x} = (A^*A)^{-1}A^*\vec{b}. \end{aligned} \tag{8.22}$$

This agrees with the uniqueness statement of Theorem 36. We can solve a positive definite system with the Cholesky factorization $A^*A = R^*R$. It costs about $\frac{1}{3}n^3$ flops, half of LU, and needs no pivoting.

8.7 (SUPPLEMENT) Further Topics

8.7.1 Why the Perturbation Argument

In Theorem 36 we moved $\vec{x}$ a little in every direction $\vec{d}$ and used that the value cannot go down. This idea, "at a minimizer, the first-order change in every direction is ≥ 0", works in many other settings:

- General differentiable functions: at a minimizer, the gradient is zero.
- Problems with constraints: we can no longer replace $\vec{d}$ by $-\vec{d}$, so the result stays an inequality. This leads to variational inequalities and the KKT conditions.
- Infinite-dimensional problems, such as finite elements. There, Galerkin orthogonality says that the residual is orthogonal to every test function v. This is the same as "for all $\vec{d}$" here.

8.7.2 A Numerical Warning About the Normal Equations

By Theorem 106, the singular values of A^*A are the squares of those of A. So

$$\kappa_2(A^*A) = \kappa_2(A)^2. \tag{8.23}$$

If A is ill-conditioned, solving $A^*A\vec{x} = A^*\vec{b}$ loses about twice as many digits as necessary.

The normal equations are faster: forming A^*A takes about mn^2 flops and Cholesky another $\frac{1}{3}n^3$. This is about half the cost of Householder QR when $m \gg n$. But the speed is only worth it when A is well-conditioned. So in practice we use a QR factorization and solve $\hat{R}\vec{x} = \hat{Q}^*\vec{b}$ instead [1, Lecture 11].

Lecture 9

Cholesky Factorization

Supplementary reading:

- [1] Lecture 23.
- [2] Section 2.7.1.
- [7] Chapter 10.

9.1 Recap and Motivation

Last lecture we solved the least squares problem $\min_{\vec{x}} \left\|A\vec{x} - \vec{b}\right\|^2$ in two ways.

- *By QR.* With a full QR factorization,

$$\left\|A\vec{x} - \vec{b}\right\|^2 = \left\|\hat{R}\vec{x} - (Q^*\vec{b})_{1:n}\right\|^2 + \left\|(Q^*\vec{b})_{n+1:m}\right\|^2.$$

  We make the first term zero by solving $\hat{R}\vec{x} = (Q^*\vec{b})_{1:n}$. The second term does not depend on $\vec{x}$. It is the squared distance from $\vec{b}$ to $\mathcal{R}(A)$.

- *By the optimality condition.* At the minimizer, the residual is orthogonal to $\mathcal{R}(A)$: $\left\langle A\vec{d}, A\vec{x}\right\rangle = \left\langle A\vec{d}, \vec{b}\right\rangle$ for all $\vec{d}$ (Theorem 36). Written as a linear system, these are the normal equations $A^*A\vec{x} = A^*\vec{b}$ (Theorem 37).

The matrix A^*A of the normal equations is a Gram matrix. We saw that Gram matrices are exactly the positive semidefinite matrices: M is Hermitian positive semidefinite if and only if $M = B^*B$ for some B (Theorem 42).

The factor B in $M = B^*B$ is not unique at all. For any unitary U, $(UB)^*(UB) = B^*U^*UB = B^*B$. Since we have so much freedom, we can ask:

If $A = B^*B$ is positive definite, can we choose B to be upper triangular?

If yes, then $A = R^*R$ with R upper triangular. Then solving $A\vec{x} = \vec{c}$ takes only two triangular solves: first $R^*\vec{y} = \vec{c}$, then $R\vec{x} = \vec{y}$. In this lecture we show that the answer is yes. The resulting factorization is called the *Cholesky factorization*. Even better, we will see that it needs no pivoting and is always backward stable.

Here is our plan.

1. Derive Cholesky from LU by making elimination *symmetric*, and turn it into an algorithm.
2. Prove that the algorithm never breaks down (existence) and that it does not amplify rounding errors (stability).
3. Look at the elimination in blocks, which leads to *Schur complements*.
4. Show that Cholesky and QR compute the same triangular factor.

Conventions. In this lecture $A \in \mathbb{K}^{m\times m}$ is a square matrix. It is not related to the $m \times n$ matrix of least squares. We say that A is *s.p.d.* if it is Hermitian positive definite, i.e., $A = A^*$ and $\vec{x}^* A\vec{x} > 0$ for all $\vec{x} \neq \vec{0}$. (In the real case this means symmetric positive definite, hence the name.) We write the Cholesky factorization as $A = LL^*$ with L lower triangular; this is the same as $A = R^*R$ with $R = L^*$.

Since A is Hermitian, its first row is the conjugate transpose of its first column. So we can split off the first row and column as

$$A = \begin{bmatrix} \alpha & \vec{a}^* \\ \vec{a} & A_{22} \end{bmatrix}, \qquad \alpha = A_{11} \in \mathbb{R}, \quad \vec{a} = A[2{:}m, 1] \in \mathbb{K}^{m-1}, \quad A_{22} = A[2{:}m, 2{:}m]. \tag{9.1}$$

9.2 Deriving Cholesky by Symmetric Elimination

The idea is simple. We already know how to factor a matrix by Gaussian elimination. We run elimination on A. But at every step we do the *same* operation on the rows and on the columns. This way the matrix stays Hermitian. Let us go through one step.

Step 1: one step of LU. The first step of LU subtracts a_i/α times row 1 from row $i + 1$, for every row below the first. In matrix form, this is multiplication from the left by a unit lower triangular L_1:

$$L_1 = \begin{bmatrix} 1 & 0 \\ -\vec{a}/\alpha & I \end{bmatrix}, \qquad L_1 A = \begin{bmatrix} \alpha & \vec{a}^* \\ 0 & A_{22} - \dfrac{\vec{a}\vec{a}^*}{\alpha} \end{bmatrix}. \tag{9.2}$$

The first column is now zero below the diagonal. The lower right block has changed to

$$S := A_{22} - \frac{\vec{a}\vec{a}^*}{\alpha}. \tag{9.3}$$

The matrix S is called a *Schur complement*. We study it in Section 9.6. Note that $\vec{a}\vec{a}^*$ is an outer product. So the update is a rank-one change of A_{22}.

Can we divide by α? Yes: $\alpha = A_{11} = \vec{e}_1^* A\vec{e}_1 > 0$ by positive definiteness.

Step 2: add the matching column operation. If we continued with LU on S, we would get $A = LU$, where L and U are not related. We want $A = LL^*$ instead. So we keep the matrix symmetric. After the row operation L_1 on the left, we apply the matching column operation L_1^* on the right. The result is

$$L_1 A L_1^* = \begin{bmatrix} \alpha & 0 \\ 0 & S \end{bmatrix}. \tag{9.4}$$

Let us check this. Right multiplication by L_1^* subtracts $\overline{a_j}/\alpha$ times column 1 from column $j + 1$.

- It does not change the lower right block, since column 1 of $L_1 A$ is zero below the diagonal.
- It makes the first row zero. Since A is Hermitian, the $(1, j+1)$ entry of $L_1 A$ is $\overline{a_j}$. So it becomes $\overline{a_j} - (\overline{a_j}/\alpha)\alpha = 0$.

This is where symmetry helps. The first row is the conjugate of the first column. So the column operation that clears the row uses the same multipliers as the row operation that cleared the column. One matrix L_1 handles both sides.

Step 3: normalize by $\sqrt{\alpha}$. Finally, we scale the top left entry to 1. We divide the first row and the first column by $\sqrt{\alpha}$. In matrix form, we replace L_1 by $\bar{L}_1 = \operatorname{diag}(1/\sqrt{\alpha}, 1, \ldots, 1) L_1$. Then

$$\bar{L}_1 A \bar{L}_1^* = \begin{bmatrix} 1 & 0 \\ 0 & S \end{bmatrix}, \qquad \bar{L}_1^{-1} = \begin{bmatrix} \sqrt{\alpha} & 0 \\ \vec{a}/\sqrt{\alpha} & I \end{bmatrix}. \tag{9.5}$$

(To invert $\bar{L}_1$, undo the scaling and then flip the sign of the multipliers.) We move $\bar{L}_1$ and $\bar{L}_1^*$ to the other side and get

$$A = \begin{bmatrix} \sqrt{\alpha} & 0 \\ \vec{a}/\sqrt{\alpha} & I \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & S \end{bmatrix} \begin{bmatrix} \sqrt{\alpha} & \vec{a}^*/\sqrt{\alpha} \\ 0 & I \end{bmatrix}. \tag{9.6}$$

Now suppose that S has a Cholesky factorization $S = L_S L_S^*$. We put it into (9.6) and multiply out:

$$A = LL^*, \qquad L = \begin{bmatrix} \sqrt{\alpha} & 0 \\ \vec{a}/\sqrt{\alpha} & L_S \end{bmatrix}. \tag{9.7}$$

In words: *the first column of L is the first column of A divided by $\sqrt{\alpha}$, and the rest of L is the Cholesky factor of the Schur complement S.* So Cholesky is a recursion: compute one column, update the remaining block to S, and repeat on S. Only one thing can go wrong: some later S might not be s.p.d., and then its top left entry might not be positive. We rule this out in Section 9.4.

Problem Solving Strategy 43 (Computing a Cholesky Factorization by Hand)**.** *Repeat for $k = 1, \ldots, m$:*

- *Let d_k be the top left entry of the current matrix.*
- *Column k of L is the current first column divided by $\sqrt{d_k}$.*
- *Replace the current matrix by its Schur complement: delete the first row and column, and subtract $\vec{a}\vec{a}^*/d_k$ from what is left.*

Example 44 (A 3×3 Example)**.** Let

$$A = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 5 & 3 \\ 2 & 3 & 6 \end{bmatrix}. \tag{9.8}$$

Step 1. $d_1 = 4$, so the first column of L is $(4, 2, 2)^\top / 2 = (2, 1, 1)^\top$. The Schur complement is

$$\begin{bmatrix} 5 & 3 \\ 3 & 6 \end{bmatrix} - \frac{1}{4} \begin{bmatrix} 2 \\ 2 \end{bmatrix} \begin{bmatrix} 2 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix}. \tag{9.9}$$

Step 2. $d_2 = 4$, so the next column is $(4, 2)^\top / 2 = (2, 1)^\top$. The Schur complement is $5 - 2 \cdot 2/4 = 4$.

Step 3. $d_3 = 4$, so the last entry is $\sqrt{4} = 2$. Altogether,

$$L = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 1 & 2 \end{bmatrix}, \tag{9.10}$$

We can check that $LL^\top = A$.

Why the square root? If we eliminate symmetrically without taking square roots, we get $A = \tilde{L}D\tilde{L}^*$. Here $\tilde{L}$ is unit lower triangular and $D = \mathrm{diag}(d_1, \ldots, d_m)$ holds the pivots. LU puts all of D into $U = D\tilde{L}^*$. Cholesky splits it evenly between the two factors:

$$A = (\tilde{L}D^{1/2})(\tilde{L}D^{1/2})^* = LL^*. \tag{9.11}$$

This needs every $d_k > 0$, so that $D^{1/2}$ is real.

9.3 The Cholesky Algorithm

The right-looking algorithm. We now turn the recursion into an in-place algorithm. Step k computes column k of L and then applies the Schur complement update to the lower right block. Since every Schur complement is Hermitian, we only store and update the lower triangle.

Algorithm 12 Cholesky factorization, right-looking, in place (lower triangle).

1: **for** $k = 1, \ldots, m$ **do**
2: $\quad A_{kk} \leftarrow \sqrt{A_{kk}}$
3: $\quad$ **for** $i = k+1, \ldots, m$ **do**
4: $\quad\quad A_{ik} \leftarrow A_{ik}/A_{kk}$ ▷ column k of L
5: $\quad$ **for** $j = k+1, \ldots, m$ **do**
6: $\quad\quad$ **for** $i = j, \ldots, m$ **do**
7: $\quad\quad\quad A_{ij} \leftarrow A_{ij} - A_{ik}\overline{A_{jk}}$ ▷ Schur complement update

In line 2, A_{kk} is the current pivot d_k, the top left entry of the current Schur complement, not an entry of the original A. In the update, A_{ik} and A_{jk} have both been divided by $\sqrt{d_k}$, so we subtract $a_i\overline{a_j}/d_k$, the (i,j) entry of $\vec{a}\vec{a}^*/\alpha$. At the end, the lower triangle of A holds L. In short form:

```
A[:,k] /= sqrt(A[k,k]),   then   A[i,j] -= A[i,k]*conj(A[j,k]).
```

Comparison with LU. Cholesky differs from LU in three ways.

- *Normalization.* Columns are divided by the square root of the pivot, as explained above.
- *Half the work and storage.* Every Schur complement is Hermitian. So we only need to keep one triangle. The other one is its conjugate transpose. The cost drops from about $\frac{2}{3}m^3$ flops for LU to about $\frac{1}{3}m^3$.
- *No pivoting.* We prove in Sections 9.4 and 9.5 that Cholesky without pivoting never breaks down and is stable.

Where positive definiteness enters. The algorithm uses positive definiteness in exactly one place: the square root $\sqrt{A_{kk}}$ in line 2. Every *updated* pivot must be positive. Then it has a real square root, and we can divide by it. Positive definiteness of A directly tells us only that the *original* diagonal entries are positive. To show that the updated pivots are also positive, we need a proof. This is the existence theorem below. The next example shows that a proof is really needed.

Example 45 (A Symmetric Matrix That Is Not Positive Definite)**.** Run Algorithm 12 on

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}. \tag{9.12}$$

For $k = 1$: $\sqrt{1} = 1$, the first column becomes $(1, 2)^\top$, and the update gives $A_{22} = 1 - 2 \cdot 2 = -3$. For $k = 2$ we need $\sqrt{-3}$, so the algorithm fails. Both original diagonal entries are 1. The problem is the updated pivot.

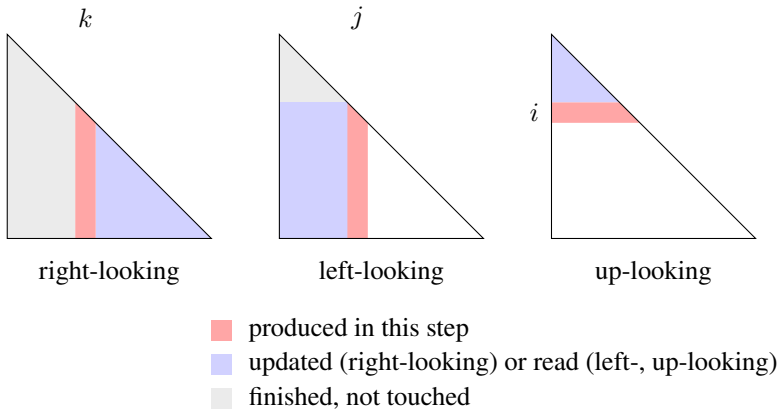
Left-looking and up-looking variants. The right-looking algorithm is not the only possible order of computation. Other orders give especially good memory access patterns. To see the options, we compare entries on both sides of $A = LL^*$ for $i \geq j$:

$$L_{jj} = \sqrt{A_{jj} - \sum_{k<j} |L_{jk}|^2}, \qquad L_{ij} = \frac{1}{L_{jj}} \left(A_{ij} - \sum_{k<j} L_{ik} \overline{L_{jk}} \right) \quad (i > j). \tag{9.13}$$

Any loop order gives a valid algorithm, as long as it evaluates these formulas using only entries of L that are already finished. All variants below do exactly the same floating-point operations. In exact arithmetic they produce the same L. They differ only in the order of the three nested loops over i, j, k.

Variant	Outer loop	Produces per step	Reads	Writes
right-looking	k	column k of L	column k	whole trailing Schur complement
left-looking	j	column j of L	columns $1, \ldots, j-1$	column j only
up-looking	i	row i of L	rows $1, \ldots, i-1$	row i only

The picture below shows one step of each variant.



Left-looking. The right-looking algorithm is *eager*. As soon as a column is finished, it applies that column's update to the whole trailing block. The left-looking algorithm is *lazy*. When it reaches column j, it applies all the updates from the finished columns to its left at once. Then it normalizes the column. So during step j it writes only to column j, and after that step column j is never touched again. In the right-looking variant, by

contrast, every step writes to the whole trailing block. The innermost loop runs down a column. This suits column-major languages (Julia, Fortran, MATLAB).

Algorithm 13 Cholesky factorization, left-looking, in place (lower triangle).

1: **for** $j = 1, \dots, m$ **do**
2: **for** $k = 1, \dots, j-1$ **do**
3: **for** $i = j, \dots, m$ **do**
4: $A_{ij} \leftarrow A_{ij} - A_{ik}\overline{A_{jk}}$
5: $s \leftarrow \sqrt{A_{jj}}$ ▷ save it: line 7 overwrites A_{jj}
6: **for** $i = j, \dots, m$ **do**
7: $A_{ij} \leftarrow A_{ij}/s$

Up-looking. Here L is computed row by row. Row i depends only on its own earlier entries and on the finished rows above it. Row i of L is column i of $R = L^*$. So a column-major code stores the upper triangular factor R instead. Then walking along a row of L becomes walking down a column of R. In *packed storage*, we store the upper triangle column by column. Column i has length i and comes right after column $i-1$:

$$\begin{bmatrix} 1 & 2 & 4 & 7 & 11 \\ & 3 & 5 & 8 & 12 \\ & & 6 & 9 & 13 \\ & & & 10 & 14 \\ & & & & 15 \end{bmatrix}. \tag{9.14}$$

Computing column i uses only earlier data. So when A grows by one row and one column, we just append one new segment at the end of the factor. Nothing computed before has to change.

Algorithm 14 Cholesky factorization, up-looking, in place (lower triangle).

1: **for** $i = 1, \dots, m$ **do**
2: **for** $j = 1, \dots, i$ **do**
3: **for** $k = 1, \dots, j-1$ **do**
4: $A_{ij} \leftarrow A_{ij} - A_{ik}\overline{A_{jk}}$
5: **if** $j < i$ **then**
6: $A_{ij} \leftarrow A_{ij}/A_{jj}$ ▷ A_{jj} already holds L_{jj}
7: **else**
8: $A_{ii} \leftarrow \sqrt{A_{ii}}$

In Algorithms 13 and 14 it is tempting to divide by $\sqrt{A_{jj}}$ in every case. This is wrong.

- In the left-looking loop, A_{jj} itself is overwritten when $i = j$. This is why that algorithm saves s first.
- In the up-looking loop, for $j < i$ the entry A_{jj} already holds L_{jj}. This is why that algorithm divides by A_{jj}.

9.4 Existence: Cholesky Never Breaks Down

We have seen that LU without pivoting can meet a zero pivot. For example, $\left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ fails at the very first step. Cholesky needs even more. The pivots must be *real and positive*, because we take their square roots. In this section we show that for s.p.d. matrices this always holds.

Here is the idea of the proof. After k steps of symmetric elimination, our current matrix is A times a nonsingular matrix on the right, and times its conjugate transpose on the left. We first show that such a "sandwich" of an s.p.d. matrix is again s.p.d. Then all its diagonal entries are positive. In particular, the next pivot is positive.

Lemma 46 (Congruence Preserves Positive Definiteness)

Let $A \in \mathbb{K}^{m\times m}$ be s.p.d. and $B \in \mathbb{K}^{m\times m}$ be nonsingular. Then B^*AB is s.p.d.

Proof. First, B^*AB is Hermitian, since $(B^*AB)^* = B^*A^*B = B^*AB$. Next we show positive definiteness. Let $\vec{x} \neq \vec{0}$. Since B is nonsingular, $B\vec{x} \neq \vec{0}$. Hence

$$\vec{x}^* B^* AB\vec{x} = (B\vec{x})^* A(B\vec{x}) > 0, \tag{9.15}$$

as desired. □

The map $A \mapsto B^*AB$ is called a *congruence*. We do need B to be nonsingular. If $B\vec{x} = \vec{0}$ for some $\vec{x} \neq \vec{0}$, then $\vec{x}^* B^* AB\vec{x} = 0$. Then we only get positive semidefiniteness.

Theorem 47 (Existence of the Cholesky Factorization)

Let $A \in \mathbb{K}^{m\times m}$ be s.p.d. Then Cholesky without pivoting meets only real, positive pivots $d_1, \dots, d_m$. Hence it produces $A = LL^*$, where L is lower triangular with positive diagonal. Also, the pivots have the lower bound

$$d_k \geq \frac{1}{\|A^{-1}\|_2} = \lambda_{\min}(A), \qquad k = 1, \dots, m. \tag{9.16}$$

Proof. We show by induction on $k = 0, \dots, m-1$ that the pivot d_{k+1} is real and positive.

Suppose that the first k pivots are positive. Then the elimination matrices $L_1, \dots, L_k$ of the first k steps are defined. (For $k = 0$ there is nothing to assume.) After k symmetric steps, we have

$$S \coloneqq L_k \cdots L_1 A L_1^* \cdots L_k^* = \begin{bmatrix} \operatorname{diag}(d_1, \dots, d_k) & 0 \\ 0 & S^{(k)} \end{bmatrix}, \tag{9.17}$$

where $S^{(k)}$ is the current Schur complement. Its top left entry is the next pivot, $d_{k+1} = S_{k+1,k+1}$.

Let $B = L_1^* \cdots L_k^*$, so that $S = B^*AB$. Each L_i is unit lower triangular, so it has determinant 1. Hence B is nonsingular. By Lemma 46, S is s.p.d. Now:

- *The pivot is real.* S is Hermitian, so $S_{k+1,k+1} = \overline{S_{k+1,k+1}}$.
- *The pivot is positive.* A standard basis vector picks out a diagonal entry, $\vec{e}_j^* S\vec{e}_j = S_{jj}$. So by positive definiteness,

$$d_{k+1} = S_{k+1,k+1} = \vec{e}_{k+1}^* S\vec{e}_{k+1} > 0. \tag{9.18}$$

This completes the induction. With all pivots positive, the construction of the previous section gives $A = LL^*$ with $L_{kk} = \sqrt{d_k} > 0$.

The lower bound (9.16) needs a fact about Schur complements. We prove it in Section 9.6. □

Note that "real" and "positive" come from different assumptions. The pivot is real because S is Hermitian. It is positive because S is positive definite. Along the way we also proved a useful fact: *every diagonal entry of a positive definite matrix is positive*. The lower bound (9.16) says even more than positivity. The pivots stay a fixed distance away from zero, for every k. So tiny pivots cannot occur.

Corollary 48 (Uniqueness of the Cholesky Factorization)**.** *An s.p.d. matrix has exactly one factorization $A = LL^*$, where L is lower triangular with positive diagonal.*

Proof. Existence is Theorem 47. Suppose $A = LL^* = MM^*$ with both factors of this form. Then $M^{-1}L = M^*L^{-*}$. The left side is lower triangular, and the right side is upper triangular. So both sides equal a diagonal matrix Δ. Its entries L_{kk}/M_{kk} are positive. Also,

$$\Delta\Delta^* = (M^{-1}L)(M^{-1}L)^* = M^{-1}LL^*M^{-*} = M^{-1}MM^*M^{-*} = I, \tag{9.19}$$

so $|\Delta_{kk}| = 1$. A positive number of absolute value 1 is 1, so $\Delta = I$ and $L = M$. □

9.5 Rounding Errors

Existence tells us that the algorithm runs to completion in exact arithmetic. We now ask what happens in floating point.

The key quantity for LU. Recall from the error analysis of LU (Theorem 16) that the computed factors satisfy $\tilde{L}\tilde{U} = A + \delta A$ with

$$|\delta A| \le cm\varepsilon_{\text{mach}} \left|\tilde{L}\right| \left|\tilde{U}\right| \quad \text{(entrywise)}. \tag{9.20}$$

So the key quantity is $|L|\,|U|$. If it is of the same size as $|A|$, the algorithm is backward stable. If it is much larger, errors are amplified. Without pivoting, $|L|\,|U|$ can be arbitrarily large. For example, for $A = \left[\begin{smallmatrix} \delta & 1 \\ 1 & 1 \end{smallmatrix}\right]$ with small δ, L contains the entry $1/\delta$.

No element growth in Cholesky. For Cholesky, $U = L^*$, so the key quantity is $|L|\,|L^*|$. We will show that it is never larger than the largest entry of A.

Proposition 49 (No Growth in the Cholesky Factor)

Let $A = LL^*$ be s.p.d. Then for all i, j,

$$|L_{ij}| \le \sqrt{A_{ii}}, \qquad (|L|\,|L^*|)_{ij} \le \sqrt{A_{ii}A_{jj}} \le \max_{k,l} |A_{kl}|\,. \tag{9.21}$$

Proof. We go in four steps.

Step 1: the diagonal of A. The (i, i) entry of $A = LL^*$ is

$$A_{ii} = \sum_k L_{ik}\overline{L_{ik}} = \sum_k |L_{ik}|^2\,.$$

So A_{ii} is the squared length of row i of L. Each $|L_{ij}|^2$ is one term of this sum, so $|L_{ij}|^2 \le A_{ii}$. This is the first claim.

Step 2: an entry of $|L|\,|L^*|$. By the rule for matrix products, with $|L^*|_{kj} = |L_{jk}|$,

$$(|L|\,|L^*|)_{ij} = \sum_k |L_{ik}|\,|L_{jk}|\,.$$

Step 3: Cauchy–Schwarz. For real numbers a_k, b_k, the Cauchy–Schwarz inequality says

$$\sum_k a_k b_k \le \left(\sum_k a_k^2\right)^{1/2} \left(\sum_k b_k^2\right)^{1/2}.$$

We take $a_k = |L_{ik}|$ and $b_k = |L_{jk}|$. By Step 1, the two sums on the right are A_{ii} and A_{jj}. So

$$\sum_k |L_{ik}|\,|L_{jk}| \le \left(\sum_k |L_{ik}|^2\right)^{1/2} \left(\sum_k |L_{jk}|^2\right)^{1/2} = \sqrt{A_{ii}}\sqrt{A_{jj}}.$$

Step 4: compare with the largest entry. The geometric mean of two numbers is at most the larger one. Both A_{ii} and A_{jj} are entries of A. So

$$\sqrt{A_{ii}A_{jj}} \le \max(A_{ii}, A_{jj}) \le \max_{k,l} |A_{kl}|\,.$$

Steps 2–4 together give the second claim. □

The same chain also shows where the large entries of an s.p.d. matrix are. Since $A_{ij} = \sum_k L_{ik}\overline{L_{jk}}$, we have $|A_{ij}| \le \sum_k |L_{ik}|\,|L_{jk}| \le \sqrt{A_{ii}}\sqrt{A_{jj}}$. So the largest entry of A is on its diagonal: $\max_{i,j} |A_{ij}| = \max_i A_{ii}$.

Here $\max_{k,l} |A_{kl}| = \|\mathrm{vec}(A)\|_\infty$ is the largest entry of A. (The vector $\mathrm{vec}(A) \in \mathbb{K}^{m^2}$ stacks the columns of A.) It is *not* the induced ∞-norm (maximum row sum) of Proposition 96. In words: the growth factor of Cholesky is 1. We put this into the backward error bound and get the main result.

Theorem 50 (Backward Stability of Cholesky)

Let $A \in \mathbb{K}^{m\times m}$ be s.p.d. Suppose Cholesky runs to completion in floating-point arithmetic and produces $\tilde{L}$. Then, to first order in $\varepsilon_{\text{mach}}$,

$$\tilde{L}\tilde{L}^* = A + \delta A, \qquad |\delta A_{ij}| \le cm\varepsilon_{\text{mach}} \max_{k,l} |A_{kl}| \tag{9.22}$$

for a modest constant c. That is, Cholesky without pivoting is backward stable.

Proof sketch. The analysis of Theorem 16 applies with $\tilde{U} = \tilde{L}^*$ and gives $|\delta A| \le cm\varepsilon_{\text{mach}} \left|\tilde{L}\right| \left|\tilde{L}^*\right|$ [7, Theorem 10.3]. Strictly speaking, Proposition 49 is about the exact factor. The computed factor satisfies $\sum_k \left|\tilde{L}_{ik}\right|^2 = A_{ii} + \delta A_{ii}$. This differs by a term of order $\varepsilon_{\text{mach}}$, which changes the bound only at second order. Hence $\left|\tilde{L}\right| \left|\tilde{L}^*\right| \le \max_{k,l} |A_{kl}|$ to first order, as desired. □

So Theorems 47 and 50 answer two different questions. The first says that the algorithm runs to completion in exact arithmetic. The second says that in floating point its answer is accurate. In LU, we need pivoting to fix both problems. For s.p.d. matrices, both hold automatically. (In floating point, completion itself is guaranteed as long as $\kappa_2(A)$ is not too close to $1/\varepsilon_{\text{mach}}$ [7, Theorem 10.7].)

Symmetric indefinite matrices. What if A is Hermitian but not positive definite? A Hermitian matrix is *indefinite* if $\vec{x}^* A\vec{x}$ takes both positive and negative values. Equivalently, it has both positive and negative eigenvalues. For such a matrix, pivots can be zero or negative, and Cholesky fails. Instead, we compute

$$PAP^\top = LDL^\top, \tag{9.23}$$

where P is a permutation and D is block diagonal with 1×1 and 2×2 blocks (LDL^* in the complex case).

Why do we use the symmetric permutation $PAP^\top$, and not PA as in LU?

- The product $PAP^\top$ swaps rows and columns in the same way. So it keeps the matrix symmetric, and we can still work on one half.
- PA swaps only rows. This destroys symmetry.

But a symmetric permutation has a limit. It only moves diagonal entries to other diagonal positions. It can never bring an off-diagonal entry onto the diagonal. For example, consider

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \tag{9.24}$$

Its diagonal is zero, so every 1×1 pivot is zero no matter how we permute. The solution is to take the whole 2×2 block as one pivot. This block is invertible, so it becomes one block of D, with $L = I$.

In general, each step moves an invertible 1×1 or 2×2 block E to the top left. Then it factors

$$PAP^\top = \begin{bmatrix} E & C^* \\ C & B \end{bmatrix} = \begin{bmatrix} I & 0 \\ CE^{-1} & I \end{bmatrix} \begin{bmatrix} E & 0 \\ 0 & B - CE^{-1}C^* \end{bmatrix} \begin{bmatrix} I & E^{-1}C^* \\ 0 & I \end{bmatrix}, \tag{9.25}$$

and repeats on the Schur complement $B - CE^{-1}C^*$, just as in the derivation of Cholesky. In practice, we choose the blocks by *Bunch–Kaufman pivoting* (LAPACK `sytrf`). If a diagonal entry is too small compared with the off-diagonal entries in its column, it uses a 2×2 block instead. This avoids small pivots. Its stability is about the same as LU with partial pivoting. It costs about $\frac{1}{3}m^3$ flops.

9.6 Schur Complements

So far we eliminated one row and column at a time. Let us now do the same thing with blocks. This leads to the Schur complement. It appears everywhere in numerical linear algebra (and also in statistics and optimization).

Block Cholesky. We partition $A = LL^*$ into 2×2 blocks. Here L_{11} and L_{22} are square and lower triangular:

$$L = \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix}, \qquad A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \qquad LL^* = \begin{bmatrix} L_{11}L_{11}^* & L_{11}L_{21}^* \\ L_{21}L_{11}^* & L_{21}L_{21}^* + L_{22}L_{22}^* \end{bmatrix}. \tag{9.26}$$

Comparing blocks, we have

$$L_{11}L_{11}^* = A_{11}, \qquad L_{21}L_{11}^* = A_{21}, \qquad L_{21}L_{21}^* + L_{22}L_{22}^* = A_{22}. \tag{9.27}$$

The top right block gives the conjugate transpose of the second equation, so it gives nothing new.

We want to find $L_{22}L_{22}^*$. The second equation gives

$$L_{21} = A_{21}L_{11}^{-*}.$$

Using $A_{21}^* = A_{12}$, we get

$$L_{21}L_{21}^* = A_{21}L_{11}^{-*}L_{11}^{-1}A_{21}^* = A_{21}(L_{11}L_{11}^*)^{-1}A_{21}^* = A_{21}A_{11}^{-1}A_{12}. \tag{9.28}$$

We put this into the third equation and get

$$L_{22}L_{22}^* = A_{22} - A_{21}A_{11}^{-1}A_{12}. \tag{9.29}$$

The matrix on the right has a name.

Definition 51 (Schur Complement)

Let $A = \left[\begin{smallmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{smallmatrix}\right]$ with A_{11} square and invertible. The *Schur complement* of A_{11} in A is

$$A/A_{11} \coloneqq A_{22} - A_{21}A_{11}^{-1}A_{12}. \tag{9.30}$$

The Schur complement S from Step 1 of Section 9.2 is the special case where $A_{11} = \alpha$ is 1×1. Then A_{11}^{-1} is just division by a scalar.

Note that the Schur complement is *defined* as $A_{22} - A_{21}A_{11}^{-1}A_{12}$. The block equations show that it is *equal* to $L_{22}L_{22}^*$. This tells us two things.

1. L_{22} is the Cholesky factor of the Schur complement. So the algorithm can recurse on blocks instead of single columns.
2. The Schur complement equals $L_{22}L_{22}^*$ with L_{22} invertible, so it is s.p.d. This gives a second proof that the recursion never breaks down. (The first proof was the congruence argument.)

The Schur complement is also exactly what the column-by-column elimination of Section 9.4 computes. Let A_{11} be the leading $k \times k$ block. Then after k steps of symmetric elimination,

$$L_k \cdots L_1 A L_1^* \cdots L_k^* = \begin{bmatrix} \operatorname{diag}(d_1, \ldots, d_k) & 0 \\ 0 & A/A_{11} \end{bmatrix}. \tag{9.31}$$

Indeed, both come from eliminating the first k rows and columns of A. The block equations above show that what is left in the lower right is $L_{22}L_{22}^* = A/A_{11}$.

In other words, *k steps of elimination produce the Schur complement of the leading $k \times k$ block.*

The inverse and the Schur complement. The following lemma is the key tool for the pivot bound. It relates the Schur complement to a block of the inverse.

Lemma 52 (Blocks of the Inverse)

Let $A = \left[\begin{smallmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{smallmatrix}\right]$ be invertible with A_{11} invertible, let $S = A_{22} - A_{21}A_{11}^{-1}A_{12}$ be invertible, and partition $\Theta \coloneqq A^{-1}$ in the same way. Then

$$\Theta_{22} = S^{-1}. \tag{9.32}$$

Proof. The idea is to compute a block LU factorization of A and invert it. Block elimination gives

$$A = \begin{bmatrix} I & 0 \\ A_{21}A_{11}^{-1} & I \end{bmatrix} \begin{bmatrix} A_{11} & 0 \\ 0 & S \end{bmatrix} \begin{bmatrix} I & A_{11}^{-1}A_{12} \\ 0 & I \end{bmatrix}. \tag{9.33}$$

(Multiply it out to check.) To invert a unit block triangular matrix, we negate its off-diagonal block. So

$$A^{-1} = \begin{bmatrix} I & -A_{11}^{-1}A_{12} \\ 0 & I \end{bmatrix} \begin{bmatrix} A_{11}^{-1} & 0 \\ 0 & S^{-1} \end{bmatrix} \begin{bmatrix} I & 0 \\ -A_{21}A_{11}^{-1} & I \end{bmatrix}. \tag{9.34}$$

To read off the lower right block of the product, we only need two pieces:

- the second block row of the left factor, which is $\begin{bmatrix} 0 & I \end{bmatrix}$;
- the second block column of the right factor, which is $\begin{bmatrix} 0 & I \end{bmatrix}^\top$.

So the lower right block of A^{-1} is

$$\begin{bmatrix} 0 & I \end{bmatrix} \begin{bmatrix} A_{11}^{-1} & 0 \\ 0 & S^{-1} \end{bmatrix} \begin{bmatrix} 0 \\ I \end{bmatrix} = S^{-1}.$$

□

The proof uses only that A_{11} and S are invertible. It needs neither symmetry nor definiteness.

In words: *to get the Schur complement, invert, take the submatrix, and invert again*:

$$A/A_{11} = \left((A^{-1})_{22}\right)^{-1}. \tag{9.35}$$

Watch the inverse here. It is easy to remember the lemma wrongly as $\Theta_{22} = A_{22} - A_{21}A_{11}^{-1}A_{12}$. But the block of the inverse is S^{-1}, not S. The slogan "invert, take submatrix, invert" helps us remember the correct version.

Proof of the pivot bound. We can now finish the proof of Theorem 47. The plan has two parts.

- The k-th pivot is a diagonal entry of a Schur complement S', so it is at least $\lambda_{\min}(S')$.
- By Lemma 52, $(S')^{-1}$ is a block of A^{-1}. This gives $\lambda_{\min}(S') \geq \lambda_{\min}(A)$.

Both parts use the Rayleigh quotient bound: for Hermitian H and $\vec{x} \neq \vec{0}$,

$$\lambda_{\min}(H) \leq \frac{\vec{x}^* H \vec{x}}{\vec{x}^* \vec{x}} \leq \lambda_{\max}(H). \tag{9.36}$$

Proof of (9.16). Fix k. Let S' be the Schur complement of the leading $(k-1) \times (k-1)$ block of A (for $k = 1$, $S' = A$). As shown above, S' is the matrix left over after $k-1$ steps of elimination, and it is s.p.d. So the k-th pivot is its top left entry, and by (9.36) with $\vec{x} = \vec{e}_1$,

$$d_k = \vec{e}_1^* S' \vec{e}_1 \geq \lambda_{\min}(S'). \tag{9.37}$$

It remains to show $\lambda_{\min}(S') \geq \lambda_{\min}(A)$. Let $\Theta = A^{-1}$. It is s.p.d., with eigenvalues $1/\lambda_i(A)$, so $\lambda_{\max}(\Theta) = 1/\lambda_{\min}(A)$. By Lemma 52, $(S')^{-1} = \Theta_{22}$, the lower right block of Θ.

Let λ be an eigenvalue of S' with eigenvector $\vec{y}$. Then $\lambda > 0$ and $(S')^{-1}\vec{y} = \lambda^{-1}\vec{y}$. Pad $\vec{y}$ with zeros:

$$\vec{x} = \begin{bmatrix} \vec{0} \\ \vec{y} \end{bmatrix}, \qquad \vec{x}^* \Theta \vec{x} = \vec{y}^* \Theta_{22} \vec{y}, \qquad \vec{x}^* \vec{x} = \vec{y}^* \vec{y}.$$

So by (9.36) applied to Θ,

$$\frac{1}{\lambda} = \frac{\vec{y}^*(S')^{-1}\vec{y}}{\vec{y}^*\vec{y}} = \frac{\vec{x}^*\Theta\vec{x}}{\vec{x}^*\vec{x}} \leq \lambda_{\max}(\Theta) = \frac{1}{\lambda_{\min}(A)}.$$

Hence $\lambda \geq \lambda_{\min}(A)$ for every eigenvalue of S'.

Putting the two parts together,

$$d_k \geq \lambda_{\min}(S') \geq \lambda_{\min}(A) = \frac{1}{\|A^{-1}\|_2}, \tag{9.38}$$

where the last equality holds because the singular values of an s.p.d. matrix are its eigenvalues (Theorem 106). □

The same argument with $\lambda_{\min}(\Theta)$ gives $\lambda \leq \lambda_{\max}(A)$. So every eigenvalue of a Schur complement of an s.p.d. matrix lies in $[\lambda_{\min}(A), \lambda_{\max}(A)]$.

9.7 Cholesky and QR

We end with a link back to least squares. The Cholesky factorization and the QR factorization are two views of the same thing. Let $B \in \mathbb{K}^{m\times n}$ have full column rank. Then B^*B is s.p.d. (Theorem 42). If $B = QR$, then

$$B^*B = R^*Q^*QR = R^*R. \tag{9.39}$$

So the R of QR is a Cholesky factor of B^*B. The next theorem states this exactly, and also gives the converse.

Theorem 53 (QR of B Is Cholesky of B^*B)

Let $B \in \mathbb{K}^{m\times n}$ have full column rank.

1. If $B = \hat{Q}\hat{R}$ is a reduced QR factorization with $r_{jj} > 0$, then $B^*B = \hat{R}^*\hat{R}$ is the Cholesky factorization of B^*B.
2. Conversely, if $B^*B = R^*R$ is the Cholesky factorization, then $B = (BR^{-1})R$ is a reduced QR factorization: BR^{-1} has orthonormal columns.

Proof. Item 1. We have $B^*B = \hat{R}^*\hat{Q}^*\hat{Q}\hat{R} = \hat{R}^*\hat{R}$, and $\hat{R}^*$ is lower triangular with positive diagonal. By Corollary 48, this is *the* Cholesky factorization.

Item 2. R is invertible because its diagonal is positive. So

$$(BR^{-1})^*(BR^{-1}) = R^{-*}B^*BR^{-1} = R^{-*}R^*RR^{-1} = I, \tag{9.40}$$

as desired. □

So QR of B and Cholesky of B^*B compute the same triangular factor. This has three consequences.

- *Uniqueness of QR.* The uniqueness of Cholesky (Corollary 48) gives the uniqueness of reduced QR with $r_{jj} > 0$ (Theorem 33). Indeed, $\hat{R}$ must be the Cholesky factor of B^*B, and then $\hat{Q} = B\hat{R}^{-1}$.
- *Least squares.* Solving the normal equations with Cholesky computes the same R as QR. But it goes through B^*B, whose condition number is $\kappa_2(B)^2$. The two methods agree in exact arithmetic, but not in floating point.
- *CholeskyQR.* Item 2 is itself an algorithm: form B^*B, factor it, and set $\hat{Q} = BR^{-1}$. It is fast, and it is mostly matrix multiplications. But for the same reason (it goes through B^*B), the computed $\hat{Q}$ loses orthogonality like $\varepsilon_{\text{mach}}\kappa_2(B)^2$. This is worse than modified Gram–Schmidt.

9.8 (SUPPLEMENT) Further Topics

9.8.1 Over $\mathbb{R}$, "Positive Definite" Does Not Imply Symmetric

If $A = B^*B$ with B real, then A is automatically symmetric. But suppose we only ask that $\vec{x}^\top A\vec{x} > 0$ for all real $\vec{x} \neq \vec{0}$. Then A need not be symmetric. For example, $A = \left[\begin{smallmatrix} 1 & 1 \\ -1 & 1 \end{smallmatrix}\right]$ has $\vec{x}^\top A\vec{x} = x_1^2 + x_2^2$.

The reason is this. Every real square matrix is the sum of a symmetric part and a skew-symmetric part K. Since $\vec{x}^\top K\vec{x} = 0$, the quadratic form only sees the symmetric part. This is why, over $\mathbb{R}$, symmetry is stated as a separate condition in "s.p.d.".

Over $\mathbb{C}$, symmetry is automatic. Suppose $\vec{x}^* A\vec{x}$ is real for all complex $\vec{x}$. Then $\vec{x}^*(A - A^*)\vec{x} = 0$ for all $\vec{x}$. Substituting $\vec{x} = \vec{e}_j + \vec{e}_k$ and $\vec{x} = \vec{e}_j + i\vec{e}_k$ (polarization) gives $A = A^*$. The example above, at $\vec{x} = (1, i)^\top$, gives $2 + 2i$. So it is not positive definite over $\mathbb{C}$.

9.8.2 Cholesky as a Test for Positive Definiteness

Congruences by unit lower triangular matrices do not change the determinants of the leading principal submatrices. So $\det A[1{:}k, 1{:}k] = d_1 \cdots d_k$. Hence the pivots are ratios of consecutive leading principal minors:

$$d_k = \frac{\det A[1{:}k, 1{:}k]}{\det A[1{:}k-1, 1{:}k-1]}. \tag{9.41}$$

Combined with Sylvester's criterion (a Hermitian matrix is positive definite if and only if all its leading principal minors are positive), this gives

$$\begin{aligned} A \text{ is positive definite} &\iff \text{all leading principal minors are } > 0 \\ &\iff \text{all pivots are } > 0 \iff \text{Cholesky completes.} \end{aligned} \tag{9.42}$$

So Cholesky is the standard test for positive definiteness. It is much cheaper than computing eigenvalues. For example, `numpy.linalg.cholesky` raises `LinAlgError` exactly at this point when the input is not positive definite.

9.8.3 Blocked Cholesky and Hardware

Libraries (LAPACK and cuSOLVER `potrf`) use a blocked version, as for LU in Section 2.9.3. We take A_{11} to be a block of size, say, 256×256, and factor it. Then:

- $L_{21} = A_{21}L_{11}^{-*}$ is a triangular solve with many right-hand sides (TRSM);
- the Schur complement update $A_{22} - L_{21}L_{21}^*$ is a matrix multiplication (SYRK/GEMM).

So almost all flops are Level 3 BLAS operations, and these run close to peak speed. This continues our discussion of memory access in the left- and up-looking variants.

9.8.4 The Normal Equations, Again

Backward stability of Cholesky means this: the computed $\tilde{R}$ is the exact Cholesky factor of $A^*A + \delta$ with a small δ. But when we solve least squares through the normal equations, the matrix we factor is A^*A. So the error in $\vec{x}$ is amplified by $\kappa(A^*A) = \kappa(A)^2$. A stable Cholesky therefore does not make "normal equations plus Cholesky" a stable least squares solver. For ill-conditioned A, QR is still the best method (Lecture 8).

Lecture 10

Eigen-like Decompositions, Computing a Single Eigenvalue

Supplementary reading:

- [1] Lectures 24, 25, and 27.
- [2] Sections 4.4.1–4.4.2 and 5.3.2.
- [5] Sections 7.3.1 and 8.2.

10.1 Eigen-Type Factorizations

LU, Cholesky, and QR are based on *elimination* or *projection*. They have two things in common.

1. Their purpose is to *solve linear systems*. They use one-sided transformations ($A \mapsto LA$, $A \mapsto Q^*A$) or congruences ($A \mapsto LAL^*$). None of these preserve eigenvalues.
2. They finish in finitely many steps, about $O(m^3)$. Without rounding, they are exact.

Starting with this lecture we study a second family, which shows the eigenstructure of a matrix: the eigen, polar, Schur, and singular value decompositions. They use *similarity transformations*

$$A \mapsto X^{-1}AX,$$

which preserve eigenvalues. The table compares them.

Factorization	Form	Exists for	Transformation
Eigen	$A = X\Lambda X^{-1}$	diagonalizable (nondefective) A	general X, possibly ill-conditioned
Polar	$A = QP$, $P = (A^*A)^{1/2} \succeq 0$	every square A	Q unitary
Schur	$A = QTQ^*$, T upper triangular	every square A	Q unitary
SVD	$A = U\Sigma V^*$	every A	U, V unitary

Only the eigendecomposition may fail to exist. The other three exist for every (square) matrix, and they use only unitary transformations, which do not amplify errors. A *normal* matrix ($A^*A = AA^*$, for example a Hermitian one) can be diagonalized by a unitary matrix, $A = Q\Lambda Q^*$. We start with the eigendecomposition of Hermitian matrices.

10.2 Eigenpairs of Hermitian Matrices

Problem. Given a Hermitian $A \in \mathbb{K}^{m\times m}$, find $\lambda_1, \dots, \lambda_m \in \mathbb{R}$ and $\vec{x}_1, \dots, \vec{x}_m \in \mathbb{K}^m$ with

$$A\vec{x}_i = \lambda_i \vec{x}_i, \qquad i = 1, \dots, m. \tag{10.1}$$

Real eigenvalues and the Rayleigh quotient. Proposition 39 showed that the eigenvalues of a Hermitian matrix are real. Here is the same proof written with inner products:

$$\lambda_i = \frac{\langle \vec{x}_i, A\vec{x}_i\rangle}{\|\vec{x}_i\|^2} = \frac{\langle A\vec{x}_i, \vec{x}_i\rangle}{\|\vec{x}_i\|^2} = \overline{\lambda_i}. \tag{10.2}$$

We check the three equalities. Recall that $\langle \vec{x}, \vec{y}\rangle = \vec{x}^*\vec{y}$ and $A = A^*$.

- *First.* We put $A\vec{x}_i = \lambda_i\vec{x}_i$ into the second slot. The scalar λ_i comes out as it is:

$$\langle \vec{x}_i, A\vec{x}_i\rangle = \vec{x}_i^*(\lambda_i\vec{x}_i) = \lambda_i\, \vec{x}_i^*\vec{x}_i = \lambda_i \|\vec{x}_i\|^2 . \tag{10.3}$$

- *Second.* Since $A = A^*$, we can move A to the other slot:

$$\langle \vec{x}_i, A\vec{x}_i\rangle = \vec{x}_i^* A\vec{x}_i = \vec{x}_i^* A^*\vec{x}_i = (A\vec{x}_i)^*\vec{x}_i = \langle A\vec{x}_i, \vec{x}_i\rangle . \tag{10.4}$$

- *Third.* We put $A\vec{x}_i = \lambda_i\vec{x}_i$ into the first slot. Now λ_i comes out conjugated:

$$\langle A\vec{x}_i, \vec{x}_i\rangle = (\lambda_i\vec{x}_i)^*\vec{x}_i = \overline{\lambda_i}\, \vec{x}_i^*\vec{x}_i = \overline{\lambda_i} \|\vec{x}_i\|^2 . \tag{10.5}$$

So $\lambda_i = \overline{\lambda_i}$, that is, λ_i is real.

The first fraction, $\vec{x}_i^* A\vec{x}_i / \|\vec{x}_i\|^2$, can be computed for *any* nonzero vector, not only for eigenvectors. At an eigenvector it gives the eigenvalue. This leads to the following definition, our main tool for estimating eigenvalues.

Definition 54 (Rayleigh Quotient)

Let $A \in \mathbb{K}^{m\times m}$. The *Rayleigh quotient* of a nonzero $\vec{x} \in \mathbb{K}^m$ is

$$r(\vec{x}) := \frac{\vec{x}^* A\vec{x}}{\vec{x}^*\vec{x}}. \tag{10.6}$$

If A is Hermitian, the Rayleigh quotient is always real. At an eigenvector it equals the eigenvalue. So if we have a good approximate eigenvector, its Rayleigh quotient is a natural guess for the eigenvalue. We will see that this guess is surprisingly accurate.

Proposition 55 (Orthogonal Eigenvectors)

Let A be Hermitian, $A\vec{x}_i = \lambda_i\vec{x}_i$ and $A\vec{x}_j = \lambda_j\vec{x}_j$ with $\lambda_i \neq \lambda_j$. Then $\vec{x}_i \perp \vec{x}_j$.

Proof. We have

$$\lambda_i \langle \vec{x}_i, \vec{x}_j\rangle = \langle A\vec{x}_i, \vec{x}_j\rangle = \langle \vec{x}_i, A\vec{x}_j\rangle = \lambda_j \langle \vec{x}_i, \vec{x}_j\rangle . \tag{10.7}$$

The first step uses that λ_i is real (Proposition 39), so it comes out of the first slot unchanged. The second uses $A = A^*$. So $(\lambda_i - \lambda_j)\langle \vec{x}_i, \vec{x}_j\rangle = 0$, and since $\lambda_i \neq \lambda_j$, $\langle \vec{x}_i, \vec{x}_j\rangle = 0$. □

10.3 A Negative Result

LU and its relatives can be computed exactly (up to rounding) in $O(m^3)$ arithmetic operations. Can we do the same for eigenvalues? No. The reason comes from algebra.

Theorem 56 (Abel–Ruffini)

For every $m \geq 5$, there is a polynomial of degree m with rational coefficients whose roots cannot be expressed in its coefficients by finitely many $+$, $-$, $\times$, $\div$, and k-th roots.

Why is this a problem for eigenvalues? We can turn any polynomial into a matrix whose eigenvalues are its roots. So an exact eigenvalue algorithm would find the roots of any polynomial exactly, which contradicts Abel.

Lemma 57 (Companion Matrix)

Let $p(z) = z^m + a_{m-1}z^{m-1} + \cdots + a_1 z + a_0$ be monic, and let

$$A = \begin{bmatrix} 0 & 1 & & & \\ & 0 & 1 & & \\ & & \ddots & \ddots & \\ & & & 0 & 1 \\ -a_0 & -a_1 & \cdots & -a_{m-2} & -a_{m-1} \end{bmatrix}, \qquad \vec{x} = \begin{bmatrix} 1 \\ z \\ z^2 \\ \vdots \\ z^{m-1} \end{bmatrix}. \tag{10.8}$$

If $p(z) = 0$, then $A\vec{x} = z\vec{x}$, so z is an eigenvalue of A with eigenvector $\vec{x} \neq \vec{0}$.

Proof. Clearly $\vec{x} \neq \vec{0}$ since $x_1 = 1$. We check $A\vec{x} = z\vec{x}$ row by row.

- *The first $m-1$ rows.* Row i of A picks out the next entry of $\vec{x}$: $(A\vec{x})_i = x_{i+1} = z^i = z \cdot z^{i-1} = zx_i$.
- *The last row.* Using $p(z) = 0$,

$$(A\vec{x})_m = -\sum_{i=0}^{m-1} a_i z^i = z^m - \underbrace{p(z)}_{=0} = z \cdot z^{m-1} = zx_m. \qquad \square$$

Expanding $\det(zI - A)$ along the last row gives $\det(zI - A) = p(z)$. So the eigenvalues of A are *exactly* the roots of p. (If p is not monic, we first divide by its leading coefficient.)

Theorem 58 (Eigenvalue Algorithms Must Be Iterative)

For $m \geq 5$, no algorithm computes the eigenvalues of every $m \times m$ matrix exactly by finitely many $+$, $-$, $\times$, $\div$, and k-th roots.

Proof. Given any monic p of degree m, form its companion matrix. This needs no arithmetic, since its entries are 0, 1, and $-a_i$. The algorithm would then return the roots of p in finitely many operations, which contradicts Theorem 56. So eigenvalue algorithms must be *iterative*. $\square$

10.4 Power Iteration

We cannot hope for exact computation. So how can we *approximate* the eigenpairs of A? The simplest idea is to multiply a vector by A again and again. To see why this works, we first look at what multiplication by A does in the eigenbasis.

Multiplying by A in the eigenbasis. Let $A = X\Lambda X^{-1}$ be diagonalizable. We write any vector as a combination of eigenvectors, $\vec{z} = X\vec{y} = \sum_i y_i \vec{x}_i$. Here $\vec{y} = X^{-1}\vec{z}$ holds the coordinates of $\vec{z}$ in the eigenbasis. Then

$$A\vec{z} = X\Lambda\vec{y}, \qquad A^2\vec{z} = X\Lambda^2\vec{y}, \qquad A^k\vec{z} = X\Lambda^k\vec{y} = \sum_{i=1}^{m} \lambda_i^k y_i \vec{x}_i. \tag{10.9}$$

In words: *in the eigenbasis, multiplication by A multiplies each coordinate by its own eigenvalue. The directions do not mix.*

Now assume that the largest eigenvalue strictly dominates, $|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_m|$. We factor out λ_1^k:

$$A^k\vec{z} = \lambda_1^k \left(y_1\vec{x}_1 + \sum_{i=2}^{m} \left(\frac{\lambda_i}{\lambda_1}\right)^k y_i \vec{x}_i \right). \tag{10.10}$$

Every ratio $|\lambda_i/\lambda_1|$ with $i \geq 2$ is less than 1. So $(\lambda_i/\lambda_1)^k \to 0$, and the *direction* of $A^k\vec{z}$ approaches $\vec{x}_1$.

Informally,

$$\Lambda^k \approx \lambda_1^k \operatorname{diag}(1, 0, \ldots, 0), \qquad A^k\vec{z} \approx \lambda_1^k y_1 \vec{x}_1.$$

These approximations hold in the relative sense, $\Lambda^k/\lambda_1^k \to \operatorname{diag}(1, 0, \ldots, 0)$. The vector $A^k\vec{z}$ itself may blow up or go to zero.

The condition $y_1 \neq 0$. All of this needs $y_1 \neq 0$. If the starting vector has no component along $\vec{x}_1$, multiplying by A never creates one. A random starting vector has $y_1 \neq 0$ with probability 1.

The algorithm. The algorithm has four ingredients.

- We compute $\vec{z}_k = A^k\vec{z} / \left\| A^k\vec{z} \right\|$ as an approximation to $\vec{x}_1$.
- We use the Rayleigh quotient $\langle \vec{z}_k, A\vec{z}_k \rangle$ as an approximation to λ_1.
- We choose $\vec{z} \sim N(0, I)$ to make sure that $y_1 \neq 0$.
- We normalize at every step to avoid overflow.

Algorithm 15 Power iteration.

1: $\vec{q} \leftarrow$ vector with i.i.d. standard normal entries; $\vec{q} \leftarrow \vec{q}/\|\vec{q}\|$
2: **for** $k = 1, 2, \ldots, k_{\max}$ **do**
3: $\quad \vec{z} \leftarrow A\vec{q}$
4: $\quad \lambda \leftarrow \vec{q}^*\vec{z}$ ▷ Rayleigh quotient
5: $\quad \vec{q} \leftarrow \vec{z}/\|\vec{z}\|$

Why normalize. We only care about the direction of $A^k\vec{z}$, and rescaling does not change it, since $A(c\vec{v}) = c \cdot A\vec{v}$. The length, however, behaves like $|\lambda_1|^k$. If $|\lambda_1| > 1$, the iterates overflow; if $|\lambda_1| < 1$, they underflow. Either happens long before the direction converges. So we rescale to unit length at every step.

Rate of convergence. We now study how fast power iteration converges, for Hermitian A. We track two errors: the error in the vector and the error in the eigenvalue. The key point is that the second error is much smaller than the first. The reason is that the Rayleigh quotient of a Hermitian matrix is *quadratically* accurate.

Lemma 59 (Quadratic Accuracy of the Rayleigh Quotient)

Let A be Hermitian with orthonormal eigenvectors $\vec{x}_1, \dots, \vec{x}_m$, and let $\vec{q}$ be a unit vector. Then for every J,

$$r(\vec{q}) - \lambda_J = \sum_{j \neq J} (\lambda_j - \lambda_J) \, |c_j|^2 \,, \qquad c_j = \langle \vec{x}_j, \vec{q} \rangle \,, \tag{10.11}$$

and hence $|r(\vec{q}) - \lambda_J| \leq (\lambda_{\max} - \lambda_{\min}) \, \|\vec{q} - \vec{x}_J\|^2$.

Proof. Expand $\vec{q}$ in the orthonormal eigenvectors:

$$\vec{q} = \sum_j c_j \vec{x}_j, \qquad \sum_j |c_j|^2 = \|\vec{q}\|^2 = 1.$$

The identity. Since $A\vec{q} = \sum_j \lambda_j c_j \vec{x}_j$ and the $\vec{x}_j$ are orthonormal,

$$r(\vec{q}) = \vec{q}^* A \vec{q} = \sum_j \lambda_j \, |c_j|^2 \,. \tag{10.12}$$

We subtract $\lambda_J = \lambda_J \sum_j |c_j|^2$. The $j = J$ term cancels, and we get

$$r(\vec{q}) - \lambda_J = \sum_j (\lambda_j - \lambda_J) \, |c_j|^2 = \sum_{j \neq J} (\lambda_j - \lambda_J) \, |c_j|^2 \,.$$

The bound. Each factor satisfies $|\lambda_j - \lambda_J| \leq \lambda_{\max} - \lambda_{\min}$, so

$$|r(\vec{q}) - \lambda_J| \leq (\lambda_{\max} - \lambda_{\min}) \sum_{j \neq J} |c_j|^2 \,.$$

It remains to bound the sum by $\|\vec{q} - \vec{x}_J\|^2$. We write

$$\vec{q} - \vec{x}_J = (c_J - 1)\vec{x}_J + \sum_{j \neq J} c_j \vec{x}_j.$$

By Pythagoras,

$$\|\vec{q} - \vec{x}_J\|^2 = |c_J - 1|^2 + \sum_{j \neq J} |c_j|^2 \geq \sum_{j \neq J} |c_j|^2 \,. \tag{10.13}$$

□

Theorem 60 (Convergence of Power Iteration)

Let A be Hermitian, with orthonormal eigenvectors $\vec{x}_1, \dots, \vec{x}_m$ and eigenvalues

$$|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_m|.$$

Let the unit starting vector satisfy $c_1 = \left\langle \vec{x}_1, \vec{q}^{(0)} \right\rangle \neq 0$. Then there are unimodular scalars s_k ($s_k = \pm 1$ in the real case) such that the iterates of Algorithm 15 satisfy

$$\left\| \vec{q}^{(k)} - s_k \vec{x}_1 \right\| = O\left(\left| \frac{\lambda_2}{\lambda_1} \right|^k \right), \tag{10.14}$$

$$\left| r(\vec{q}^{(k)}) - \lambda_1 \right| = O\left(\left| \frac{\lambda_2}{\lambda_1} \right|^{2k} \right). \tag{10.15}$$

Proof. Write $\rho = |\lambda_2/\lambda_1| < 1$. Normalizing at every step only rescales, so $\vec{q}^{(k)}$ is $A^k \vec{q}^{(0)}$ scaled to unit length.

Expand. By (10.10) with $y_j = c_j$,

$$A^k \vec{q}^{(0)} = \lambda_1^k c_1 (\vec{x}_1 + \vec{e}_k), \qquad \vec{e}_k = \sum_{j=2}^{m} \left(\frac{\lambda_j}{\lambda_1} \right)^k \frac{c_j}{c_1} \vec{x}_j.$$

Each $|\lambda_j/\lambda_1| \leq \rho$ and $\sum_j |c_j|^2 = 1$, so

$$\|\vec{e}_k\| \leq \frac{\rho^k}{|c_1|}.$$

Normalize. Let $s_k = \lambda_1^k c_1 / \left| \lambda_1^k c_1 \right|$ and $n_k = \|\vec{x}_1 + \vec{e}_k\|$. Then

$$\vec{q}^{(k)} = \frac{s_k (\vec{x}_1 + \vec{e}_k)}{n_k}.$$

Since $\vec{e}_k \perp \vec{x}_1$, we have $n_k = \sqrt{1 + \|\vec{e}_k\|^2}$, so $1 \leq n_k \leq 1 + \|\vec{e}_k\|^2/2$. Hence

$$\vec{q}^{(k)} - s_k \vec{x}_1 = s_k \left[\left(\frac{1}{n_k} - 1 \right) \vec{x}_1 + \frac{\vec{e}_k}{n_k} \right],$$

and

$$\left\| \vec{q}^{(k)} - s_k \vec{x}_1 \right\| \leq \left(1 - \frac{1}{n_k} \right) + \|\vec{e}_k\| \leq \frac{\|\vec{e}_k\|^2}{2} + \|\vec{e}_k\| = O(\rho^k).$$

Eigenvalue. $s_k \vec{x}_1$ is also a unit eigenvector for λ_1. So Lemma 59, with $\vec{x}_1$ replaced by $s_k \vec{x}_1$, squares the vector error:

$$\left| r(\vec{q}^{(k)}) - \lambda_1 \right| \leq (\lambda_{\max} - \lambda_{\min}) \left\| \vec{q}^{(k)} - s_k \vec{x}_1 \right\|^2 = O(\rho^{2k}). \qquad \square$$

In words, if $|\lambda_1| > |\lambda_2|$ and $c_1 \neq 0$, power iteration converges:

- the direction of $\vec{q}^{(k)}$ converges to $\vec{x}_1$ (up to sign), and the error shrinks by about $|\lambda_2/\lambda_1|$ per step;

- the eigenvalue estimate $r(\vec{q}^{(k)})$ converges to λ_1 twice as fast, since its error is the square of the vector error.

For example, if $|\lambda_2/\lambda_1| = 0.5$, then after 10 steps the vector error is about $0.5^{10} \approx 10^{-3}$ and the eigenvalue error is about $0.5^{20} \approx 10^{-6}$. A random start gives $c_1 \neq 0$ with probability 1.

The square needs A Hermitian (Lemma 59). For a general diagonalizable matrix, the eigenvalue error is only $O(|\lambda_2/\lambda_1|^k)$.

What happens when $|\lambda_2/\lambda_1|$ is close to 1? There are two very different cases: $\lambda_2 \approx \lambda_1$ and $\lambda_2 \approx -\lambda_1$.

When $\lambda_2 \approx \lambda_1$: the eigenvalue still converges. In short, in this case *the eigenvalue converges well enough, but the eigenvector converges slowly.*

Suppose $|\lambda_1| \sim |\lambda_2| \gg |\lambda_3|$. In the space spanned by $\vec{x}_1, \vec{x}_2, \vec{x}_3$, the iterates move in two phases.

1. They first fall quickly into the plane $\mathrm{span}(\vec{x}_1, \vec{x}_2)$, at the rate $|\lambda_3/\lambda_1|^k$.
2. Then they turn slowly toward $\vec{x}_1$ within the plane, at the rate $|\lambda_2/\lambda_1|^k$.

After the first phase, $\vec{q} \approx c_1\vec{x}_1 + c_2\vec{x}_2$. The Rayleigh quotient

$$\lambda_1 |c_1|^2 + \lambda_2 |c_2|^2$$

is then a weighted average of λ_1 and λ_2. Its error is at most $|\lambda_1 - \lambda_2|$, which is already small.

When $\lambda_2 \approx -\lambda_1$: the real failure. Now only the plane matters. Let $\vec{q} = c_1\vec{x}_1 + c_2\vec{x}_2$. One multiplication by A gives

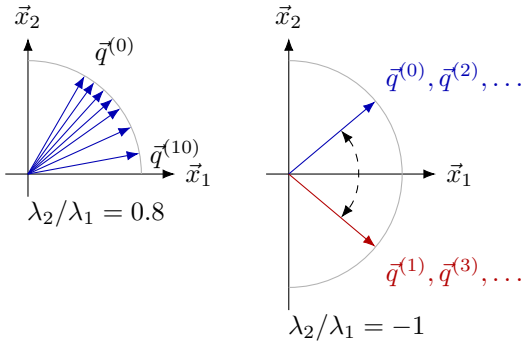
$$A\vec{q} \approx \lambda_1(c_1\vec{x}_1 - c_2\vec{x}_2).$$

Geometrically this is a *reflection across the $\vec{x}_1$ axis*. The iterates jump back and forth between two directions that are symmetric about $\vec{x}_1$.

If $\lambda_2 = -\lambda_1$ exactly, $|c_2/c_1|$ never changes, and there is no convergence at all. The Rayleigh quotient

$$\vec{q}^* A\vec{q} \approx \lambda_1 \left(|c_1|^2 - |c_2|^2\right) \tag{10.16}$$

is fixed by the starting vector. It may lie anywhere in $[-\lambda_1, \lambda_1]$.



Left: with $\lambda_2/\lambda_1 = 0.8$ the iterates turn slowly but steadily toward $\vec{x}_1$. Right: with $\lambda_2 = -\lambda_1$ each step reflects $\vec{q}$ across the $\vec{x}_1$ axis, and the iteration never converges.

So the truly bad case is eigenvalues with *similar modulus but far-apart values*; close values do no harm. The cure is to change the spectrum, which is the topic of the next section.

10.5 Matrix Functions: Changing the Spectrum

Power iteration has two weaknesses.

- It always finds the eigenvalue of largest modulus, which may not be the one we want.
- It is slow when another eigenvalue has nearly the same modulus.

We keep the algorithm and change the matrix instead. So we ask: *how can we change the eigenvalues of a matrix, while keeping its eigenvectors, so that the one we want stands out?*

Polynomials keep the eigenvectors. Let $A\vec{x} = \lambda\vec{x}$. Then

$$A^2\vec{x} = A(\lambda\vec{x}) = \lambda^2\vec{x}, \qquad \text{and in general} \qquad A^k\vec{x} = \lambda^k\vec{x}.$$

A polynomial $p(z) = a_0 + a_1 z + \cdots + a_d z^d$ is a linear combination of powers, so

$$p(A)\vec{x} = a_0\vec{x} + a_1\lambda\vec{x} + \cdots + a_d\lambda^d\vec{x} = p(\lambda)\vec{x}.$$

So $\vec{x}$ is still an eigenvector of $p(A)$, and its eigenvalue is now $p(\lambda)$. Some special cases:

Transformation	Eigenvalues	Eigenvectors
$A \to A^k$	$\lambda \to \lambda^k$	unchanged
$A \to \alpha A$	$\lambda \to \alpha\lambda$	unchanged
$A \to A + \beta I$	$\lambda \to \lambda + \beta$	unchanged
$A \to p(A)$	$\lambda \to p(\lambda)$	unchanged

- A^k makes large eigenvalues larger relative to small ones. This is exactly what power iteration does.
- αA scales all eigenvalues by the same factor, so the ratios $|\lambda_j/\lambda_1|$ do not change. It acts like normalization and does not help.
- $A + \beta I$ shifts all eigenvalues by the same amount. This *does* change the ratios, and it is the main tool below.
- $p(A)$ is the general case.

Power iteration on $p(A)$. We run power iteration with $p(A)$ in place of A. By Theorem 60, it converges to the eigenvector $\vec{x}_i$ for which $|p(\lambda_i)|$ is largest, at the rate

$$\max_{j \neq i} \frac{|p(\lambda_j)|}{|p(\lambda_i)|}.$$

Since $\vec{x}_i$ is also an eigenvector of A, the Rayleigh quotient of A (not of $p(A)$) gives the eigenvalue we want:

$$r(\vec{q}^{(k)}) = \vec{q}^{(k)*} A \vec{q}^{(k)} \to \lambda_i.$$

So p acts as a *filter*: we want $|p|$ large at the wanted eigenvalue and small at all the others. This is called *polynomial filtering*. Plain power iteration is the case $p(z) = z$; after k steps it has applied the filter z^k.

Shifting. Suppose an isolated eigenvalue sits on the positive axis, and a cluster sits on the negative axis with similar modulus, say

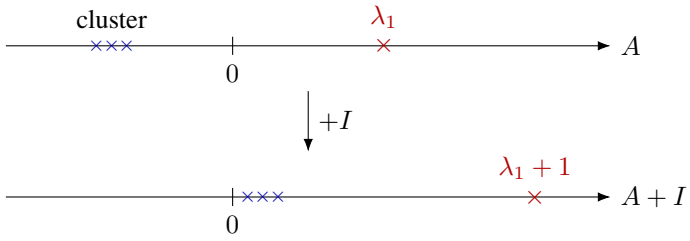
$$\lambda_1 = 1, \qquad \lambda_2 = -0.9, \qquad \lambda_3 = -0.8, \quad \dots$$

Then $|\lambda_2/\lambda_1| = 0.9$, and power iteration is slow. The problem is not that λ_1 and λ_2 are close; they are 1.9 apart. The problem is that their moduli are close.

Now iterate with $A + I$ instead. The eigenvalues become

$$2, \qquad 0.1, \qquad 0.2, \quad \dots,$$

and the ratio drops to $0.2/2 = 0.1$. The shift moves the cluster toward the origin and the isolated eigenvalue farther out.



The limit of shifting. For real eigenvalues, $|\lambda + \beta|$ is largest at one of the two ends of the spectrum, whatever β is. So a shift can only make the smallest or the largest eigenvalue stand out. To reach an eigenvalue in the middle, close to some μ, we need a function that is large near μ and small elsewhere, such as $f(z) = 1/(z - \mu)$. This leads to inverse iteration.

10.6 Inverse Iteration

A shift can only make an eigenvalue at an end of the spectrum stand out. Now suppose we want an eigenvalue in the *middle*: the one closest to a given number $\mu \in \mathbb{R}$. Following the previous section, we look for a function f that is large near μ and small far from μ. The simplest choice is

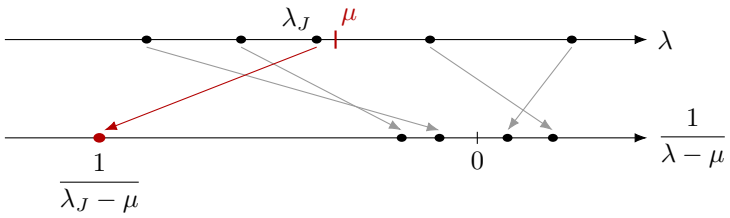
$$f(z) = \frac{1}{z - \mu}. \tag{10.17}$$

The matching matrix is $f(A) = (A - \mu I)^{-1}$. It has the same eigenvectors as A:

$$(A - \mu I)\vec{x}_j = (\lambda_j - \mu)\vec{x}_j \quad \Longrightarrow \quad (A - \mu I)^{-1}\vec{x}_j = \frac{1}{\lambda_j - \mu}\vec{x}_j.$$

What f does to the spectrum. Let λ_J be the eigenvalue closest to μ, and λ_K the second closest.

- λ_J is mapped to $1/(\lambda_J - \mu)$. Its denominator is the smallest, so it has the largest modulus.
- λ_K is mapped to the second largest modulus, $1/|\lambda_K - \mu|$.
- The far eigenvalues are squeezed toward 0. Those with $\lambda_j < \mu$ go to the negative axis, and those with $\lambda_j > \mu$ go to the positive axis.



The map $\lambda \mapsto 1/(\lambda - \mu)$ sends the eigenvalue closest to μ far away and squeezes all the others toward 0.

The algorithm. So power iteration on $(A - \mu I)^{-1}$ finds $\vec{x}_J$. This is *inverse iteration*. We never form the inverse; we compute $\vec{z} = (A - \mu I)^{-1}\vec{q}$ by solving $(A - \mu I)\vec{z} = \vec{q}$. Since μ is fixed, we factor $A - \mu I$ once, and each step costs $O(m^2)$ (Section 10.8.1).

Algorithm 16 Inverse iteration with a fixed shift μ.

$\vec{q} \leftarrow$ random unit vector
for $k = 1, 2, \ldots, k_{\max}$ **do**
    solve $(A - \mu I)\vec{z} = \vec{q}$
    $\vec{q} \leftarrow \vec{z} / \|\vec{z}\|$
    $\lambda \leftarrow \vec{q}^* A \vec{q}$ ▷ Rayleigh quotient of A

Theorem 61 (Convergence of Inverse Iteration)

Let A be Hermitian. Let λ_J be the eigenvalue closest to μ and λ_K the second closest, with $|\lambda_J - \mu| < |\lambda_K - \mu|$, and let $\left\langle \vec{x}_J, \vec{q}^{(0)} \right\rangle \neq 0$. Then there are unimodular s_k such that the iterates of Algorithm 16 satisfy

$$\left\| \vec{q}^{(k)} - s_k \vec{x}_J \right\| = O\left(\left| \frac{\lambda_J - \mu}{\lambda_K - \mu} \right|^k \right), \tag{10.18}$$

$$\left| \lambda^{(k)} - \lambda_J \right| = O\left(\left| \frac{\lambda_J - \mu}{\lambda_K - \mu} \right|^{2k} \right). \tag{10.19}$$

Proof. Up to normalization, Algorithm 16 is Algorithm 15 applied to $(A - \mu I)^{-1}$. This matrix is Hermitian, with the same orthonormal eigenvectors as A. Its largest eigenvalue in modulus is $1/(\lambda_J - \mu)$, and its second largest is $1/(\lambda_K - \mu)$. So Theorem 60 applies with the ratio

$$\left| \frac{1/(\lambda_K - \mu)}{1/(\lambda_J - \mu)} \right| = \left| \frac{\lambda_J - \mu}{\lambda_K - \mu} \right|.$$

For the eigenvalue, $\lambda^{(k)}$ is the Rayleigh quotient of A itself, so Lemma 59 squares the vector error. □

The role of μ. The closer μ is to λ_J, the smaller the ratio: *the rate is controlled by our choice of μ.* For example, $|\lambda_J - \mu| = 0.01$ and $|\lambda_K - \mu| = 1$ give a factor 100 per step. Where does a good μ come from? The next section takes μ to be the current eigenvalue estimate.

10.7 Rayleigh Quotient Iteration

In some problems we know a good μ in advance. But what if we don't? The idea of Rayleigh quotient iteration (RQI) is to *choose μ to be our present guess of the eigenvalue*, that is, the Rayleigh quotient of the current vector.

Algorithm 17 Rayleigh quotient iteration.

1: $\vec{q} \leftarrow$ random unit vector; $\mu \leftarrow \vec{q}^* A \vec{q}$
2: **for** $k = 1, 2, \ldots, k_{\max}$ **do**
3: solve $(A - \mu I)\vec{z} = \vec{q}$
4: $\vec{q} \leftarrow \vec{z} / \|\vec{z}\|$
5: $\mu \leftarrow \vec{q}^* A \vec{q}$

The two updates help each other.

- A better $\vec{q}$ gives a better Rayleigh quotient μ.
- A better μ gives a smaller inverse iteration ratio $|\lambda_J - \mu| \,/\, |\lambda_K - \mu|$, so the next $\vec{q}$ improves more.

To see how fast this goes, we first ask how good the Rayleigh quotient is.

10.7.1 The Rayleigh Quotient Near an Eigenvector

Let $A \in \mathbb{R}^{m \times m}$ be symmetric. We define the Rayleigh quotient as

$$r(\vec{x}) = \frac{\vec{x}^\top A \vec{x}}{\vec{x}^\top \vec{x}}, \qquad \vec{x} \neq \vec{0}.$$

In inverse iteration and RQI, the eigenvalue estimate is $\mu_k = r(\vec{q}^{(k)})$.

For an eigenpair $(\lambda_J, \vec{x}_J)$, we have $r(\vec{x}_J) = \lambda_J$. So $\vec{y} \approx \vec{x}_J$ implies $r(\vec{y}) \approx \lambda_J$. How close? We compute the Taylor expansion around $\vec{x}_J$:

$$r(\vec{y}) = r(\vec{x}_J) + \nabla r(\vec{x}_J)^\top (\vec{y} - \vec{x}_J) + \tfrac{1}{2}(\vec{y} - \vec{x}_J)^\top H(\vec{x}_J)(\vec{y} - \vec{x}_J) + O(\|\vec{y} - \vec{x}_J\|^3), \tag{10.20}$$

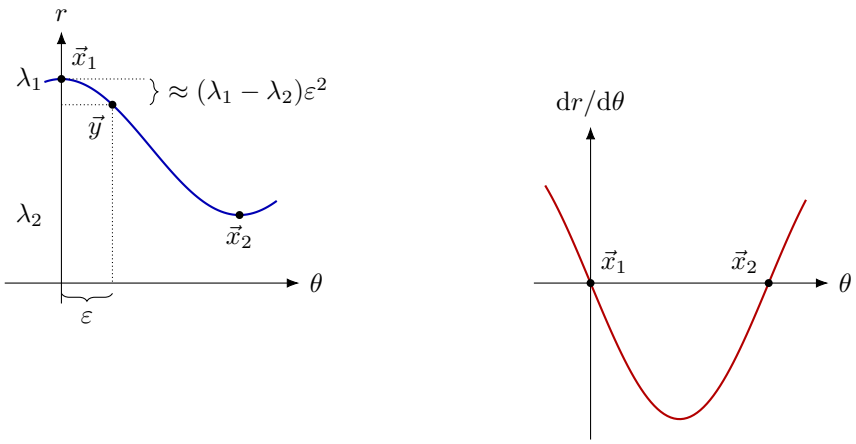
where H is the Hessian of r. By the quotient rule, the gradient is

$$\nabla r(\vec{x}) = \frac{2A\vec{x}\,(\vec{x}^\top \vec{x}) - 2\vec{x}\,(\vec{x}^\top A \vec{x})}{(\vec{x}^\top \vec{x})^2} = \frac{2}{\vec{x}^\top \vec{x}}(A\vec{x} - r(\vec{x})\vec{x}). \tag{10.21}$$

At an eigenvector, $A\vec{x}_J = \lambda_J \vec{x}_J = r(\vec{x}_J)\vec{x}_J$, so $\nabla r(\vec{x}_J) = \vec{0}$. The linear term drops out, and

$$r(\vec{y}) = \lambda_J + O(\|\vec{y} - \vec{x}_J\|^2). \tag{10.22}$$

In fact, $\nabla r(\vec{x}) = \vec{0}$ exactly when $\vec{x}$ is an eigenvector: *the eigenvectors are the critical points of r*. Lemma 59 gives the same result for complex Hermitian A, with an explicit constant.



The Rayleigh quotient on the unit circle $\vec{q} = \cos\theta\,\vec{x}_1 + \sin\theta\,\vec{x}_2$, for $\lambda_1 = 3$, $\lambda_2 = 1$, is $r(\theta) = \lambda_2 + (\lambda_1 - \lambda_2)\cos^2\theta$. The derivative is zero at the eigenvectors. So an angle error ε moves r by only $(\lambda_1 - \lambda_2)\sin^2\varepsilon \approx (\lambda_1 - \lambda_2)\varepsilon^2$.

This is the good news: *getting $\vec{q}$ close to $\vec{x}_J$ gets μ even closer to λ_J.* A vector error ε gives an eigenvalue error $O(\varepsilon^2)$. RQI is similar to Newton's method (see the supplement).

10.7.2 Cubic Convergence

We now bound one step of RQI,

$$\vec{q}^{(k+1)} = \frac{(A - \mu I)^{-1}\vec{q}^{(k)}}{\left\|(A - \mu I)^{-1}\vec{q}^{(k)}\right\|}, \qquad \mu = r(\vec{q}^{(k)}).$$

Assume $\left\|\vec{q}^{(k)} - \vec{x}_J\right\| \le \varepsilon$ with ε small. The argument has two parts.

- One step of inverse iteration gives $\left\|\vec{q}^{(k+1)} - \vec{x}_J\right\| = O(\varepsilon\,|\lambda_J - \mu|)$. The normalization does not change this order, by the estimate
$$\frac{1}{a+\varepsilon} - \frac{1}{a} \approx -\frac{\varepsilon}{a^2}:$$
dividing by a length that changes by a small amount changes the result only by the same order.
- The Rayleigh quotient gives $|\lambda_J - \mu| = O(\varepsilon^2)$.

Together, $\left\|\vec{q}^{(k+1)} - \vec{x}_J\right\| = O(\varepsilon^3)$: *cubic convergence*. The number of correct digits roughly triples at every step, for example $10^{-2} \to 10^{-6} \to 10^{-18}$. This is a *local* result: it applies only once $\vec{q}^{(k)}$ is close enough to some eigenvector.

The proposition below makes one step precise. Since $s\vec{x}_J$ is again a unit eigenvector for λ_J, it applies at every step. With $\varepsilon_k = \left\|\vec{q}^{(k)} - s_k\vec{x}_J\right\|$ and $C = 4s_A/g$, it gives

$$\varepsilon_{k+1} \le C\varepsilon_k^3, \qquad \left|\mu^{(k)} - \lambda_J\right| \le s_A\varepsilon_k^2.$$

Proposition 62 (One Step of RQI)

Let A be Hermitian with orthonormal eigenvectors $\vec{x}_1, \dots, \vec{x}_m$, and let λ_J be a simple eigenvalue. Define the gap and the spread

$$g := \min_{j \neq J} |\lambda_j - \lambda_J| > 0, \qquad s_A := \lambda_{\max} - \lambda_{\min}.$$

Let $\vec{q}$ be a unit vector with

$$\|\vec{q} - \vec{x}_J\| \leq \varepsilon, \qquad \varepsilon \leq \tfrac{1}{2}, \qquad s_A \varepsilon^2 \leq \tfrac{g}{2}.$$

Take one RQI step from $\vec{q}$:

$$\mu = r(\vec{q}) \neq \lambda_J, \qquad \vec{q}^{+} = \frac{(A - \mu I)^{-1}\vec{q}}{\|(A - \mu I)^{-1}\vec{q}\|}.$$

Then there is a unimodular s with

$$\left\|\vec{q}^{+} - s\vec{x}_J\right\| \leq \frac{4 s_A}{g}\varepsilon^3. \tag{10.23}$$

Proof. Expand. Write

$$\vec{q} = \alpha \vec{x}_J + \beta \vec{v}, \qquad \alpha = \langle \vec{x}_J, \vec{q} \rangle, \quad \|\vec{v}\| = 1, \quad \vec{v} \perp \vec{x}_J.$$

By (10.13) and since $\varepsilon \leq \frac{1}{2}$,

$$|\beta| \leq \|\vec{q} - \vec{x}_J\| \leq \varepsilon, \qquad |\alpha - 1| = |\langle \vec{x}_J, \vec{q} - \vec{x}_J \rangle| \leq \varepsilon \implies |\alpha| \geq \tfrac{1}{2}.$$

Apply $(A - \mu I)^{-1}$. Since $\vec{x}_J$ is an eigenvector of $(A - \mu I)^{-1}$ with eigenvalue $1/(\lambda_J - \mu)$,

$$(A - \mu I)^{-1}\vec{q} = \frac{\alpha}{\lambda_J - \mu}\vec{x}_J + \beta (A - \mu I)^{-1}\vec{v} = \frac{\alpha}{\lambda_J - \mu}(\vec{x}_J + \vec{d}),$$

where

$$\vec{d} = \frac{\beta(\lambda_J - \mu)}{\alpha}(A - \mu I)^{-1}\vec{v} \perp \vec{x}_J.$$

To bound $\vec{d}$, we bound each factor. By Lemma 59, $|\lambda_J - \mu| \leq s_A \varepsilon^2 \leq g/2$. So for every $j \neq J$,

$$|\lambda_j - \mu| \geq |\lambda_j - \lambda_J| - |\lambda_J - \mu| \geq g - \tfrac{g}{2} = \tfrac{g}{2}.$$

Since $\vec{v} \perp \vec{x}_J$, only these eigenvalues act on $\vec{v}$, and $\|(A - \mu I)^{-1}\vec{v}\| \leq 2/g$. Hence

$$\left\|\vec{d}\right\| \leq \frac{\varepsilon}{1/2} \cdot s_A \varepsilon^2 \cdot \frac{2}{g} = \frac{4 s_A}{g}\varepsilon^3. \tag{10.24}$$

Normalize. Let s be the phase of $\alpha/(\lambda_J - \mu)$. Then

$$\vec{q}^{+} = s\vec{u}, \qquad \vec{u} = \frac{\vec{x}_J + \vec{d}}{n}, \qquad n = \left\|\vec{x}_J + \vec{d}\right\| = \sqrt{1 + \left\|\vec{d}\right\|^2},$$

where the last equality uses $\vec{d} \perp \vec{x}_J$. Hence

$$\|\vec{u} - \vec{x}_J\|^2 = \left(1 - \frac{1}{n}\right)^2 + \frac{\left\|\vec{d}\right\|^2}{n^2} = 2 - \frac{2}{n} \leq \left\|\vec{d}\right\|^2,$$

where the last step uses $1/\sqrt{1+t} \geq 1 - t/2$. So $\|\vec{q}^{+} - s\vec{x}_J\| = \|\vec{u} - \vec{x}_J\| \leq \left\|\vec{d}\right\|$. □

10.8 (SUPPLEMENT) Further Topics

10.8.1 Implementing Inverse Iteration

Factor once. Since μ is fixed, we compute the factorization once, outside the loop. If we solved from scratch in every step, each step would cost $O(m^3)$ instead of $O(m^2)$. (Rayleigh quotient iteration cannot do this, since its shift changes at every step.)

Ill-conditioning is harmless. The closer μ is to λ_J, the closer $A - \mu I$ is to singular, and the larger the error in the solve. Is this a problem?

No. This error lies almost entirely along $\vec{x}_J$, which is exactly the direction we want, and normalization removes it.

An exact zero pivot (when μ is exactly an eigenvalue) almost never happens in floating point. If it does, it is enough to perturb μ by about $\varepsilon_{\text{mach}}$.

10.8.2 RQI and Newton's Method

RQI is similar to Newton's method, and we can make this precise. An eigenpair is a root of

$$F(\vec{x}, \lambda) = \begin{bmatrix} (A - \lambda I)\vec{x} \\ \frac{1}{2}(1 - \vec{x}^\top \vec{x}) \end{bmatrix}, \qquad F'(\vec{x}, \lambda) = \begin{bmatrix} A - \lambda I & -\vec{x} \\ -\vec{x}^\top & 0 \end{bmatrix}. \tag{10.25}$$

The first block row of the Newton step $F'(\vec{x}, \lambda)(\Delta\vec{x}, \Delta\lambda) = -F(\vec{x}, \lambda)$ reads

$$(A - \lambda I)\Delta\vec{x} - \Delta\lambda\, \vec{x} = -(A - \lambda I)\vec{x},$$

i.e.,

$$\vec{x} + \Delta\vec{x} = \Delta\lambda\, (A - \lambda I)^{-1}\vec{x}. \tag{10.26}$$

So the Newton update of the vector is parallel to one step of inverse iteration with shift λ. RQI takes this direction and normalizes it. Then it replaces Newton's $\lambda + \Delta\lambda$ by the Rayleigh quotient of the new vector.

Newton's method converges quadratically, because the error of the linearization is quadratic. For symmetric A, the stationarity of r adds one more order.

10.8.3 Why Not Go Through the Characteristic Polynomial

Even in low degrees, where root formulas exist, we do not compute $\det(zI - A)$ and then its roots. The reason: going from a matrix to the coefficients of its polynomial is extremely ill-conditioned. Take the Wilkinson polynomial $\prod_{i=1}^{20}(z - i)$. If we change the coefficient of z^{19} by $2^{-23} \approx 10^{-7}$, several roots move into the complex plane, by $O(1)$. Yet the eigenvalues of $\mathrm{diag}(1, \ldots, 20)$ are perfectly well-conditioned under perturbations of the matrix. So all iterative methods of this lecture work directly on the matrix, using only matrix-vector products or linear solves. They use $p(A)\vec{x}_i = p(\lambda_i)\vec{x}_i$, and do not approximate roots of a polynomial.

Lecture 11

Schur Factorization

Supplementary reading:

- [1] Lecture 24.
- [2] Sections 4.2–4.3.
- [4] Sections 5A, 5C, 6B, 7B, and 8A–8C (linear algebra background).

11.1 Recap and Motivation

Lecture 10 gave methods that compute *one* eigenpair: power iteration, inverse iteration and Rayleigh quotient iteration. Now we want *all* eigenpairs: we ask how to compute them efficiently, and what single object should describe them.

The plan of this chapter:

1. *Deflation* (Section 11.2) removes an eigenpair we already know, so we can compute the eigenpairs one by one.
2. This is slow. Iterating on many vectors at once is faster (Section 11.3). So we want all eigenpairs as one matrix factorization.
3. The eigenvalue decomposition $A = X\Lambda X^{-1}$ does not always exist (Sections 11.4 and 11.5).
4. The Jordan form always exists, but it is unstable (Section 11.6).
5. The Schur factorization $A = QTQ^*$ always exists and is stable (Section 11.7). It is the right object.

11.2 Deflation

We have a fast way to compute one eigenpair. What if we want all of them?

Suppose we have computed the largest eigenpair $(\lambda_1, \vec{x}_1)$. If we run power iteration again, it finds $\vec{x}_1$ again. How do we avoid computing the same thing twice?

The idea: work with

$$B \coloneqq A - \lambda_1 \frac{\vec{x}_1 \vec{x}_1^*}{\vec{x}_1^* \vec{x}_1}. \tag{11.1}$$

It has the same eigenpairs as A, except that λ_1 becomes 0. This works when A is Hermitian.

Proposition 63 (Hotelling Deflation)

Let $A \in \mathbb{K}^{m \times m}$ be Hermitian with eigenpairs $(\lambda_i, \vec{x}_i)$, where the $\vec{x}_i$ are pairwise orthogonal. Then $B\vec{x}_1 = \vec{0}$ and $B\vec{x}_i = \lambda_i \vec{x}_i$ for $i = 2, \ldots, m$.

Proof. For $i = 1$: $B\vec{x}_1 = \lambda_1 \vec{x}_1 - \lambda_1 \vec{x}_1 \frac{\vec{x}_1^* \vec{x}_1}{\vec{x}_1^* \vec{x}_1} = \vec{0}$.

For $i \geq 2$: $\vec{x}_1^* \vec{x}_i = 0$, so the correction term vanishes, and $B\vec{x}_i = A\vec{x}_i = \lambda_i \vec{x}_i$. □

Power iteration on B now converges to the eigenpair with the next largest modulus. Repeating this gives all eigenpairs, one by one.

Two remarks.

- The proof needs orthogonal eigenvectors. Hermitian A has them (Proposition 55). For non-Hermitian A, $\vec{x}_1^* \vec{x}_i \neq 0$ in general, and the other eigenpairs change.
- We never form B; if A is sparse, B is dense. We compute $B\vec{z} = A\vec{z} - \lambda_1 \vec{x}_1 (\vec{x}_1^* \vec{z})/(\vec{x}_1^* \vec{x}_1)$, at the cost of one product with A plus $O(m)$.

11.3 Subspace Iteration: Motivation

So far we can compute single eigenpairs. With deflation we can, in principle, compute them all one by one. The problem is speed.

- N eigenpairs need at least N matrix-vector products (in practice many more).
- These products happen *serially*: we cannot start eigenpair $k+1$ before eigenpair k is done.
- Each product loads all of A from RAM to do only $2m^2$ flops. Its intensity is $q \approx 2$ (Section 2.9.2). Memory bandwidth limits it.

So we load the matrix from RAM at least N times.

The alternative: run power iteration on *many vectors at the same time*. For $Z \in \mathbb{K}^{m \times n}$, the product AZ does $2m^2 n$ flops but reads A once. Matrix-vector products become matrix-matrix products, which run near peak speed (Lecture 2). This is *subspace iteration*.

We must keep the columns independent. Otherwise every column converges to $\vec{x}_1$. So we orthonormalize them after each step, for example by a QR factorization (Lecture 12).

What should such an iteration converge to? To answer this, we first describe all eigenpairs as one matrix factorization.

11.4 Eigenvalue Problems as Matrix Factorizations

Proposition 64 (Distinct Eigenvalues Give an Eigenvalue Decomposition)

If $A \in \mathbb{K}^{m\times m}$ has m distinct eigenvalues $\lambda_1, \dots, \lambda_m$ with eigenvectors $\vec{x}_1, \dots, \vec{x}_m$, then

$$A = X\Lambda X^{-1}, \qquad X = \begin{bmatrix} \vec{x}_1 & \cdots & \vec{x}_m \end{bmatrix}, \quad \Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_m). \tag{11.2}$$

Proof. Column i of AX is $A\vec{x}_i$. Column i of $X\Lambda$ is $\lambda_i\vec{x}_i$. So the m equations $A\vec{x}_i = \lambda_i\vec{x}_i$ say $AX = X\Lambda$. Eigenvectors for distinct eigenvalues are linearly independent (Lemma 76). So X is invertible, and $A = X\Lambda X^{-1}$. □

So computing all eigenpairs means computing the factorization $A = X\Lambda X^{-1}$.

For self-adjoint matrices ($A^* = A$, i.e., Hermitian), we can do better: X can be chosen orthogonal.

Theorem 65 (Spectral Theorem)

If $A \in \mathbb{K}^{m\times m}$ is self-adjoint, there is a unitary X with

$$A = X\Lambda X^*, \tag{11.3}$$

and Λ is real [1, Thm. 24.7], [4, 7.29, 7.31].

Then $X^{-1} = X^*$ is free, and $\kappa_2(X) = 1$: changing to eigenvector coordinates does not amplify errors. The proof follows from the Schur factorization (Corollary 72).

11.5 Defective Matrices and Invariant Subspaces

Are all matrices diagonalizable? No. Consider

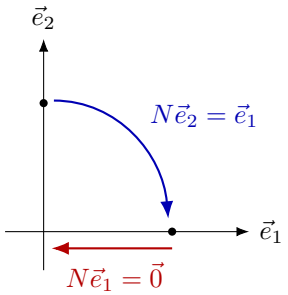
$$J = \begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}. \tag{11.4}$$

What are its eigenpairs?

- J is triangular, so its only eigenvalue is λ.
- $(J - \lambda I)\vec{x} = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]\vec{x} = (x_2, 0)^\top$. This is zero only if $x_2 = 0$.

So $(\lambda, \vec{e}_1)$ is the only eigenpair, up to scaling. We need two independent eigenvectors to build X, but we have only one. Such matrices are called *defective*. (For the precise definition via algebraic and geometric multiplicity, see Section 11.9.2.)

To see what happens, take $\lambda = 0$: $N = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$. Then $N\vec{e}_2 = \vec{e}_1$ and $N\vec{e}_1 = \vec{0}$. So N pushes $\vec{e}_2$ to $\vec{e}_1$, and $\vec{e}_1$ to $\vec{0}$. Only $\vec{e}_1$ is mapped to a multiple of itself.



The nilpotent matrix N maps $\vec{e}_2$ to $\vec{e}_1$ and $\vec{e}_1$ to $\vec{0}$. Only $\operatorname{span}(\vec{e}_1)$ is an eigenspace.

Intuition. Still, all vectors show "λ-growth" under J. For example, $J\vec{e}_2 = \lambda\vec{e}_2 + \vec{e}_1$. It is not a multiple of $\vec{e}_2$, but it stays in $\operatorname{span}(\vec{e}_1, \vec{e}_2)$, and the diagonal of J is still λ. (Concretely, $J^k = \begin{bmatrix} \lambda^k & k\lambda^{k-1} \\ 0 & \lambda^k \end{bmatrix}$ grows like $|\lambda|^k$, up to a factor k.)

Idea. Generalize the λ-eigenspace to an *invariant subspace with λ-growth*: a subspace that A maps into itself. On an eigenspace, A acts as λI. On an invariant subspace, A may act as any matrix B.

Definition 66 (Invariant Subspace)

Let $X = [\vec{x}_1 \ \cdots \ \vec{x}_n] \in \mathbb{K}^{m\times n}$ have linearly independent columns. The space $\mathcal{R}(X)$ spanned by its columns is *A-invariant* if and only if there is $B \in \mathbb{K}^{n\times n}$ with

$$AX = XB. \tag{11.5}$$

How to read it:

- Column j says $A\vec{x}_j = X\vec{b}_j$. So A maps every basis vector back into $\mathcal{R}(X)$. Equivalently, $A\,\mathcal{R}(X) \subseteq \mathcal{R}(X)$.
- B is the matrix of A on this subspace, in the basis X.
- Special cases: an eigenpair ($n = 1$, $B = \lambda$); a diagonalizable A (X all eigenvectors, $B = \Lambda$); the matrix J ($X = I$, $B = J$).

Proposition 67 (Eigenvalues on an Invariant Subspace)

Let $AX = XB$ with X of full column rank. Then every eigenvalue of B is an eigenvalue of A. If the columns of $X = Q$ are orthonormal, then $B = Q^*AQ$ [2, Prop. 4.3].

Proof. If $B\vec{v} = \lambda\vec{v}$ with $\vec{v} \neq \vec{0}$, then $A(X\vec{v}) = XB\vec{v} = \lambda X\vec{v}$, and $X\vec{v} \neq \vec{0}$. If $Q^*Q = I$, multiply $AQ = QB$ by Q^* on the left. □

11.6 The Jordan Normal Form

If we split the whole space into invariant subspaces of this kind, we get the Jordan normal form.

Theorem 68 (Jordan Normal Form)

For every $A \in \mathbb{K}^{m\times m}$ there is a nonsingular $S \in \mathbb{C}^{m\times m}$ with

$$A = SJS^{-1}, \qquad J = \begin{bmatrix} J_1 & & \\ & \ddots & \\ & & J_k \end{bmatrix}, \qquad J_\ell = \begin{bmatrix} \lambda_\ell & 1 & & \\ & \lambda_\ell & \ddots & \\ & & \ddots & 1 \\ & & & \lambda_\ell \end{bmatrix} \in \mathbb{C}^{n_\ell \times n_\ell}. \tag{11.6}$$

J is unique up to the order of the blocks [2, Thm. 4.1], [4, 8.46].

The Jordan form decomposes the space into A-invariant subspaces. Split $S = [S_1 \ \cdots \ S_k]$ to match the blocks. Then $AS_\ell = S_\ell J_\ell$: each block is an invariant subspace with $B = J_\ell$. A is diagonalizable exactly when all blocks are 1×1.

Proposition 69 (One Eigenvector per Jordan Block)

Every Jordan block J_ℓ has, up to scaling, only one eigenvector, $\vec{e}_1$ [2, Prop. 4.2].

Proof. $J_\ell - \lambda_\ell I$ shifts the entries up: $(J_\ell - \lambda_\ell I)\vec{x} = (x_2, \ldots, x_{n_\ell}, 0)^\top$. This is zero only if $\vec{x} \in \operatorname{span}(\vec{e}_1)$. □

Problem. The Jordan form is not (numerically) stable, for two reasons.

First, it is discontinuous in A.

Example 70 (A Tiny Perturbation Changes the Jordan Form)**.** Perturb $N = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$ to $A_\varepsilon = \left[\begin{smallmatrix} \varepsilon & 1 \\ 0 & 2\varepsilon \end{smallmatrix}\right]$, $\varepsilon > 0$. Its eigenvalues $\varepsilon, 2\varepsilon$ are distinct, with eigenvectors $\vec{e}_1$ and $(1, \varepsilon)^\top$. So

$$A_\varepsilon = S_\varepsilon \begin{bmatrix} \varepsilon & 0 \\ 0 & 2\varepsilon \end{bmatrix} S_\varepsilon^{-1}, \qquad S_\varepsilon = \begin{bmatrix} 1 & 1 \\ 0 & \varepsilon \end{bmatrix}. \tag{11.7}$$

As $\varepsilon \to 0$, $A_\varepsilon \to N$, but its Jordan form $\operatorname{diag}(\varepsilon, 2\varepsilon) \to 0 \neq N$.

In floating point, perturbations of size $\varepsilon_{\text{mach}} \|A\|$ are always present. So no computation can decide whether A has a nontrivial Jordan block.

Second, S can be arbitrarily ill-conditioned. The columns of S_ε are nearly parallel, and $\kappa_2(S_\varepsilon) \approx 2/\varepsilon$; no rescaling of the columns helps (Section 11.9.4). Even if we computed J with only a tiny error δJ, the backward error $\delta A = S\,\delta J\,S^{-1}$ satisfies

$$\|\delta A\| \leq \kappa(S)\,\|\delta J\|, \tag{11.8}$$

which can be much larger than $\varepsilon_{\text{mach}} \|A\|$. So the Jordan form cannot be computed in a backward stable way [2, Example 4.3].

So we should only use *unitary* similarity transformations, with $\kappa_2 = 1$. We cannot reach a form as simple as J that way. But we get something almost as good.

11.7 The Schur Factorization

Theorem 71 (Schur Factorization)

For every $A \in \mathbb{K}^{m\times m}$ there exist a unitary Q and an upper triangular T with

$$A = QTQ^*. \tag{11.9}$$

The eigenvalues of A are the diagonal entries of T [1, Thm. 24.9], [2, Thm. 4.2], [4, 6.38].

Compared with the other two factorizations:

- unlike $X\Lambda X^{-1}$, it exists for *every* matrix, defective or not;
- unlike SJS^{-1}, it uses a unitary Q, so it is stable;
- the price: T is only triangular, not diagonal.

Proof. We use the fact that every matrix has at least one eigenvalue. Induction on m; for $m = 1$ take $Q = [1]$.

Let $(\lambda, \vec{u})$ be an eigenpair with $\|\vec{u}\| = 1$. Extend $\vec{u}$ to a square unitary $U = [\vec{u}\ \tilde{U}]$. Then

$$U^*AU = \begin{bmatrix} \vec{u}^*A\vec{u} & \vec{u}^*A\tilde{U} \\ \tilde{U}^*A\vec{u} & \tilde{U}^*A\tilde{U} \end{bmatrix} = \begin{bmatrix} \lambda & \vec{u}^*A\tilde{U} \\ \vec{0} & \tilde{U}^*A\tilde{U} \end{bmatrix}, \tag{11.10}$$

since $A\vec{u} = \lambda\vec{u}$, $\vec{u}^*\vec{u} = 1$ and $\tilde{U}^*\vec{u} = \vec{0}$.

Then recurse on the bottom right part. By induction, $\tilde{U}^*A\tilde{U} = \tilde{Q}\tilde{T}\tilde{Q}^*$. With $Q := U\,\mathrm{diag}(1, \tilde{Q})$,

$$Q^*AQ = \begin{bmatrix} \lambda & \vec{u}^*A\tilde{U}\tilde{Q} \\ \vec{0} & \tilde{T} \end{bmatrix} \tag{11.11}$$

is upper triangular. We call it T. A and T are similar, so they have the same eigenvalues: the diagonal of T. □

The first step uses the fundamental theorem of algebra, which is where $\mathbb{C}$ enters. A real matrix may have no real eigenvalue (a rotation of the plane). So Q and T are complex in general, even for real A.

For Hermitian A, the spectral theorem now follows in two lines.

Corollary 72 (Normal Matrices and the Spectral Theorem)**.** *If A is normal ($A^*A = AA^*$), then the triangular factor T of any Schur factorization of A is diagonal, so $A = Q\Lambda Q^*$. If A is Hermitian, Λ is also real. In particular, Theorem 65 holds [1, Thm. 24.8].*

Proof. Hermitian case. $T^* = Q^*A^*Q = Q^*AQ = T$. A Hermitian upper triangular matrix is diagonal, with real diagonal.

Normal case. See Theorem 81 in Lecture 12. □

11.7.1 The Real Schur Form

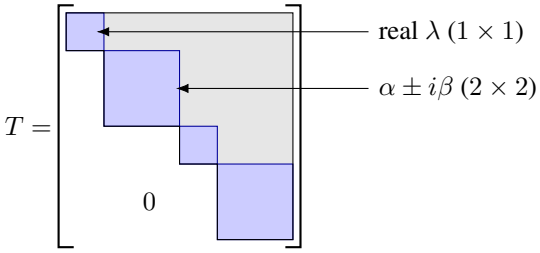
When we work with real matrices, we prefer to stay with real numbers. Complex arithmetic costs more, and rounding would break the exact conjugate symmetry of the eigenvalues.

Theorem 73 (Real Schur Form)

If $A \in \mathbb{R}^{m\times m}$, there exist an orthogonal Q and a block upper triangular T with 1×1 and 2×2 diagonal blocks such that

$$A = QTQ^\top. \tag{11.12}$$

The 1×1 blocks are the real eigenvalues of A. Each 2×2 block holds a pair of complex conjugate eigenvalues [2, Thm. 4.3].



The quasi-upper triangular factor of the real Schur form. Shaded entries may be nonzero.

Key idea. The non-real eigenvalues of a real matrix appear in complex conjugate pairs. Each pair gives a *real* two-dimensional invariant subspace:

Lemma 74 (A Conjugate Pair Gives a Real Invariant Plane)

Let $A \in \mathbb{R}^{m\times m}$ and $A\vec{x} = \lambda\vec{x}$ with $\lambda = \alpha + i\beta$, $\beta \neq 0$, $\vec{x} = \vec{a} + i\vec{b}$ ($\vec{a}, \vec{b}$ real). Then $\vec{a}, \vec{b}$ are linearly independent and

$$A\begin{bmatrix}\vec{a} & \vec{b}\end{bmatrix} = \begin{bmatrix}\vec{a} & \vec{b}\end{bmatrix}\begin{bmatrix}\alpha & \beta\\ -\beta & \alpha\end{bmatrix}. \tag{11.13}$$

The 2×2 matrix has eigenvalues $\alpha \pm i\beta$.

Proof. Take real and imaginary parts of $A(\vec{a} + i\vec{b}) = (\alpha + i\beta)(\vec{a} + i\vec{b})$: $A\vec{a} = \alpha\vec{a} - \beta\vec{b}$ and $A\vec{b} = \beta\vec{a} + \alpha\vec{b}$. If $\vec{a}, \vec{b}$ were dependent, $\vec{x}$ would be a complex multiple of a real vector $\vec{r}$, and $A\vec{r} = \lambda\vec{r}$ would force λ to be real. □

Proof of Theorem 73. Same induction as for Theorem 71, in real arithmetic. A real eigenvalue gives a 1×1 block as before. Otherwise, orthonormalize $[\vec{a}\ \vec{b}]$ from Lemma 74 to $\hat{Q} \in \mathbb{R}^{m\times 2}$. Its span is invariant, so $A\hat{Q} = \hat{Q}\hat{B}$ with $\hat{B} = \hat{Q}^\top A\hat{Q}$ (Proposition 67). Extend to an orthogonal $U = [\hat{Q}\ \tilde{Q}]$. Then $\tilde{Q}^\top A\hat{Q} = \tilde{Q}^\top\hat{Q}\hat{B} = 0$, so

$$U^\top AU = \begin{bmatrix}\hat{B} & \hat{Q}^\top A\tilde{Q}\\ 0 & \tilde{Q}^\top A\tilde{Q}\end{bmatrix}, \tag{11.14}$$

and we recurse on $\tilde{Q}^\top A\tilde{Q}$. □

11.8 Computing Eigenvectors from the Schur Form

The Schur factorization gives the eigenvalues. What about the eigenvectors? With $A = QTQ^*$,

$$T\vec{x} = \lambda\vec{x} \iff (QTQ^*)(Q\vec{x}) = \lambda Q\vec{x}. \tag{11.15}$$

So we need the eigenvectors of the triangular matrix T, and then multiply by Q.

Let $\lambda = t_{kk}$ be a simple eigenvalue. Partition $T - \lambda I$ around row and column k, and solve $(T - \lambda I)\vec{x} = \vec{0}$:

$$\vec{0} = \begin{bmatrix} T_{11} - \lambda I & \vec{t}_{12} & T_{13} \\ & 0 & \vec{t}_{23}^* \\ & & T_{33} - \lambda I \end{bmatrix} \begin{bmatrix} \vec{x}_1 \\ x_2 \\ \vec{x}_3 \end{bmatrix} = \begin{bmatrix} (T_{11} - \lambda I)\vec{x}_1 + \vec{t}_{12}x_2 + T_{13}\vec{x}_3 \\ \vec{t}_{23}^*\vec{x}_3 \\ (T_{33} - \lambda I)\vec{x}_3 \end{bmatrix}. \tag{11.16}$$

Since λ is simple, $T_{11} - \lambda I$ and $T_{33} - \lambda I$ are nonsingular. We solve from the bottom up:

- the third row gives $\vec{x}_3 = \vec{0}$, and then the second row holds;
- x_2 is free; we choose $x_2 = 1$;
- the first row gives $\vec{x}_1 = (\lambda I - T_{11})^{-1}\vec{t}_{12}$.

Proposition 75 (Eigenvectors of a Triangular Matrix)

For a simple eigenvalue $\lambda = t_{kk}$ of the upper triangular T,

$$\vec{x} = \begin{bmatrix} (\lambda I - T_{11})^{-1}\vec{t}_{12} \\ 1 \\ \vec{0} \end{bmatrix} \tag{11.17}$$

is an eigenvector of T, and $Q\vec{x}$ is an eigenvector of A [2, §4.2.1].

Cost. One triangular solve of size $k - 1$, about $(k - 1)^2$ flops, plus $2mk$ flops for $Q\vec{x}$. For all k, this is $O(m^3)$, the same order as the Schur form itself. For $k = 1$, $\vec{x} = \vec{e}_1$: the first Schur vector $\vec{q}_1$ is always an eigenvector.

Caveats.

- If λ is repeated on the diagonal of T_{11}, then $T_{11} - \lambda I$ is singular. This is the defective case again.
- If λ is close to another diagonal entry of T_{11}, then $(\lambda I - T_{11})^{-1}$ is large and $\vec{x}$ is sensitive. This is not a flaw of the method: the eigenvector itself is ill-conditioned.

11.9 (SUPPLEMENT) Further Topics

11.9.1 Eigenvectors for Distinct Eigenvalues Are Independent

Lemma 76 (Eigenvectors for Distinct Eigenvalues)

Eigenvectors $\vec{x}_1, \dots, \vec{x}_k$ of A corresponding to distinct eigenvalues $\lambda_1, \dots, \lambda_k$ are linearly independent [4, 5.11].

Proof. Suppose not. Among all vanishing combinations with nonzero coefficients, pick one with the fewest terms: $\sum_{i \in S} c_i \vec{x}_i = \vec{0}$, all $c_i \neq 0$. Then S has at least two elements, since eigenvectors are nonzero. Fix $j \in S$ and apply $A - \lambda_j I$:

$$\sum_{i \in S \setminus \{j\}} c_i (\lambda_i - \lambda_j) \vec{x}_i = \vec{0}. \tag{11.18}$$

The term $i = j$ dropped out. All remaining coefficients are nonzero, since $\lambda_i \neq \lambda_j$. This is a shorter vanishing combination, a contradiction. □

11.9.2 Algebraic and Geometric Multiplicity

This subsection follows [1, Lecture 24]. Throughout, $A \in \mathbb{C}^{m \times m}$.

- The *eigenspace* $E_\lambda = \mathcal{N}(A - \lambda I)$ is an invariant subspace. Its dimension is the *geometric multiplicity* of λ: the largest number of independent eigenvectors for λ.
- The *characteristic polynomial* $p_A(z) = \det(zI - A)$ is monic of degree m. λ is an eigenvalue if and only if $\lambda I - A$ is singular, i.e., $p_A(\lambda) = 0$. So a real matrix may have complex eigenvalues.
- By the fundamental theorem of algebra, $p_A(z) = (z - \lambda_1) \cdots (z - \lambda_m)$. The multiplicity of λ as a root is its *algebraic multiplicity*. So A has m eigenvalues, counted with algebraic multiplicity.
- Similar matrices A and $X^{-1}AX$ have the same characteristic polynomial, since $\det(zI - X^{-1}AX) = \det(X^{-1}) \det(zI - A) \det(X)$. They also have the same geometric multiplicities, since X^{-1} maps $E_\lambda(A)$ onto $E_\lambda(X^{-1}AX)$.

Definition 77 (Defective)

An eigenvalue whose algebraic multiplicity exceeds its geometric multiplicity is *defective*. A matrix with a defective eigenvalue is a *defective matrix*.

Example [1, Ex. 24.1]: $2I_3$ and $\begin{bmatrix} 2 & 1 & \\ & 2 & 1 \\ & & 2 \end{bmatrix}$ both have $p(z) = (z-2)^3$. The first has three independent eigenvectors; the second only $\vec{e}_1$, so it is defective.

Theorem 78 ([1, Thm. 24.4])

The algebraic multiplicity of an eigenvalue is at least its geometric multiplicity.

Proof. Let $n = \dim E_\lambda$, and let $\hat{V}$ hold an orthonormal basis of E_λ. Extend to a unitary $V = [\hat{V}\ \tilde{V}]$. Since

$A\hat{V} = \lambda\hat{V}$,

$$V^*AV = \begin{bmatrix} \lambda I_n & C \\ 0 & D \end{bmatrix}, \qquad \det(zI - V^*AV) = (z-\lambda)^n \det(zI - D). \tag{11.19}$$

So λ is a root of p_A of multiplicity at least n. □

Theorem 79 ([1, Thm. 24.5])

A has an eigenvalue decomposition $A = X\Lambda X^{-1}$ if and only if A is nondefective.

Proof. ($\Rightarrow$) A is similar to Λ, so it has the same multiplicities, and a diagonal matrix is nondefective.

($\Leftarrow$) Take a basis of each eigenspace. Their sizes add up to the algebraic multiplicities, so to m. Together they are independent: group a vanishing combination by eigenvalue as $\vec{u}_1 + \cdots + \vec{u}_r = \vec{0}$ with $\vec{u}_s \in E_{\mu_s}$; by Lemma 76 every $\vec{u}_s = \vec{0}$, so all coefficients vanish. These m vectors form an invertible X with $AX = X\Lambda$. □

Another word for nondefective is *diagonalizable*.

11.9.3 Powers on an Invariant Subspace

From $AX = XB$ we get $A^kX = XB^k$. So on an invariant subspace, powers of A (and power iteration) behave like powers of B. For the 2×2 block, $J = \lambda I + N$ with $N^2 = 0$, and the binomial formula gives $J^k = \lambda^k I + k\lambda^{k-1}N$. Even for $|\lambda| < 1$, $\left\|J^k\right\|$ can first grow before it decays (see the supplement of Lecture 10).

11.9.4 No Column Scaling Makes S_ε Well-Conditioned

Normalize the columns of S_ε to unit vectors $\vec{u}, \vec{v}$ with angle θ, $\tan\theta = \varepsilon$. Then $\kappa_2 = \cot(\theta/2) \approx 2/\varepsilon$. For any scaling $SD = [a\vec{u}\ b\vec{v}]$:

- with $\vec{w} = (1/a, -1/b)^\top$ we get $SD\vec{w} = \vec{u} - \vec{v}$ and $\|\vec{w}\| \geq 1/\min(|a|, |b|)$, so

$$\sigma_{\min}(SD) \leq \min(|a|, |b|)\, \|\vec{u} - \vec{v}\|;$$

- $\sigma_{\max}(SD) \geq \max(|a|, |b|)$.

Hence $\kappa_2(SD) \geq 1/\|\vec{u} - \vec{v}\| = 1/(2\sin(\theta/2)) \approx 1/\varepsilon$. Since A_ε is diagonalizable, this also shows that eigenvector matrices of matrices close to defective ones are badly conditioned.

11.9.5 Schur versus Jordan

Both forms exist for every matrix. Jordan uses an S that may be ill-conditioned, and it is discontinuous. Schur uses a unitary Q with $\kappa_2(Q) = 1$. The price is that T is only triangular: its strictly upper part may be dense.

11.9.6 Nested Invariant Subspaces

We write $AQ = QT$ and keep the first k columns. Since T is upper triangular, this gives $AQ_{:,1:k} = Q_{:,1:k}T_{1:k,1:k}$. So $\operatorname{span}(\vec{q}_1) \subset \operatorname{span}(\vec{q}_1, \vec{q}_2) \subset \cdots$ is a nested sequence of invariant subspaces (Definition 66), with orthonormal bases. Conversely, if all leading spans of a basis are invariant, the basis gives an upper triangular matrix [4, 5.39]. This is the goal of subspace iteration with orthonormalization (Section 11.3).

11.9.7 How Ill-Conditioned Eigenvalues Near a Jordan Block Are

In Example 70, the eigenvalues moved by only $O(\varepsilon)$, while the Jordan structure and S broke down. A different perturbation moves the eigenvalues themselves much more. Put ε in the bottom left corner of an $n \times n$ Jordan block with eigenvalue 0. The result is the companion matrix (Lemma 57) of $z^n - \varepsilon$. Its eigenvalues are the n complex n-th roots of ε, all of modulus $\varepsilon^{1/n}$ [2, Example 4.4]. For $n = 10$ and $\varepsilon = 10^{-10}$, a perturbation of size 10^{-10} moves the eigenvalues by 0.1. A multiple eigenvalue in a nontrivial Jordan block has infinite condition number.

11.9.8 The Bauer–Fike Theorem

We can make the role of $\kappa(X)$ quantitative. Let $A = X\Lambda X^{-1}$ be diagonalizable, and let μ be an eigenvalue of $A + E$. Then

$$\min_i |\mu - \lambda_i| \le \kappa_2(X)\, \|E\|_2 \,. \tag{11.20}$$

To see this, assume μ is not an eigenvalue of A. Since $A + E - \mu I$ is singular, so is $X^{-1}(A + E - \mu I)X = (\Lambda - \mu I) + X^{-1}EX$. Hence $I + (\Lambda - \mu I)^{-1}X^{-1}EX$ is also singular. A matrix $I + M$ can only be singular if $\|M\| \ge 1$. So

$$1 \le \left\|(\Lambda - \mu I)^{-1}\right\| \left\|X^{-1}\right\| \|E\| \, \|X\| = \frac{\kappa_2(X)\, \|E\|_2}{\min_i |\lambda_i - \mu|}. \tag{11.21}$$

For normal matrices, X can be unitary. Then $\kappa_2(X) = 1$, and the eigenvalues move by at most $\|E\|_2$. When A approaches a defective matrix, $\kappa_2(X) \to \infty$ and the bound becomes useless. This agrees with the $\varepsilon^{1/n}$ behavior above. See [2, §4.3].

Lecture 12

Orthogonal Iteration

Supplementary reading:

- [1] Lectures 24 and 28.
- [2] Section 4.4.3.

12.1 Recap and Motivation

Last lecture (Lecture 11) covered deflation, the need for subspace iteration (matrix-vector products become matrix-matrix products), eigenvalue problems as matrix factorizations $A = X\Lambda X^{-1}$, defective matrices and Jordan blocks, and the Schur factorization $A = QTQ^*$. Now we ask when T is diagonal, and we start computing several eigenpairs at once.

The plan of this chapter:

1. The Schur factorization is a diagonalization exactly for *normal* matrices (Section 12.2).
2. *Orthogonal iteration* with two vectors: power iteration plus one projection (Section 12.3).
3. The same with r vectors, using a QR factorization (Section 12.4).
4. The columns converge to the Q factor of the eigenvector matrix (Section 12.5).
5. With $r = m$, this gives the Schur factorization (Section 12.6).

12.2 From Schur Factorization to Diagonalization

The Schur factorization $A = QTQ^*$ always exists, but T is only upper triangular. When is it a diagonalization, that is, when is T diagonal?

Definition 80 (Normal Matrix)

$A \in \mathbb{K}^{m\times m}$ is *normal* if $AA^* = A^*A$.

Theorem 81 (Normal Matrices Are Unitarily Diagonalizable)

$A \in \mathbb{K}^{m\times m}$ is normal if and only if it has an orthogonal eigenvalue decomposition $A = Q\Lambda Q^*$ with Q unitary and Λ diagonal. In this case the triangular factor T of every Schur factorization $A = QTQ^*$ is diagonal.

Proof. If $A = Q\Lambda Q^*$, then $AA^* = Q\Lambda\overline{\Lambda}Q^* = Q\overline{\Lambda}\Lambda Q^* = A^*A$, since diagonal matrices commute.

Now let A be normal, and take a Schur factorization $A = QTQ^*$ (Theorem 71). Then

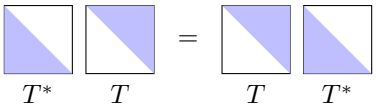
$$QTT^*Q^* = AA^* = A^*A = QT^*TQ^* \quad \Longrightarrow \quad TT^* = T^*T. \tag{12.1}$$

We compare the $(1,1)$ entries. The first column of T is $t_{11}\vec{e}_1$, so

$$|t_{11}|^2 = (T^*T)_{11} = (TT^*)_{11} = \sum_{i=1}^{m} |t_{1i}|^2 . \tag{12.2}$$

Hence $t_{1i} = 0$ for $i \geq 2$, and $T = \operatorname{diag}(t_{11}, T_2)$. The lower right block of $TT^* = T^*T$ says that T_2 is normal again. By induction on m, T is diagonal. □

In pictures: lower times upper equals upper times lower.



On the left, the $(1,1)$ entry uses only the first column of T, which is just t_{11}. On the right, it uses the whole first row. So the rest of the first row must be zero.

Hermitian, skew-Hermitian and unitary matrices are normal (Section 12.7.1).

12.3 Orthogonal Iteration

In this section, $A \in \mathbb{R}^{m\times m}$ is diagonalizable, with

$$|\lambda_1| > |\lambda_2| > \cdots > |\lambda_m| . \tag{12.3}$$

Non-real eigenvalues of a real matrix come in conjugate pairs of equal modulus. So all eigenvalues and eigenvectors here are real.

Assume we want to compute $r = 2$ eigenpairs at once. We apply power iteration to two vectors at once.

- *Problem.* Both vectors converge to the same eigenvector $\vec{x}_1$.
- *Idea.* Project $\vec{q}_2$ onto the orthogonal complement of $\vec{q}_1$.

Algorithm 18 Orthogonal iteration with two vectors.

1: $\vec{q}_1, \vec{q}_2 \leftarrow$ random vectors
2: **while** not converged **do**
3: $\quad \vec{q}_1 \leftarrow A\vec{q}_1, \quad \vec{q}_2 \leftarrow A\vec{q}_2$
4: $\quad \vec{q}_2 \leftarrow \left(I - \dfrac{\vec{q}_1\vec{q}_1^{\top}}{\vec{q}_1^{\top}\vec{q}_1}\right)\vec{q}_2$ ▷ project out $\vec{q}_1$
5: $\quad \vec{q}_1 \leftarrow \vec{q}_1/\|\vec{q}_1\|, \quad \vec{q}_2 \leftarrow \vec{q}_2/\|\vec{q}_2\|$

The last two lines are Gram–Schmidt on $[A\vec{q}_1\ A\vec{q}_2]$, i.e., a reduced QR factorization (Lecture 5).

12.3.1 What Do the Two Vectors Converge To?

The iteration of $\vec{q}_1$ is unchanged:

$$\vec{q}_1^{(k)} = \frac{A^k \vec{q}_1^{(0)}}{\left\| A^k \vec{q}_1^{(0)} \right\|}. \tag{12.4}$$

So $\vec{q}_1$ performs power iteration (Algorithm 15) and converges to $\vec{x}_1$ (normalized, up to sign).

Now assume that $\vec{q}_1 \approx \vec{x}_1$ already, with $\|\vec{x}_1\| = 1$. Then, up to normalization,

$$\vec{q}_2^{(k)} \approx (I - \vec{x}_1\vec{x}_1^\top)A\vec{q}_2^{(k-1)} = (I - \vec{x}_1\vec{x}_1^\top)A(I - \vec{x}_1\vec{x}_1^\top)\vec{q}_2^{(k-1)}. \tag{12.5}$$

The equality holds because $\vec{q}_2^{(k-1)}$ is already orthogonal to $\vec{x}_1$. So $\vec{q}_2$ does power iteration with the matrix $(I - \vec{x}_1\vec{x}_1^\top)A(I - \vec{x}_1\vec{x}_1^\top)$.

The normal case. For normal A, this matrix is the deflated matrix of Proposition 63:

$$(I - \vec{x}_1\vec{x}_1^\top)A(I - \vec{x}_1\vec{x}_1^\top) = A - 2\lambda_1\vec{x}_1\vec{x}_1^\top + \lambda_1\vec{x}_1\vec{x}_1^\top = A - \lambda_1\vec{x}_1\vec{x}_1^\top. \tag{12.6}$$

To get the middle expression, we expand the product and use $A\vec{x}_1 = \lambda_1\vec{x}_1$ and $\vec{x}_1^\top A = \lambda_1\vec{x}_1^\top$. The second identity needs normality (Section 12.7.2).

So orthogonal iteration does deflation automatically. Unlike Section 11.2, it projects out the current $\vec{q}_1$, not the converged $\vec{x}_1$, so both vectors iterate at the same time. The eigenvalues of $A - \lambda_1\vec{x}_1\vec{x}_1^\top$ are $0, \lambda_2, \ldots, \lambda_m$, so $\vec{q}_2 \to \vec{x}_2$ at the rate $|\lambda_3/\lambda_2|$.

The general case. Write $P \coloneqq I - \vec{x}_1\vec{x}_1^\top$. Since $P\vec{x}_1 = \vec{0}$,

$$PAP\vec{x}_i = P(\lambda_i\vec{x}_i - \lambda_1\vec{x}_1(\vec{x}_1^\top\vec{x}_i)) = \lambda_i P\vec{x}_i. \tag{12.7}$$

Read the left side as PA applied to $P\vec{x}_i$. Together with $PA\vec{x}_1 = \lambda_1 P\vec{x}_1 = \vec{0}$, this gives the eigenpairs of $(I - \vec{x}_1\vec{x}_1^\top)A$:

$$(0, \vec{x}_1) \qquad \text{and} \qquad (\lambda_i, (I - \vec{x}_1\vec{x}_1^\top)\vec{x}_i), \quad i = 2, \ldots, m. \tag{12.8}$$

The dominant one is $(\lambda_2, P\vec{x}_2)$. So, normalized and up to sign,

$$\vec{q}_2^{(k)} \longrightarrow (I - \vec{x}_1\vec{x}_1^\top)\vec{x}_2 \neq \vec{x}_2 \qquad (\text{if } \vec{x}_1 \not\perp \vec{x}_2). \tag{12.9}$$

This makes sense: $\vec{q}_1 \perp \vec{q}_2$ by construction, but $\vec{x}_1$ and $\vec{x}_2$ need not be orthogonal.

But the subspace converges:

$$\operatorname{span}(\vec{q}_1^{(k)}, \vec{q}_2^{(k)}) \longrightarrow \operatorname{span}(\vec{x}_1, \vec{x}_2), \tag{12.10}$$

since $P\vec{x}_2 = \vec{x}_2 - (\vec{x}_1^\top\vec{x}_2)\vec{x}_1$. This is an invariant subspace of A, and $\vec{q}_1, \vec{q}_2$ are the first two columns of a Schur factorization (Section 11.7).

12.3.2 Extracting the Eigenvalues

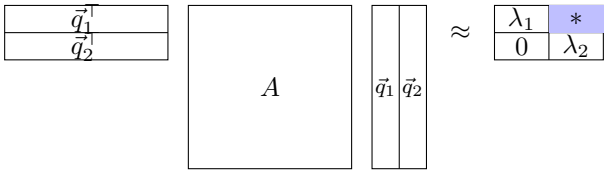
How do we find the eigenvalues? We collect the iterates and project A onto their span:

$$Q_k \coloneqq \begin{bmatrix} \vec{q}_1^{(k)} & \vec{q}_2^{(k)} \end{bmatrix} \in \mathbb{R}^{m\times 2}, \qquad \text{compute } Q_k^\top A Q_k \in \mathbb{R}^{2\times 2}. \tag{12.11}$$

Proposition 82 (Projected Matrix)

As $k \to \infty$,

$$Q_k^\top A Q_k = \begin{bmatrix} \vec{q}_1^\top A\vec{q}_1 & \vec{q}_1^\top A\vec{q}_2 \\ \vec{q}_2^\top A\vec{q}_1 & \vec{q}_2^\top A\vec{q}_2 \end{bmatrix} \longrightarrow \begin{bmatrix} \lambda_1 & * \\ 0 & \lambda_2 \end{bmatrix}. \tag{12.12}$$



Proof sketch. Write $Q = Q_k$. For the $(2,1)$ entry, we use $\vec{q}_1 \approx \vec{x}_1$ twice:

$$\vec{q}_2^\top A\vec{q}_1 \approx \vec{q}_2^\top A\vec{x}_1 = \lambda_1 \vec{q}_2^\top \vec{x}_1 \approx \lambda_1 \vec{q}_2^\top \vec{q}_1 = 0. \tag{12.13}$$

For the eigenvalues, let $\vec{v}_i \in \mathbb{R}^2$ with $Q\vec{v}_i \approx \vec{x}_i$, $i = 1, 2$. It exists because $\vec{x}_i$ lies (approximately) in $\mathcal{R}(Q)$. With $Q^\top Q = I_2$,

$$Q^\top A Q\vec{v}_i \approx Q^\top A\vec{x}_i = \lambda_i Q^\top \vec{x}_i \approx \lambda_i Q^\top Q\vec{v}_i = \lambda_i \vec{v}_i. \tag{12.14}$$

So λ_1, λ_2 are the eigenvalues of the 2×2 matrix. The $(1,1)$ entry is the Rayleigh quotient of $\vec{q}_1 \approx \vec{x}_1$, so it tends to λ_1, and the $(2,2)$ entry tends to λ_2. □

The proof also gives the eigenvectors: solve the small problem $Q_k^\top A Q_k \vec{v}_i = \lambda_i \vec{v}_i$, then lift, $\vec{x}_i \approx Q_k \vec{v}_i$.

12.4 Orthogonal Iteration in General

To compute $r > 2$ eigenpairs at once, we start with r random vectors, apply A, and reorthogonalize. In matrix form: we repeatedly multiply with A and orthogonalize with a QR factorization.

Algorithm 19 Orthogonal iteration (A, Q).

Input: $A \in \mathbb{K}^{m\times m}$, starting $Q \in \mathbb{K}^{m\times r}$ (e.g., random)

1: **for** $k = 1, \ldots,$ iters **do**
2: $\quad Q \leftarrow AQ$ ▷ Level 3: A is read once for all r columns
3: $\quad Q, R \leftarrow \mathrm{qr}(Q)$ ▷ reduced QR, $Q \in \mathbb{K}^{m\times r}$

In the QR factorization, column j is $A\vec{q}_j$ with its components along $\vec{q}_1, \ldots, \vec{q}_{j-1}$ removed. So the first two columns do exactly what Algorithm 18 does.

12.5 Convergence of Eigenvector Estimates

Let $Q_k = [\vec{q}_1^{(k)} \cdots \vec{q}_r^{(k)}]$ be the iterate after k steps. The first i columns of a QR factorization depend only on the first i columns of the matrix. So the first i columns of Q_k are orthogonal iteration with i vectors, and as in Section 12.3,

$$\operatorname{span}(\vec{q}_1^{(k)}, \ldots, \vec{q}_i^{(k)}) \longrightarrow \operatorname{span}(\vec{x}_1, \ldots, \vec{x}_i), \qquad i = 1, \ldots, r. \tag{12.15}$$

We prove this in Section 13.3.

Eigenvalues. By the same argument as before, with $Q_k \vec{v}_i \approx \vec{x}_i$,

$$Q_k^\top A Q_k \vec{v}_i \approx Q_k^\top A \vec{x}_i = \lambda_i Q_k^\top \vec{x}_i \approx \lambda_i Q_k^\top Q_k \vec{v}_i = \lambda_i \vec{v}_i. \tag{12.16}$$

What about the eigenvectors?

Symmetric case. If A is symmetric, then as before $\vec{q}_1^{(k)} \to \vec{x}_1$ and $\vec{q}_2^{(k)} \to \vec{x}_2$. By the same argument,

$$\vec{q}_i^{(k)} \longrightarrow \vec{x}_i, \qquad 2 < i \le r, \tag{12.17}$$

up to sign. Once $\vec{q}_1, \ldots, \vec{q}_{i-1}$ have converged, column i does power iteration with $\sum_{j \ge i} \lambda_j \vec{x}_j \vec{x}_j^\top$, whose dominant eigenvector is $\vec{x}_i$.

Non-symmetric case. Now let $A = X \Lambda X^{-1}$ be diagonalizable, but not symmetric (or normal). We take a reduced QR factorization of the first r eigenvectors:

$$[\vec{x}_1 \cdots \vec{x}_r] =: Q^X R^X, \qquad Q^X = [\vec{q}_1^X \cdots \vec{q}_r^X]. \tag{12.18}$$

R^X is upper triangular and invertible. So, as in Equation (13.4), $\operatorname{span}(\vec{q}_1^X, \ldots, \vec{q}_i^X) = \operatorname{span}(\vec{x}_1, \ldots, \vec{x}_i)$ for every i.

Proposition 83 (Limit of the Orthogonal Iterates)

Assume Equation (12.15). Then $Q_k \to Q^X$, up to the signs of the columns.

Proof. We check column by column, and write $\vec{q}_i$ for the limit of $\vec{q}_i^{(k)}$. First, $\vec{q}_1^{(k)} \to \vec{q}_1 = \vec{x}_1$ (normalized), and $\vec{q}_1^X = \vec{x}_1$ (normalized).

Now

$$\operatorname{span}(\vec{x}_1, \vec{x}_2) = \operatorname{span}(\vec{q}_1, \vec{q}_2) = \operatorname{span}(\vec{q}_1^X, \vec{q}_2^X). \tag{12.19}$$

So $\vec{q}_2$ is a unit vector in the orthogonal complement of $\vec{q}_1$ in this plane, and so is $\vec{q}_2^X$. This complement is a line, so $\vec{q}_2 = \pm \vec{q}_2^X$.

We repeat the argument for $\operatorname{span}(\vec{x}_1, \ldots, \vec{x}_i) = \operatorname{span}(\vec{q}_1, \ldots, \vec{q}_i) = \operatorname{span}(\vec{q}_1^X, \ldots, \vec{q}_i^X)$. □

For symmetric A the eigenvectors are orthonormal, so $Q^X = [\vec{x}_1 \cdots \vec{x}_r]$, and we get the symmetric case back.

12.6 Convergence to Schur Factorization

Now take $r = m$, so Q_k and Q^X are square and orthogonal. Consider

$$Q_k^\top A Q_k \longrightarrow Q^{X\top} A Q^X, \tag{12.20}$$

using

$$Q^X R^X = X \quad \Longleftrightarrow \quad Q^X = X(R^X)^{-1} \quad \Longleftrightarrow \quad Q^{X\top} = R^X X^{-1}. \tag{12.21}$$

Then

$$Q^{X\top} A Q^X = R^X X^{-1} A X (R^X)^{-1} = R^X X^{-1} X \Lambda (R^X)^{-1} = \underbrace{R^X \Lambda (R^X)^{-1}}_{\text{upper triangular!}}. \tag{12.22}$$

Its diagonal is $r_{ii}\lambda_i r_{ii}^{-1} = \lambda_i$. Its eigenpairs are

$$(\lambda_i, R^X \vec{e}_i), \qquad \text{since} \quad R^X \Lambda (R^X)^{-1} R^X \vec{e}_i = \lambda_i R^X \vec{e}_i. \tag{12.23}$$

(The eigenvector is $R^X\vec{e}_i$, not $(R^X)^{-1}\vec{e}_i$. Lifting it gives $Q^X R^X \vec{e}_i = \vec{x}_i$.) The column signs do not matter: $Q_k \approx Q^X D$ with $D = \mathrm{diag}(\pm 1)$ only flips the signs of some entries above the diagonal.

In summary,

$$Q_k, \quad Q_k^\top A Q_k, \quad Q_k^\top \tag{12.24}$$

converge to the Schur factorization of A. We can read off the eigenvalues from the diagonal of $Q_k^\top A Q_k$. Theorem 84 gives the exact conditions, and Section 13.5 computes $Q_k^\top A Q_k$ without forming Q_k.

12.7 (SUPPLEMENT) Further Topics

12.7.1 Examples of Normal Matrices

Hermitian ($A^* = A$), skew-Hermitian ($A^* = -A$) and unitary ($A^* = A^{-1}$) matrices are normal: in each case A^* commutes with A. The nilpotent $N = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$ is not:

$$NN^* = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \neq \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = N^* N. \tag{12.25}$$

Over $\mathbb{C}$, an "orthogonal" eigenvalue decomposition means that the eigenvector matrix is unitary.

12.7.2 Eigenvectors of Normal Matrices Are Also Left Eigenvectors

In Equation (12.6) we used $\vec{x}_1^* A = \lambda_1 \vec{x}_1^*$. Here is why. $B \coloneqq A - \lambda_1 I$ is normal, since A is. For any normal B and any $\vec{v}$,

$$\|B\vec{v}\|^2 = \vec{v}^* B^* B \vec{v} = \vec{v}^* B B^* \vec{v} = \|B^*\vec{v}\|^2. \tag{12.26}$$

So $B\vec{x}_1 = \vec{0}$ implies $B^*\vec{x}_1 = \vec{0}$, i.e., $A^*\vec{x}_1 = \overline{\lambda_1}\vec{x}_1$. Taking the conjugate transpose gives $\vec{x}_1^* A = \lambda_1 \vec{x}_1^*$. Then three of the four terms in the expansion

$$A - \vec{x}_1(\vec{x}_1^* A) - (A\vec{x}_1)\vec{x}_1^* + \vec{x}_1(\vec{x}_1^* A \vec{x}_1)\vec{x}_1^* \tag{12.27}$$

equal $\pm\lambda_1 \vec{x}_1 \vec{x}_1^*$.

In the general case, $P\vec{x}_i = \vec{x}_i - (\vec{x}_1^*\vec{x}_i)\vec{x}_1 \neq \vec{0}$ for $i \geq 2$, since the $\vec{x}_i$ are independent. For normal A, $\vec{x}_i \perp \vec{x}_1$, so $P\vec{x}_i = \vec{x}_i$, which agrees with Equation (12.6). Overall, the error of $\vec{q}_2$ decays like $\max(|\lambda_2/\lambda_1|, |\lambda_3/\lambda_2|)^k$, and the error of $\vec{q}_i$ is limited by the slowest ratio $|\lambda_{j+1}/\lambda_j|$ with $j \leq i$.

12.7.3 The Complex Case

Over $\mathbb{C}$, the argument of Proposition 83 fixes each column only up to a factor $e^{\mathrm{i}\theta}$ instead of a sign: a complex line contains a whole circle of unit vectors.

12.7.4 The Rayleigh–Ritz Procedure

Take any $Q \in \mathbb{K}^{m\times r}$ with orthonormal columns, and form the projected matrix $H \coloneqq Q^* A Q$, as in Proposition 82. This matrix generalizes the Rayleigh quotient from one vector to a subspace. Its eigenvalues θ_i are called *Ritz values*, and the lifted eigenvectors $Q\vec{v}_i$ are called *Ritz vectors*.

Among all $R \in \mathbb{K}^{r\times r}$, the choice $R = H$ gives the smallest residual $\|AQ - QR\|_2$. To see this, let $E \coloneqq AQ - QH$. Then $Q^*E = 0$, so for every $\vec{y}$,

$$\|(AQ - QR)\vec{y}\|^2 = \|E\vec{y} + Q(H - R)\vec{y}\|^2 = \|E\vec{y}\|^2 + \|(H - R)\vec{y}\|^2 \geq \|E\vec{y}\|^2 . \tag{12.28}$$

For Hermitian A, there are r eigenvalues of A, each within $\|E\|_2$ of a different Ritz value. These are not necessarily the largest eigenvalues [2, §7.2].

So in subspace iteration for Hermitian matrices, we often replace Q_k by its Ritz vectors $Q_k V$, where $Q_k^* A Q_k = V \Theta V^*$. The Ritz vectors depend only on $\operatorname{span}(Q_k) = \operatorname{span}(A^k Q_0)$. There, the components along $\vec{x}_j$, $j > r$, are smaller than the component along $\vec{x}_i$ by roughly $|\lambda_{r+1}/\lambda_i|^k$. So the ith Ritz vector converges like $|\lambda_{r+1}/\lambda_i|^k$, not like $|\lambda_{i+1}/\lambda_i|^k$: a few extra columns speed up convergence. Without the Rayleigh–Ritz step, extra columns do not help, since the leading columns of Algorithm 19 never see the later ones.

12.7.5 Why Orthonormalize at Every Step

Mathematically, $\operatorname{span}(Q_k) = \operatorname{span}(A^k Q_0)$. So we could compute $A^k Q_0$ and orthonormalize only once at the end. In floating point this fails. Every column of $A^k Q_0$ converges to a multiple of $\vec{x}_1$, so the columns become nearly parallel. The information about $\vec{x}_2, \ldots, \vec{x}_r$ sits in components smaller by factors $|\lambda_i/\lambda_1|^k$, and rounding destroys it [1, Lecture 28]. Orthonormalizing after every multiplication keeps a well-conditioned basis of the same subspace.

With $Q_0 = I$ and $r = m$, the product of all the triangular factors gives the QR factorization of A^k itself. This is the starting point of the QR algorithm in Lecture 13.

Lecture 13

Convergence of Orthogonal Iteration and the Basic QR Iteration

Supplementary reading:

- [1] Lecture 28.
- [2] Sections 4.4.3 and 4.4.4.

13.1 Recap and Motivation

Last lecture (Lecture 12) showed that the Schur factorization is a diagonalization exactly for normal matrices, and introduced orthogonal iteration: power iteration on several vectors, kept orthonormal by a QR factorization. Now we prove that it converges to the Schur factorization, and rewrite it as an iteration on a single matrix.

The plan of this chapter:

1. The link between the eigendecomposition and the Schur form: $X = QR$ (Section 13.2).
2. $\operatorname{span}(Q_k)$ converges to the span of the dominant eigenvectors (Section 13.3).
3. With $Q_0 = I$, $Q_k^* A Q_k$ converges to the Schur form (Section 13.4).
4. The same iterates come from "factor as QR, multiply as RQ": the basic QR iteration (Section 13.5).

13.2 Schur and Eigendecomposition

In this lecture we consider

$$A = X\Lambda X^{-1}, \qquad |\lambda_1| > |\lambda_2| > \cdots > |\lambda_m|\,. \tag{13.1}$$

This assumption is *generic*: an arbitrarily small perturbation makes it true. Every A has a Schur factorization $A = QTQ^*$ with the eigenvalues on the diagonal of T (Theorem 71). We add a small perturbation to T, and hence to A:

$$\tilde{A} \coloneqq Q(T + \operatorname{diag}(\varepsilon_1, \ldots, \varepsilon_m))Q^* = A + Q \operatorname{diag}(\varepsilon_1, \ldots, \varepsilon_m)Q^*. \tag{13.2}$$

- This is a Schur factorization of $\tilde{A}$. So its eigenvalues are $t_{ii} + \varepsilon_i$, and we can choose the ε_i so that they have distinct magnitude.
- The perturbation is small: $\left\|\tilde{A} - A\right\|_2 = \max_i |\varepsilon_i|$, since Q is unitary.
- Distinct eigenvalues imply diagonalizability. So $\tilde{A}$ satisfies Equation (13.1).

(For real matrices the situation is different; see Section 13.6.1.)

Now, given $A = X\Lambda X^{-1}$, how do we obtain the Schur form? We take a QR factorization of the eigenvector matrix:

$$X = QR \quad \Longrightarrow \quad A = Q \underbrace{R\Lambda R^{-1}}_{\text{upper triangular}} Q^*. \tag{13.3}$$

R and R^{-1} are upper triangular, so $R\Lambda R^{-1}$ is too, with diagonal $r_{ii}\lambda_i r_{ii}^{-1} = \lambda_i$. So Equation (13.3) is a Schur factorization, with the eigenvalues in the order of Λ.

Since R is upper triangular and invertible, $\vec{x}_j$ is a combination of $\vec{q}_1, \ldots, \vec{q}_j$, and vice versa:

$$\operatorname{span}(\vec{q}_1, \ldots, \vec{q}_j) = \operatorname{span}(\vec{x}_1, \ldots, \vec{x}_j), \qquad j = 1, \ldots, m. \tag{13.4}$$

This needs X, so it is not an algorithm. But it shows the target: orthogonal iteration should converge to these nested subspaces.

13.3 Convergence of Orthogonal Iteration

How should we think about convergence of orthogonal iteration (Algorithm 19)? Let $Q_k \in \mathbb{K}^{m\times p}$ be the iterate after k steps. Consider $A = X\Lambda X^{-1}$ diagonalizable, with the eigenvalues ordered by modulus and a gap after the pth one:

$$|\lambda_p| > |\lambda_{p+1}|. \tag{13.5}$$

The QR step does not change the column space. So

$$\operatorname{span}(Q_{k+1}) = \operatorname{span}(AQ_k) = \cdots = \operatorname{span}(A^{k+1}Q_0) = \operatorname{span}(X\Lambda^{k+1}X^{-1}Q_0). \tag{13.6}$$

We pull out the factor λ_p^{k+1}, which does not change the span:

$$\operatorname{span}(Q_{k+1}) = \operatorname{span}\left(\lambda_p^{k+1} X \begin{bmatrix} (\lambda_1/\lambda_p)^{k+1} & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & & \ddots & \\ & & & & (\lambda_m/\lambda_p)^{k+1} \end{bmatrix} X^{-1}Q_0 \right). \tag{13.7}$$

For $j \le p$, $|\lambda_j/\lambda_p| \ge 1$; for $j > p$, $|\lambda_j/\lambda_p| < 1$. So the last $m - p$ rows decay:

$$(\Lambda/\lambda_p)^{k+1} X^{-1}Q_0 = \begin{bmatrix} V_k \\ W_k \end{bmatrix}, \qquad V_k \in \mathbb{K}^{p\times p}, \qquad W_k \longrightarrow 0. \tag{13.8}$$

We split $X = [X_p \; \tilde{X}_p]$ the same way, with $X_p = [\vec{x}_1 \cdots \vec{x}_p]$. Then

$$\operatorname{span}(Q_{k+1}) = \operatorname{span}(X\Lambda^{k+1}X^{-1}Q_0) = \operatorname{span}(X_pV_k + \tilde{X}_pW_k) \longrightarrow \operatorname{span}(X_p). \tag{13.9}$$

So $\operatorname{span}(Q_k)$ converges to the invariant subspace of the p dominant eigenvectors, at the rate $|\lambda_{p+1}/\lambda_p|$ (Section 13.6.2).

13.4 Convergence to Schur Factorization

Theorem 84 (Orthogonal Iteration Converges to Schur Form)

Consider orthogonal iteration (Algorithm 19) on $A = X\Lambda X^{-1}$ with $p = m$ and $Q_0 = I$. If all eigenvalues of A have distinct absolute values and the leading blocks $(X^{-1})_{1:j,1:j}$ have full rank for $j = 1, \dots, m$, then

$$T_k \coloneqq Q_k^* A Q_k \tag{13.10}$$

converges to the Schur form of A.

The rank condition is on X^{-1}, not on X; a condition on $X_{1:j,1:j}$ is not enough in general (Section 13.6.3).

Proof sketch. Fix $p < m$ and split $Q_k = [\bar{Q}_k \; \tilde{Q}_k]$ with $\bar{Q}_k \in \mathbb{K}^{m\times p}$. Then

$$T_k = Q_k^* A Q_k = \begin{bmatrix} \bar{Q}_k^* A \bar{Q}_k & \bar{Q}_k^* A \tilde{Q}_k \\ \tilde{Q}_k^* A \bar{Q}_k & \tilde{Q}_k^* A \tilde{Q}_k \end{bmatrix}. \tag{13.11}$$

The first p columns of a QR factorization depend only on the first p columns of the matrix. So $\bar{Q}_k$ is orthogonal iteration with p columns, and by Section 13.3,

$$\operatorname{span}(\bar{Q}_k) \longrightarrow \operatorname{span}(\vec{x}_1, \dots, \vec{x}_p) \quad \Longrightarrow \quad \operatorname{span}(A\bar{Q}_k) \longrightarrow \operatorname{span}(A[\vec{x}_1 \cdots \vec{x}_p]) = \operatorname{span}(\vec{x}_1, \dots, \vec{x}_p). \tag{13.12}$$

So the columns of $A\bar{Q}_k$ lie (asymptotically) in $\operatorname{span}(\bar{Q}_k)$, and $\tilde{Q}_k$ is orthogonal to it. Hence

$$\tilde{Q}_k^* A \bar{Q}_k \longrightarrow 0. \tag{13.13}$$

This is true for all subdiagonal blocks: every entry (i, j) with $i > j$ lies in the lower left block for $p = j$. So

$$Q_k^* A Q_k \longrightarrow \text{upper triangular.} \tag{13.14}$$

□

The diagonal then holds the eigenvalues, ordered by decreasing modulus (Section 13.6.4).

13.5 Basic QR Iteration

Orthogonal iteration allows computing

$$T_k \coloneqq Q_k^* A Q_k, \tag{13.15}$$

which converges to the Schur factorization (Theorem 84). But computing T_{k+1} requires Q_k and a matrix-matrix product with A. Can we derive an iteration $T_k \to T_{k+1}$?

Observe that

$$T_k = Q_k^* A Q_k, \qquad T_{k+1} = Q_{k+1}^* A Q_{k+1}. \tag{13.16}$$

By orthogonal iteration (with Q_k square, so $Q_k Q_k^* = I$),

$$Q_{k+1} R_{k+1} = A Q_k \quad \Longrightarrow \quad R_{k+1} Q_k^* = Q_{k+1}^* A. \tag{13.17}$$

We put the first form into T_k and the second into T_{k+1}:

$$T_k = Q_k^* (Q_{k+1} R_{k+1}), \qquad T_{k+1} = (R_{k+1} Q_k^*) Q_{k+1}. \tag{13.18}$$

With $U_{k+1} \coloneqq Q_k^* Q_{k+1}$:

$$T_k = U_{k+1} R_{k+1} \quad \xrightarrow{QR \to RQ} \quad T_{k+1} = R_{k+1} U_{k+1}. \tag{13.19}$$

U_{k+1} is unitary and R_{k+1} is upper triangular, so $T_k = U_{k+1} R_{k+1}$ is a QR factorization of T_k. Also, $T_{k+1} = U_{k+1}^* T_k U_{k+1}$, so every T_k has the eigenvalues of A.

This gives a simple algorithm: compute a QR factorization and swap the order of the factors. With $Q_0 = I$ we start from $T_0 = A$.

Algorithm 20 Basic QR iteration (QR algorithm without shifts).

Input: $A \in \mathbb{K}^{m \times m}$

$T \leftarrow A$
for $k = 1, \dots,$ maxiter **do**
  $U, R \leftarrow \mathrm{qr}(T)$
  $T \leftarrow RU$
return $\mathrm{diag}(T)$ ▷ approximate eigenvalues $\lambda_1, \dots, \lambda_m$

How to obtain Q? By definition, $U_{k+1} = Q_k^* Q_{k+1}$. Implicitly $Q_0 = I$, so the product telescopes:

$$U_1 U_2 \cdots U_{k+1} = Q_0^* Q_1 \, Q_1^* Q_2 \cdots Q_k^* Q_{k+1} = Q_{k+1}. \tag{13.20}$$

So we start from $Q \leftarrow I$ and update $Q \leftarrow QU$ after each step. At the end, $A \approx QTQ^*$.

Close to convergence, $Q_{k+1} \approx Q_k$, so $U_{k+1} = Q_k^* Q_{k+1}$ is a small rotation (up to signs, Section 13.6.5). The U_k are a series of rotational corrections transforming I to Q.

13.6 (SUPPLEMENT) Further Topics

13.6.1 Real Matrices: Distinct Moduli Are Not Generic

The perturbation in Section 13.2 is complex in general. For real A, non-real eigenvalues come in conjugate pairs $\lambda, \overline{\lambda}$ with $|\lambda| = \left|\overline{\lambda}\right|$, and a small real perturbation keeps them a pair. So among real matrices, Equation (13.1) fails on an open set, for example near every rotation of $\mathbb{R}^2$ by an angle $\theta \notin \{0, \pi\}$. Also, eigenvalues depend continuously on the matrix, so in the complex case the strict inequalities survive small perturbations of $\tilde{A}$.

13.6.2 The Convergence Rate of the Subspace

Write $B \coloneqq X^{-1} Q_0 = \begin{bmatrix} B_1 \\ B_2 \end{bmatrix}$ with $B_1 \in \mathbb{K}^{p \times p}$, and $\Lambda = \mathrm{diag}(\Lambda_1, \Lambda_2)$ with $\Lambda_1 = \mathrm{diag}(\lambda_1, \dots, \lambda_p)$. Note $\lambda_p \neq 0$ by Equation (13.5). Then

$$V_k = (\Lambda_1 / \lambda_p)^{k+1} B_1, \qquad W_k = (\Lambda_2 / \lambda_p)^{k+1} B_2. \tag{13.21}$$

Assume B_1 is invertible: Q_0 has a component in every direction $\vec{x}_1, \dots, \vec{x}_p$. This is the analogue of $\vec{x}_1^* \vec{q}^{(0)} \neq 0$ for power iteration, and it holds with probability one for random Q_0. Then

$$\mathrm{span}(X_p V_k + \tilde{X}_p W_k) = \mathrm{span}\big(X_p + \tilde{X}_p W_k V_k^{-1}\big), \tag{13.22}$$

and since $V_k^{-1} = B_1^{-1}(\lambda_p \Lambda_1^{-1})^{k+1}$ has $\left\|V_k^{-1}\right\| \le \left\|B_1^{-1}\right\|$,

$$\left\|\tilde{X}_p W_k V_k^{-1}\right\| \le \left\|\tilde{X}_p\right\| \|B_2\| \left\|B_1^{-1}\right\| \left|\frac{\lambda_{p+1}}{\lambda_p}\right|^{k+1} \longrightarrow 0. \tag{13.23}$$

So the rate is the gap between the last eigenvalue we keep and the first one we drop.

13.6.3 Why the Condition Is on X^{-1}

The argument of Section 13.6.2 needs $B_1 = (X^{-1}Q_0)_{1:j,1:j} = (X^{-1})_{1:j,1:j}$ invertible. Asking that $X_{1:j,1:j}$ be invertible is not enough. Take

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \qquad X^{-1} = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix}. \tag{13.24}$$

Here $X_{11} = 1$ and X are nonsingular, but $(X^{-1})_{11} = 0$. The starting vector $\vec{e}_1 = \vec{x}_2$ is an eigenvector with no component along $\vec{x}_1$, and in exact arithmetic $\vec{q}_1^{(k)} = \vec{x}_2$ forever.

For normal $A = Q\Lambda Q^*$ the two conditions agree: $X^{-1} = Q^*$ and $(Q^*)_{1:j,1:j} = (Q_{1:j,1:j})^*$. This is why Trefethen and Bau, who treat real symmetric A, can state the condition with the leading blocks of Q [1, Thm. 28.4]. Some books state it with $X_{1:j,1:j}$ for general A (e.g., [2, Thm. 4.8]).

13.6.4 The Diagonal of T_k

In the proof of Theorem 84, $\bar{Q}_k^* A \bar{Q}_k$ is the matrix of A restricted to $\operatorname{span}(\bar{Q}_k) \to \operatorname{span}(\vec{x}_1, \ldots, \vec{x}_p)$. So its eigenvalues tend to $\lambda_1, \ldots, \lambda_p$, as in Proposition 82. This block becomes upper triangular, so its diagonal holds these eigenvalues. Comparing p with $p-1$ shows that $(T_k)_{pp} \to \lambda_p$.

A precise version of "the columns of $A\bar{Q}_k$ lie in $\operatorname{span}(\bar{Q}_k)$": since $\tilde{Q}_k^* \bar{Q}_k = 0$,

$$\tilde{Q}_k^* A \bar{Q}_k = \tilde{Q}_k^* (I - \bar{Q}_k \bar{Q}_k^*) A \bar{Q}_k, \tag{13.25}$$

and $(I - \bar{Q}_k \bar{Q}_k^*) A \bar{Q}_k$, the part of $A\bar{Q}_k$ outside $\operatorname{span}(\bar{Q}_k)$, tends to zero. This uses $\lambda_1, \ldots, \lambda_p \neq 0$.

13.6.5 Sign Choices and the Phases of U_k

A library QR routine may not return the same factors as orthogonal iteration. For invertible T, QR factors are unique only up to signs (phases) of the columns of U and the rows of R (Lecture 7). Every choice gives $T_{k+1} = U^* T_k U$, and different choices give iterates that differ by a diagonal unitary similarity $D^* T_k D$. This changes neither the diagonal nor the moduli of the entries.

Near convergence, T_k is nearly upper triangular, so $U_{k+1} = T_k R_{k+1}^{-1}$ is nearly upper triangular. Being unitary, it is nearly diagonal: $U_{k+1} \approx D$. With $r_{ii} > 0$, $D \approx \operatorname{diag}(\lambda_1/|\lambda_1|, \ldots, \lambda_m/|\lambda_m|)$. So a negative eigenvalue flips the sign of its column of Q_k in every step, as in power iteration.

13.6.6 The QR Algorithm Factors the Powers of A

Let $\underline{R}_k \coloneqq R_k R_{k-1} \cdots R_1$. Then

$$A^k = Q_k \underline{R}_k, \qquad T_k = Q_k^* A Q_k, \qquad Q_k = U_1 U_2 \cdots U_k. \tag{13.26}$$

The first identity follows by induction: $A^k = A\, Q_{k-1} \underline{R}_{k-1} = Q_k R_k \underline{R}_{k-1} = Q_k \underline{R}_k$. So the QR algorithm computes a QR factorization of A^k without forming A^k, whose columns would all be nearly parallel to $\vec{x}_1$. The two identities in Equation (13.26) explain the algorithm [1, Lecture 28].

- The leading j columns of Q_k are an orthonormal basis of the span of the leading j columns of A^k: simultaneous power iteration on $\vec{e}_1, \ldots, \vec{e}_m$.
- The diagonal entries $(T_k)_{jj} = \vec{q}_j^{(k)*} A \vec{q}_j^{(k)}$ are Rayleigh quotients. So for Hermitian A they converge twice as fast as the columns of Q_k (Lecture 10).

13.6.7 Convergence Rates of Individual Entries

By Section 13.3, the block $(T_k)_{p+1:m,1:p}$ decays like $|\lambda_{p+1}/\lambda_p|^k$. In particular:

- the subdiagonal entry $(T_k)_{j+1,j}$ decays like $|\lambda_{j+1}/\lambda_j|^k$;
- the diagonal entry $(T_k)_{pp}$ converges to λ_p like $\max(|\lambda_{p+1}/\lambda_p|^k, |\lambda_p/\lambda_{p-1}|^k)$ [2, §4.4.3].

The overall rate is the worst ratio $\max_j |\lambda_{j+1}/\lambda_j|$ [1, Thm. 28.4]. If two eigenvalues have nearly equal modulus, the unshifted algorithm becomes arbitrarily slow.

For example, take $\Lambda = \mathrm{diag}(30, 6, 2, 1)$ and a random X with $\kappa(X) \approx 20$. After 19 steps, the subdiagonal entries are about $7 \cdot 10^{-13}$, $1 \cdot 10^{-9}$, and $5 \cdot 10^{-6}$ [2, Example 4.5]. This agrees with $(1/5)^{19} \approx 5 \cdot 10^{-14}$, $(1/3)^{19} \approx 9 \cdot 10^{-10}$, and $(1/2)^{19} \approx 2 \cdot 10^{-6}$, up to constants.

The most important block is the last row, $(T_k)_{m,1:m-1}$, which decays like $|\lambda_m/\lambda_{m-1}|^k$. Once it is below a tolerance such as $\varepsilon_{\mathrm{mach}} \|A\|$, we accept $(T_k)_{mm}$ as an eigenvalue and continue on the leading $(m-1)\times(m-1)$ block. This is deflation.

13.6.8 Eigenvalues of Equal Modulus

Without a gap $|\lambda_p| > |\lambda_{p+1}|$, the block at the split after column p need not decay. The extreme case is a unitary A: all its eigenvalues have modulus 1. Its QR factorization is $A = A \cdot I$, so $T_1 = I \cdot A = A$, and the iteration does not move [2, Example 4.6]. An example is $A = \left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$, with eigenvalues ± 1.

For real A, the generic case is a pair of complex conjugate eigenvalues (Section 13.6.1). All U_k, R_k, T_k are real, so T_k cannot converge to a triangular matrix with non-real diagonal entries. Splits between eigenvalues of different moduli still have a gap, so those blocks still decay. If the only ties in modulus come from conjugate pairs, T_k converges to the real Schur form (Theorem 73). The entries of each 2×2 block keep changing, but its eigenvalues converge to $\lambda, \overline{\lambda}$.

Lecture 14

Hessenberg QR

Supplementary reading:

- [1] Lecture 26.
- [2] Sections 4.4.5, 4.4.6, and 4.4.8.

14.1 Recap and Motivation

Last lecture (Lecture 13) linked the Schur form to the eigendecomposition ($X = QR$), proved that orthogonal iteration converges to the Schur factorization, and turned it into the basic QR iteration $T_k = U_{k+1}R_{k+1} \to T_{k+1} = R_{k+1}U_{k+1}$. But each step costs $O(m^3)$. Now we make each step cost $O(m^2)$.

The plan of this chapter:

1. Reduce A once to Hessenberg form $H = Q^*AQ$ (Sections 14.2 and 14.3).
2. On H, one QR step takes $m-1$ Givens rotations and costs $O(m^2)$ (Section 14.4).
3. Bulge chasing does the same step in one sweep (Section 14.5).
4. The implicit Q theorem will show that bulge chasing gives the same T_k (Section 14.6).

14.2 Improvements to QR

The bottleneck: each step of Algorithm 20 is a dense QR factorization, at cost $O(m^3)$, and we possibly need m of them. So the cost is $O(m^4)$, even if every eigenvector converges in $O(1)$ iterations.

The idea: preprocess A once, to simplify the QR factorizations. The goal is to find Q such that

$$H = Q^*AQ \tag{14.1}$$

is as close to triangular as possible. This must be a similarity, so that the eigenvalues do not change, and we take Q unitary, so that the conditioning does not get worse (Theorem 103).

Definition 85 (Hessenberg Matrix)

$H \in \mathbb{K}^{m\times m}$ is *upper Hessenberg* if $h_{ij} = 0$ for $i > j + 1$: it has only one nonzero lower diagonal.

$$H = \begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix}. \tag{14.2}$$

14.3 Reduction to Hessenberg Form

We compute Q using Householder reflections (Lecture 6). Let $P_1 \in \mathbb{K}^{(m-1)\times(m-1)}$ be the reflector that maps

$$\vec{v} \coloneqq A_{2:m,1} \quad \text{to} \quad \|\vec{v}\|\,\vec{e}_1, \qquad \text{and set} \qquad Q_1 \coloneqq \begin{bmatrix} 1 & 0 \\ 0 & P_1 \end{bmatrix}. \tag{14.3}$$

Left multiplication by Q_1 keeps the first row and zeros out column 1 below the subdiagonal:

$$\underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix}}_{A} \xrightarrow{Q_1 \cdot} \underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{bmatrix}}_{Q_1 A}. \tag{14.4}$$

Right multiplication by Q_1^* acts only on columns $2, \ldots, m$. So the new zeros in column 1 stay:

$$\underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{bmatrix}}_{Q_1 A} \xrightarrow{\cdot\, Q_1^*} \underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{bmatrix}}_{Q_1 A Q_1^*}. \tag{14.5}$$

This is why the reflector starts at row 2 and leaves a_{21} in place. A reflector on all rows would also zero out a_{21}, but its adjoint on the right would mix all columns and fill column 1 back in.

We repeat on the $(2{:}m, 2{:}m)$ submatrix, and so on, with $Q_k = \operatorname{diag}(I_k, P_k)$. Left and right multiplication by Q_k only touch rows and columns $k+1, \ldots, m$, so earlier zeros stay. After $m-2$ steps we gather the reflectors:

$$Q_{m-2} \cdots Q_1\, A\, Q_1 \cdots Q_{m-2} = Q^* A Q = H, \qquad Q \coloneqq Q_1 \cdots Q_{m-2}. \tag{14.6}$$

Each Q_k is Hermitian, so $Q^* = Q_{m-2} \cdots Q_1$. In practice we never form Q; we store the reflection vectors (Algorithm 21).

Why not upper triangular? Because it is impossible. A unitary similarity to triangular form in finitely many steps would compute all eigenvalues exactly, i.e., the roots of any polynomial. This contradicts Abel's theorem (Section 10.3).

14.4 QR Iteration on H

We now run Algorithm 20 on $T_0 = H$.

First step: compute U_k, R_k with $U_k R_k = T_{k-1}$. We have seen that $O(m)$ Givens rotations suffice (Section 7.3). G_j^* acts on rows $j, j+1$ and zeros out the $(j+1, j)$ entry:

$$\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix} \xrightarrow{G_1^*} \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix} \xrightarrow{G_2^*} \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix} \xrightarrow{G_3^*} \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{bmatrix}. \tag{14.7}$$

Each rotation costs $O(m)$, so the QR factorization costs $O(m^2)$:

$$R_k = G_{m-1}^* \cdots G_1^* T_{k-1}, \qquad U_k = G_1 G_2 \cdots G_{m-1}. \tag{14.8}$$

Second step: compute $T_k = R_k U_k$. We keep U_k in factored form:

$$T_k = R_k G_1 G_2 \cdots G_{m-1}. \tag{14.9}$$

Again each rotation changes two columns, and the cost is $O(m^2)$.

So the total cost is $O(m^3)$ for m iterations, the same order as the reduction to Hessenberg form.

Why is T_k upper Hessenberg? Right multiplication by G_j forms linear combinations of columns j and $j+1$. Column $j+1$ has nonzeros down to row $j+1$, so this creates one new nonzero at $(j+1, j)$, on the subdiagonal:

$$\underbrace{\begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{bmatrix}}_{R_k} \xrightarrow{\cdot G_1} \begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{bmatrix} \xrightarrow{\cdot G_2} \begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & 0 & \times \end{bmatrix} \xrightarrow{\cdot G_3} \underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix}}_{T_k}. \tag{14.10}$$

So the Hessenberg form is kept, and every step costs $O(m^2)$.

14.5 Bulge Chasing

So far, one step applies all rotations from the left, then all from the right:

$$T_k = \left(G_{m-1}^* \cdots G_1^* T_{k-1}\right) G_1 \cdots G_{m-1}. \tag{14.11}$$

Can we apply left and right simultaneously?

$$T_k = G_{m-1}^* \Big(\cdots G_2^* (G_1^* T_{k-1} G_1) G_2 \cdots \Big) G_{m-1}. \tag{14.12}$$

Problem. We need $G_1^* T_{k-1}$ to compute G_2: G_2^* must zero out its $(3,2)$ entry. How can we compute G_2 from $G_1^* T_{k-1} G_1$ instead?

What happens when we compute $G_1^* T_{k-1} G_1$? G_1^* combines rows $1, 2$ and keeps the Hessenberg form. Then G_1 combines columns $1, 2$, and column 2 has a nonzero in row 3. So we get a nonzero at $(3, 1)$, the *bulge*:

$$\underbrace{\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & 0 & \times & \times \end{bmatrix}}_{T_{k-1}} \xrightarrow{G_1^* \cdot G_1} \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ + & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & 0 & \times & \times \end{bmatrix}. \tag{14.13}$$

Idea. Choose G_2 such that it removes the bulge. G_2^* acts on rows $2, 3$ and zeros out $(3, 1)$; then G_2 combines columns $2, 3$ and creates a new bulge at $(4, 2)$. So the bulge moves further down the off-diagonal:

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ + & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & 0 & \times & \times \end{bmatrix} \xrightarrow{G_2^* \cdot G_2} \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & + & \times & \times & \times \\ 0 & 0 & 0 & \times & \times \end{bmatrix} \xrightarrow{G_3^* \cdot G_3} \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & + & \times & \times \end{bmatrix} \xrightarrow{G_4^* \cdot G_4} \text{Hessenberg.} \tag{14.14}$$

The last rotation chases the bulge off the bottom. Each step changes two rows and two columns, so one QR step still costs $O(m^2)$, now in a single sweep (Algorithm 22).

We need to show that this leads to the same T_k. This follows from the implicit Q theorem.

14.6 The Implicit Q Theorem

An upper Hessenberg H is *unreduced* if all subdiagonal entries are nonzero, $h_{j+1,j} \neq 0$, i.e., H is not block upper triangular. (If some $h_{j+1,j} = 0$, the eigenvalue problem splits into the two diagonal blocks.)

Theorem 86 (Implicit Q Theorem)

Assume that Q is orthogonal and $Q^\top AQ = H$ is unreduced upper Hessenberg. Then columns $2, \dots, m$ of Q are uniquely determined by column 1, up to signs.

So to know Q, it is enough to know its first column and that the result is Hessenberg. In bulge chasing, $U_k \vec{e}_1 = G_1 \vec{e}_1$ as in the two-sweep algorithm, and both results are Hessenberg. We prove the theorem and apply it in Sections 15.2 and 15.3.

14.7 (SUPPLEMENT) Further Topics

14.7.1 The Householder Reduction Algorithm and Its Cost

Algorithm 21 Householder reduction to Hessenberg form [1, Algorithm 26.1].

Input: $A \in \mathbb{K}^{m \times m}$, overwritten by $H = Q^* AQ$

1: **for** $k = 1, \dots, m-2$ **do**
2: $\quad \vec{x} \leftarrow A_{k+1:m,k}$
3: $\quad \vec{v}_k \leftarrow \operatorname{sign}(x_1) \|\vec{x}\| \vec{e}_1 + \vec{x}, \quad \vec{v}_k \leftarrow \vec{v}_k / \|\vec{v}_k\|$
4: $\quad A_{k+1:m,k:m} \leftarrow A_{k+1:m,k:m} - 2\vec{v}_k(\vec{v}_k^* A_{k+1:m,k:m})$ ▷ left: rows $k+1, \dots, m$
5: $\quad A_{1:m,k+1:m} \leftarrow A_{1:m,k+1:m} - 2(A_{1:m,k+1:m}\vec{v}_k)\vec{v}_k^*$ ▷ right: columns $k+1, \dots, m$

Here the reflector maps $\vec{x}$ to $-\operatorname{sign}(x_1)\|\vec{x}\|\vec{e}_1$ rather than $\|\vec{x}\|\vec{e}_1$, to avoid cancellation (Lecture 6). Either choice gives a Hessenberg matrix. The left updates cost about $\frac{4}{3}m^3$ flops; the right updates act on full columns and cost about $2m^3$. In total,

$$\text{work for Hessenberg reduction} \sim \tfrac{10}{3}m^3 \text{ flops } [1, (26.1)], \tag{14.15}$$

paid once, before the QR iteration starts.

14.7.2 Bulge Chasing as an Algorithm

Algorithm 22 One step of the QR iteration on an upper Hessenberg matrix by bulge chasing.

Input: upper Hessenberg $T \in \mathbb{K}^{m\times m}$, overwritten by the next iterate

choose G_1 acting on rows $1, 2$ such that $(G_1^* T)_{21} = 0$
$T \leftarrow G_1^* T G_1$ ▷ creates the bulge at $(3, 1)$
for $j = 2, \dots, m-1$ **do**
    choose G_j acting on rows $j, j+1$ such that $(G_j^* T)_{j+1,j-1} = 0$ ▷ remove the bulge
    $T \leftarrow G_j^* T G_j$ ▷ bulge moves to $(j+2, j)$, or leaves if $j = m-1$

14.7.3 The Hermitian Case: Tridiagonal Form

If A is Hermitian, then so is $H = Q^* A Q$. A Hermitian Hessenberg matrix is tridiagonal. Using the zeros and the symmetry, the reduction costs $\sim \frac{4}{3}m^3$ flops instead of $\frac{10}{3}m^3$. After that, each QR step on a tridiagonal matrix costs only $O(m)$ [1, Lecture 26], [2, Section 4.4.7].

14.7.4 Stability

Like Householder QR, Algorithm 21 is backward stable. Let $\tilde{H}$ be the computed result, and $\tilde{Q}$ the exactly unitary matrix defined by the computed reflection vectors. Then $\tilde{Q}\tilde{H}\tilde{Q}^* = A + \delta A$ with $\|\delta A\| / \|A\| = O(\varepsilon_{\text{mach}})$ [1, Theorem 26.1]. So the eigenvalues of $\tilde{H}$ are the exact eigenvalues of a nearby matrix.

Lecture 15

Shifted QR

Supplementary reading:

- [2] Section 4.4.8 (Theorem 4.9, the implicit Q theorem).
- [2] Sections 4.4.4 and 4.4.5 (QR iteration with a shift, making it practical).
- [2] Section 5.3.1 (Wilkinson's shift).

15.1 Recap and Motivation

Last lecture (Lecture 14) reduced A to Hessenberg form $H = Q^* A Q$, ran the QR iteration on H at $O(m^2)$ per step, and did each step in one sweep by bulge chasing. Now we prove that bulge chasing gives the same T_k, and add shifts to make the iteration converge fast.

The plan of this chapter:

1. Prove the implicit Q theorem, and apply it to bulge chasing (Sections 15.2 and 15.3).
2. The shifted QR iteration (Section 15.4).
3. Choosing the shift, and the practical QR iteration (Section 15.5).
4. The QR iteration is both power and inverse power iteration (Section 15.6).

15.2 Proof of the Implicit Q Theorem

Recall the statement (Theorem 86): if Q is orthogonal and $Q^\top A Q = H$ is unreduced upper Hessenberg, then the first column of Q determines columns $2, \dots, m$, up to signs.

Proof. Let both $Q^\top A Q = H$ and $V^\top A V = G$ be such, with equal first column $\vec{q}_1 = \vec{v}_1$. We want to show that

$$W \coloneqq V^\top Q = \operatorname{diag}(\pm 1, \dots, \pm 1). \tag{15.1}$$

Then $Q = VW$, i.e., $\vec{q}_i = \pm \vec{v}_i$.

W is orthogonal, and $W\vec{e}_1 = V^\top \vec{q}_1 = \vec{e}_1$. Using $GV^\top = V^\top A$ and $AQ = QH$, we have

$$GW = GV^\top Q = V^\top AQ = V^\top QH = WH. \tag{15.2}$$

Take column i, with $\vec{w}_i$ the ith column of W. Since H is upper Hessenberg,

$$G\vec{w}_i = (GW)_{:,i} = (WH)_{:,i} = \sum_{j=1}^{i+1} \vec{w}_j H_{ji} \quad \Longrightarrow \quad H_{i+1,i}\,\vec{w}_{i+1} = G\vec{w}_i - \sum_{j=1}^{i} H_{ji}\vec{w}_j. \tag{15.3}$$

By induction, $\vec{w}_i$ is zero below entry i:

$$\vec{w}_i = \begin{bmatrix} \times \\ \vdots \\ \times \\ 0 \\ \vdots \\ 0 \end{bmatrix} \leftarrow i \tag{15.4}$$

This holds for $\vec{w}_1 = \vec{e}_1$. If it holds up to i, then on the right of (15.3), $G\vec{w}_i$ is zero below row $i+1$ (since G is Hessenberg), and the sum is zero below row i. Since H is unreduced, $H_{i+1,i} \neq 0$, so $\vec{w}_{i+1}$ is zero below row $i+1$.

So W is upper triangular and orthogonal. Then $W^{-1} = W^\top$ is both upper and lower triangular, so W is diagonal, with $w_{ii}^2 = 1$. □

15.3 Application to Bulge Chasing

Both the two-sweep algorithm and bulge chasing compute $T_k = U_k^\top T_{k-1} U_k$ with $U_k = G_1 G_2 \cdots G_{m-1}$.

- *Same first column.* Only G_1 touches row 1, so $U_k\vec{e}_1 = G_1\vec{e}_1$. And G_1 is the same in both, since only the first column of T_{k-1} decides it.
- *Hessenberg form.* Both results are upper Hessenberg.

If T_k is unreduced (if not, we split), Theorem 86 says the two U_k agree up to column signs. So the two T_k agree up to $D^\top T_k D$ with $D = \mathrm{diag}(\pm 1)$, which changes neither the diagonal nor the moduli of the entries.

15.4 The Shifted QR Iteration

The QR iteration can converge slowly: its rate is $|\lambda_{p+1}/\lambda_p|$ (Section 13.3). As for inverse iteration (Algorithm 16), the fix is a shift:

$$\text{normal QR:} \quad \begin{aligned} T_{k-1} &= U_k R_k, \\ T_k &= R_k U_k, \end{aligned} \qquad \text{shifted QR:} \quad \begin{aligned} T_{k-1} - \mu_k I &= U_k R_k, \\ T_k &= R_k U_k + \mu_k I. \end{aligned} \tag{15.5}$$

- *It preserves upper Hessenberg form.* The shift only changes the diagonal, so $T_{k-1} - \mu_k I$ is Hessenberg (Givens QR, $O(m^2)$), and $R_k U_k$ is Hessenberg (Section 14.4).
- *It preserves the eigenvalues.* See below.
- *How do we track the eigenvectors?*

From $R_k = U_k^*(T_{k-1} - \mu_k I)$,

$$T_k = R_k U_k + \mu_k I = U_k^*(T_{k-1} - \mu_k I)U_k + \mu_k I = U_k^* T_{k-1} U_k. \tag{15.6}$$

So T_k is a unitary similarity of T_{k-1}. (Without adding back $\mu_k I$, every step would shift the eigenvalues by $-\mu_k$.) This is the same relation as in the unshifted case, so we can track Q_k as before (Section 13.5): update $Q \leftarrow QU_k$, starting from the Q of the Hessenberg reduction.

15.5 Choosing the Shift

How does the shift affect convergence? Which μ_k should we choose?

Exact shift. Assume μ_k is an eigenvalue. Then $T_{k-1} - \mu_k I$ is singular. Assume also that the off-diagonal entries $t_{j+1,j}$ are nonzero.

The first $m-1$ columns of $T_{k-1} - \mu_k I$ are independent: rows $2, \dots, m$ of these columns form an upper triangular matrix with the nonzero $t_{j+1,j}$ on its diagonal. In $T_{k-1} - \mu_k I = U_k R_k$, column j is a combination of $\vec{u}_1, \dots, \vec{u}_j$ with coefficient r_{jj} on $\vec{u}_j$. So $r_{11}, \dots, r_{m-1,m-1} \neq 0$. But R_k is singular, so the last entry must be zero, $r_{mm} = 0$:

$$\underbrace{T_{k-1} - \mu_k I}_{\text{unreduced Hessenberg}} = U_k \underbrace{\begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 \end{bmatrix}}_{R_k}. \tag{15.7}$$

Now what happens to T_k? The last row of $R_k U_k$ is zero, so

$$T_k = R_k U_k + \mu_k I = \begin{bmatrix} & \hat{T} & & \vec{t} \\ 0 & \cdots & 0 & \mu_k \end{bmatrix}. \tag{15.8}$$

With an exact shift, one step splits off the eigenvalue μ_k. We do not have to worry about the last entry any more, and continue on $\hat{T}$. A unitary Q acting on $\hat{T}$ acts on the full matrix as $\mathrm{diag}(Q, 1)$:

$$\begin{bmatrix} Q^* & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \hat{T} & \vec{t} \\ 0 & \mu_k \end{bmatrix} \begin{bmatrix} Q & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} Q^* \hat{T} Q & Q^* \vec{t} \\ 0 & \mu_k \end{bmatrix}. \tag{15.9}$$

15.5.1 Practical QR Iteration

Goal. Choose $\mu_k \approx$ an eigenvalue, and split up the problem whenever a subdiagonal entry approaches 0:

$$T_k \approx \begin{bmatrix} (T_k)_{11} & (T_k)_{12} \\ 0 & (T_k)_{22} \end{bmatrix}. \tag{15.10}$$

The eigenvalues of T_k are those of the two diagonal blocks. Both are again Hessenberg, so we recurse on $(T_k)_{11}$ and $(T_k)_{22}$.

Which eigenvalue? We choose $\mu_k \approx \lambda_m$, for two reasons.

1. We want $|\lambda_i - \mu| \ll |\lambda_j - \mu|$ for $j \neq i$. Without a shift, this is easiest if λ_i is small.
2. The eigenvalues are ordered by size on the diagonal of T_k (Theorem 84), with λ_m at the bottom right. Reordering the eigenvalues slows down convergence.

So the shift should be close to the eigenvalue already converging in the bottom right corner, and there we check $t_{m,m-1} \approx 0$.

Algorithm 23 Practical QR iteration.

Input: $A \in \mathbb{K}^{m \times m}$

compute $H = Q^* A Q$ ▷ Algorithm 21, once
$T_0 \leftarrow H$, $k \leftarrow 0$
while not converged **do**
    $k \leftarrow k + 1$, choose μ_k
    $T_{k-1} - \mu_k I = U_k R_k$ ▷ Givens QR, $O(m^2)$
    $T_k \leftarrow R_k U_k + \mu_k I$
    if some subdiagonal entry of T_k is ≈ 0 **then**
        split T_k into $(T_k)_{11}$ and $(T_k)_{22}$ and recurse on both

15.6 Relationship with (Inverse) Power Iteration

We have seen that the QR iteration is orthogonal iteration with $Q_0 = I$ (Section 13.5). So the shifted QR iteration is a *shifted* orthogonal iteration. Start with $T_0 = H$:

$$\begin{aligned} U_1 U_2 R_2 R_1 &= U_1 (T_1 - \mu_2 I) R_1 = U_1 (R_1 U_1 + \mu_1 I - \mu_2 I) R_1 \\ &= (H - \mu_1 I)\big((H - \mu_1 I) + \mu_1 I - \mu_2 I\big) \\ &= (H - \mu_1 I)(H - \mu_2 I), \end{aligned} \tag{15.11}$$

using $U_1 R_1 = H - \mu_1 I$ twice in the second line. With $\bar{Q}_k \coloneqq U_1 \cdots U_k$ and $\bar{R}_k \coloneqq R_k \cdots R_1$, we can write

$$(H - \mu_k I) \cdots (H - \mu_1 I) = \bar{Q}_k \bar{R}_k. \tag{15.12}$$

By induction, using $T_{k-1} = \bar{Q}_{k-1}^* H \bar{Q}_{k-1}$,

$$\bar{Q}_k \bar{R}_k = \bar{Q}_{k-1} (T_{k-1} - \mu_k I) \bar{R}_{k-1} = (H - \mu_k I) \bar{Q}_{k-1} \bar{R}_{k-1}. \tag{15.13}$$

So (15.12) is a QR factorization of the polynomial $p_k(H) = (H - \mu_k I) \cdots (H - \mu_1 I)$. In the unshifted case, $\bar{Q}_k$ is the Q factor of H^k, which scales the eigenvector component for λ_j by λ_j^k. Now it is scaled by $p_k(\lambda_j) = \prod_i (\lambda_j - \mu_i)$, which is tiny if the shifts are close to λ_j.

What about inverse iteration? Invert (15.12) and take the adjoint:

$$\bar{Q}_k \bar{R}_k^{-*} = (H - \mu_k I)^{-*} \cdots (H - \mu_1 I)^{-*}, \tag{15.14}$$

with $X^{-*} \coloneqq (X^{-1})^*$. The factors commute, so their order does not matter. The right side is shifted inverse iteration (Algorithm 16) on H^*, with shifts $\bar{\mu}_i$.

Now $\bar{R}_k^{-*}$ is *lower* triangular. So its last column is $(\bar{R}_k^{-*})_{mm}\vec{e}_m$, and multiplying (15.14) by $\vec{e}_m$ gives

$$\bar{\vec{q}}_m = \overline{(\bar{R}_k)_{mm}}\,(H-\mu_k I)^{-*}\cdots(H-\mu_1 I)^{-*}\vec{e}_m, \tag{15.15}$$

where $\bar{\vec{q}}_m$ is the last column of $\bar{Q}_k$. So the same $\bar{Q}_k$ does two iterations at once:

factorization	triangular factor	columns of $\bar{Q}_k$
$p_k(H) = \bar{Q}_k\bar{R}_k$	upper	first columns: orthogonal (power) iteration
$p_k(H)^{-*} = \bar{Q}_k\bar{R}_k^{-*}$	lower	last columns: shifted inverse iteration

The QR iteration is both power and inverse power iteration. This is why the bottom right corner of T_k converges first, and why the shift should be close to λ_m: a shift close to an eigenvalue makes inverse iteration converge fast (Theorem 61). For a single step, see [2, Section 4.4.4].

15.7 (SUPPLEMENT) Further Topics

15.7.1 Comparison with the Uniqueness of QR

The proof of Theorem 86 works like the proof that the QR factorization is unique (Theorem 33): take two solutions, look at the matrix W that maps one to the other, show that it is triangular and orthogonal, hence diagonal with entries of modulus one. These entries are the sign choices we cannot fix.

For $\mathbb{K} = \mathbb{C}$, the same proof works with Q, V unitary and $\top$ replaced by $*$. Then each column of Q is fixed up to a factor of modulus one.

15.7.2 The Rayleigh Quotient Shift

A natural estimate of λ_m is the bottom right entry,

$$\mu_k = t_{mm}^{(k-1)} = \vec{e}_m^* T_{k-1}\vec{e}_m,$$

the Rayleigh quotient of T_{k-1} at $\vec{e}_m$. By the inverse iteration view, the last column of $\bar{Q}_k$ then does Rayleigh quotient iteration (Algorithm 17). This gives local *quadratic* convergence [2, Section 4.4.4]:

- Let $\eta = \|T_{k-1}(m, 1{:}m-1)\| \,/\, \|A\| \ll 1$.
- Setting the off-diagonal part of the last row to zero changes a well separated eigenvalue by $O(\eta\,\|A\|)$. So $|\lambda_m - t_{mm}| = O(\eta\,\|A\|)$.
- The next step shrinks the last row by the factor $|\lambda_m - \mu| \,/\, \min_{j\neq m}|\lambda_j - \mu| = O(\eta)$, giving $O(\eta^2)$.

15.7.3 The Wilkinson Shift

The Rayleigh quotient shift can get stuck. Take $T_0 = \left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$. Then $\mu_1 = t_{22} = 0$. T_0 is orthogonal, so $U_1 = T_0$, $R_1 = I$, and $T_1 = T_0$. Nothing changes, because the eigenvalues ± 1 are equally far from the shift.

The *Wilkinson shift* takes the eigenvalue of the last 2×2 block of T_{k-1} that is closer to t_{mm}. In the example this is ± 1, an exact eigenvalue, so one step is enough. For symmetric tridiagonal matrices, QR iteration with the Wilkinson shift converges globally, at least linearly, and for almost all matrices cubically in the limit [2, Theorem 5.8].

15.7.4 When Is a Subdiagonal Entry ≈ 0?

T_k comes from A by unitary similarities in floating point, so it already carries rounding errors of size $O(\varepsilon_{\text{mach}})\,\|A\|$. A subdiagonal entry smaller than this may be set to zero; this is a backward stable perturbation [2, Section 4.4.5]. The iteration stops when all diagonal blocks are 1×1 (or 2×2 for complex conjugate pairs of a real matrix).

15.7.5 Why the Implicit Q Theorem Matters for Shifts

With a shift, the first column of U_k is parallel to the first column of $T_{k-1} - \mu I$, which is $(t_{11} - \mu, t_{21}, 0, \ldots, 0)^\top$. By Theorem 86, this column fixes U_k. So:

- we compute G_1 from these two numbers;
- the rest of the step is the bulge chasing of Algorithm 22 on T_{k-1} itself;
- we never form $T_{k-1} - \mu I$.

This is the *implicitly shifted* QR algorithm. For a real matrix with complex conjugate eigenvalues, two such steps with shifts μ and $\bar{\mu}$ combine into one *double-shift* step in real arithmetic (the Francis step) [2, Section 4.4.8].

Appendix A

Linear Algebra Basics

This appendix collects the first-course linear algebra that is used without comment throughout the course: matrix products, transposes, inverses, determinants, subspaces, eigenvalues, orthogonality, and symmetric matrices. Proofs are omitted; results proved in the main text are cross-referenced. Throughout, A and B are matrices of compatible sizes, c is a scalar, and $\vec{x}, \vec{y}$ are column vectors. Supplementary reading:

- [3] Chapters 2–7.
- [1] Lectures 1 and 2.
- [4] Section 7A.

A.1 Matrix Multiplication

Write $\vec{a}_j$ for the columns of $A \in \mathbb{K}^{m\times n}$ and $\vec{b}_i^{\top}$ for the rows of $B \in \mathbb{K}^{n\times p}$. The product AB can be read in four equivalent ways:

- *entries*: $(AB)_{ij} = \sum_k A_{ik}B_{kj}$;
- *columns*: $A\vec{x} = \sum_j x_j \vec{a}_j$ is a linear combination of the columns of A, and the j-th column of AB is A times the j-th column of B;
- *rows*: the i-th row of AB is the i-th row of A times B;
- *outer products*: $AB = \sum_{k=1}^{n} \vec{a}_k \vec{b}_k^{\top}$, a sum of rank-one matrices.

Multiplication is associative and distributive but not commutative. Multiplying by a diagonal matrix D on the left scales the rows (DA), on the right the columns (AD). Block matrices multiply like ordinary matrices as long as the block sizes are compatible.

A.2 Transpose and Conjugate Transpose

Definition 87 (Transpose and Conjugate Transpose)

For $A \in \mathbb{K}^{m\times n}$, the *transpose* $A^\top \in \mathbb{K}^{n\times m}$ and the *conjugate transpose* (or *adjoint*) $A^* \in \mathbb{K}^{n\times m}$ are

$$(A^\top)_{ij} = A_{ji}, \qquad (A^*)_{ij} = \overline{A_{ji}}. \tag{A.1}$$

For real matrices the two coincide.

Proposition 88 (Rules for the Transpose)

1. $(A^\top)^\top = A$.
2. $(A+B)^\top = A^\top + B^\top$ and $(cA)^\top = cA^\top$.
3. $(AB)^\top = B^\top A^\top$, and more generally $(A_1 A_2 \cdots A_k)^\top = A_k^\top \cdots A_2^\top A_1^\top$.
4. If A is invertible, so is $A^\top$, and $(A^\top)^{-1} = (A^{-1})^\top$, written $A^{-\top}$.
5. $\det A^\top = \det A$, $\operatorname{tr} A^\top = \operatorname{tr} A$, and $\operatorname{rank} A^\top = \operatorname{rank} A$.
6. $A^\top$ and A have the same characteristic polynomial, hence the same eigenvalues, but in general different eigenvectors.

The order reversal in (3) is the same as in $(AB)^{-1} = B^{-1}A^{-1}$: the factor applied last is undone first. Rule (4) follows from (3) by transposing $AA^{-1} = I$.

A.2.1 Vectors and Inner Products

Since a column vector is an $n \times 1$ matrix, the rules above apply to it directly.

- $(A\vec{x})^\top = \vec{x}^\top A^\top$.
- $\vec{x}^\top \vec{y} = \vec{y}^\top \vec{x}$, because $\vec{x}^\top \vec{y}$ is a 1×1 matrix, and a scalar is its own transpose.
- $(A\vec{x})^\top \vec{y} = \vec{x}^\top (A^\top \vec{y})$, i.e., $\langle A\vec{x}, \vec{y}\rangle = \left\langle \vec{x}, A^\top \vec{y}\right\rangle$. This *adjoint relation* moves a matrix from one side of an inner product to the other. It is the key step in deriving the normal equations (Lemma 34).

A.2.2 Block Matrices

A block matrix is transposed by transposing each block *and* the arrangement of the blocks:

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^\top = \begin{bmatrix} A^\top & C^\top \\ B^\top & D^\top \end{bmatrix}. \tag{A.2}$$

The off-diagonal blocks B and C swap places.

A.2.3 The Complex Case

For complex matrices the conjugate transpose $A^* = \overline{A}^\top$ plays the role of the transpose. All rules of Proposition 88 carry over, with complex conjugates wherever a scalar appears:

- $(cA)^* = \bar{c}A^*$ and $(AB)^* = B^*A^*$;
- $\det A^* = \overline{\det A}$, and the eigenvalues of A^* are the complex conjugates of those of A;
- $\vec{x}^*\vec{y} = \overline{\vec{y}^*\vec{x}}$, which in general is *not* equal to $\vec{y}^*\vec{x}$;
- $\langle A\vec{x}, \vec{y}\rangle = \langle \vec{x}, A^*\vec{y}\rangle$ for the standard inner product $\langle \vec{x}, \vec{y}\rangle = \vec{x}^*\vec{y}$ (Lecture 11).

Hermitian versus symmetric. A matrix is *Hermitian* if $A^* = A$ and *symmetric* if $A^\top = A$. For real matrices $A^* = A^\top$, so the two notions coincide, and every result about Hermitian matrices applies to real symmetric ones. For complex matrices they differ: a complex symmetric matrix is in general not Hermitian. For example,

$$A = \begin{bmatrix} 1 & i \\ i & 1 \end{bmatrix} \tag{A.3}$$

satisfies $A^\top = A$, but $A^* = \left[\begin{smallmatrix} 1 & -i \\ -i & 1 \end{smallmatrix}\right] \neq A$, and its eigenvalues $1 \pm i$ are not real. Complex symmetric matrices therefore enjoy none of the special properties of Hermitian matrices (real eigenvalues, unitary diagonalization, the Rayleigh quotient characterization). In these notes $\mathbb{K}$ is $\mathbb{R}$ or $\mathbb{C}$, and "Hermitian" means "symmetric" when $\mathbb{K} = \mathbb{R}$.

A.2.4 Useful Consequences

- A^*A and AA^* are Hermitian and positive semidefinite, since $\vec{x}^*A^*A\vec{x} = \|A\vec{x}\|_2^2 \geq 0$.
- Q is unitary if and only if $Q^*Q = I$, i.e., $Q^* = Q^{-1}$ (Section B.2).
- A^* and A have the same singular values, so $\|A^*\|_2 = \|A\|_2$ and $\kappa_2(A^*) = \kappa_2(A)$.
- $\|A^*\|_1 = \|A\|_\infty$, since transposing turns column sums into row sums (Proposition 96).

Remark 89 (Common Mistakes)**.**

- Writing $(AB)^\top = A^\top B^\top$. The order must be reversed.
- Assuming that the product of two symmetric matrices is symmetric. In fact $(AB)^\top = B^\top A^\top = BA$, which in general differs from AB.
- Using $^\top$ where * is meant for complex matrices, e.g., writing $A^\top = A$ for "Hermitian".

A.3 Inverses, Elementary and Triangular Matrices

A square matrix A is invertible if $AB = BA = I$ for some $B = A^{-1}$; for square matrices a one-sided inverse is automatically two-sided. Then $(AB)^{-1} = B^{-1}A^{-1}$ and $(A^{-1})^{-1} = A$.

Theorem 90 (Invertible Matrix Theorem)

For $A \in \mathbb{K}^{n\times n}$ the following are equivalent:

1. A is invertible;
2. $A\vec{x} = \vec{b}$ has a unique solution for every $\vec{b}$;
3. $A\vec{x} = \vec{0}$ has only the solution $\vec{x} = \vec{0}$;
4. the columns (equivalently, the rows) of A are linearly independent;
5. $\operatorname{rank} A = n$;
6. $\det A \neq 0$;
7. 0 is not an eigenvalue of A;
8. $\sigma_{\min}(A) > 0$.

Elementary matrices. Each elementary row operation (swap two rows, scale a row, add a multiple of one row to another) is left multiplication by the matrix obtained by applying that operation to I; the corresponding column operation is right multiplication. Gaussian elimination is therefore a sequence of left multiplications, which is the starting point of LU factorization (Lecture 3). A *permutation matrix* P, a row-permuted identity, satisfies $P^{-1} = P^{\top}$.

Triangular matrices. Products and inverses of lower (upper) triangular matrices are lower (upper) triangular, and if the diagonal entries are all 1 they stay 1. A triangular matrix is invertible if and only if all diagonal entries are nonzero, and its eigenvalues are its diagonal entries.

A.4 Determinants

- For triangular A, $\det A = \prod_i a_{ii}$.
- Row operations: swapping two rows flips the sign, scaling a row by c scales $\det A$ by c, and adding a multiple of one row to another leaves $\det A$ unchanged. Hence if $PA = LU$ with unit lower triangular L, then $\det A = \pm \prod_i u_{ii}$, which is how determinants are computed in practice.
- $\det(AB) = \det A \det B$, $\det A^{-1} = 1/\det A$, $\det(cA) = c^n \det A$ for $A \in \mathbb{K}^{n\times n}$, and $|\det Q| = 1$ for unitary Q. In general $\det(A + B) \neq \det A + \det B$.
- For block triangular matrices, $\det \left[\begin{smallmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{smallmatrix}\right] = \det A_{11} \det A_{22}$.
- $\det A = \prod_i \lambda_i$ and $\operatorname{tr} A = \sum_i \lambda_i$, with eigenvalues counted with algebraic multiplicity.

A.5 Subspaces and Rank

For $A \in \mathbb{K}^{m\times n}$, the *range* (column space) $\mathcal{R}(A) = \{A\vec{x}\} \subseteq \mathbb{K}^m$ and the *null space* $\mathcal{N}(A) = \{\vec{x} : A\vec{x} = \vec{0}\} \subseteq \mathbb{K}^n$ are subspaces. A *basis* of a subspace is a linearly independent spanning set; all bases have the same size, the *dimension*.

- $\operatorname{rank} A = \dim \mathcal{R}(A)$, and column rank equals row rank, so $\operatorname{rank} A \le \min(m, n)$.
- *Rank–nullity*: $\operatorname{rank} A + \dim \mathcal{N}(A) = n$.
- $\mathcal{R}(A)^\perp = \mathcal{N}(A^*)$, so $\mathbb{K}^m = \mathcal{R}(A) \oplus \mathcal{N}(A^*)$ as an orthogonal direct sum.
- $\mathcal{N}(A^*A) = \mathcal{N}(A)$, hence $\operatorname{rank} A^*A = \operatorname{rank} A$, and A^*A is invertible if and only if A has linearly independent columns (Lecture 8).
- $\operatorname{rank}(AB) \le \min(\operatorname{rank} A, \operatorname{rank} B)$, and multiplying by an invertible matrix does not change the rank.

A.6 Eigenvalues and Diagonalization

λ is an *eigenvalue* of $A \in \mathbb{K}^{n\times n}$ with *eigenvector* $\vec{x} \neq \vec{0}$ if $A\vec{x} = \lambda\vec{x}$. The eigenvalues are the roots of the characteristic polynomial $\det(\lambda I - A)$, so there are exactly n of them in $\mathbb{C}$, counted with multiplicity.

- A real matrix may have complex eigenvalues; they come in conjugate pairs $\lambda, \bar{\lambda}$, with conjugate eigenvectors.
- If $A\vec{x} = \lambda\vec{x}$, then $A^k\vec{x} = \lambda^k\vec{x}$, $(A - \sigma I)\vec{x} = (\lambda - \sigma)\vec{x}$, and $A^{-1}\vec{x} = \lambda^{-1}\vec{x}$ for invertible A. These are the facts behind the power iteration, shifts, and inverse iteration (Lecture 10).
- *Similar* matrices $B = S^{-1}AS$ have the same characteristic polynomial, hence the same eigenvalues; eigenvectors transform as $\vec{x} \mapsto S^{-1}\vec{x}$. Row operations alone are *not* similarity transformations and change the eigenvalues, which is why eigenvalue algorithms transform $A \mapsto Q^*AQ$ from both sides.
- Eigenvectors belonging to distinct eigenvalues are linearly independent.
- The *algebraic multiplicity* of λ is its multiplicity as a root of the characteristic polynomial, and the *geometric multiplicity* is $\dim \mathcal{N}(A - \lambda I)$; always $1 \le$ geometric $\le$ algebraic.
- A is *diagonalizable*, $A = X\Lambda X^{-1}$, if and only if it has n linearly independent eigenvectors, if and only if the two multiplicities agree for every eigenvalue. Then $A^k = X\Lambda^k X^{-1}$. A matrix with n distinct eigenvalues is diagonalizable; a matrix that is not is called *defective* (Lecture 11).

A.7 Orthogonality and Projections

- A set of nonzero, mutually orthogonal vectors is linearly independent. In an orthonormal basis $\vec{q}_1, \ldots, \vec{q}_n$ the coordinates are inner products: $\vec{y} = \sum_j (\vec{q}_j^*\vec{y})\vec{q}_j$.
- If $Q \in \mathbb{K}^{m\times n}$ has orthonormal columns, then $Q^*Q = I_n$ and $\|Q\vec{x}\|_2 = \|\vec{x}\|_2$. For $m > n$, however, $QQ^* \neq I_m$: it is the orthogonal projector onto $\mathcal{R}(Q)$.
- *Projection onto a line*: the projection of $\vec{y}$ onto $\operatorname{span}\{\vec{a}\}$ is $\frac{\vec{a}^*\vec{y}}{\vec{a}^*\vec{a}}\vec{a}$.
- *Orthogonal decomposition and best approximation*: for a subspace W, every $\vec{y}$ splits uniquely as $\vec{y} = \hat{\vec{y}} + \vec{z}$ with $\hat{\vec{y}} \in W$ and $\vec{z} \perp W$, and $\hat{\vec{y}}$ is the point of W closest to $\vec{y}$. If the columns of Q form an orthonormal basis of W, then $\hat{\vec{y}} = QQ^*\vec{y}$.
- *Gram–Schmidt* turns linearly independent columns into orthonormal ones and yields $A = QR$ (Lectures 5 and 7).

- *Least squares*: $\min_{\vec{x}} \left\| A\vec{x} - \vec{b} \right\|_2$ is attained exactly when $A^* A\vec{x} = A^*\vec{b}$; with $A = QR$ this becomes $R\vec{x} = Q^*\vec{b}$ (Lectures 5 and 8).

A.8 Hermitian Matrices, Quadratic Forms, and the SVD

- *Spectral theorem* (Theorem 65): a Hermitian A has real eigenvalues and an orthonormal basis of eigenvectors, $A = Q\Lambda Q^*$ with Q unitary and Λ real diagonal. Equivalently $A = \sum_i \lambda_i \vec{q}_i \vec{q}_i^*$ (spectral decomposition).
- A Hermitian A is *positive definite* if $\vec{x}^* A\vec{x} > 0$ for all $\vec{x} \neq \vec{0}$, equivalently if all its eigenvalues are positive; *semidefinite* versions use $\geq$. *Negative* (semi)definite is defined in the same way. A is *indefinite* if $\vec{x}^* A\vec{x}$ takes both signs, equivalently if it has both positive and negative eigenvalues. For example, $\left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ is indefinite: for real $\vec{x}$, $\vec{x}^\top A\vec{x} = 2x_1x_2$ is positive along $(1, 1)$ and negative along $(1, -1)$. Changing variables $\vec{x} = Q\vec{y}$ turns the quadratic form into $\sum_i \lambda_i |y_i|^2$ (principal axes theorem).
- On the unit sphere, $\vec{x}^* A\vec{x}$ ranges over $[\lambda_{\min}, \lambda_{\max}]$, attained at the corresponding eigenvectors (Definition 54).
- *SVD*: every $A \in \mathbb{K}^{m\times n}$ factors as $A = U\Sigma V^*$ with U, V unitary and Σ diagonal with entries $\sigma_1 \geq \cdots \geq 0$. The σ_i^2 are the eigenvalues of A^*A, the columns $\vec{v}_i$ of V are its orthonormal eigenvectors, and $\vec{u}_i = A\vec{v}_i/\sigma_i$ for $\sigma_i > 0$. The number of nonzero σ_i is $\operatorname{rank} A$ (Section B.3).

Appendix B

Norms and Unitary Matrices

This appendix collects facts about unitary matrices and matrix norms that are used throughout the course, starting from Lecture 6. We begin with vector and matrix norms in general, following [1, Lecture 3], and then specialize to the 2-norm and the Frobenius norm. Supplementary reading:

- [1] Lectures 2, 3, and 5.
- [2] Sections 3.2.3 and 3.4.

B.1 Vector and Matrix Norms

Norms are the yardsticks of numerical linear algebra: every statement about the size of an error, the accuracy of an approximation, or the convergence of an iteration is made in some norm.

B.1.1 Vector Norms

Definition 91 (Vector Norm)

A *norm* on $\mathbb{K}^m$ is a function $\|\cdot\| : \mathbb{K}^m \to \mathbb{R}$ such that for all $\vec{x}, \vec{y} \in \mathbb{K}^m$ and all $\alpha \in \mathbb{K}$:

1. $\|\vec{x}\| \geq 0$, and $\|\vec{x}\| = 0$ only if $\vec{x} = \vec{0}$;
2. $\|\vec{x} + \vec{y}\| \leq \|\vec{x}\| + \|\vec{y}\|$ (the *triangle inequality*);
3. $\|\alpha\vec{x}\| = |\alpha| \, \|\vec{x}\|$.

In words, a nonzero vector has positive length, the length of a sum is at most the sum of the lengths, and scaling a vector scales its length by the same amount. The most important norms are the p-norms.

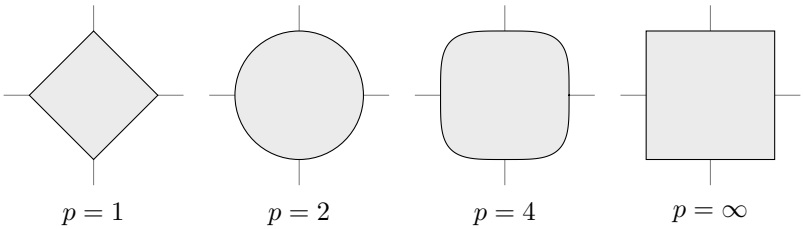
Definition 92 (p-Norms)

For $\vec{x} \in \mathbb{K}^m$,

$$\|\vec{x}\|_1 = \sum_{i=1}^{m} |x_i|\,, \qquad \|\vec{x}\|_2 = \left(\sum_{i=1}^{m} |x_i|^2\right)^{1/2} = \sqrt{\vec{x}^*\vec{x}}, \qquad \|\vec{x}\|_\infty = \max_{1\le i\le m} |x_i|\,, \tag{B.1}$$

and more generally $\|\vec{x}\|_p = (\sum_{i=1}^m |x_i|^p)^{1/p}$ for $1 \le p < \infty$.

A norm is determined by its closed unit ball $\{\vec{x} : \|\vec{x}\| \le 1\}$. For $m = 2$, this gives the following picture.



Unit balls of $\mathbb{R}^2$ in the p-norms. As p grows, the ball grows from the diamond ($p = 1$) to the square ($p = \infty$).

The 2-norm is the Euclidean length, and its unit ball is round. Since the balls are nested, the p-norms decrease as p increases; Corollary 100 below makes this precise, with constants depending on the dimension.

Aside from the p-norms, the most useful norms are the *weighted norms*. Given any norm $\|\cdot\|$ and a diagonal matrix $W = \operatorname{diag}(w_1, \dots, w_m)$ with all $w_i \ne 0$, we set

$$\|\vec{x}\|_W := \|W\vec{x}\|\,. \tag{B.2}$$

For example, the weighted 2-norm is $\|\vec{x}\|_W = \left(\sum_i |w_i x_i|^2\right)^{1/2}$, and its unit ball is an axis-aligned ellipse with semiaxes $1/|w_i|$. More generally, $\|W\vec{x}\|$ is a norm for *any* nonsingular W: properties 2 and 3 are inherited from $\|\cdot\|$ because W is linear, and $\|W\vec{x}\| = 0$ forces $W\vec{x} = \vec{0}$, hence $\vec{x} = \vec{0}$. Weighted norms are natural when the entries of $\vec{x}$ are measured in different units.

B.1.2 Induced Matrix Norms

An $m \times n$ matrix can be viewed as a vector in an mn-dimensional space, so any vector norm on $\mathbb{K}^{mn}$ measures the size of a matrix. More useful, however, are norms that measure what A *does* as a map from $\mathbb{K}^n$ to $\mathbb{K}^m$.

Definition 93 (Induced Matrix Norm)

Let $\|\cdot\|_{(n)}$ and $\|\cdot\|_{(m)}$ be norms on $\mathbb{K}^n$ and $\mathbb{K}^m$. The *induced matrix norm* of $A \in \mathbb{K}^{m\times n}$ is the smallest C such that $\|A\vec{x}\|_{(m)} \le C\,\|\vec{x}\|_{(n)}$ for all $\vec{x} \in \mathbb{K}^n$, that is,

$$\|A\|_{(m,n)} := \sup_{\vec{x}\ne\vec{0}} \frac{\|A\vec{x}\|_{(m)}}{\|\vec{x}\|_{(n)}} = \sup_{\|\vec{x}\|_{(n)}=1} \|A\vec{x}\|_{(m)}\,. \tag{B.3}$$

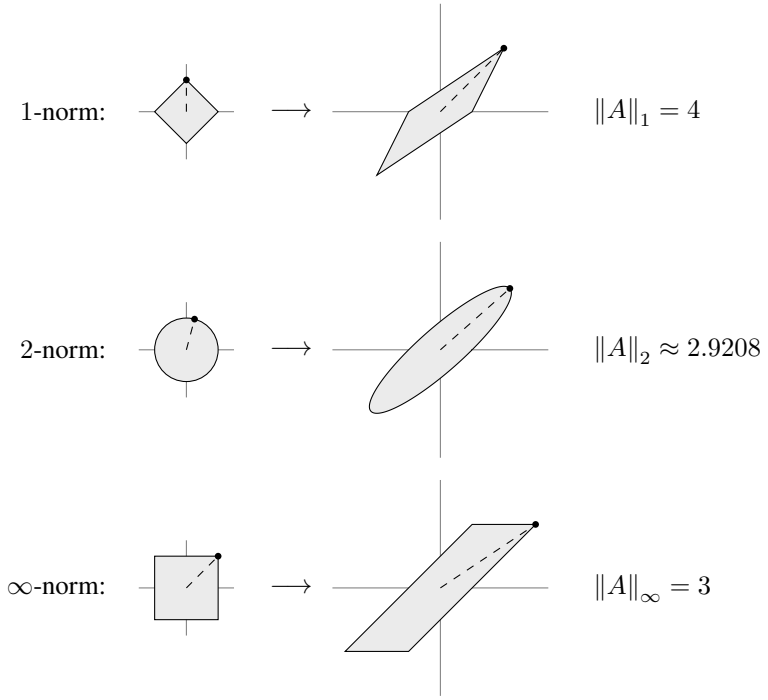
When both norms are p-norms, we write $\|A\|_p$.

So $\|A\|_{(m,n)}$ is the largest factor by which A can stretch a vector. The two expressions agree by property 3 of Definition 91: scaling $\vec{x}$ does not change the ratio, so it suffices to look at unit vectors. In particular, $\|A\|$ is determined by the image of the unit ball under A, which makes induced norms easy to visualize.

Example 94 (A 2×2 Matrix in Three Norms)**.** Take

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 2 \end{bmatrix}. \tag{B.4}$$

Since A is real, we may draw it as a map of $\mathbb{R}^2$. It sends $\vec{e}_1$ to its first column $(1,0)^\top$ and $\vec{e}_2$ to its second column $(2,2)^\top$. Mapping the unit balls of the 1-, 2-, and ∞-norms gives the following picture.



Left: unit balls of $\mathbb{R}^2$. Right: their images under A. The dashed vectors are the ones that A stretches the most in each norm.

In the 1-norm, the most amplified unit vector is $\vec{e}_2$, which is mapped to $(2,2)^\top$ with 1-norm 4. In the ∞-norm, it is $(1,1)^\top$, which is mapped to $(3,2)^\top$ with ∞-norm 3. In the 2-norm, the image of the unit circle is an ellipse, and $\|A\|_2$ is the length of its longest semiaxis, about 2.9208. This must be at least $\|A\vec{e}_2\|_2 = \sqrt{8} \approx 2.8284$; the exact value is $\sigma_{\max}(A)$ by Theorem 106.

Example 95 (The p-Norm of a Diagonal Matrix)**.** Let $D = \operatorname{diag}(d_1, \ldots, d_m)$. For $1 \le p < \infty$,

$$\|D\vec{x}\|_p^p = \sum_{i=1}^{m} |d_i x_i|^p \le \left(\max_i |d_i|\right)^p \|\vec{x}\|_p^p, \tag{B.5}$$

with equality for $\vec{x} = \vec{e}_i$ where i maximizes $|d_i|$; the case $p = \infty$ is the same argument with sums replaced by maxima. Hence $\|D\|_p = \max_i |d_i|$ for every p. Geometrically, D maps the 2-norm unit sphere to an ellipsoid with semiaxes $|d_i|$, and $\|D\|_2$ is the longest semiaxis. The SVD shows that *every* matrix maps the unit sphere to an ellipsoid; only the axes may be rotated.

For $p = 1$ and $p = \infty$, the induced norm has a closed form in terms of the entries.

Proposition 96 (The 1- and ∞-Norms of a Matrix)

Let $A \in \mathbb{K}^{m\times n}$ have columns $\vec{a}_1, \dots, \vec{a}_n$ and rows $\vec{\alpha}_1^*, \dots, \vec{\alpha}_m^*$. Then

$$\|A\|_1 = \max_{1\le j\le n} \|\vec{a}_j\|_1 \quad \text{(maximum column sum)}, \qquad \|A\|_\infty = \max_{1\le i\le m} \|\vec{\alpha}_i\|_1 \quad \text{(maximum row sum)}. \tag{B.6}$$

Proof. For the 1-norm, let $\|\vec{x}\|_1 = 1$. By the triangle inequality,

$$\|A\vec{x}\|_1 = \left\| \sum_{j=1}^n x_j \vec{a}_j \right\|_1 \le \sum_{j=1}^n |x_j| \, \|\vec{a}_j\|_1 \le \max_j \|\vec{a}_j\|_1 , \tag{B.7}$$

since the weights $|x_j|$ sum to 1. Taking $\vec{x} = \vec{e}_j$ for the column j with the largest 1-norm attains the bound.

For the ∞-norm, let $\|\vec{x}\|_\infty = 1$. Each entry of $A\vec{x}$ satisfies $|(A\vec{x})_i| \le \sum_j |A_{ij}| \, |x_j| \le \|\vec{\alpha}_i\|_1$, so $\|A\vec{x}\|_\infty \le \max_i \|\vec{\alpha}_i\|_1$. To attain it, pick the row i with the largest 1-norm and set $x_j = \overline{A_{ij}} / |A_{ij}|$ (and $x_j = 1$ if $A_{ij} = 0$). Then $\|\vec{x}\|_\infty = 1$ and $(A\vec{x})_i = \sum_j |A_{ij}| = \|\vec{\alpha}_i\|_1$. □

In the example above, the column sums are 1 and 4 and the row sums are 3 and 2, matching the picture. Both norms cost only $O(mn)$ operations, whereas $\|A\|_2$ requires the largest singular value. Computing $\|A\|_p$ for other p is harder still.

B.1.3 Cauchy–Schwarz and Hölder Inequalities

To bound matrix norms when $p \neq 1, \infty$, we bound inner products by p-norms.

Theorem 97 (Hölder and Cauchy–Schwarz Inequalities)

Let $1 \le p, q \le \infty$ with $1/p + 1/q = 1$. Then for all $\vec{x}, \vec{y} \in \mathbb{K}^m$,

$$|\vec{x}^* \vec{y}| \le \|\vec{x}\|_p \, \|\vec{y}\|_q . \tag{B.8}$$

The case $p = q = 2$ is the *Cauchy–Schwarz inequality* $|\vec{x}^*\vec{y}| \le \|\vec{x}\|_2 \, \|\vec{y}\|_2$.

Both bounds are tight: for every $\vec{x}$ there is a $\vec{y} \neq \vec{0}$ attaining equality. For Cauchy–Schwarz, $\vec{y} = \vec{x}$ works. Proofs can be found in any linear algebra text. These inequalities immediately give the 2-norm of two simple kinds of matrices.

Example 98 (The 2-Norm of a Row Vector)**.** Let $A = \vec{a}^*$ be a single row. By Cauchy–Schwarz, $\|A\vec{x}\|_2 = |\vec{a}^*\vec{x}| \le \|\vec{a}\|_2 \, \|\vec{x}\|_2$, with equality for $\vec{x} = \vec{a}$. Hence $\|\vec{a}^*\|_2 = \|\vec{a}\|_2$.

Example 99 (The 2-Norm of an Outer Product)**.** Let $A = \vec{u}\vec{v}^*$ with $\vec{u} \in \mathbb{K}^m$ and $\vec{v} \in \mathbb{K}^n$. For every $\vec{x} \in \mathbb{K}^n$,

$$\|A\vec{x}\|_2 = \|\vec{u}(\vec{v}^*\vec{x})\|_2 = \|\vec{u}\|_2 \, |\vec{v}^*\vec{x}| \le \|\vec{u}\|_2 \, \|\vec{v}\|_2 \, \|\vec{x}\|_2 , \tag{B.9}$$

with equality for $\vec{x} = \vec{v}$. Hence $\|\vec{u}\vec{v}^*\|_2 = \|\vec{u}\|_2 \, \|\vec{v}\|_2$. The Frobenius norm gives the same value: $\|\vec{u}\vec{v}^*\|_F^2 = \sum_{i,j} |u_i|^2 \, |v_j|^2 = \|\vec{u}\|_2^2 \, \|\vec{v}\|_2^2$, consistent with item 3 of Proposition 108 for $r = 1$.

Cauchy–Schwarz also lets us compare the p-norms in a fixed dimension.

Corollary 100 (Comparing p-Norms)**.** *For $\vec{x} \in \mathbb{K}^m$,*

$$\|\vec{x}\|_\infty \le \|\vec{x}\|_2 \le \|\vec{x}\|_1 \le \sqrt{m}\, \|\vec{x}\|_2 \le m\, \|\vec{x}\|_\infty , \tag{B.10}$$

and for $A \in \mathbb{K}^{m\times n}$,

$$\frac{1}{\sqrt{n}} \|A\|_\infty \le \|A\|_2 \le \sqrt{m}\, \|A\|_\infty , \qquad \frac{1}{\sqrt{m}} \|A\|_1 \le \|A\|_2 \le \sqrt{n}\, \|A\|_1 . \tag{B.11}$$

Proof. The first two inequalities hold because $\max_i |x_i|^2 \le \sum_i |x_i|^2 \le (\sum_i |x_i|)^2$. The third is Cauchy–Schwarz applied to the vector of the $|x_i|$ and the all-ones vector, and the fourth is $\sum_i |x_i|^2 \le m \max_i |x_i|^2$.

For the matrix bounds, chain the vector bounds through the definition. For instance,

$$\|A\vec{x}\|_2 \le \sqrt{m}\, \|A\vec{x}\|_\infty \le \sqrt{m}\, \|A\|_\infty \|\vec{x}\|_\infty \le \sqrt{m}\, \|A\|_\infty \|\vec{x}\|_2 \tag{B.12}$$

gives $\|A\|_2 \le \sqrt{m}\, \|A\|_\infty$, and $\|A\vec{x}\|_\infty \le \|A\vec{x}\|_2 \le \|A\|_2 \|\vec{x}\|_2 \le \sqrt{n}\, \|A\|_2 \|\vec{x}\|_\infty$ gives the other bound. The 1-norm bounds follow in the same way. □

In particular, all p-norms agree up to factors that depend only on the dimension. Such factors are harmless in $O(\varepsilon_{\text{mach}})$ statements, so a method that is stable in one p-norm is stable in all of them.

B.1.4 Bounding $\|AB\|$

Proposition 101 (Submultiplicativity of Induced Norms)

Let $\|\cdot\|_{(\ell)}$, $\|\cdot\|_{(m)}$, $\|\cdot\|_{(n)}$ be norms on $\mathbb{K}^\ell$, $\mathbb{K}^m$, $\mathbb{K}^n$, and let $A \in \mathbb{K}^{\ell\times m}$ and $B \in \mathbb{K}^{m\times n}$. Then

$$\|AB\|_{(\ell,n)} \le \|A\|_{(\ell,m)}\, \|B\|_{(m,n)} . \tag{B.13}$$

Proof. Apply the definition twice: $\|AB\vec{x}\|_{(\ell)} \le \|A\|_{(\ell,m)}\, \|B\vec{x}\|_{(m)} \le \|A\|_{(\ell,m)}\, \|B\|_{(m,n)}\, \|\vec{x}\|_{(n)}$. □

In particular, $\left\|A^k\right\| \le \|A\|^k$ for a square matrix in any induced norm. This is generally not an equality. For example, $A = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$ has $\|A\|_p = 1$ for every p, but $A^2 = 0$.

A related bound concerns eigenvalues. If $A\vec{x} = \lambda\vec{x}$ with $\vec{x} \ne \vec{0}$, then $|\lambda|\, \|\vec{x}\| = \|A\vec{x}\| \le \|A\|\, \|\vec{x}\|$. So the *spectral radius* $\rho(A) \coloneqq \max\{|\lambda| : \lambda \text{ an eigenvalue of } A\}$ satisfies

$$\rho(A) \le \|A\| \tag{B.14}$$

in every induced norm. The same example shows that the inequality can be strict: $\rho(A) = 0$ but $\|A\| = 1$.

B.1.5 General Matrix Norms

Matrix norms do not have to be induced by vector norms. In general, a *matrix norm* is any function on $\mathbb{K}^{m\times n}$ that satisfies the three conditions of Definition 91 on this mn-dimensional vector space:

$$\|A\| \ge 0 \text{ with equality only if } A = 0, \qquad \|A + B\| \le \|A\| + \|B\| , \qquad \|\alpha A\| = |\alpha|\, \|A\| . \tag{B.15}$$

The most important matrix norm that is not induced by a vector norm is the Frobenius norm, the subject of Section B.4. Finally, the 2-norm and the Frobenius norm are both invariant under multiplication by unitary matrices; this is Theorem 103 below.

B.2 Unitary Transformations

The error analysis of Lecture 6 uses one property of unitary matrices: they do not change the length of a vector. In this section we collect this and a few related facts, which we also use in Lectures 5 and 7.

Definition 102 (Unitary and Orthogonal Matrices)

We say that a square matrix $Q \in \mathbb{K}^{m\times m}$ is *unitary* if $Q^*Q = I$. When $\mathbb{K} = \mathbb{R}$, this condition reads $Q^\top Q = I$, and we call Q *orthogonal*. We say that a tall matrix $\hat{Q} \in \mathbb{K}^{m\times n}$ has *orthonormal columns* if $\hat{Q}^*\hat{Q} = I_n$.

The condition $Q^*Q = I$ just says that the columns of Q are orthonormal. Since Q is square, it also gives $Q^{-1} = Q^*$ and $QQ^* = I$. In particular, inverting a unitary matrix costs nothing: the solution of $Q\vec{x} = \vec{b}$ is $\vec{x} = Q^*\vec{b}$.

Theorem 103 (Unitary Matrices Preserve the 2-Norm)

Let $Q \in \mathbb{K}^{m\times m}$ be unitary. Then for all $\vec{x}, \vec{y} \in \mathbb{K}^m$,

$$\langle Q\vec{x}, Q\vec{y}\rangle = \langle \vec{x}, \vec{y}\rangle\,, \qquad \|Q\vec{x}\|_2 = \|\vec{x}\|_2\,. \tag{B.16}$$

As a consequence, for any $A \in \mathbb{K}^{m\times n}$ and unitary $U \in \mathbb{K}^{m\times m}$, $V \in \mathbb{K}^{n\times n}$:

1. $\|UAV\|_2 = \|A\|_2$ and $\|UAV\|_F = \|A\|_F$.
2. UAV has the same singular values as A. In particular, $\kappa_2(UAV) = \kappa_2(A)$.
3. $\|Q\|_2 = \|Q^{-1}\|_2 = 1$ and $\kappa_2(Q) = 1$. All singular values of Q are 1, every eigenvalue of Q satisfies $|\lambda| = 1$, and $|\det Q| = 1$.

Proof. We have $\langle Q\vec{x}, Q\vec{y}\rangle = \vec{x}^*Q^*Q\vec{y} = \vec{x}^*\vec{y}$. Taking $\vec{y} = \vec{x}$ gives $\|Q\vec{x}\|_2 = \|\vec{x}\|_2$.

For item 1, the norm identity gives $\|UA\vec{x}\|_2 = \|A\vec{x}\|_2$ for every $\vec{x}$, so $\|UA\|_2 = \|A\|_2$. For the right factor, note that $\vec{y} = V\vec{x}$ runs over all unit vectors as $\vec{x}$ does, so

$$\|AV\|_2 = \sup_{\|\vec{x}\|_2=1} \|AV\vec{x}\|_2 = \sup_{\|\vec{y}\|_2=1} \|A\vec{y}\|_2 = \|A\|_2\,. \tag{B.17}$$

The statement for the Frobenius norm is item 1 of Proposition 108.

For item 2, let $A = W\Sigma Z^*$ be an SVD. Then $UAV = (UW)\Sigma(V^*Z)^*$. Products of unitary matrices are unitary, so this is an SVD of UAV with the same Σ.

For item 3, $Q = Q \cdot I \cdot I^*$ is an SVD with $\Sigma = I$. If $Q\vec{x} = \lambda\vec{x}$ with $\vec{x} \neq \vec{0}$, then $\|\vec{x}\|_2 = \|Q\vec{x}\|_2 = |\lambda|\,\|\vec{x}\|_2$, so $|\lambda| = 1$. Finally, $|\det Q|^2 = \det(Q^*Q) = \det I = 1$. □

The converse is also true: if $\|Q\vec{x}\|_2 = \|\vec{x}\|_2$ for every $\vec{x}$, then Q is unitary. (One recovers inner products from norms by the polarization identity.) In other words, unitary matrices are exactly the linear maps that preserve lengths. Geometrically, they are rotations and reflections.

B.2.1 Examples

Several matrices we have already seen are unitary.

- *Householder reflectors* $F = I - 2\vec{v}\vec{v}^*/(\vec{v}^*\vec{v})$. They are Hermitian, satisfy $F^2 = I$, and have $\det F = -1$.
- *Givens rotations* (Lecture 7). They have $\det = 1$ and change only two coordinates.
- *Permutation matrices*. So the row swaps of partial pivoting (Lecture 4) do not change any norms, and all of the instability of LU comes from the triangular factors.
- *Diagonal matrices* $D = \operatorname{diag}(d_1, \ldots, d_m)$ with $|d_i| = 1$. These describe exactly the freedom in the QR factorization, since $A = (QD)(D^*R)$. The condition $r_{jj} > 0$ removes this freedom (Theorem 33).

B.2.2 Tall Matrices with Orthonormal Columns

Now let $\hat{Q} \in \mathbb{K}^{m\times n}$ with $m > n$ have orthonormal columns. The same one-line proof shows that $\left\|\hat{Q}\vec{x}\right\|_2 = \|\vec{x}\|_2$, and hence, as in item 1 of Theorem 103, $\left\|\hat{Q}B\right\|_2 = \|B\|_2$ and $\left\|\hat{Q}B\right\|_F = \|B\|_F$ for every $B \in \mathbb{K}^{n\times p}$. Taking conjugate transposes, the same holds for multiplication on the right by a matrix with orthonormal rows. However, $\hat{Q}^*$ does *not* preserve lengths. The matrix $\hat{Q}\hat{Q}^*$ is not the identity; it is the orthogonal projector onto $\mathcal{R}(\hat{Q})$. So $\left\|\hat{Q}^*\vec{y}\right\|_2 \le \|\vec{y}\|_2$, with equality only if $\vec{y} \in \mathcal{R}(\hat{Q})$.

This is exactly what makes QR useful for least squares. Take a full QR factorization $A = QR = \begin{bmatrix}\hat{Q} & \hat{Q}_\perp\end{bmatrix}\begin{bmatrix}\hat{R}\\ 0\end{bmatrix}$. Since Q^* preserves lengths, we can apply it inside the norm:

$$\left\|A\vec{x} - \vec{b}\right\|_2^2 = \left\|Q^*(A\vec{x} - \vec{b})\right\|_2^2 = \left\|\hat{R}\vec{x} - \hat{Q}^*\vec{b}\right\|_2^2 + \left\|\hat{Q}_\perp^*\vec{b}\right\|_2^2. \tag{B.18}$$

The second term does not depend on $\vec{x}$. So the minimizer solves $\hat{R}\vec{x} = \hat{Q}^*\vec{b}$, and the minimal residual is $\left\|\hat{Q}_\perp^*\vec{b}\right\|_2$ (compare Lecture 5).

B.2.3 Why Unitary Transformations Are Good for Computation

1. *One step does not amplify errors.* In Theorem 26 we have $\kappa(Q_k) = 1$, so each step has relative backward error $O(\varepsilon_{\text{mach}})$ with no extra factor.
2. *Many steps do not amplify errors.* A product of unitary matrices is unitary. So when we pull an error back through all the previous steps, its size does not change. The backward errors of n steps add up to $O(n\varepsilon_{\text{mach}})$ instead of multiplying (see Section 6.7).
3. *No element growth.* Every intermediate matrix has the same 2-norm as A, so there is nothing like the growth factor of LU. Compare this with Example 27: a triangular matrix with entries bounded by 1 can still have an exponentially large inverse, while $\left\|Q^{-1}\right\|_2 = \|Q\|_2 = 1$ always.
4. *Errors look the same before and after.* An error of size δ in $\vec{x}$ is an error of size δ in $Q\vec{x}$. So we can move errors between the input and the output without losing anything.

Note that this invariance holds for the 2-norm and the Frobenius norm, but not for other norms. For example, rotating $\vec{e}_1$ by $45°$ gives $(1/\sqrt{2}, 1/\sqrt{2})^\top$, whose ∞-norm is $1/\sqrt{2}$ and whose 1-norm is $\sqrt{2}$. This is why stability results for orthogonal methods are stated in the 2-norm. In other norms they still hold, but with extra constants that depend on the dimension.

As a final note, everything above assumes that the transformation is *exactly* unitary. A matrix that we compute column by column, such as the $\hat{Q}$ of modified Gram–Schmidt, is only unitary up to $O(\varepsilon_{\text{mach}}\kappa(A))$, and it

loses these advantages to the same extent. Householder's Q is different. Each reflector built from a computed vector $\tilde{\vec{v}}_k$ is exactly unitary, so rounding errors only change *which* unitary matrix we get, not *whether* it is unitary (Lecture 7).

B.3 The 2-Norm, Singular Values, and Condition Numbers

Most error bounds in this course are stated in the 2-norm. Here we record how it is expressed through the SVD; these facts are proved in Homework 1, Problem 3.

Definition 104 (Full and Reduced SVD)

Let $A \in \mathbb{K}^{m\times n}$ and $p = \min(m,n)$. A *full SVD* is a factorization

$$A = U\Sigma V^*, \qquad U \in \mathbb{K}^{m\times m}, \quad \Sigma \in \mathbb{R}^{m\times n}, \quad V \in \mathbb{K}^{n\times n}, \tag{B.19}$$

with U and V unitary and Σ "diagonal" of the same shape as A: only $\Sigma_{11}, \dots, \Sigma_{pp}$ can be nonzero, and these are the *singular values* $\sigma_1 \geq \cdots \geq \sigma_p \geq 0$. Dropping the zero blocks of Σ together with the columns of U or V they multiply gives the *reduced SVD*

$$A = \hat{U}\hat{\Sigma}\hat{V}^*, \qquad \hat{U} \in \mathbb{K}^{m\times p}, \quad \hat{\Sigma} = \operatorname{diag}(\sigma_1, \dots, \sigma_p) \in \mathbb{R}^{p\times p}, \quad \hat{V} \in \mathbb{K}^{n\times p}, \tag{B.20}$$

where $\hat{U}$ and $\hat{V}$ have orthonormal columns.

In the full SVD, Σ has a zero block below or beside its square diagonal part $D = \operatorname{diag}(\sigma_1, \dots, \sigma_p)$:

$$\Sigma = \begin{bmatrix} D \\ 0 \end{bmatrix} \ (m > n), \qquad \Sigma = \begin{bmatrix} D & 0 \end{bmatrix} \ (m < n). \tag{B.21}$$

The reduced form is more economical for computation, while the full form is more convenient in proofs, because square unitary U and V preserve norms in both directions (Theorem 103). Every matrix has an SVD, and the singular values are unique [1, Theorem 4.1].

Definition 105 (Induced 2-Norm)

The 2-*norm* of $A \in \mathbb{K}^{m\times n}$ is the matrix norm induced by the vector 2-norm (Definition 93),

$$\|A\|_2 \coloneqq \sup_{\vec{x}\neq\vec{0}} \frac{\|A\vec{x}\|_2}{\|\vec{x}\|_2}. \tag{B.22}$$

Theorem 106 (The 2-Norm Through the SVD)

Let $A \in \mathbb{K}^{m\times n}$ have singular values $\sigma_{\max} = \sigma_1 \geq \cdots \geq \sigma_{\min(m,n)} = \sigma_{\min}$. Then:

1. $\|A\|_2 = \sigma_{\max}$.
2. If $m \geq n$, then $\|A\vec{x}\|_2 \geq \sigma_{\min} \|\vec{x}\|_2$ for all $\vec{x}$, with equality for the last right singular vector.
3. If A is square and invertible, then $\left\|A^{-1}\right\|_2 = 1/\sigma_{\min}$, and hence

$$\kappa_2(A) \coloneqq \|A\|_2 \left\|A^{-1}\right\|_2 = \frac{\sigma_{\max}}{\sigma_{\min}}. \tag{B.23}$$

Proof. Take an SVD $A = U\Sigma V^*$ and put $\vec{y} = V^*\vec{x}$. Since U and V are unitary (Theorem 103), $\|A\vec{x}\|_2 = \|\Sigma\vec{y}\|_2$ and $\|\vec{y}\|_2 = \|\vec{x}\|_2$. Now

$$\|\Sigma\vec{y}\|_2^2 = \sum_{i=1}^{\min(m,n)} \sigma_i^2 |y_i|^2 . \tag{B.24}$$

The sum stops at $\min(m,n)$, not at n: for $m < n$ the last $n-m$ entries of $\vec{y}$ are multiplied by the zero block of Σ and do not contribute. In particular a wide matrix has a nontrivial null space, which is why item 2 needs $m \geq n$. This is at most $\sigma_{\max}^2 \|\vec{y}\|_2^2$, with equality for $\vec{y} = \vec{e}_1$, which gives item 1. If $m \geq n$, the sum runs over all n entries of $\vec{y}$, so it is at least $\sigma_{\min}^2 \|\vec{y}\|_2^2$, with equality for $\vec{y} = \vec{e}_n$; this gives item 2. For item 3, $A^{-1} = V\Sigma^{-1}U^*$ is an SVD up to reordering, so the singular values of A^{-1} are the $1/\sigma_i$, and the largest of them is $1/\sigma_{\min}$. □

For a tall matrix with full column rank we *define* $\kappa_2(A) \coloneqq \sigma_{\max}/\sigma_{\min}$; this is the condition number that governs least squares (Lecture 5). In both cases, $\kappa_2(A)$ is the worst-case condition number of the map $\vec{x} \mapsto A\vec{x}$ from Lecture 1:

$$\sup_{\vec{x}\neq\vec{0}} \frac{\|A\|_2 \|\vec{x}\|_2}{\|A\vec{x}\|_2} = \frac{\sigma_{\max}}{\sigma_{\min}}, \tag{B.25}$$

since $\|A\vec{x}\|_2 / \|\vec{x}\|_2$ is smallest, equal to $\sigma_{\min}$, for the last right singular vector. In words, $\kappa_2(A)$ is the ratio of the largest to the smallest stretching factor of A.

B.4 The Frobenius Norm

In Lecture 6 we bounded $\|L\|_2$ through the Frobenius norm. It is often the easiest matrix norm to compute and to estimate.

Definition 107 (Frobenius Norm)

The *Frobenius norm* of $A \in \mathbb{K}^{m\times n}$ is

$$\|A\|_F \coloneqq \left(\sum_{i=1}^{m} \sum_{j=1}^{n} |A_{ij}|^2 \right)^{1/2}. \tag{B.26}$$

In other words, $\|A\|_F$ is the Euclidean norm of A if we stack its columns into one long vector of length mn. Equivalently, $\|A\|_F^2 = \sum_j \|\vec{a}_j\|_2^2$ is the sum of the squared column norms. These squared column norms are

exactly the diagonal entries of A^*A, and the squared row norms are the diagonal entries of AA^*, so

$$\|A\|_F^2 = \operatorname{trace}(A^*A) = \operatorname{trace}(AA^*). \tag{B.27}$$

Proposition 108 (Properties of the Frobenius Norm)

Let $A \in \mathbb{K}^{m\times n}$ have rank r and singular values $\sigma_1 \geq \cdots \geq \sigma_r > 0$. Then:

1. *Unitary invariance.* $\|UAV\|_F = \|A\|_F$ for all unitary $U \in \mathbb{K}^{m\times m}$ and $V \in \mathbb{K}^{n\times n}$.
2. *Singular values.* $\|A\|_F = (\sigma_1^2 + \cdots + \sigma_r^2)^{1/2}$.
3. *Comparison with the 2-norm.* $\|A\|_2 \leq \|A\|_F \leq \sqrt{r}\,\|A\|_2 \leq \sqrt{\min(m,n)}\,\|A\|_2$.
4. *Compatibility and submultiplicativity.* $\|A\vec{x}\|_2 \leq \|A\|_F\,\|\vec{x}\|_2$, and $\|AB\|_F \leq \|A\|_2\,\|B\|_F \leq \|A\|_F\,\|B\|_F$.
5. *Only absolute values matter.* $\|A\|_F = \||A|\|_F$, and if $|A| \leq |B|$ entrywise, then $\|A\|_F \leq \|B\|_F$.

Proof. For item 1, using $U^*U = I$ and the cyclic property of the trace,

$$\|UAV\|_F^2 = \operatorname{trace}(V^*A^*U^*UAV) = \operatorname{trace}(V^*A^*AV) = \operatorname{trace}(A^*A) = \|A\|_F^2\,. \tag{B.28}$$

For item 2, take an SVD $A = U\Sigma V^*$. By item 1, $\|A\|_F = \|\Sigma\|_F = (\sum_i \sigma_i^2)^{1/2}$.

For item 3, use item 2 and $\|A\|_2 = \sigma_1$: we have $\sigma_1^2 \leq \sum_i \sigma_i^2 \leq r\sigma_1^2$.

For item 4, let $\vec{\alpha}_i^*$ be the i-th row of A. By the Cauchy–Schwarz inequality, $|\vec{\alpha}_i^*\vec{x}|^2 \leq \|\vec{\alpha}_i\|_2^2\,\|\vec{x}\|_2^2$. Summing over i gives $\|A\vec{x}\|_2^2 \leq \|A\|_F^2\,\|\vec{x}\|_2^2$. For the product, apply $\left\|A\vec{b}_j\right\|_2 \leq \|A\|_2\left\|\vec{b}_j\right\|_2$ to each column $\vec{b}_j$ of B and sum the squares. The last inequality then follows from $\|A\|_2 \leq \|A\|_F$.

Item 5 follows directly from the definition. □

Note that the Frobenius norm is *not* induced by any vector norm. Every induced norm satisfies $\|I\| = \sup_{\vec{x}\neq\vec{0}} \|\vec{x}\| / \|\vec{x}\| = 1$, but $\|I_n\|_F = \sqrt{n}$. Still, by item 4 it is compatible with the Euclidean norm on vectors.

B.4.1 Why It Is Useful in Error Analysis

- *It is cheap.* Computing $\|A\|_F$ takes $O(mn)$ operations, while $\|A\|_2 = \sigma_1$ requires the SVD (or an estimate of it). By item 3, the two norms differ by at most a factor of $\sqrt{\min(m,n)}$, and such dimension-dependent constants do not matter in $O(\varepsilon_{\text{mach}})$ statements.
- *It turns entrywise bounds into norm bounds.* The backward error of LU from Lecture 4 satisfies $|E| \leq m\varepsilon_{\text{mach}}\,|L|\,|U|$. By items 5 and 4,

$$\|E\|_F \leq m\varepsilon_{\text{mach}}\,\||L|\,|U|\|_F \leq m\varepsilon_{\text{mach}}\,\|L\|_F\,\|U\|_F\,. \tag{B.29}$$

  With partial pivoting, $\|L\|_F \leq m$ (Lecture 6), so everything depends on $\|U\|_F$, i.e., on the growth factor.
- *It is unitarily invariant.* Just like the 2-norm, the Frobenius norm of an error does not change under orthogonal transformations such as Householder reflectors.

B.4.2 Low-Rank Approximation

Since $\|A-B\|_F^2 = \sum_{i,j} |A_{ij} - B_{ij}|^2$ is a sum of squared entrywise errors, approximating a matrix in the Frobenius norm is a least squares problem. The Eckart–Young theorem also holds in this norm: the truncated SVD $A_k = \sum_{i \le k} \sigma_i \vec{u}_i \vec{v}_i^*$ satisfies

$$\min_{\operatorname{rank}(B) \le k} \|A - B\|_F = \|A - A_k\|_F = \left(\sigma_{k+1}^2 + \cdots + \sigma_r^2\right)^{1/2}. \tag{B.30}$$

In the 2-norm, the corresponding error is $\|A - A_k\|_2 = \sigma_{k+1}$ [2, §3.2.3]. We will come back to this when the course covers low-rank approximation.

Appendix C

The Floating-Point Model

This appendix collects background on floating-point arithmetic that is used throughout the course. Supplementary reading:

- [1] Lectures 13 and 14.
- [2] Section 1.5.
- [5] Section 2.4.

C.1 IEEE 754 Formats

The formats in common use are listed below; b_p counts the implicit bit, so a significand field with $b_p - 1$ stored bits provides b_p bits of precision.

Format	Total bits	Exponent bits	Stored significand bits	b_p	Bias	Decimal digits
Float64	64	11	52	53	1023	≈ 15.9
Float32	32	8	23	24	127	≈ 7.2
Float16	16	5	10	11	15	≈ 3.3
BFloat16	16	8	7	8	127	≈ 2.4

The exponent field uses a biased encoding, and its two extreme values are reserved:

- An all-zeros exponent encodes ± 0 (if the significand is also zero) or a *subnormal* number, which drops the implicit leading 1 and uses $0.f \times 2^{E_{\min}}$. Subnormals provide *gradual underflow*, which guarantees $x \neq y \implies x - y \neq 0$.
- An all-ones exponent encodes $\pm\infty$ (zero significand) or `NaN` (nonzero significand).

The normal range of Float64 is therefore roughly 2.2×10^{-308} to 1.8×10^{308}. IEEE defines five exceptions (invalid, division by zero, overflow, underflow, inexact), and specifies that ∞ and `NaN` propagate through arithmetic instead of halting it. Demmel [2, §1.5] stresses the value of this: an algorithm can run to completion and check its result at the end, instead of branching at every step. Note that `NaN != NaN` is always true, so one must test with `isnan`.

C.2 Machine Epsilon and the Model of Arithmetic

C.2.1 Two Conventions for Machine Epsilon

Two different quantities are both called "machine epsilon" in the literature, and they differ by a factor of 2.

Definition 109 (Machine Epsilon and Unit Roundoff)

Let b_p be the precision of a floating-point format.

- The *machine epsilon* $\varepsilon = \varepsilon_{\text{mach}} := 2^{-(b_p-1)}$ is the distance from 1 to the next larger floating-point number. For Float64, $\varepsilon = 2^{-52} \approx 2.22 \times 10^{-16}$. This is the convention we use, as do Julia's `eps(Float64)`, and of C's `DBL_EPSILON`.
- The *unit roundoff* $u := 2^{-b_p} = \varepsilon/2$ is the largest relative rounding error, i.e., half a spacing. For Float64, $u = 2^{-53} \approx 1.11 \times 10^{-16}$. Trefethen and Bau [1] and Demmel [2] call this quantity *machine epsilon* ($\epsilon_{\text{machine}}$, resp. ε), and Golub and Van Loan [5] call it the unit roundoff. So when copying a bound from these books, their "machine epsilon" is our $u = \varepsilon_{\text{mach}}/2$.

The two quantities answer different questions: ε describes how dense the floating-point grid is, while u describes how much error rounding to that grid causes. They differ by a factor of 2 because rounding goes to the *nearest* floating-point number. On $[2^e, 2^{e+1})$ the spacing is $2^e\varepsilon$, a real number x in this interval is at most half a spacing from its nearest neighbour, and therefore

$$\frac{|\mathrm{fl}(x) - x|}{|x|} \le \frac{2^e\varepsilon/2}{2^e} = \frac{\varepsilon}{2} = u. \tag{C.1}$$

Both conventions survive for historical and practical reasons. Many early machines rounded by *chopping*, i.e., discarding the extra bits, and then the largest relative error is a whole spacing, so the two quantities coincided. They separated only once IEEE 754 made round to nearest the default. Since then, authors on error analysis (Trefethen–Bau, Higham, Golub–Van Loan) use u, because what they need is the bound on δ in $\mathrm{fl}(x) = x(1+\delta)$, whereas programming languages report ε, because the gap from 1 to `nextfloat(1.0)` is what can be observed directly.

Any statement of the form $O(\varepsilon)$ is unaffected by the choice, but constants copied from different textbooks, such as $|\delta| \le u$ or $\gamma_n = nu/(1-nu)$, must be read with care.

The IEEE default rounding mode is *round to nearest, ties to even*, under which both orders in Example 1 (Lecture 1) give 4. Non-associativity does not depend on the rounding mode, however. With ties to even,

$$(1+u)+u = 1 \qquad \text{but} \qquad 1+(u+u) = 1+\varepsilon, \tag{C.2}$$

since on the left each intermediate result is a tie that rounds back to 1, while on the right $u+u = \varepsilon$ and $1+\varepsilon$ are both exact.

C.2.2 The Fundamental Axiom

Error analysis needs rounding to be captured by a rule we can reason with. Let $\mathrm{fl}(x)$ denote the floating-point number nearest to x. Because the relative spacing is bounded, every x in the normal range satisfies

$$\mathrm{fl}(x) = x(1+\delta), \qquad |\delta| \le u. \tag{C.3}$$

IEEE requires $+$, $-$, $\times$, $\div$, and $\sqrt{\cdot}$ to be *correctly rounded*: the exact result is computed in infinite precision and then rounded once. This gives the following model, stated in the same form in [1, Lecture 13], [5, §2.4], and [2, §1.5].

Theorem 110 (Fundamental Axiom of Floating-Point Arithmetic)

For all floating-point numbers x, y and $\circledast \in \{+, -, \times, \div\}$,

$$\mathrm{fl}(x \circledast y) = (x \circledast y)(1+\delta), \qquad |\delta| \le u. \tag{C.4}$$

The axiom has an immediate *backward* reading. For addition,

$$\mathrm{fl}(x+y) = (x+y)(1+\delta) = x(1+\delta) + y(1+\delta), \tag{C.5}$$

so the computed sum is *exactly* the sum of two slightly perturbed inputs. Pushing errors back onto the inputs in this way is the main thread of error analysis in this course; see Section C.3.

C.2.3 Accumulation of Errors

A single operation incurs an error of size $O(u)$. For n operations, Golub and Van Loan write

$$\gamma_n \coloneqq \frac{nu}{1-nu} \approx nu \qquad (nu \ll 1). \tag{C.6}$$

Proposition 111 (Error in Inner Products)

For $\vec{x}, \vec{y} \in \mathbb{R}^n$,

$$\left|\mathrm{fl}(\vec{x}^\top \vec{y}) - \vec{x}^\top \vec{y}\right| \le \gamma_n \sum_{i=1}^{n} |x_i|\,|y_i| = \gamma_n \left|\vec{x}\right|^\top |\vec{y}|\,. \tag{C.7}$$

Similarly, for matrices, $|\mathrm{fl}(AB) - AB| \le \gamma_n |A|\,|B|$ entrywise.

The *shape* of this bound matters much more than its constant. The right-hand side involves $|\vec{x}|^\top |\vec{y}|$, not $\left|\vec{x}^\top \vec{y}\right|$. If all terms have the same sign, the two agree and the relative error is $O(nu)$, which is fine. If instead $\vec{x}^\top \vec{y}$ is a small number obtained by cancelling many terms of both signs, then $|\vec{x}|^\top |\vec{y}| \gg \left|\vec{x}^\top \vec{y}\right|$ and the relative error can be arbitrarily bad. The linear factor n is almost never the real issue (it is a very pessimistic worst case, and random errors grow more like $\sqrt{n}$). The real issue is always *cancellation*.

C.3 Backward Stability

The textbooks [1, Lecture 14] mostly use a stronger notion than the stability defined in Section 1.7 of Lecture 1.

Definition 112 (Backward Stability)

An algorithm $\hat{f}$ for f is *backward stable* if for every x there exists $\tilde{x}$ with

$$\hat{f}(x) = f(\tilde{x}) \qquad \text{and} \qquad \frac{|\tilde{x} - x|}{|x|} = O(u), \tag{C.8}$$

where $\tilde{x}$ is a hypothetical input constructed in the analysis, never computed.

In words: *the computed answer is the exact answer to a nearby problem*. Both halves of the definition matter.

- The two sides of $\hat{f}(x) = f(\tilde{x})$ are computed differently. The left side runs the algorithm on the computer, with rounding, on the true input x. The right side evaluates the exact mathematical function f, with no rounding at all, on the perturbed input $\tilde{x}$.
- The perturbation must be small, $|\tilde{x} - x| \,/\, |x| = O(u)$. Without this condition the definition would be empty, since for most f some $\tilde{x}$ reproduces any output whatsoever.

Backward stability does not make the rounding errors disappear; it is a way of *accounting* for them. All errors committed inside the algorithm are pushed back to the input and reinterpreted as a perturbation $\tilde{x} - x$ of the data. The backward reading of Theorem 110 is the simplest example: $\mathrm{fl}(x + y)$ is the exact sum of $\tilde{x} = x(1 + \delta)$ and $\tilde{y} = y(1 + \delta)$, which are within relative distance u of x and y.

This accounting is useful because the input carries errors of size $O(u)$ anyway, from rounding when it is stored or from measurement. A backward stable algorithm gives the exact answer for some input that is indistinguishable from the one we have, which is as much as can be asked of any algorithm. Whatever error remains is due to the problem, not the algorithm. Backward stability is the gold standard of numerical linear algebra because it cleanly separates the problem from the algorithm, and it yields the classical rule of thumb

$$\text{forward relative error} \lesssim \kappa_f(x) \cdot u, \tag{C.9}$$

or, in a form that is easier to remember,

$$\text{accuracy} \approx \text{condition number (problem)} \times \text{backward error (algorithm)}. \tag{C.10}$$

A backward stable algorithm still returns inaccurate answers for ill-conditioned problems, and this is *not the algorithm's fault*; it cannot be improved upon. Conversely, when a result is inaccurate, the first step should be to estimate the condition number, not to suspect the code.

Concept	Belongs to	Question it answers	Measure
Conditioning	problem f and input x	how much input perturbations are intrinsically amplified	$\kappa_f(x)$
Stability	algorithm $\hat{f}$	whether the implementation amplifies errors further	backward error $O(u)$
Accuracy	final result	whether the computed number is right	$\lesssim \kappa \cdot u$

Some common condition numbers:

Problem	κ	Remark
$f(x) = \sqrt{x}$	$1/2$	always well-conditioned; even shrinks errors
$f(x) = x^n$	n	grows linearly with the power
$f(x) = e^x$	$\lvert x\rvert$	ill-conditioned for large $\lvert x\rvert$
$f(x_1, x_2) = x_1 - x_2$	$\dfrac{\lvert x_1\rvert + \lvert x_2\rvert}{\lvert x_1 - x_2\rvert}$	cancellation (componentwise perturbations)
$\vec{x} \mapsto A\vec{x}$	$\dfrac{\lVert A\rVert \, \lVert\vec{x}\rVert}{\lVert A\vec{x}\rVert} \le \kappa(A)$	depends on the direction of $\vec{x}$
solve $A\vec{x} = \vec{b}$	$\kappa(A) = \lVert A\rVert \, \lVert A^{-1}\rVert$	
polynomial roots	can be enormous	Wilkinson's polynomial

For $f : \mathbb{R}^n \to \mathbb{R}^m$ one replaces f' by the Jacobian $J(\vec{x})$: the absolute condition number is $\hat{\kappa} = \lVert J(\vec{x})\rVert$ and the relative condition number is $\kappa = \lVert J(\vec{x})\rVert \, \lVert\vec{x}\rVert \, / \, \lVert f(\vec{x})\rVert$ [1, Lecture 12].

Bibliography

[1] L. N. Trefethen and D. Bau III. *Numerical Linear Algebra*. SIAM, 1997.

[2] J. W. Demmel. *Applied Numerical Linear Algebra*. SIAM, 1997.

[3] D. C. Lay, S. R. Lay, and J. J. McDonald. *Linear Algebra and Its Applications*, 6th edition. Pearson, 2021.

[4] S. Axler. *Linear Algebra Done Right*, 4th edition. Springer, 2024.

[5] G. H. Golub and C. F. Van Loan. *Matrix Computations*, 3rd edition. Johns Hopkins University Press, 1996.

[6] M. L. Overton. *Numerical Computing with IEEE Floating Point Arithmetic*. SIAM, 2001.

[7] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2nd edition. SIAM, 2002.

[8] N. J. Higham. *Functions of Matrices: Theory and Computation*. SIAM, 2008.